Corso completo di C++ moderno

Dalle basi al C++20/23: strutture dati, algoritmi, web, mobile, videogiochi, embedded, sistemi operativi, reti e database

Con esempi svolti, esercizi risolti e problemi consigliati da LeetCode per ogni capitolo

Indice

Parte 1 - Fondamenti del linguaggio 4

Capitolo 1 - Introduzione e ambiente di lavoro . . . 4
Capitolo 2 - Tipi, variabili e operatori . . . 6
Capitolo 3 - Controllo di flusso e funzioni . . . 9
Capitolo 4 - Puntatori, riferimenti e memoria . . . 11
Capitolo 5 - Stringhe, I/O e file . . . 14

Parte 2 - OOP, template e C++ moderno 18

Capitolo 6 - Programmazione orientata agli oggetti . . . 18
Capitolo 7 - Template e programmazione generica . . . 23
Capitolo 8 - Il C++ moderno: C++11, 14, 17, 20, 23 . . . 25

Parte 3 - STL e strutture dati 29

Capitolo 9 - La Standard Template Library . . . 29
Capitolo 10 - Strutture dati classiche (implementarle da zero) . . . 32

Parte 4 - Algoritmi principali 39

Capitolo 11 - Analisi e tecniche di base . . . 39
Capitolo 12 - Ordinamento . . . 42
Capitolo 13 - Ricorsione, backtracking, greedy . . . 44
Capitolo 14 - Programmazione dinamica . . . 48
Capitolo 15 - Algoritmi su grafi . . . 50
Capitolo 16 - Stringhe, matematica e bit manipulation . . . 53

Parte 5 - Concorrenza, prestazioni e librerie 57

Capitolo 17 - Concorrenza e parallelismo . . . 57
Capitolo 18 - Prestazioni e ottimizzazione . . . 60
Capitolo 19 - Le principali librerie del C++ . . . 62

Parte 6 - Programmazione web e mobile 66

Capitolo 20 - Programmazione web con C++ . . . 66
Capitolo 21 - Programmazione mobile . . . 70

Parte 7 - Videogiochi e grafica 75

Capitolo 22 - Fondamenti di game programming . . . 75
Capitolo 23 - Librerie e motori . . . 80
Capitolo 24 - IA, rete e ottimizzazione nei giochi . . . 82

Parte 8 - Sistemi embedded 87

Capitolo 25 - Fondamenti dell'embedded con C++ . . . 87
Capitolo 26 - Protocolli, RTOS e sistemi connessi . . . 91

Parte 9 - Sistemi operativi 97

Capitolo 27 - Programmazione di sistema (POSIX/Linux e Windows) . . . 97
Capitolo 28 - Concetti di sistemi operativi . . . 100
Capitolo 29 - Costruire un sistema operativo . . . 103

Parte 10 - Programmazione di reti 109

Capitolo 30 - Fondamenti di networking . . . 109
Capitolo 31 - Server scalabili . . . 112
Capitolo 32 - Algoritmi e progetti di rete . . . 116

Parte 11 - Programmazione di database 121

Capitolo 33 - Usare database da C++ . . . 121
Capitolo 34 - Costruire un database: strutture e algoritmi . . . 126

Parte 12 - Percorso di studio e progetti finali 134

Capitolo 35 - Piano di studio e preparazione ai colloqui . . . 134

Appendice - Soluzioni degli esercizi 137

Soluzioni - Parte 1 . . . 137
Soluzioni - Parte 2 . . . 146
Soluzioni - Parte 3 . . . 155
Soluzioni - Parte 4 . . . 162
Soluzioni - Parte 5 . . . 178
Soluzioni - Parte 6 . . . 184
Soluzioni - Parte 7 . . . 191
Soluzioni - Parte 8 . . . 203
Soluzioni - Parte 9 . . . 209
Soluzioni - Parte 10 . . . 221
Soluzioni - Parti 11 e 12 . . . 232

Parte 1 - Fondamenti del linguaggio

Capitolo 1 - Introduzione e ambiente di lavoro

Lezione 1.1 - Cos'e' il C++ e perche' impararlo

Il C++ e' un linguaggio compilato, staticamente tipizzato e multi-paradigma (procedurale, orientato agli oggetti, generico, funzionale). Creato da Bjarne Stroustrup nel 1979 come "C with Classes", e' standardizzato da ISO: C++98, C++03, C++11, C++14, C++17, C++20, C++23. Questo corso usa lo stile del **C++ moderno** (da C++11 in poi): niente `new`/`delete` manuali, niente puntatori grezzi proprietari, molto uso della libreria standard.

Il C++ e' usato dove servono prestazioni e controllo: motori di gioco (Unreal), browser (Chrome, Firefox), database (MySQL, MongoDB, ClickHouse), sistemi operativi e driver, firmware embedded, finanza ad alta frequenza, grafica e machine learning (backend di PyTorch, TensorFlow).

- **Zero-cost abstraction**: non paghi per cio' che non usi.
- **Controllo della memoria**: gestione deterministica delle risorse tramite RAII.
- **Ecosistema enorme**: STL, Boost, Qt, Eigen, Asio, SDL e molto altro.

Lezione 1.2 - Compilatori, toolchain e primo programma

Compilatori principali: GCC (`g++`), Clang (`clang++`), MSVC. Per seguire il corso installa GCC 13+ o Clang 17+ e CMake. Il processo di build ha quattro fasi: preprocessore, compilazione, assemblaggio, linking.

```
#include <iostream>

int main() {
    std::cout << "Ciao, mondo!\n";
    return 0;
}
```

```
g++ -std=c++20 -Wall -Wextra -O2 main.cpp -o main
./main
```

Opzioni utili: `-Wall -Wextra -Wpedantic` (avvisi), `-g` (debug), `-O2` (ottimizzazione), `-fsanitize=address,undefined` (sanitizer, indispensabili durante lo sviluppo).

> **Nota**
>
> Compila sempre con gli avvisi attivi e con i sanitizer in debug: individuano la maggior parte degli errori di memoria e di comportamento indefinito.

Lezione 1.3 - Strumenti: CMake, debugger, IDE

CMake e' lo standard de facto per descrivere progetti C++ multipiattaforma.

```
cmake_minimum_required(VERSION 3.20)
project(corso LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
add_executable(corso src/main.cpp)
target_compile_options(corso PRIVATE -Wall -Wextra)
```

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j
```

Altri strumenti: **gdb/lldb** (debugger), **valgrind**, **clang-format** (formattazione), **clang-tidy** (analisi statica), **Conan/vcpkg** (gestione dipendenze), **Catch2/GoogleTest** (test), IDE come CLion, VS Code (con clangd) e Visual Studio.

Lezione 1.4 - Esempi svolti ed esercizi

Esempio svolto 1.1 - Un programma che legge la riga di comando

Problema. Scrivere un programma che riceve dei numeri come argomenti (`./somma 3 4 5`) e ne stampa quantita', somma e media; se un argomento non e' un numero va segnalato senza interrompere il programma.

Ragionamento. `main` puo' ricevere `argc` (numero di argomenti, compreso il nome del programma) e `argv` (array di stringhe C). Per convertire usiamo `std::from_chars` (C++17): non lancia eccezioni, non dipende dal locale e ci dice esattamente fin dove ha letto. Per rendere l'esempio riproducibile simuliamo gli argomenti con un array.

```
#include <charconv>
#include <iostream>
#include <string_view>

int elabora(int argc, const char* argv[]) {
    int validi = 0;
    long long somma = 0;
    for (int i = 1; i < argc; ++i) {
        std::string_view s = argv[i];
        long long v = 0;
        auto [fine, err] = std::from_chars(s.data(), s.data() + s.size(), v);
        if (err != std::errc{} || fine != s.data() + s.size()) {
            std::cerr << "ignoro '" << s << "'\n";
            continue;
        }
        somma += v;
        ++validi;
    }
    std::cout << "numeri: " << validi << ", somma: " << somma;
    if (validi > 0) std::cout << ", media: " << static_cast<double>(somma) / validi;
    std::cout << '\n';
    return validi > 0 ? 0 : 1;
}

int main() {
    const char* args[] = {"somma", "3", "4", "x7", "5"};   // come da shell
    return elabora(5, args);
}
```

```
numeri: 3, somma: 12, media: 4
```

Nel programma reale si scrive `int main(int argc, char* argv[]) { ... }`. Il codice di uscita (0 = successo) e' visibile nella shell con `echo $?` ed e' cio' che usano script e sistemi di CI.

Esempio svolto 1.2 - Trovare un bug con i sanitizer

Problema. Il programma seguente "funziona" sulla macchina dell'autore ma a volte stampa valori casuali. Trovare l'errore.

```
// versione con bug - non eseguire
#include <iostream>
#include <vector>
int main() {
    std::vector<int> v{1, 2, 3, 4};
    int somma = 0;
    for (std::size_t i = 0; i <= v.size(); ++i) somma += v[i];   // <= invece di <
    std::cout << somma << '\n';
}
```

Ragionamento. Compiliamo con `-fsanitize=address -g`: AddressSanitizer inserisce controlli su ogni accesso alla memoria e al primo accesso fuori limiti stampa un rapporto con file e riga.

```
$ g++ -std=c++20 -g -fsanitize=address,undefined bug.cpp -o bug && ./bug
==1234==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x502000000020
READ of size 4 at 0x502000000020 thread T0
    #0 0x55d1 in main bug.cpp:7
```

Il rapporto indica la riga 7: con `i <= v.size()` l'ultima iterazione legge `v[4]`, che non esiste. La correzione migliore elimina l'indice: `for (int x : v) somma += x;` oppure `std::accumulate(v.begin(), v.end(), 0)`. Se un indice e' davvero necessario, `v.at(i)` lancia `std::out_of_range` invece di produrre comportamento indefinito.

Esercizi (soluzioni nell'Appendice)

Esercizio 1.1 [Base] - Modifica "Ciao, mondo!" perche' saluti il nome passato come primo argomento (`./ciao Anna` stampa `Ciao, Anna!`) e usi "mondo" se non viene passato nulla.

Esercizio 1.2 [Base] - Compila con `-Wall -Wextra` il frammento `int f(int x) { int y; if (x > 0) return x; }` e spiega i due avvisi che ottieni. Correggilo.

Esercizio 1.3 [Medio] - Scrivi un `CMakeLists.txt` per un progetto con una libreria `matematica` (file `src/matematica.cpp` e `include/matematica.hpp`) e un eseguibile `app` che la usa. La libreria deve esporre la cartella `include` a chi la collega.

Esercizio 1.4 [Medio] - Esegui manualmente le quattro fasi della compilazione di `main.cpp` (`-E`, `-S`, `-c`, linking) e descrivi cosa contiene ciascun file prodotto.

Capitolo 2 - Tipi, variabili e operatori

Lezione 2.1 - Tipi fondamentali

Tipi interi: `int`, `long`, `long long`, `short`, `char` e varianti `unsigned`. Per dimensioni garantite usa `<cstdint>`: `int32_t`, `uint64_t`, ecc. Tipi in virgola mobile: `float`, `double`, `long double`. Altri: `bool`, `char`, `wchar_t`, `char8_t` (C++20), `std::byte`.

```
#include <cstdint>
#include <iostream>
int main() {
    int a = 42;
    double pi = 3.14159;
    std::uint8_t byte = 255;
    bool ok = true;
    auto x = 10u;          // deduzione: unsigned int
    int b{7};              // inizializzazione uniforme (vieta narrowing)
    constexpr int N = 100; // valutata a compile time
    std::cout << sizeof(int) << ' ' << sizeof(long long) << '\n';
}
```

Preferisci l'inizializzazione con parentesi graffe `{}`: impedisce le conversioni con perdita (`int x{3.5};` e' un errore). `auto` deduce il tipo, `const` e `constexpr` esprimono immutabilita'.

Esercizi consigliati su LeetCode

7 - Reverse Integer (overflow e limiti dei tipi)

9 - Palindrome Number

13 - Roman to Integer

Lezione 2.2 - Operatori e conversioni

Aritmetici (`+ - * / %`), confronto, logici (`&& || !`), bit a bit (`& | ^ ~ << >>`), assegnazione composta, ternario `?:`. Attenzione a: divisione tra interi (tronca), overflow dei signed (comportamento indefinito), confronti signed/unsigned. Usa `static_cast<T>(x)` per le conversioni esplicite; evita i cast in stile C.

```
int n = 7;
double r = static_cast<double>(n) / 2;   // 3.5
unsigned flags = 0b1010;
flags |= (1u << 0);                      // imposta il bit 0
bool bit3 = flags & (1u << 3);
```

Il C++20 introduce l'operatore a tre vie `<=>` (spaceship) che genera automaticamente tutti i confronti.

Esercizi consigliati su LeetCode

191 - Number of 1 Bits

231 - Power of Two

136 - Single Number (XOR)

338 - Counting Bits

190 - Reverse Bits

Lezione 2.3 - Esempi svolti ed esercizi

Esempio svolto 2.1 - Invertire le cifre senza overflow (LeetCode 7)

Problema. Data un intero a 32 bit, restituire le sue cifre in ordine inverso; se il risultato non sta in un `int32_t` restituire 0. Non e' consentito usare un tipo piu' grande.

Ragionamento. Ad ogni passo togliamo l'ultima cifra con `% 10` e la accodiamo con `r = r * 10 + cifra`. L'overflow dei signed e' comportamento indefinito, quindi va **evitato prima** che accada: confrontiamo `r` con `INT_MAX / 10` (e `INT_MIN / 10`) prima della moltiplicazione. Nota: in C++ `%` su un negativo da' un resto negativo (`-123 % 10 == -3`), il che semplifica il codice.

```
#include <cstdint>
#include <iostream>
#include <limits>

std::int32_t inverti(std::int32_t x) {
    constexpr auto MAX = std::numeric_limits<std::int32_t>::max();
    constexpr auto MIN = std::numeric_limits<std::int32_t>::min();
    std::int32_t r = 0;
    while (x != 0) {
        int cifra = x % 10;
        x /= 10;
        if (r > MAX / 10 || (r == MAX / 10 && cifra > 7)) return 0;   // MAX = 2147483647
        if (r < MIN / 10 || (r == MIN / 10 && cifra < -8)) return 0;  // MIN = -2147483648
        r = r * 10 + cifra;
    }
    return r;
}

int main() {
    for (std::int32_t x : {123, -120, 1534236469, 0})
        std::cout << x << " -> " << inverti(x) << '\n';
}
```

```
123 -> 321
-120 -> -21
1534236469 -> 0
0 -> 0
```

Complessita': O(numero di cifre) = O(log10 x) tempo, O(1) spazio.

Esempio svolto 2.2 - Flag a bit con enum class

Problema. Rappresentare i permessi di un file (lettura, scrittura, esecuzione) in un solo byte, in modo type-safe.

Ragionamento. Ogni permesso e' un bit. Con `enum class` i valori non si convertono implicitamente in interi, quindi definiamo noi gli operatori `|` e `&`. `std::to_underlying` (C++23) o `static_cast` estraggono il valore numerico; `std::popcount` (in `<bit>`, C++20) conta i bit accesi.

```
#include <bit>
#include <cstdint>
#include <iostream>

enum class Permesso : std::uint8_t {
    Nessuno = 0, Lettura = 1, Scrittura = 2, Esecuzione = 4
};

constexpr Permesso operator|(Permesso a, Permesso b) {
    auto x = static_cast<std::uint8_t>(a) | static_cast<std::uint8_t>(b);
    return static_cast<Permesso>(x);
}
constexpr bool ha(Permesso insieme, Permesso p) {
    return (static_cast<std::uint8_t>(insieme) & static_cast<std::uint8_t>(p)) != 0;
}

int main() {
    Permesso p = Permesso::Lettura | Permesso::Esecuzione;
    auto bit = static_cast<std::uint8_t>(p);
    std::cout << "valore: " << int(bit) << ", attivi: " << std::popcount(bit) << '\n';
    std::cout << "scrittura? " << std::boolalpha << ha(p, Permesso::Scrittura) << '\n';
    static_assert(ha(Permesso::Lettura | Permesso::Scrittura, Permesso::Scrittura));
}
```

```
valore: 5, attivi: 2
scrittura? false
```

Esercizi (soluzioni nell'Appendice)

Esercizio 2.1 [Base] - Stampa una tabella di conversione da Celsius a Fahrenheit (F = C * 9/5 + 32) per C da -20 a 40 a passi di 10. Attenzione: `9/5` tra interi vale 1.

Esercizio 2.2 [Base] - Scrivi `bool bisestile(int anno)`: un anno e' bisestile se divisibile per 4, ma non per 100, a meno che sia divisibile per 400.

Esercizio 2.3 [Medio] - Conta i bit a 1 di un `uint32_t` senza `std::popcount`, usando il trucco di Kernighan `n &= n - 1`. Perche' il ciclo esegue tante iterazioni quanti sono i bit accesi?

Esercizio 2.4 [Medio] - Prevedi e poi verifica l'output di: `std::cout << (-1 < 0u) << ' ' << (7 / 2) << ' ' << (-7 / 2) << ' ' << (-7 % 3);`. Spiega ogni risultato.

Esercizio 2.5 [Medio] - Converti un numero romano in intero (LeetCode 13): `"MCMXCIV"` vale 1994.

Capitolo 3 - Controllo di flusso e funzioni

Lezione 3.1 - Istruzioni condizionali e cicli

`if`, `else`, `switch`, `for`, `while`, `do-while`, `break`, `continue`. Il C++17 consente l'inizializzazione nell'`if` e nello `switch`; il C++11 introduce il ciclo range-based.

```
#include <vector>
#include <map>
#include <iostream>
int main() {
    std::vector<int> v{1, 2, 3, 4};
    for (int x : v) std::cout << x << ' ';

    std::map<std::string, int> m{{"a", 1}, {"b", 2}};
    for (const auto& [chiave, valore] : m)        // structured bindings (C++17)
        std::cout << chiave << '=' << valore << '\n';

    if (auto it = m.find("a"); it != m.end())      // init-statement (C++17)
        std::cout << it->second;
}
```

Esercizi consigliati su LeetCode

1 - Two Sum

26 - Remove Duplicates from Sorted Array

27 - Remove Element

412 - Fizz Buzz

1480 - Running Sum of 1d Array

Lezione 3.2 - Funzioni, overloading e lambda

Una funzione ha tipo di ritorno, nome e parametri. Passaggio per valore, per riferimento (`T&`), per riferimento costante (`const T&`). L'overloading permette piu' funzioni con lo stesso nome ma parametri diversi. Gli argomenti di default riducono la duplicazione.

```
#include <algorithm>
#include <vector>
int somma(int a, int b = 0) { return a + b; }
double somma(double a, double b) { return a + b; }

[[nodiscard]] constexpr int fattoriale(int n) {
    return n <= 1 ? 1 : n * fattoriale(n - 1);
}

int main() {
    std::vector<int> v{5, 2, 9, 1};
    int soglia = 3;
    auto grandi = std::count_if(v.begin(), v.end(), [soglia](int x) { return x > soglia; });
    auto quadrato = [](auto x) { return x * x; };   // lambda generica
    static_assert(fattoriale(5) == 120);
}
```

La ricorsione e' una tecnica fondamentale: ogni funzione ricorsiva ha un caso base e un passo ricorsivo.

Esercizi consigliati su LeetCode

509 - Fibonacci Number

70 - Climbing Stairs

50 - Pow(x, n)

344 - Reverse String

Lezione 3.3 - Esempi svolti ed esercizi

Esempio svolto 3.1 - Numeri primi con una funzione e un ciclo

Problema. Stampare i numeri primi minori di 50, dieci per riga.

Ragionamento. Un numero `n >= 2` e' primo se nessun `d` con `d * d <= n` lo divide: basta provare fino alla radice, perche' se `n = a * b` almeno uno dei due fattori e' `<= sqrt(n)`. Scriviamo una funzione `constexpr` (cosi' possiamo anche verificarla a compile time) e un ciclo che va a capo ogni dieci numeri.

```
#include <iostream>

constexpr bool primo(int n) {
    if (n < 2) return false;
    for (int d = 2; d * d <= n; ++d)
        if (n % d == 0) return false;
    return true;
}
static_assert(primo(97) && !primo(91));   // 91 = 7 * 13

int main() {
    int trovati = 0;
    for (int n = 2; n < 50; ++n) {
        if (!primo(n)) continue;
        std::cout << n << (++trovati % 10 == 0 ? '\n' : ' ');
    }
    std::cout << "\ntotale: " << trovati << '\n';
}
```

```
2 3 5 7 11 13 17 19 23 29
31 37 41 43 47
totale: 15
```

Esempio svolto 3.2 - Fibonacci: ricorsione, memoizzazione, iterazione

Problema. Calcolare F(n) (F(0)=0, F(1)=1) per n fino a 90 e confrontare tre implementazioni.

Ragionamento. La ricorsione diretta ricalcola gli stessi valori un numero esponenziale di volte: O(phi^n). Memorizzando i risultati (memoizzazione) ogni valore si calcola una volta: O(n). La versione iterativa tiene solo gli ultimi due valori: O(n) tempo e O(1) spazio. F(90) sta in un `long long` (F(92) e' l'ultimo che ci sta).

```
#include <iostream>
#include <vector>

long long fibRic(int n) { return n < 2 ? n : fibRic(n - 1) + fibRic(n - 2); }

long long fibMemo(int n, std::vector<long long>& memo) {
    if (n < 2) return n;
    if (memo[n] != 0) return memo[n];
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}

long long fibIter(int n) {
    long long a = 0, b = 1;
    for (int i = 0; i < n; ++i) { long long c = a + b; a = b; b = c; }
    return a;
}

int main() {
    std::vector<long long> memo(91, 0);
    std::cout << fibRic(25) << ' ' << fibMemo(25, memo) << ' ' << fibIter(25) << '\n';
    std::cout << "F(90) = " << fibIter(90) << '\n';   // fibRic(90) non terminerebbe mai
}
```

```
75025 75025 75025
F(90) = 2880067194370816120
```

La stessa idea (non ricalcolare i sottoproblemi) e' la base della programmazione dinamica del Capitolo 14.

Esercizi (soluzioni nell'Appendice)

Esercizio 3.1 [Base] - Scrivi `mcd(a, b)` con l'algoritmo di Euclide (iterativo) e `mcm(a, b)` usando `a / mcd(a, b) * b`. Perche' si divide prima di moltiplicare?

Esercizio 3.2 [Base] - Dato `std::vector<std::string>` di parole, conta con `std::count_if` e una lambda quante hanno piu' di `k` lettere, con `k` letto da una variabile catturata.

Esercizio 3.3 [Medio] - Two Sum (LeetCode 1): restituisci gli indici di due elementi con somma `target`. Scrivi prima la versione O(n^2) con due cicli, poi quella O(n) con `std::unordered_map`.

Esercizio 3.4 [Medio] - Torre di Hanoi: stampa le mosse per spostare `n` dischi dal piolo A al piolo C. Quante mosse servono per `n` dischi?

Esercizio 3.5 [Avanzato] - Implementa `double potenza(double x, long long n)` in O(log n) (LeetCode 50), gestendo anche `n` negativo e `n = LLONG_MIN`.

Capitolo 4 - Puntatori, riferimenti e memoria

Lezione 4.1 - Riferimenti e puntatori

Un riferimento e' un alias di un oggetto esistente. Un puntatore contiene un indirizzo e puo' essere riassegnato o nullo (`nullptr`). Usa riferimenti quando possibile, puntatori quando serve opzionalita' o riassegnazione.

```
int x = 10;
int& r = x;        // riferimento
int* p = &x;       // puntatore
*p = 20;           // x vale 20
r++;               // x vale 21
int arr[3] = {1, 2, 3};
int* q = arr;      // decadimento array -> puntatore
q[1] = 5;          // aritmetica dei puntatori: *(q+1)
```

Lezione 4.2 - Stack, heap, durata e RAII

Gli oggetti automatici vivono nello **stack** e sono distrutti all'uscita dallo scope. Gli oggetti dinamici vivono nell'**heap**. **RAII** (Resource Acquisition Is Initialization) lega la durata di una risorsa (memoria, file, mutex, socket) alla vita di un oggetto: il distruttore la rilascia automaticamente, anche in presenza di eccezioni.

Errori classici: memory leak, dangling pointer, double free, buffer overflow, use-after-free. Il C++ moderno li evita con gli smart pointer e i contenitori della STL.

```
#include <memory>
struct Nodo { int valore; std::unique_ptr<Nodo> next; };

auto a = std::make_unique<int>(42);        // proprieta' esclusiva
auto b = std::make_shared<int>(7);         // proprieta' condivisa (conteggio)
std::weak_ptr<int> w = b;                  // osservatore non proprietario
```

> **Nota**
>
> Regola d'oro del C++ moderno: non scrivere `new` e `delete` nel codice applicativo. Usa `make_unique`, `make_shared` e i contenitori.

> **Esercizi consigliati su LeetCode**
>
> 141 - Linked List Cycle
>
> 206 - Reverse Linked List
>
> 21 - Merge Two Sorted Lists
>
> 876 - Middle of the Linked List

Lezione 4.3 - Esempi svolti ed esercizi

Esempio svolto 4.1 - Lista concatenata con unique_ptr e inversione in loco

Problema. Costruire una lista singolarmente concatenata da un vettore, stamparla, invertirla senza allocare nuovi nodi (LeetCode 206) e liberarla senza memory leak.

Ragionamento. Ogni nodo possiede il successivo tramite `std::unique_ptr`, quindi distruggendo la testa si distrugge tutta la lista. Per invertire scorriamo la lista spostando (`std::move`) i puntatori: `prec` e' la parte gia' invertita, `cur` quella da invertire. Usiamo puntatori grezzi solo per **osservare** (`const Nodo*` nella stampa), mai per possedere.

```
#include <iostream>
#include <memory>
#include <vector>

struct Nodo {
    int val;
    std::unique_ptr<Nodo> next;
};

std::unique_ptr<Nodo> daVettore(const std::vector<int>& v) {
    std::unique_ptr<Nodo> testa;
    for (auto it = v.rbegin(); it != v.rend(); ++it)        // inserimento in testa
        testa = std::make_unique<Nodo>(Nodo{*it, std::move(testa)});
    return testa;
}

void stampa(const Nodo* n) {
    for (; n; n = n->next.get()) std::cout << n->val << (n->next ? " -> " : "\n");
}

std::unique_ptr<Nodo> inverti(std::unique_ptr<Nodo> cur) {
    std::unique_ptr<Nodo> prec;
    while (cur) {
        auto succ = std::move(cur->next);   // stacca il resto
        cur->next = std::move(prec);        // aggancia alla parte invertita
        prec = std::move(cur);
        cur = std::move(succ);
    }
    return prec;
}

int main() {
    auto lista = daVettore({1, 2, 3, 4, 5});
    stampa(lista.get());
    lista = inverti(std::move(lista));
    stampa(lista.get());
}   // qui unique_ptr libera tutti i nodi
```

```
1 -> 2 -> 3 -> 4 -> 5
5 -> 4 -> 3 -> 2 -> 1
```

> **Nota**
>
> Con liste lunghissime (milioni di nodi) la distruzione ricorsiva dei `unique_ptr` puo' esaurire lo stack: in quel caso si scrive un distruttore iterativo che stacca i nodi uno alla volta.

Esempio svolto 4.2 - RAII e ordine di distruzione anche con eccezioni

Problema. Mostrare che le risorse acquisite in un blocco vengono rilasciate in ordine inverso anche quando viene lanciata un'eccezione.

Ragionamento. Gli oggetti automatici vengono distrutti all'uscita dallo scope, in ordine inverso di costruzione, sia in uscita normale sia durante lo **stack unwinding** di un'eccezione. Una piccola classe che stampa in costruttore e distruttore lo rende visibile.

```
#include <iostream>
#include <stdexcept>
#include <string>

class Risorsa {
    std::string nome_;
public:
    explicit Risorsa(std::string n) : nome_(std::move(n)) {
        std::cout << "apro " << nome_ << '\n';
    }
    ~Risorsa() { std::cout << "chiudo " << nome_ << '\n'; }
    Risorsa(const Risorsa&) = delete;
    Risorsa& operator=(const Risorsa&) = delete;
};

void lavora() {
    Risorsa file("file");
    Risorsa conn("connessione");
    throw std::runtime_error("errore di rete");
    Risorsa mai("mai creata");      // non viene raggiunta
}

int main() {
    try { lavora(); }
    catch (const std::exception& e) { std::cout << "gestito: " << e.what() << '\n'; }
}
```

```
apro file
apro connessione
chiudo connessione
chiudo file
gestito: errore di rete
```

Lo stesso meccanismo chiude file (`std::fstream`), rilascia mutex (`std::lock_guard`) e libera memoria (`std::unique_ptr`) senza `try/finally`.

Esercizi (soluzioni nell'Appendice)

Esercizio 4.1 [Base] - Scrivi due versioni di `scambia`: una con puntatori (`void scambia(int* a, int* b)`) e una con riferimenti. Quale e' piu' sicura e perche'?

Esercizio 4.2 [Base] - Trova l'errore: `const std::string& saluto() { std::string s = "ciao"; return s; }`. Come lo segnala il compilatore? Correggilo.

Esercizio 4.3 [Medio] - Implementa una classe template `ScopeGuard` che esegue una lambda nel distruttore, con un metodo `annulla()`. Usala per ripristinare una variabile globale.

Esercizio 4.4 [Medio] - Trova il nodo centrale di una lista (LeetCode 876) in una sola passata con due puntatori, uno che avanza di 1 e uno di 2.

Esercizio 4.5 [Avanzato] - Due oggetti `Genitore` e `Figlio` si puntano a vicenda con `std::shared_ptr`: dimostra con stampe nel distruttore che non vengono mai distrutti, poi risolvi con `std::weak_ptr`.

Capitolo 5 - Stringhe, I/O e file

Lezione 5.1 - std::string e std::string_view

`std::string` gestisce testo con memoria dinamica; `std::string_view` (C++17) e' una vista non proprietaria, ideale per parametri in sola lettura. Funzioni utili: `substr`, `find`, `replace`, `append`, `std::to_string`, `std::stoi`.

```
#include <string>
#include <string_view>
#include <format>      // C++20
#include <iostream>

int contaVocali(std::string_view s) {
    int n = 0;
    for (char c : s) if (std::string_view("aeiouAEIOU").find(c) != std::string_view::npos) ++n;
    return n;
}
int main() {
    std::string s = "Ciao mondo";
    std::cout << std::format("'{}' ha {} vocali\n", s, contaVocali(s));
}
```

Esercizi consigliati su LeetCode

14 - Longest Common Prefix

125 - Valid Palindrome

242 - Valid Anagram

387 - First Unique Character in a String

49 - Group Anagrams

5 - Longest Palindromic Substring

Lezione 5.2 - Stream e file

Le classi `std::ifstream`, `std::ofstream` e `std::fstream` leggono e scrivono file, chiudendoli automaticamente (RAII). C++17 aggiunge `std::filesystem` per percorsi e directory.

```
#include <fstream>
#include <filesystem>
#include <sstream>
#include <iostream>
namespace fs = std::filesystem;

int main() {
    { std::ofstream out("dati.txt"); out << "riga 1\nriga 2\n"; }
    std::ifstream in("dati.txt");
    std::string riga;
    while (std::getline(in, riga)) std::cout << riga << '\n';
    for (auto& e : fs::directory_iterator(".")) std::cout << e.path() << '\n';
}
```

Lezione 5.3 - Esempi svolti ed esercizi

Esempio svolto 5.1 - Frequenza delle parole in un testo

Problema. Dato un testo, contare le parole ignorando maiuscole e punteggiatura e stampare le tre piu' frequenti (a parita' di frequenza, in ordine alfabetico).

Ragionamento. Scorriamo i caratteri: le lettere si accumulano (in minuscolo) nella parola corrente, qualunque altro carattere chiude la parola. Contiamo con una `std::map<std::string, int>`; poi copiamo le coppie in un vettore e ordiniamo con un comparatore a due criteri. `std::tolower` richiede un `unsigned char` per evitare comportamento indefinito con caratteri negativi.

```
#include <algorithm>
#include <cctype>
#include <iostream>
#include <map>
#include <string>
#include <vector>

std::map<std::string, int> conta(const std::string& testo) {
    std::map<std::string, int> freq;
    std::string parola;
    for (char c : testo + ' ') {                // lo spazio finale chiude l'ultima parola
        auto u = static_cast<unsigned char>(c);
        if (std::isalpha(u)) parola += static_cast<char>(std::tolower(u));
        else if (!parola.empty()) { ++freq[parola]; parola.clear(); }
    }
    return freq;
}

int main() {
    std::string t = "Il gatto e il cane. Il cane dorme; il gatto no! Cane, gatto, topo.";
    auto freq = conta(t);
    std::vector<std::pair<std::string, int>> v(freq.begin(), freq.end());
    std::sort(v.begin(), v.end(), [](const auto& a, const auto& b) {
        return a.second != b.second ? a.second > b.second : a.first < b.first;
    });
    for (std::size_t i = 0; i < 3 && i < v.size(); ++i)
        std::cout << v[i].first << ": " << v[i].second << '\n';
}
```

```
il: 4
cane: 3
gatto: 3
```

Esempio svolto 5.2 - Leggere e analizzare un file CSV

Problema. Scrivere un file `voti.csv` con righe `nome,voto1,voto2,voto3`, rileggerlo e stampare la media di ogni studente, scartando le righe malformate.

Ragionamento. `std::getline(in, riga)` legge una riga alla volta; un `std::istringstream` sulla riga permette di usare `std::getline(ss, campo, ',')` per separare i campi. `std::stod` converte e lancia un'eccezione se il campo non e' un numero: la catturiamo per scartare la riga.

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

int main() {
    {
        std::ofstream out("voti.csv");
        out << "Anna,8,7.5,9\nLuca,6,x,7\nSara,10,9,9.5\n";
    }   // file chiuso qui (RAII)

    std::ifstream in("voti.csv");
    std::string riga;
    int numRiga = 0;
    while (std::getline(in, riga)) {
        ++numRiga;
        std::istringstream ss(riga);
        std::string nome, campo;
        std::getline(ss, nome, ',');
        std::vector<double> voti;
        try {
            while (std::getline(ss, campo, ',')) voti.push_back(std::stod(campo));
        } catch (const std::exception&) {
            std::cout << "riga " << numRiga << " scartata\n";
            continue;
        }
        double somma = 0;
        for (double v : voti) somma += v;
        std::cout << nome << ": " << std::fixed << std::setprecision(2)
                  << somma / voti.size() << '\n';
    }
}
```

```
Anna: 8.17
riga 2 scartata
Sara: 9.50
```

Esercizi (soluzioni nell'Appendice)

Esercizio 5.1 [Base] - Valid Palindrome (LeetCode 125): verifica se una stringa e' palindroma considerando solo lettere e cifre e ignorando le maiuscole. Usa due indici, senza creare copie.

Esercizio 5.2 [Base] - Scrivi `std::vector<std::string_view> dividi(std::string_view s, char sep)`. Che cosa succede se la stringa originale viene distrutta prima di usare il risultato?

Esercizio 5.3 [Medio] - Cifrario di Cesare: implementa `cifra(s, k)` e `decifra(s, k)` che spostano solo le lettere (maiuscole e minuscole) di `k` posizioni con wrap-around.

Esercizio 5.4 [Medio] - Scrivi un piccolo `wc`: dato il nome di un file, stampa numero di righe, parole e caratteri.

Esercizio 5.5 [Avanzato] - Group Anagrams (LeetCode 49): raggruppa le parole che sono anagrammi tra loro usando come chiave la parola ordinata. Qual e' la complessita'?

Parte 2 - OOP, template e C++ moderno

Capitolo 6 - Programmazione orientata agli oggetti

Lezione 6.1 - Classi, costruttori e distruttori

Una classe incapsula dati e comportamento. `struct` e `class` differiscono solo per la visibilita' di default (pubblica vs privata). I costruttori inizializzano, il distruttore libera le risorse.

```
#include <string>
#include <utility>

class Conto {
public:
    Conto(std::string titolare, double saldo = 0.0)
        : titolare_(std::move(titolare)), saldo_(saldo) {}
    void versa(double v) { saldo_ += v; }
    [[nodiscard]] double saldo() const noexcept { return saldo_; }
private:
    std::string titolare_;
    double saldo_;
};
```

- Usa la **lista di inizializzazione** dei membri.
- Marca `const` i metodi che non modificano lo stato.
- `explicit` sui costruttori a un argomento evita conversioni implicite.

Lezione 6.2 - Regola del 5 / del 0, copia e move

Se una classe gestisce una risorsa servono: distruttore, costruttore di copia, assegnazione di copia, costruttore di move, assegnazione di move (**regola del 5**). Meglio ancora: usa membri che gestiscono la risorsa (`vector`, `unique_ptr`) e non scrivere nulla (**regola del 0**).

La **move semantics** (C++11) trasferisce le risorse da oggetti temporanei (rvalue) invece di copiarle: `std::move` converte in rvalue reference `T&&`.

```
class Buffer {
    std::size_t n_; int* d_;
public:
    explicit Buffer(std::size_t n) : n_(n), d_(new int[n]{}) {}
    ~Buffer() { delete[] d_; }
    Buffer(const Buffer& o) : n_(o.n_), d_(new int[o.n_]) { std::copy(o.d_, o.d_ + n_, d_); }
    Buffer& operator=(Buffer o) noexcept { std::swap(n_, o.n_); std::swap(d_, o.d_); return *this; }
    Buffer(Buffer&& o) noexcept : n_(std::exchange(o.n_, 0)), d_(std::exchange(o.d_, nullptr)) {}
};
```

Lezione 6.3 - Ereditarieta' e polimorfismo

Il polimorfismo a runtime usa funzioni `virtual`. La classe base deve avere distruttore virtuale. `override` e `final` rendono esplicite le intenzioni. Preferisci la composizione all'ereditarieta' quando possibile.

```
#include <memory>
#include <vector>
#include <numbers>
struct Forma {
    virtual ~Forma() = default;
    virtual double area() const = 0;   // classe astratta
};
struct Cerchio : Forma {
    double r;
    explicit Cerchio(double r) : r(r) {}
    double area() const override { return std::numbers::pi * r * r; }
};
struct Rettangolo : Forma {
    double w, h;
    Rettangolo(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
};
int main() {
    std::vector<std::unique_ptr<Forma>> v;
    v.push_back(std::make_unique<Cerchio>(2));
    v.push_back(std::make_unique<Rettangolo>(3, 4));
    double tot = 0; for (auto& f : v) tot += f->area();
}
```

Lezione 6.4 - Overloading degli operatori, eccezioni, principi SOLID

Si possono ridefinire operatori (`+`, `==`, `<<`, `[]`, `()`) per tipi definiti dall'utente: fallo solo quando il significato e' naturale. Le eccezioni (`try`/`catch`/`throw`) segnalano errori eccezionali; con RAII garantiscono la pulizia. In alternativa: `std::optional`, `std::expected` (C++23), codici di errore.

I principi **SOLID** (Single responsibility, Open/closed, Liskov, Interface segregation, Dependency inversion) guidano un design manutenibile. Pattern importanti: Strategy, Observer, Factory, Builder, Singleton (con cautela), Visitor, Decorator, Adapter.

Esercizi consigliati su LeetCode

146 - LRU Cache (design di classi)

155 - Min Stack

232 - Implement Queue using Stacks

225 - Implement Stack using Queues

705 - Design HashSet

706 - Design HashMap

208 - Implement Trie (Prefix Tree)

355 - Design Twitter

Lezione 6.5 - Esempi svolti ed esercizi

Esempio svolto 6.1 - Una classe valore: Frazione

Problema. Progettare un tipo `Frazione` che si comporti come un numero: somma, prodotto, confronti e stampa, mantenendo sempre la forma ridotta con denominatore positivo.

Ragionamento. L'**invariante** della classe (ridotta, denominatore > 0) viene stabilito nel costruttore e preservato da ogni operazione, quindi il resto del codice puo' darlo per scontato. Gli operatori aritmetici si scrivono come funzioni libere `friend` (cosi' funzionano anche con un `int` a sinistra grazie al costruttore implicito); `operator<=>` di default non andrebbe bene, perche' confronterebbe i campi e non i valori,

quindi lo definiamo con la moltiplicazione incrociata.

```
#include <compare>
#include <iostream>
#include <numeric>
#include <stdexcept>

class Frazione {
    long long num_, den_;
public:
    Frazione(long long n = 0, long long d = 1) : num_(n), den_(d) {
        if (d == 0) throw std::invalid_argument("denominatore nullo");
        if (den_ < 0) { num_ = -num_; den_ = -den_; }
        long long g = std::gcd(num_, den_);
        num_ /= g; den_ /= g;
    }
    long long num() const { return num_; }
    long long den() const { return den_; }

    friend Frazione operator+(const Frazione& a, const Frazione& b) {
        return {a.num_ * b.den_ + b.num_ * a.den_, a.den_ * b.den_};
    }
    friend Frazione operator*(const Frazione& a, const Frazione& b) {
        return {a.num_ * b.num_, a.den_ * b.den_};
    }
    friend bool operator==(const Frazione&, const Frazione&) = default;  // forma ridotta
    friend std::strong_ordering operator<=>(const Frazione& a, const Frazione& b) {
        return a.num_ * b.den_ <=> b.num_ * a.den_;   // den > 0: il verso non cambia
    }
    friend std::ostream& operator<<(std::ostream& os, const Frazione& f) {
        return f.den_ == 1 ? os << f.num_ : os << f.num_ << '/' << f.den_;
    }
};

int main() {
    Frazione a{1, 2}, b{3, -4};
    std::cout << a << " + " << b << " = " << a + b << '\n';
    std::cout << a << " * " << b << " = " << a * b << '\n';
    std::cout << "2 + a = " << 2 + a << '\n';
    std::cout << std::boolalpha << (Frazione{2, 4} == a) << ' ' << (b < a) << '\n';
}
```

```
1/2 + -3/4 = -1/4
1/2 * -3/4 = -3/8
2 + a = 5/2
true true
```

Esempio svolto 6.2 - Min Stack (LeetCode 155)

Problema. Progettare una pila che supporti `push`, `pop`, `top` e `getMin` tutte in O(1).

Ragionamento. Cercare il minimo a ogni chiamata costerebbe O(n). Trucco: accanto a ogni elemento memorizziamo il minimo della pila **fino a quel punto**. Quando togliamo un elemento, il minimo precedente e' gia' pronto nella coppia sottostante. L'interfaccia nasconde la rappresentazione: chi usa `MinStack` non sa (e non deve sapere) che dentro ci sono coppie.

```
#include <algorithm>
#include <iostream>
#include <stdexcept>
#include <utility>
#include <vector>

class MinStack {
    std::vector<std::pair<int, int>> d_;   // (valore, minimo fino a qui)
public:
    void push(int x) { d_.emplace_back(x, d_.empty() ? x : std::min(x, d_.back().second)); }
    void pop() {
        if (d_.empty()) throw std::out_of_range("pila vuota");
        d_.pop_back();
    }
    int top() const { return d_.back().first; }
    int getMin() const { return d_.back().second; }
    bool empty() const { return d_.empty(); }
};

int main() {
    MinStack s;
    for (int x : {5, 3, 7, 2, 8}) s.push(x);
    while (!s.empty()) {
        std::cout << "top " << s.top() << " min " << s.getMin() << '\n';
        s.pop();
    }
}
```

```
top 8 min 2
top 2 min 2
top 7 min 3
top 3 min 3
top 5 min 5
```

Esempio svolto 6.3 - Polimorfismo e pattern Strategy

Problema. Un carrello calcola il totale applicando uno sconto che puo' cambiare a runtime (nessuno, percentuale, "3x2"). Aggiungere nuovi sconti non deve richiedere modifiche al carrello (principio Open/Closed).

Ragionamento. Definiamo un'interfaccia astratta `Sconto` e la iniettiamo nel carrello tramite `std::unique_ptr` (dependency inversion: il carrello dipende dall'astrazione, non dalle classi concrete).

```
#include <algorithm>
#include <iostream>
#include <memory>
#include <vector>

struct Sconto {
    virtual ~Sconto() = default;
    virtual double applica(const std::vector<double>& prezzi) const = 0;
};
struct NessunoSconto : Sconto {
    double applica(const std::vector<double>& p) const override {
        double t = 0; for (double x : p) t += x; return t;
    }
};
struct Percentuale : Sconto {
    double perc;
    explicit Percentuale(double p) : perc(p) {}
    double applica(const std::vector<double>& p) const override {
        return NessunoSconto{}.applica(p) * (1 - perc / 100);
    }
};
struct TrePerDue : Sconto {          // ogni 3 articoli il meno caro e' gratis (semplificato)
    double applica(const std::vector<double>& p) const override {
        double t = NessunoSconto{}.applica(p), minimo = p.empty() ? 0 : p[0];
        for (double x : p) minimo = std::min(minimo, x);
        return p.size() >= 3 ? t - minimo : t;
    }
};

class Carrello {
    std::vector<double> prezzi_;
    std::unique_ptr<Sconto> sconto_ = std::make_unique<NessunoSconto>();
public:
    void aggiungi(double p) { prezzi_.push_back(p); }
    void impostaSconto(std::unique_ptr<Sconto> s) { sconto_ = std::move(s); }
    double totale() const { return sconto_->applica(prezzi_); }
};

int main() {
    Carrello c;
    for (double p : {10.0, 20.0, 5.0}) c.aggiungi(p);
    std::cout << c.totale() << ' ';
    c.impostaSconto(std::make_unique<Percentuale>(10));
    std::cout << c.totale() << ' ';
    c.impostaSconto(std::make_unique<TrePerDue>());
```

```
    std::cout << c.totale() << '\n';
}
```

```
35 31.5 30
```

Esercizi (soluzioni nell'Appendice)

Esercizio 6.1 [Base] - Estendi la classe `Conto` della Lezione 6.1 con `preleva(double)` che lancia `std::invalid_argument` per importi non positivi e `std::runtime_error` se il saldo non basta. Scrivi un `main` che gestisce entrambe le eccezioni.

Esercizio 6.2 [Base] - Crea una gerarchia `Animale` (astratta, con `virtual std::string verso() const = 0`) e le classi `Cane`, `Gatto`, `Mucca`. Riempi un `std::vector<std::unique_ptr<Animale>>` e stampa i versi.

Esercizio 6.3 [Medio] - Scrivi una classe `Traccia` che stampa un messaggio in ognuna delle sei funzioni speciali (costruttore, distruttore, copia, assegnazione di copia, move, assegnazione di move). Usala per scoprire cosa succede con `push_back`, `emplace_back`, `std::move` e la restituzione per valore.

Esercizio 6.4 [Medio] - Implementa una coda con due pile (LeetCode 232) e dimostra che `pop` costa O(1) ammortizzato.

Esercizio 6.5 [Avanzato] - LRU Cache (LeetCode 146): `get` e `put` in O(1) con capacita' fissa, usando `std::list` e `std::unordered_map` di iteratori.

Capitolo 7 - Template e programmazione generica

Lezione 7.1 - Template di funzione e di classe

I template generano codice per tipi diversi a compile time senza overhead a runtime.

```
template <typename T>
T massimo(T a, T b) { return a < b ? b : a; }

template <typename T, std::size_t N>
class Array {
    T dati_[N]{};
public:
    constexpr T& operator[](std::size_t i) { return dati_[i]; }
    constexpr std::size_t size() const { return N; }
};

template <typename... Args>             // variadic template
auto somma(Args... args) { return (args + ...); }   // fold expression (C++17)
```

Lezione 7.2 - Concepts, SFINAE e metaprogrammazione

I **concepts** (C++20) esprimono vincoli sui parametri template con messaggi d'errore chiari, sostituendo il vecchio SFINAE. `if constexpr` (C++17) sceglie il ramo a compile time. `<type_traits>` offre `is_integral_v`, `enable_if`, `remove_reference_t`, ecc.

```
#include <concepts>
template <std::integral T>
T mcd(T a, T b) { return b == 0 ? a : mcd(b, a % b); }

template <typename T>
concept Stampabile = requires(T t, std::ostream& os) { os << t; };

template <Stampabile T>
void stampa(const T& x) { std::cout << x << '\n'; }

template <typename T>
auto descrivi(const T& v) {
    if constexpr (std::is_pointer_v<T>) return "puntatore";
    else return "valore";
}
```

Esercizi consigliati su LeetCode

703 - Kth Largest Element in a Stream

295 - Find Median from Data Stream

Lezione 7.3 - Esempi svolti ed esercizi

Esempio svolto 7.1 - Un algoritmo generico vincolato da concepts

Problema. Scrivere una funzione che restituisce l'elemento massimo di **qualsiasi** range (vettore, array, lista, `string`...) purche' i suoi elementi siano confrontabili, con un errore di compilazione chiaro negli altri casi.

Ragionamento. Il concept `std::ranges::forward_range` vincola il contenitore; `std::totally_ordered` vincola il tipo degli elementi, ricavato con `std::ranges::range_value_t`. Restituiamo `std::optional` perche' un range vuoto non ha massimo.

```
#include <concepts>
#include <iostream>
#include <list>
#include <optional>
#include <ranges>
#include <string>
#include <vector>

template <std::ranges::forward_range R>
    requires std::totally_ordered<std::ranges::range_value_t<R>>
std::optional<std::ranges::range_value_t<R>> massimo(const R& r) {
    auto it = std::ranges::begin(r);
    if (it == std::ranges::end(r)) return std::nullopt;
    auto m = *it;
    for (; it != std::ranges::end(r); ++it)
        if (m < *it) m = *it;
    return m;
}

struct Punto { int x, y; };   // nessun operator<: non soddisfa totally_ordered

int main() {
    std::vector<int> v{3, 9, 2};
    std::list<std::string> l{"pera", "mela", "uva"};
    std::cout << *massimo(v) << ' ' << *massimo(l) << ' ' << massimo(std::string("ciao")).value()
              << ' ' << massimo(std::vector<double>{}).has_value() << '\n';
    // massimo(std::vector<Punto>{});  // errore: vincoli non soddisfatti (totally_ordered)
}
```

```
9 uva o 0
```

Esempio svolto 7.2 - Template variadici, fold expression e if constexpr

Problema. Scrivere `stampaTutto(args...)` che stampa un numero qualsiasi di argomenti separati da virgole, racchiudendo le stringhe tra virgolette e i booleani come `vero/falso`.

Ragionamento. Un **pack** di parametri si espande con una fold expression sull'operatore virgola. Per ogni elemento, `if constexpr` sceglie a compile time come formattarlo: i rami scartati non vengono nemmeno istanziati, quindi possono contenere codice che per quel tipo non compilerebbe.

```
#include <iostream>
#include <string>
#include <string_view>
#include <type_traits>

template <typename T>
void stampaUno(const T& x) {
    using D = std::decay_t<T>;
    if constexpr (std::is_same_v<D, bool>) std::cout << (x ? "vero" : "falso");
    else if constexpr (std::is_convertible_v<D, std::string_view>) std::cout << '"' << x << '"';
    else if constexpr (std::is_floating_point_v<D>) std::cout << x << 'f';
    else std::cout << x;
}

template <typename... Args>
void stampaTutto(const Args&... args) {
    std::size_t i = 0;
    ((std::cout << (i++ ? ", " : ""), stampaUno(args)), ...);   // fold sulla virgola
    std::cout << " (" << sizeof...(args) << " argomenti)\n";
}

int main() {
    stampaTutto(42, "ciao", 3.5, true, std::string("C++"), 'x');
}
```

```
42, "ciao", 3.5f, vero, "C++", x (6 argomenti)
```

Esercizi (soluzioni nell'Appendice)

Esercizio 7.1 [Base] - Scrivi un template di classe `Coppia<T, U>` con membri `primo` e `secondo` e una funzione `scambiata()` che restituisce `Coppia<U, T>`. Verifica che `Coppia c{1, 2.5};` funzioni grazie alla CTAD.

Esercizio 7.2 [Base] - Definisci un concept `Numerico` (interi o virgola mobile, ma non `bool`) e una funzione `media(std::span<const T>)` vincolata da esso.

Esercizio 7.3 [Medio] - Scrivi un template di classe `Pila<T, N>` a capacita' fissa basato su `std::array`, con `push` che restituisce `false` quando e' piena. Aggiungi uno `static_assert(N > 0)`.

Esercizio 7.4 [Medio] - Kth Largest Element in a Stream (LeetCode 703): una classe che riceve numeri uno alla volta e restituisce in ogni momento il k-esimo piu' grande. Rendila generica sul tipo e sul comparatore.

Esercizio 7.5 [Avanzato] - Calcola a compile time la tabella dei primi 20 numeri di Fibonacci in un `std::array` con una funzione `consteval`, e verificane un valore con `static_assert`.

Capitolo 8 - Il C++ moderno: C++11, 14, 17, 20, 23

Lezione 8.1 - Novita' di C++11/14/17

- **C++11**: `auto`, range-for, lambda, `nullptr`, `move`, smart pointer, `constexpr`, `thread`, `unordered_map`, `enum class`, `override`, `= default`/`= delete`, inizializzazione uniforme.
- **C++14**: lambda generiche, `make_unique`, `decltype(auto)`, variable template, separatori numerici `1'000'000`.
- **C++17**: structured bindings, `std::optional`, `std::variant`, `std::string_view`, `std::filesystem`, `if constexpr`, fold expression, algoritmi paralleli, CTAD, `inline` variables.

```
#include <optional>
#include <variant>
#include <string>
std::optional<int> parseInt(const std::string& s) {
    try { return std::stoi(s); } catch (...) { return std::nullopt; }
}
using Valore = std::variant<int, double, std::string>;
struct Visitatore {
    void operator()(int i) const { /* ... */ }
    void operator()(double d) const { /* ... */ }
    void operator()(const std::string& s) const { /* ... */ }
};
// std::visit(Visitatore{}, valore);
```

Lezione 8.2 - Novita' di C++20/23: concepts, ranges, coroutine, moduli

- **Ranges**: pipeline componibili e pigre.
- **Coroutine**: `co_await`, `co_yield`, `co_return` per codice asincrono e generatori.
- **Moduli**: `import std;` sostituisce `#include` e velocizza la compilazione.
- `std::format`, `std::span`, `std::jthread`, `<=>`, `consteval`, `std::source_location`.
- **C++23**: `std::expected`, `std::print`, `std::mdspan`, `std::generator`, `deducing this`, `std::flat_map`.

```
#include <ranges>
#include <vector>
#include <print>   // C++23
namespace vw = std::views;
int main() {
    std::vector<int> v{1,2,3,4,5,6,7,8,9,10};
    auto r = v | vw::filter([](int x){ return x % 2 == 0; })
               | vw::transform([](int x){ return x * x; })
               | vw::take(3);
    for (int x : r) std::print("{} ", x);   // 4 16 36
}
```

Lezione 8.3 - Buone pratiche: C++ Core Guidelines

Segui le **C++ Core Guidelines** (Stroustrup e Sutter): `const` per default, parametri di ingresso per `const&` o per valore se piccoli, restituire per valore (grazie a copy elision/RVO), `noexcept` sulle funzioni di move, evitare le variabili globali, usare `std::span` invece di coppie puntatore+dimensione, `gsl::not_null`, `[[nodiscard]]`. Strumenti: clang-tidy (`modernize-*`, `cppcoreguidelines-*`), cppcheck, sanitizer.

Lezione 8.4 - Esempi svolti ed esercizi

Esempio svolto 8.1 - Configurazione con optional, variant e visit

Problema. Leggere righe `chiave = valore` e memorizzare il valore con il tipo giusto (intero, reale, booleano o stringa); poi stampare ogni voce con il suo tipo. Le righe senza `=` vanno ignorate.

Ragionamento. `std::variant` e' un'unione type-safe: contiene esattamente uno dei tipi elencati. Per visitarlo usiamo il classico idioma `overloaded`, che eredita da piu' lambda e ne unisce gli `operator()`. `std::optional` esprime il "forse": una riga non valida restituisce `std::nullopt` invece di un codice di errore.

```
#include <charconv>
#include <iostream>
#include <map>
#include <optional>
#include <string>
#include <variant>

using Valore = std::variant<long, double, bool, std::string>;

template <typename... F> struct overloaded : F... { using F::operator()...; };

std::string ripulisci(std::string s) {
    auto a = s.find_first_not_of(' '), b = s.find_last_not_of(' ');
    return a == std::string::npos ? "" : s.substr(a, b - a + 1);
}

Valore interpreta(const std::string& s) {
    if (s == "true" || s == "false") return s == "true";
    long l; auto fine = s.data() + s.size();
    if (auto [p, e] = std::from_chars(s.data(), fine, l); e == std::errc{} && p == fine) return l;
    double d;
    if (auto [p, e] = std::from_chars(s.data(), fine, d); e == std::errc{} && p == fine) return d;
    return s;
}

std::optional<std::pair<std::string, Valore>> leggiRiga(const std::string& riga) {
    auto pos = riga.find('=');
    if (pos == std::string::npos) return std::nullopt;
    return std::pair{ripulisci(riga.substr(0, pos)), interpreta(ripulisci(riga.substr(pos + 1)))};
}

int main() {
    std::map<std::string, Valore> conf;
    for (std::string r : {"porta = 8080", "ratio=0.75", "debug = true", "commento", "nome = srv"})
        if (auto kv = leggiRiga(r)) conf.insert(*kv);

    for (const auto& [k, v] : conf)
        std::visit(overloaded{
            [&](long x) { std::cout << k << ": intero " << x << '\n'; },
            [&](double x) { std::cout << k << ": reale " << x << '\n'; },
            [&](bool x) { std::cout << k << ": bool " << std::boolalpha << x << '\n'; },
            [&](const std::string& x) { std::cout << k << ": stringa " << x << '\n'; },
        }, v);
}
```

```
debug: bool true
nome: stringa srv
porta: intero 8080
ratio: reale 0.75
```

Esempio svolto 8.2 - Elaborare dati con ranges e proiezioni

Problema. Data una lista di studenti con nome, corso e voto, stampare in ordine di voto decrescente i nomi dei promossi (voto >= 18) del corso "C++", e calcolarne la media.

Ragionamento. Gli algoritmi `std::ranges::` accettano una **proiezione**: `std::ranges::sort(v, std::greater{}, &Studente::voto)` ordina per voto senza scrivere un comparatore. Le viste (`views::filter`, `views::transform`) sono pigre: non creano copie, calcolano gli elementi solo quando li si scorre.

```
#include <algorithm>
#include <functional>
#include <iostream>
#include <numeric>
#include <ranges>
#include <string>
#include <vector>

struct Studente { std::string nome, corso; int voto; };

int main() {
    std::vector<Studente> v{{"Anna", "C++", 28}, {"Bruno", "Java", 30}, {"Carla", "C++", 17},
                            {"Dario", "C++", 24}, {"Elena", "C++", 30}};
    std::ranges::sort(v, std::greater{}, &Studente::voto);

    auto promossi = v | std::views::filter([](const Studente& s) {
                            return s.corso == "C++" && s.voto >= 18; });
    for (const auto& nome : promossi | std::views::transform(&Studente::nome))
        std::cout << nome << ' ';

    auto voti = promossi | std::views::transform(&Studente::voto);
    double somma = std::accumulate(voti.begin(), voti.end(), 0.0);
    std::cout << "| media " << somma / std::ranges::distance(promossi) << '\n';
}
```

```
Elena Anna Dario | media 27.3333
```

Esercizi (soluzioni nell'Appendice)

Esercizio 8.1 [Base] - Modernizza questo codice C++03: `std::map<std::string, std::vector<int> >::const_iterator it; for (it = m.begin(); it != m.end(); ++it) { int* p = new int(it->second.size()); std::cout << it->first << *p; delete p; }`.

Esercizio 8.2 [Base] - Con `std::views::iota`, `filter`, `transform` e `take` stampa i quadrati dei primi 5 numeri dispari maggiori di 10.

Esercizio 8.3 [Medio] - Riscrivi `parseInt` della Lezione 8.1 restituendo `std::expected<int, std::string>` (C++23) con messaggi di errore diversi per "non e' un numero" e "fuori intervallo".

Esercizio 8.4 [Medio] - Data una `std::map<std::string, int>` di prodotti e quantita', trova con `std::ranges::max_element` e una proiezione il prodotto con quantita' massima e stampalo con structured bindings.

Esercizio 8.5 [Avanzato] - Modella una forma geometrica come `std::variant<Cerchio, Rettangolo, Triangolo>` e calcola l'area con `std::visit`, senza ereditarieta' ne' funzioni virtuali. Confronta pregi e difetti con la soluzione della Lezione 6.3.

Parte 3 - STL e strutture dati

Capitolo 9 - La Standard Template Library

Lezione 9.1 - Contenitori sequenziali

La STL si compone di **contenitori**, **iteratori** e **algoritmi**. Contenitori sequenziali: `std::array` (dimensione fissa), `std::vector` (array dinamico, il default), `std::deque`, `std::list` (lista doppia), `std::forward_list`.

Contenitore	Accesso	Inserimento in coda	Inserimento in mezzo
vector	O(1)	O(1) ammortizzato	O(n)
deque	O(1)	O(1)	O(n)
list	O(n)	O(1)	O(1) con iteratore

Nota pratica: `std::vector` e' quasi sempre piu' veloce di `std::list` grazie alla localita' di cache. Usa `reserve()` se conosci la dimensione e `emplace_back` per costruire in loco.

```
std::vector<int> v;
v.reserve(100);
for (int i = 0; i < 100; ++i) v.emplace_back(i);
v.erase(std::remove_if(v.begin(), v.end(), [](int x){ return x % 2; }), v.end()); // erase-remove
std::erase_if(v, [](int x){ return x > 50; });   // C++20
```

Esercizi consigliati su LeetCode

88 - Merge Sorted Array

283 - Move Zeroes

66 - Plus One

53 - Maximum Subarray

121 - Best Time to Buy and Sell Stock

238 - Product of Array Except Self

189 - Rotate Array

Lezione 9.2 - Contenitori associativi e hash

`std::map`/`std::set` (albero rosso-nero, O(log n), ordinati), `std::unordered_map`/`unordered_set` (tabella hash, O(1) medio), `multimap`, `multiset`. C++23 aggiunge `flat_map`/`flat_set`.

```
#include <unordered_map>
#include <string>
std::unordered_map<std::string, int> freq;
for (const std::string& w : parole) ++freq[w];
if (auto it = freq.find("ciao"); it != freq.end()) { /* trovato */ }
// chiave custom: servono std::hash e operator==
struct Punto { int x, y; bool operator==(const Punto&) const = default; };
template<> struct std::hash<Punto> {
    std::size_t operator()(const Punto& p) const noexcept { return std::hash<int>()(p.x) * 31 + std::hash<int>()(p.y); }
};
```

Esercizi consigliati su LeetCode

217 - Contains Duplicate

219 - Contains Duplicate II

347 - Top K Frequent Elements

560 - Subarray Sum Equals K

128 - Longest Consecutive Sequence

3 - Longest Substring Without Repeating Characters

4 - Median of Two Sorted Arrays

Lezione 9.3 - Adattatori, iteratori e algoritmi

Adattatori: `stack`, `queue`, `priority_queue`. Categorie di iteratori: input, forward, bidirectional, random access, contiguous. Algoritmi in `<algorithm>` e `<numeric>`: `sort`, `stable_sort`, `binary_search`, `lower_bound`, `find`, `accumulate`, `transform`, `partition`, `unique`, `reverse`, `rotate`, `next_permutation`, `iota`, `reduce`. In C++17 si possono eseguire in parallelo con `std::execution::par`.

```
#include <algorithm>
#include <numeric>
#include <queue>
std::vector<int> v{5, 3, 8, 1};
std::sort(v.begin(), v.end(), std::greater<>{});
int somma = std::accumulate(v.begin(), v.end(), 0);
std::priority_queue<int, std::vector<int>, std::greater<>> minHeap;
auto it = std::lower_bound(v.begin(), v.end(), 4);
```

Esercizi consigliati su LeetCode

215 - Kth Largest Element in an Array

1046 - Last Stone Weight

56 - Merge Intervals

75 - Sort Colors

31 - Next Permutation

912 - Sort an Array

Lezione 9.4 - Esempi svolti ed esercizi

Esempio svolto 9.1 - Merge Intervals (LeetCode 56)

Problema. Dato un elenco di intervalli `[inizio, fine]`, fondere quelli sovrapposti. Esempio: `[1,3] [2,6] [8,10] [15,18]` diventa `[1,6] [8,10] [15,18]`.

Ragionamento. Dopo aver ordinato per inizio, due intervalli che si sovrappongono sono per forza **consecutivi**. Scorriamo una volta: se l'intervallo corrente inizia prima della fine dell'ultimo fuso, estendiamo quest'ultimo; altrimenti ne apriamo uno nuovo. Costo: O(n log n) per l'ordinamento, O(n) per la scansione. Un `std::pair` si ordina gia' lessicograficamente, quindi basta `std::sort` senza comparatore.

```
#include <algorithm>
#include <iostream>
#include <utility>
#include <vector>

using Intervallo = std::pair<int, int>;

std::vector<Intervallo> fondi(std::vector<Intervallo> v) {   // per valore: lo ordiniamo
    std::sort(v.begin(), v.end());
    std::vector<Intervallo> out;
    for (const auto& [a, b] : v) {
        if (!out.empty() && a <= out.back().second)
            out.back().second = std::max(out.back().second, b);
        else
            out.emplace_back(a, b);
    }
    return out;
}

int main() {
    for (auto [a, b] : fondi({{8, 10}, {1, 3}, {15, 18}, {2, 6}, {17, 20}}))
        std::cout << '[' << a << ',' << b << "] ";
    std::cout << '\n';
}
```

```
[1,6] [8,10] [15,20]
```

Esempio svolto 9.2 - Top K Frequent Elements (LeetCode 347)

Problema. Restituire i `k` elementi piu' frequenti di un vettore.

Ragionamento. Primo passo: contare le occorrenze con `std::unordered_map` (O(n)). Secondo passo: scegliere i k migliori. Ordinare tutte le coppie costerebbe O(m log m) con m valori distinti; `std::partial_sort` ordina solo i primi k, in O(m log k). A parita' di frequenza ordiniamo per valore, cosi' l'output e' deterministico (l'ordine di iterazione di una `unordered_map` non lo e').

```
#include <algorithm>
#include <iostream>
#include <unordered_map>
#include <utility>
#include <vector>

std::vector<int> topK(const std::vector<int>& a, std::size_t k) {
    std::unordered_map<int, int> freq;
    for (int x : a) ++freq[x];
    std::vector<std::pair<int, int>> v(freq.begin(), freq.end());   // (valore, frequenza)
    k = std::min(k, v.size());
    std::partial_sort(v.begin(), v.begin() + k, v.end(), [](const auto& x, const auto& y) {
        return x.second != y.second ? x.second > y.second : x.first < y.first;
    });
    std::vector<int> out;
    for (std::size_t i = 0; i < k; ++i) out.push_back(v[i].first);
    return out;
}

int main() {
    for (int x : topK({4, 1, 1, 1, 2, 2, 3, 4, 4, 5, 2, 2}, 3)) std::cout << x << ' ';
    std::cout << '\n';
}
```

```
2 1 4
```

Alternativa da conoscere: **bucket sort** per frequenza (vettore di liste indicizzato dalla frequenza) che porta il costo a O(n).

Esempio svolto 9.3 - Scegliere il contenitore giusto

Problema. Un sistema riceve eventi `(timestamp, utente)` e deve rispondere a: (a) "quanti utenti distinti?", (b) "quali eventi tra t1 e t2?", (c) "qual e' l'ultimo evento?". Quali contenitori usare?

Ragionamento. (a) appartenenza senza ordine: `std::unordered_set` (O(1) medio). (b) ricerca per intervallo su chiavi ordinate: `std::multimap` (piu' eventi possono avere lo stesso timestamp) con `lower_bound`/`upper_bound` in O(log n). (c) l'ultimo elemento di una mappa ordinata e' `rbegin()`.

```
#include <iostream>
#include <map>
#include <string>
#include <unordered_set>

int main() {
    std::multimap<int, std::string> eventi{{10, "anna"}, {12, "bruno"}, {12, "anna"},
                                           {15, "carla"}, {20, "dario"}};
    std::unordered_set<std::string> utenti;
    for (const auto& [t, u] : eventi) utenti.insert(u);
    std::cout << "distinti: " << utenti.size() << '\n';

    for (auto it = eventi.lower_bound(11); it != eventi.upper_bound(15); ++it)
        std::cout << it->first << ':' << it->second << ' ';
    std::cout << "\nultimo: " << eventi.rbegin()->second << '\n';
}
```

```
distinti: 4
12:bruno 12:anna 15:carla
ultimo: dario
```

Esercizi (soluzioni nell'Appendice)

Esercizio 9.1 [Base] - Move Zeroes (LeetCode 283): sposta gli zeri in fondo mantenendo l'ordine degli altri elementi. Risolvilo prima con un indice di scrittura, poi con un solo algoritmo della STL.

Esercizio 9.2 [Base] - Contains Duplicate II (LeetCode 219): esistono `i != j` con `a[i] == a[j]` e `|i - j| <= k`? Usa una `unordered_map` che ricorda l'ultimo indice di ogni valore.

Esercizio 9.3 [Medio] - Longest Substring Without Repeating Characters (LeetCode 3) in O(n) con una finestra scorrevole e un array di 256 posizioni.

Esercizio 9.4 [Medio] - Conta i punti distinti di un `std::vector<Punto>` con `std::unordered_set<Punto>`, definendo `operator==` e una specializzazione di `std::hash` (vedi Lezione 9.2). Poi risolvi lo stesso problema con `std::set` e confronta i requisiti.

Esercizio 9.5 [Avanzato] - Product of Array Except Self (LeetCode 238) senza divisione, in O(n) tempo e O(1) spazio extra (escluso il vettore risultato).

Capitolo 10 - Strutture dati classiche (implementarle da zero)

Lezione 10.1 - Liste concatenate, pile e code

Implementare le strutture a mano insegna la gestione della memoria. Esempio: pila con lista singola e unique_ptr.

```
template <typename T>
class Pila {
    struct Nodo { T val; std::unique_ptr<Nodo> next; };
    std::unique_ptr<Nodo> testa_;
    std::size_t n_ = 0;
public:
    void push(T v) { testa_ = std::unique_ptr<Nodo>(new Nodo{std::move(v), std::move(testa_)}); ++n_; }
    T pop() { T v = std::move(testa_->val); testa_ = std::move(testa_->next); --n_; return v; }
    const T& top() const { return testa_->val; }
    bool empty() const { return n_ == 0; }
};
```

Temi: lista doppia, lista circolare, coda circolare (ring buffer), deque. Tecniche: due puntatori (lento/veloce), nodo sentinella, inversione in loco.

Esercizi consigliati su LeetCode

20 - Valid Parentheses

234 - Palindrome Linked List

19 - Remove Nth Node From End of List

160 - Intersection of Two Linked Lists

142 - Linked List Cycle II

2 - Add Two Numbers

23 - Merge k Sorted Lists

25 - Reverse Nodes in k-Group

739 - Daily Temperatures

84 - Largest Rectangle in Histogram

239 - Sliding Window Maximum

Lezione 10.2 - Alberi binari e di ricerca

Un albero binario ha nodi con al massimo due figli. Un BST mantiene `sinistro < nodo < destro`. Visite: pre-order, in-order, post-order, per livelli (BFS). Alberi bilanciati: AVL, rosso-nero (usato da `std::map`), B-tree (database e file system).

```
struct Nodo { int val; Nodo *sx = nullptr, *dx = nullptr; };

void inorder(const Nodo* n, std::vector<int>& out) {
    if (!n) return;
    inorder(n->sx, out); out.push_back(n->val); inorder(n->dx, out);
}
int altezza(const Nodo* n) { return n ? 1 + std::max(altezza(n->sx), altezza(n->dx)) : 0; }
```

Esercizi consigliati su LeetCode

104 - Maximum Depth of Binary Tree

94 - Binary Tree Inorder Traversal

102 - Binary Tree Level Order Traversal

226 - Invert Binary Tree

100 - Same Tree

101 - Symmetric Tree

98 - Validate Binary Search Tree

230 - Kth Smallest Element in a BST

235 - Lowest Common Ancestor of a BST

236 - Lowest Common Ancestor of a Binary Tree

105 - Construct Binary Tree from Preorder and Inorder Traversal

124 - Binary Tree Maximum Path Sum

297 - Serialize and Deserialize Binary Tree

Lezione 10.3 - Heap, tabelle hash, trie, union-find

- **Heap** (coda con priorita'): inserimento/estrazione O(log n); `std::priority_queue`, `make_heap`.
- **Tabella hash**: funzione di hash + gestione collisioni (concatenamento o indirizzamento aperto); fattore di carico e rehash.
- **Trie**: albero di prefissi per dizionari e autocompletamento.
- **Union-Find (DSU)**: insiemi disgiunti con path compression e union by rank, quasi O(1).
- **Segment tree / Fenwick tree**: query su intervalli in O(log n).
- **Skip list, Bloom filter, LRU cache**: strutture usate nei database e nei sistemi distribuiti.

```
struct DSU {
    std::vector<int> p, r;
    explicit DSU(int n) : p(n), r(n, 0) { std::iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unisci(int a, int b) {
        a = find(a); b = find(b); if (a == b) return false;
        if (r[a] < r[b]) std::swap(a, b);
        p[b] = a; if (r[a] == r[b]) ++r[a]; return true;
    }
};
```

Esercizi consigliati su LeetCode

684 - Redundant Connection

547 - Number of Provinces

721 - Accounts Merge

307 - Range Sum Query - Mutable

211 - Design Add and Search Words Data Structure

212 - Word Search II

460 - LFU Cache

295 - Find Median from Data Stream

Lezione 10.4 - Grafi

Rappresentazioni: matrice di adiacenza, liste di adiacenza (`vector<vector<int>>`), lista di archi. Visite: BFS (cammini minimi non pesati, livelli) e DFS (componenti connesse, cicli, ordinamento topologico).

```
std::vector<int> bfs(const std::vector<std::vector<int>>& g, int s) {
    std::vector<int> dist(g.size(), -1);
    std::queue<int> q; q.push(s); dist[s] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (dist[v] < 0) { dist[v] = dist[u] + 1; q.push(v); }
    }
    return dist;
}
```

Esercizi consigliati su LeetCode

200 - Number of Islands

133 - Clone Graph

207 - Course Schedule

210 - Course Schedule II

417 - Pacific Atlantic Water Flow

994 - Rotting Oranges

127 - Word Ladder

785 - Is Graph Bipartite?

261 - Graph Valid Tree

Lezione 10.5 - Esempi svolti ed esercizi

Esempio svolto 10.1 - Parentesi bilanciate con una pila (LeetCode 20)

Problema. Verificare che in una stringa con `()[]{}` ogni parentesi aperta sia chiusa dal tipo giusto e nell'ordine giusto.

Ragionamento. L'ultima parentesi aperta deve essere la prima a chiudersi: e' il comportamento LIFO di una pila. Per ogni apertura facciamo `push` della chiusura attesa; per ogni chiusura controlliamo che coincida con la cima. Alla fine la pila deve essere vuota.

```
#include <iostream>
#include <stack>
#include <string_view>

bool bilanciata(std::string_view s) {
    std::stack<char> attese;
    for (char c : s) {
        switch (c) {
            case '(': attese.push(')'); break;
            case '[': attese.push(']'); break;
            case '{': attese.push('}'); break;
            case ')': case ']': case '}':
                if (attese.empty() || attese.top() != c) return false;
                attese.pop();
                break;
            default: break;                // altri caratteri ignorati
        }
    }
    return attese.empty();
}

int main() {
    for (auto s : {"{[()]}", "([)]", "((", "f(a[i]) { return; }"})
        std::cout << s << " -> " << (bilanciata(s) ? "ok" : "errore") << '\n';
}
```

```
{[()]} -> ok
([)] -> errore
(( -> errore
f(a[i]) { return; } -> ok
```

Esempio svolto 10.2 - Un albero binario di ricerca completo

Problema. Implementare un BST di interi con inserimento, ricerca, visita in ordine, visita per livelli e altezza, senza memory leak.

Ragionamento. Ogni nodo possiede i figli con `std::unique_ptr`. L'inserimento ricorsivo prende un **riferimento** al puntatore da riempire, cosi' non serve distinguere il caso "albero vuoto". La visita in ordine di un BST produce le chiavi ordinate; quella per livelli usa una coda (BFS). In un albero bilanciato le operazioni costano O(log n), ma con inserimenti gia' ordinati degenera in una lista: O(n).

```
#include <iostream>
#include <memory>
#include <queue>
#include <vector>

class BST {
    struct Nodo {
        int k;
        std::unique_ptr<Nodo> sx, dx;
    };
    std::unique_ptr<Nodo> radice_;

    static void inserisci(std::unique_ptr<Nodo>& n, int k) {
        if (!n) n = std::make_unique<Nodo>(Nodo{k, nullptr, nullptr});
        else if (k < n->k) inserisci(n->sx, k);
        else if (k > n->k) inserisci(n->dx, k);        // duplicati ignorati
    }
    static void inordine(const Nodo* n, std::vector<int>& out) {
        if (!n) return;
        inordine(n->sx.get(), out); out.push_back(n->k); inordine(n->dx.get(), out);
    }
    static int altezza(const Nodo* n) {
        return n ? 1 + std::max(altezza(n->sx.get()), altezza(n->dx.get())) : 0;
    }
public:
    void inserisci(int k) { inserisci(radice_, k); }
    bool contiene(int k) const {
        for (const Nodo* n = radice_.get(); n; n = k < n->k ? n->sx.get() : n->dx.get())
            if (n->k == k) return true;
        return false;
    }
    std::vector<int> ordinati() const { std::vector<int> v; inordine(radice_.get(), v); return v; }
    int altezza() const { return altezza(radice_.get()); }
    void stampaLivelli() const {
        std::queue<const Nodo*> q;
        if (radice_) q.push(radice_.get());
        while (!q.empty()) {
            for (auto n = q.size(); n > 0; --n) {      // un livello alla volta
                const Nodo* x = q.front(); q.pop();
                std::cout << x->k << ' ';
                if (x->sx) q.push(x->sx.get());
                if (x->dx) q.push(x->dx.get());
            }
            std::cout << '\n';
        }
```

```
    }
};

int main() {
    BST t;
    for (int k : {50, 30, 70, 20, 40, 60, 80, 35}) t.inserisci(k);
    for (int k : t.ordinati()) std::cout << k << ' ';
    std::cout << "\naltezza " << t.altezza() << ", contiene 35? " << t.contiene(35)
              << ", contiene 36? " << t.contiene(36) << '\n';
    t.stampaLivelli();
}
```

```
20 30 35 40 50 60 70 80
altezza 4, contiene 35? 1, contiene 36? 0
50
30 70
20 40 60 80
35
```

Esempio svolto 10.3 - Ordinamento topologico: Course Schedule (LeetCode 207/210)

Problema. Ci sono `n` corsi e coppie `(a, b)` che significano "per seguire a devi aver seguito b". Trovare un ordine valido in cui seguirli, o stabilire che non esiste (dipendenze circolari).

Ragionamento. E' un grafo orientato: arco `b -> a`. L'algoritmo di **Kahn** parte dai corsi senza prerequisiti (grado entrante 0), li "segue" e decrementa il grado entrante dei successori; chi arriva a 0 entra in coda. Se alla fine non abbiamo seguito tutti i corsi, c'e' un ciclo. Costo O(V + E).

```
#include <iostream>
#include <queue>
#include <utility>
#include <vector>

std::vector<int> ordineCorsi(int n, const std::vector<std::pair<int, int>>& prereq) {
    std::vector<std::vector<int>> g(n);
    std::vector<int> gradoIn(n, 0);
    for (auto [a, b] : prereq) { g[b].push_back(a); ++gradoIn[a]; }

    std::queue<int> q;
    for (int i = 0; i < n; ++i) if (gradoIn[i] == 0) q.push(i);
    std::vector<int> ordine;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        ordine.push_back(u);
        for (int v : g[u]) if (--gradoIn[v] == 0) q.push(v);
    }
    if ((int)ordine.size() != n) return {};                // ciclo
    return ordine;
}

int main() {
    auto o = ordineCorsi(6, {{1, 0}, {2, 0}, {3, 1}, {3, 2}, {4, 3}, {5, 4}});
    for (int c : o) std::cout << c << ' ';
    std::cout << "\nciclo? " << ordineCorsi(3, {{0, 1}, {1, 2}, {2, 0}}).empty() << '\n';
}
```

```
0 1 2 3 4 5
ciclo? 1
```

Esercizi (soluzioni nell'Appendice)

Esercizio 10.1 [Base] - Implementa un template `CodaCircolare<T>` (ring buffer, LeetCode 622) con capacita' fissata nel costruttore e metodi `push`, `pop`, `front`, `piena`, `vuota`, usando un `std::vector<T>` e due indici.

Esercizio 10.2 [Medio] - Implementa un Trie (LeetCode 208) con `inserisci`, `cerca` e `iniziaCon`, usando nodi con `std::array<std::unique_ptr<Nodo>, 26>`.

Esercizio 10.3 [Medio] - Number of Provinces (LeetCode 547): data una matrice di adiacenza, conta le componenti connesse usando la struttura `DSU` della Lezione 10.3.

Esercizio 10.4 [Medio] - Daily Temperatures (LeetCode 739): per ogni giorno, quanti giorni bisogna aspettare per una temperatura piu' alta? Risolvilo in O(n) con una pila monotona di indici.

Esercizio 10.5 [Avanzato] - Validate BST (LeetCode 98): verifica che un albero binario sia un BST. Perche' non basta controllare che ogni nodo sia maggiore del figlio sinistro e minore del destro?

Esercizio 10.6 [Avanzato] - Number of Islands (LeetCode 200): conta le isole di `'1'` in una griglia, con una BFS che marca le celle visitate.

Parte 4 - Algoritmi principali

Capitolo 11 - Analisi e tecniche di base

Lezione 11.1 - Complessita' computazionale

La notazione Big-O descrive la crescita del tempo e dello spazio al crescere dell'input: O(1), O(log n), O(n), O(n log n), O(n^2), O(2^n), O(n!). Con n fino a 10^5 servono algoritmi O(n log n); con n fino a 20 e' accettabile O(2^n). Si analizzano caso migliore, medio, peggiore e costo ammortizzato (es. `push_back`).

Lezione 11.2 - Ricerca, two pointers, sliding window, prefix sum

Tecniche fondamentali sugli array: ricerca binaria (anche sulla risposta), due puntatori, finestra scorrevole, somme prefisse.

```
int ricercaBinaria(const std::vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;      // evita overflow
        if (a[mid] == x) return mid;
        (a[mid] < x ? lo : hi) = (a[mid] < x ? mid + 1 : mid - 1);
    }
    return -1;
}
```

Esercizi consigliati su LeetCode

704 - Binary Search

35 - Search Insert Position

33 - Search in Rotated Sorted Array

153 - Find Minimum in Rotated Sorted Array

875 - Koko Eating Bananas

167 - Two Sum II

15 - 3Sum

11 - Container With Most Water

42 - Trapping Rain Water

209 - Minimum Size Subarray Sum

424 - Longest Repeating Character Replacement

76 - Minimum Window Substring

438 - Find All Anagrams in a String

303 - Range Sum Query - Immutable

724 - Find Pivot Index

Lezione 11.3 - Esempi svolti ed esercizi

Esempio svolto 11.1 - Ricerca binaria sulla risposta: Koko Eating Bananas (LeetCode 875)

Problema. Ci sono pile di banane `p[i]`; Koko mangia al massimo `v` banane all'ora (una pila per ora, se ne avanzano aspetta l'ora successiva) e ha `h` ore. Qual e' la velocita' minima `v` che le basta?

Ragionamento. Le ore necessarie `ore(v) = somma(ceil(p[i] / v))` **decrescono** al crescere di v: la proprieta' "v basta" e' monotona (falsa, falsa, ..., vera, vera). Su una proprieta' monotona si puo' fare ricerca binaria sull'intervallo delle risposte `[1, max(p)]`, cercando il primo valore vero. Costo: O(n log max(p)) invece di O(n * max(p)).

```
#include <algorithm>
#include <iostream>
#include <vector>

long long ore(const std::vector<int>& p, long long v) {
    long long h = 0;
    for (int x : p) h += (x + v - 1) / v;           // ceil(x / v) tra interi
    return h;
}

int velocitaMinima(const std::vector<int>& p, long long h) {
    int lo = 1, hi = *std::max_element(p.begin(), p.end());
    while (lo < hi) {                               // invariante: la risposta e' in [lo, hi]
        int mid = lo + (hi - lo) / 2;
        if (ore(p, mid) <= h) hi = mid;             // mid basta: forse anche meno
        else lo = mid + 1;                          // mid non basta
    }
    return lo;
}

int main() {
    std::cout << velocitaMinima({3, 6, 7, 11}, 8) << ' '
              << velocitaMinima({30, 11, 23, 4, 20}, 5) << ' '
              << velocitaMinima({30, 11, 23, 4, 20}, 6) << '\n';
}
```

```
4 30 23
```

Lo schema `while (lo < hi)` con `hi = mid` / `lo = mid + 1` trova il **primo** valore che soddisfa la condizione ed e' quello da memorizzare: si adatta a moltissimi problemi ("capacita' minima", "tempo minimo", "distanza massima minima").

Esempio svolto 11.2 - Somme prefisse e hash map: Subarray Sum Equals K (LeetCode 560)

Problema. Contare i sottoarray contigui la cui somma e' esattamente `k` (gli elementi possono essere negativi).

Ragionamento. Sia `S[i]` la somma dei primi i elementi. La somma del sottoarray `(j, i]` vale `S[i] - S[j]`; vogliamo quindi, per ogni i, il numero di j precedenti con `S[j] = S[i] - k`. Teniamo in una hash map quante volte abbiamo visto ogni somma prefissa. La finestra scorrevole non funziona perche' con numeri negativi allargare la finestra non fa necessariamente crescere la somma.

```
#include <iostream>
#include <unordered_map>
#include <vector>

int contaSottoarray(const std::vector<int>& a, int k) {
    std::unordered_map<long long, int> visti{{0, 1}};   // somma vuota: vista una volta
    long long s = 0;
    int tot = 0;
    for (int x : a) {
        s += x;
        if (auto it = visti.find(s - k); it != visti.end()) tot += it->second;
        ++visti[s];
    }
    return tot;
}

int main() {
    std::cout << contaSottoarray({1, 1, 1}, 2) << ' '
              << contaSottoarray({1, 2, 3}, 3) << ' '
              << contaSottoarray({3, 4, 7, 2, -3, 1, 4, 2}, 7) << '\n';
}
```

```
2 2 4
```

Esempio svolto 11.3 - Finestra scorrevole: Minimum Size Subarray Sum (LeetCode 209)

Problema. Dati numeri **positivi** e un obiettivo `t`, trovare la lunghezza minima di un sottoarray contiguo con somma `>= t`.

Ragionamento. Con soli positivi, allargare la finestra aumenta la somma e restringerla la diminuisce. Allarghiamo a destra; appena la somma raggiunge `t` restringiamo da sinistra finche' resta valida, aggiornando il minimo. Ogni indice entra ed esce dalla finestra una volta sola: O(n).

```
#include <algorithm>
#include <climits>
#include <iostream>
#include <vector>

int minimaLunghezza(int t, const std::vector<int>& a) {
    int best = INT_MAX, sx = 0, somma = 0;
    for (int dx = 0; dx < (int)a.size(); ++dx) {
        somma += a[dx];
        while (somma >= t) {
            best = std::min(best, dx - sx + 1);
            somma -= a[sx++];
        }
    }
    return best == INT_MAX ? 0 : best;
}

int main() {
    std::cout << minimaLunghezza(7, {2, 3, 1, 2, 4, 3}) << ' '
              << minimaLunghezza(11, {1, 1, 1, 1}) << '\n';
}
```

```
2 0
```

Esercizi (soluzioni nell'Appendice)

Esercizio 11.1 [Base] - Scrivi a mano `lowerBound(a, x)`: il primo indice con `a[i] >= x` (o `a.size()` se non esiste). Verificalo confrontandolo con `std::lower_bound` su molti input casuali.

Esercizio 11.2 [Base] - Two Sum II (LeetCode 167): su un array ordinato trova due elementi con somma `target` usando due puntatori. Perche' e' corretto spostare il puntatore sinistro quando la somma e' troppo piccola?

Esercizio 11.3 [Medio] - Container With Most Water (LeetCode 11) in O(n) con due puntatori.

Esercizio 11.4 [Medio] - Find Pivot Index (LeetCode 724): l'indice in cui la somma a sinistra e' uguale a quella a destra.

Esercizio 11.5 [Medio] - Indica la complessita' di: (a) due cicli annidati `for i < n`, `for j < i`; (b) `for (i = 1; i < n; i *= 2)`; (c) un ciclo `for i < n` che contiene una ricerca binaria su n elementi; (d) la generazione di tutti i sottoinsiemi di n elementi.

Esercizio 11.6 [Avanzato] - Find All Anagrams in a String (LeetCode 438): tutti gli indici di `s` in cui inizia un anagramma di `p`, in O(|s|) con una finestra di lunghezza fissa e due array di conteggi.

Capitolo 12 - Ordinamento

Lezione 12.1 - Algoritmi di ordinamento

Elementari O(n^2): bubble, selection, insertion (ottimo su piccoli input quasi ordinati). Efficienti O(n log n): merge sort (stabile), quicksort (in media il piu' veloce), heap sort. Lineari non basati su confronto: counting sort, radix sort, bucket sort. `std::sort` e' un introsort (quick + heap + insertion).

```
void mergeSort(std::vector<int>& a, int l, int r, std::vector<int>& tmp) {
    if (r - l < 2) return;
    int m = (l + r) / 2;
    mergeSort(a, l, m, tmp); mergeSort(a, m, r, tmp);
    std::merge(a.begin() + l, a.begin() + m, a.begin() + m, a.begin() + r, tmp.begin() + l);
    std::copy(tmp.begin() + l, tmp.begin() + r, a.begin() + l);
}
```

Esercizi consigliati su LeetCode

912 - Sort an Array

148 - Sort List

169 - Majority Element

347 - Top K Frequent Elements

451 - Sort Characters By Frequency

164 - Maximum Gap (radix/bucket)

315 - Count of Smaller Numbers After Self

Lezione 12.2 - Esempi svolti ed esercizi

Esempio svolto 12.1 - Quicksort con pivot casuale, verificato contro std::sort

Problema. Implementare quicksort e verificarne la correttezza in modo automatico.

Ragionamento. Usiamo la partizione di **Hoare**: due indici che avanzano l'uno verso l'altro scambiando gli elementi fuori posto. Un pivot scelto a caso rende improbabile il caso peggiore O(n^2) (che con pivot fisso si ha su input gia' ordinati). Per verificare, confrontiamo il risultato con `std::sort` su molti vettori casuali: e' un semplice **test differenziale**, utilissimo per qualunque algoritmo che abbia un'implementazione di riferimento.

```
#include <algorithm>
#include <iostream>
#include <random>
#include <vector>

std::mt19937 rng(42);

void quicksort(std::vector<int>& a, int lo, int hi) {         // ordina a[lo..hi]
    if (lo >= hi) return;
    int pivot = a[std::uniform_int_distribution<int>(lo, hi)(rng)];
    int i = lo - 1, j = hi + 1;
    for (;;) {                                                // partizione di Hoare
        do ++i; while (a[i] < pivot);
        do --j; while (a[j] > pivot);
        if (i >= j) break;
        std::swap(a[i], a[j]);
    }
    quicksort(a, lo, j);
    quicksort(a, j + 1, hi);
}

int main() {
    std::uniform_int_distribution<int> len(0, 200), val(-50, 50);
    int errori = 0;
    for (int t = 0; t < 1000; ++t) {
        std::vector<int> a(len(rng));
        for (int& x : a) x = val(rng);
        auto atteso = a;
        std::sort(atteso.begin(), atteso.end());
        quicksort(a, 0, (int)a.size() - 1);
        if (a != atteso) ++errori;
    }
    std::cout << "test falliti: " << errori << " su 1000\n";
}
```

```
test falliti: 0 su 1000
```

Esempio svolto 12.2 - Ordinamenti stabili e comparatori a piu' criteri

Problema. Ordinare dei candidati per punteggio decrescente; a parita' di punteggio deve restare l'ordine di iscrizione.

Ragionamento. Un ordinamento e' **stabile** se mantiene l'ordine relativo degli elementi equivalenti. `std::sort` non lo garantisce, `std::stable_sort` (merge sort) si'. In alternativa si puo' usare `std::sort` con un comparatore che include esplicitamente il criterio di spareggio (qui l'indice di iscrizione). Il comparatore deve essere un **ordinamento stretto debole**: usare `>=` invece di `>` e' comportamento indefinito.

```
#include <algorithm>
#include <iostream>
#include <string>
#include <tuple>
#include <vector>

struct Candidato { std::string nome; int punti; int iscrizione; };

int main() {
    std::vector<Candidato> v{{"Ugo", 80, 1}, {"Ada", 95, 2}, {"Bea", 80, 3},
                             {"Ciro", 95, 4}, {"Dina", 70, 5}};
    auto a = v;
    std::stable_sort(a.begin(), a.end(),
                     [](const auto& x, const auto& y) { return x.punti > y.punti; });
    auto b = v;
    std::sort(b.begin(), b.end(), [](const auto& x, const auto& y) {
        return std::tie(y.punti, x.iscrizione) < std::tie(x.punti, y.iscrizione);
    });
    for (const auto& c : a) std::cout << c.nome << ' ';
    std::cout << "| ";
    for (const auto& c : b) std::cout << c.nome << ' ';
    std::cout << '\n';
}
```

```
Ada Ciro Ugo Bea Dina | Ada Ciro Ugo Bea Dina
```

`std::tie(y.punti, x.iscrizione) < std::tie(x.punti, y.iscrizione)` confronta lessicograficamente: prima i punti (invertiti, quindi decrescenti), poi l'iscrizione (crescente).

Esercizi (soluzioni nell'Appendice)

Esercizio 12.1 [Base] - Implementa insertion sort e conta gli spostamenti eseguiti su un vettore ordinato, uno casuale e uno ordinato al contrario. Cosa noti?

Esercizio 12.2 [Medio] - Conta le inversioni (coppie `i < j` con `a[i] > a[j]`) in O(n log n) modificando il merge sort della Lezione 12.1.

Esercizio 12.3 [Medio] - Sort Colors (LeetCode 75): ordina un vettore di 0, 1 e 2 in una sola passata con l'algoritmo della bandiera olandese.

Esercizio 12.4 [Medio] - Majority Element (LeetCode 169) in O(n) tempo e O(1) spazio con il voto di Boyer-Moore.

Esercizio 12.5 [Avanzato] - Implementa il radix sort LSD per `uint32_t` con base 256 (quattro passate di counting sort) e confrontane i tempi con `std::sort` su 10 milioni di elementi.

Capitolo 13 - Ricorsione, backtracking, greedy

Lezione 13.1 - Backtracking e divide et impera

Il backtracking esplora lo spazio delle soluzioni scartando i rami non validi: permutazioni, combinazioni, sottoinsiemi, N-regine, sudoku. Divide et impera: dividere, risolvere i sottoproblemi, combinare (merge sort, quicksort, Karatsuba).

```
void sottoinsiemi(const std::vector<int>& a, int i, std::vector<int>& cur, std::vector<std::vector<int>>& out) {
    if (i == (int)a.size()) { out.push_back(cur); return; }
    sottoinsiemi(a, i + 1, cur, out);                // escludi
    cur.push_back(a[i]); sottoinsiemi(a, i + 1, cur, out); cur.pop_back(); // includi
}
```

Esercizi consigliati su LeetCode

78 - Subsets
90 - Subsets II
46 - Permutations
47 - Permutations II
39 - Combination Sum
40 - Combination Sum II
17 - Letter Combinations of a Phone Number
22 - Generate Parentheses
79 - Word Search
131 - Palindrome Partitioning
51 - N-Queens
37 - Sudoku Solver

Lezione 13.2 - Algoritmi greedy

Un algoritmo greedy compie la scelta localmente ottima sperando in un ottimo globale; va dimostrata la correttezza. Esempi: selezione di attivita', codifica di Huffman, monete (per alcuni sistemi), Kruskal e Prim.

Esercizi consigliati su LeetCode

455 - Assign Cookies
55 - Jump Game
45 - Jump Game II
134 - Gas Station
435 - Non-overlapping Intervals
452 - Minimum Number of Arrows to Burst Balloons
763 - Partition Labels
621 - Task Scheduler
135 - Candy

Lezione 13.3 - Esempi svolti ed esercizi

Esempio svolto 13.1 - N regine (LeetCode 51/52)

Problema. In quanti modi si possono disporre N regine su una scacchiera N x N senza che due si attacchino? Stampare anche la prima soluzione per N = 6.

Ragionamento. Mettiamo una regina per riga (cosi' le righe sono gia' diverse) e proviamo ogni colonna. Una casella `(r, c)` e' libera se non sono occupate la colonna `c`, la diagonale `r - c` e l'antidiagonale `r + c`: tre vettori di booleani rendono il controllo O(1). Se nessuna colonna va bene si torna indietro (**backtracking**), annullando le scelte fatte.

```
#include <iostream>
#include <string>
#include <vector>

struct Regine {
    int n, soluzioni = 0;
    std::vector<bool> col, diag, anti;
    std::vector<int> pos;                       // pos[r] = colonna della regina in riga r
    std::vector<int> prima;

    explicit Regine(int n) : n(n), col(n), diag(2 * n), anti(2 * n), pos(n) {}

    void risolvi(int r) {
        if (r == n) {
            if (soluzioni++ == 0) prima = pos;
            return;
        }
        for (int c = 0; c < n; ++c) {
            if (col[c] || diag[r - c + n] || anti[r + c]) continue;
            col[c] = diag[r - c + n] = anti[r + c] = true;     // scelta
            pos[r] = c;
            risolvi(r + 1);
            col[c] = diag[r - c + n] = anti[r + c] = false;    // annulla la scelta
        }
    }
};

int main() {
    for (int n : {4, 6, 8}) {
        Regine q(n);
        q.risolvi(0);
        std::cout << n << " regine: " << q.soluzioni << " soluzioni\n";
        if (n == 6)
            for (int c : q.prima)
                std::cout << std::string(c, '.') << 'Q' << std::string(n - c - 1, '.') << '\n';
    }
}
```

```
4 regine: 2 soluzioni
6 regine: 4 soluzioni
.Q....
...Q..
.....Q
Q.....
..Q...
....Q.
8 regine: 92 soluzioni
```

Esempio svolto 13.2 - Generate Parentheses (LeetCode 22)

Problema. Generare tutte le stringhe di n coppie di parentesi ben formate.

Ragionamento. Costruiamo la stringa un carattere alla volta e **potiamo** i rami impossibili invece di generare tutte le 2^(2n) stringhe: possiamo aggiungere (se ne abbiamo aperte meno di n, e) solo se ci sono parentesi aperte non ancora chiuse. Ogni foglia raggiunta e' una soluzione valida; il loro numero e' il numero di Catalan C(n).

```
#include <iostream>
#include <string>
#include <vector>

void genera(int n, int aperte, int chiuse, std::string& cur, std::vector<std::string>& out) {
    if ((int)cur.size() == 2 * n) { out.push_back(cur); return; }
    if (aperte < n) { cur += '('; genera(n, aperte + 1, chiuse, cur, out); cur.pop_back(); }
    if (chiuse < aperte) { cur += ')'; genera(n, aperte, chiuse + 1, cur, out); cur.pop_back(); }
}

int main() {
    std::vector<std::string> out;
    std::string cur;
    genera(3, 0, 0, cur, out);
    for (const auto& s : out) std::cout << s << ' ';
    std::cout << "(" << out.size() << ")\n";
}
```

```
((())) (()()) (())() ()(()) ()()() (5)
```

Esempio svolto 13.3 - Greedy dimostrato: Non-overlapping Intervals (LeetCode 435)

Problema. Dato un insieme di intervalli, quanti bisogna rimuoverne, al minimo, perche' i rimanenti non si sovrappongano?

Ragionamento. Equivale a tenerne il **massimo** numero di compatibili (il classico problema della selezione di attivita'). Scelta greedy: tra gli intervalli compatibili con quelli gia' presi, prendere quello che **finisce prima**. Dimostrazione per scambio: in una soluzione ottima si puo' sempre sostituire il primo intervallo con quello che finisce prima senza creare conflitti, quindi la scelta greedy non peggiora mai l'ottimo. Ordinare per inizio, invece, sarebbe sbagliato: un intervallo lunghissimo che inizia per primo bloccherebbe tutti gli altri.

```
#include <algorithm>
#include <climits>
#include <iostream>
#include <utility>
#include <vector>

int daRimuovere(std::vector<std::pair<int, int>> v) {
    std::sort(v.begin(), v.end(), [](auto& a, auto& b) { return a.second < b.second; });
    int tenuti = 0, fineUltimo = INT_MIN;
    for (auto [inizio, fine] : v)
        if (inizio >= fineUltimo) { ++tenuti; fineUltimo = fine; }
    return (int)v.size() - tenuti;
}

int main() {
    std::cout << daRimuovere({{1, 2}, {2, 3}, {3, 4}, {1, 3}}) << ' '
              << daRimuovere({{1, 100}, {2, 3}, {4, 5}, {6, 7}}) << ' '
              << daRimuovere({{1, 2}, {1, 2}, {1, 2}}) << '\n';
}
```

```
1 1 2
```

Esercizi (soluzioni nell'Appendice)

Esercizio 13.1 [Base] - Permutations (LeetCode 46): genera tutte le permutazioni di un vettore di interi distinti con il backtracking, poi verifica il risultato con `std::next_permutation`.

Esercizio 13.2 [Medio] - Combination Sum (LeetCode 39): tutte le combinazioni (con ripetizione) di numeri candidati che sommano a `target`. Come eviti di generare la stessa combinazione in ordini diversi?

Esercizio 13.3 [Medio] - Jump Game (LeetCode 55): ogni `a[i]` e' il salto massimo da i; si puo' raggiungere l'ultimo indice? Risolvilo in O(n) con un greedy sul punto piu' lontano raggiungibile.

Esercizio 13.4 [Medio] - Mostra che il greedy "prendi sempre la moneta piu' grande" non e' ottimo per le monete {1, 3, 4} e l'importo 6. Per quali sistemi di monete funziona?

Esercizio 13.5 [Avanzato] - Sudoku Solver (LeetCode 37): risolvi un sudoku con il backtracking usando tre array di bitmask (righe, colonne, riquadri) e scegliendo per prima la casella con meno candidati.

Capitolo 14 - Programmazione dinamica

Lezione 14.1 - Concetti e schemi classici

La DP risolve problemi con sottoproblemi sovrapposti e sottostruttura ottima, con memoizzazione (top-down) o tabulazione (bottom-up). Passi: definire lo stato, la ricorrenza, i casi base, l'ordine di calcolo, ottimizzare lo spazio.

```
int knapsack(const std::vector<int>& w, const std::vector<int>& v, int W) {
    std::vector<int> dp(W + 1, 0);
    for (std::size_t i = 0; i < w.size(); ++i)
        for (int c = W; c >= w[i]; --c) dp[c] = std::max(dp[c], dp[c - w[i]] + v[i]);
    return dp[W];
}
```

Schemi: 1D (scalini, house robber), 2D su griglia, zaino 0/1 e illimitato, sequenze (LIS, LCS, edit distance), intervalli, bitmask, alberi, stringhe.

Esercizi consigliati su LeetCode

198 - House Robber

213 - House Robber II

322 - Coin Change

518 - Coin Change II

300 - Longest Increasing Subsequence

1143 - Longest Common Subsequence

72 - Edit Distance

62 - Unique Paths

64 - Minimum Path Sum

416 - Partition Equal Subset Sum

494 - Target Sum

139 - Word Break

152 - Maximum Product Subarray

91 - Decode Ways

312 - Burst Balloons

10 - Regular Expression Matching

5 - Longest Palindromic Substring

746 - Min Cost Climbing Stairs

Lezione 14.2 - Esempi svolti ed esercizi

Esempio svolto 14.1 - Coin Change passo passo (LeetCode 322)

Problema. Dato un insieme di monete (in quantita' illimitata) e un importo, trovare il numero minimo di monete per formarlo, oppure -1. Stampare anche quali monete usare.

Ragionamento. Seguiamo i passi della Lezione 14.1.

- **Stato**: `dp[x]` = numero minimo di monete per formare l'importo x.
- **Ricorrenza**: l'ultima moneta usata e' una qualsiasi `m <= x`, quindi `dp[x] = 1 + min(dp[x - m])`.
- **Caso base**: `dp[0] = 0`; gli importi non formabili valgono "infinito".
- **Ordine**: x crescente, perche' `dp[x]` dipende da valori piu' piccoli.
- **Ricostruzione**: salviamo in `scelta[x]` la moneta che ha dato il minimo e la seguiamo all'indietro.

Costo O(importo * numero di monete). Con le monete {1, 3, 4} e importo 6 il greedy darebbe 4+1+1 (3 monete), la DP trova 3+3.

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> monete{1, 3, 4};
    int importo = 6;
    const int INF = importo + 1;                        // piu' di qualsiasi soluzione reale
    std::vector<int> dp(importo + 1, INF), scelta(importo + 1, -1);
    dp[0] = 0;
    for (int x = 1; x <= importo; ++x)
        for (int m : monete)
            if (m <= x && dp[x - m] + 1 < dp[x]) { dp[x] = dp[x - m] + 1; scelta[x] = m; }

    for (int x = 0; x <= importo; ++x) std::cout << "dp[" << x << "]=" << dp[x] << ' ';
    std::cout << '\n';
    if (dp[importo] >= INF) { std::cout << "-1\n"; return 0; }
    std::cout << dp[importo] << " monete:";
    for (int x = importo; x > 0; x -= scelta[x]) std::cout << ' ' << scelta[x];
    std::cout << '\n';
}
```

```
dp[0]=0 dp[1]=1 dp[2]=2 dp[3]=1 dp[4]=1 dp[5]=2 dp[6]=2
2 monete: 3 3
```

Esempio svolto 14.2 - Longest Common Subsequence con ricostruzione (LeetCode 1143)

Problema. Trovare la piu' lunga sottosequenza comune (non necessariamente contigua) di due stringhe, e stamparla.

Ragionamento. `dp[i][j]` = lunghezza della LCS tra i primi i caratteri di `a` e i primi j di `b`. Se `a[i-1] == b[j-1]` il carattere fa parte della LCS: `dp[i][j] = dp[i-1][j-1] + 1`; altrimenti si scarta l'ultimo carattere di una delle due: `max(dp[i-1][j], dp[i][j-1])`. Per ricostruire si parte da `dp[n][m]` e si ripercorrono le scelte. Costo O(n m) tempo e spazio (lo spazio scende a O(m) se basta la lunghezza).

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

std::string lcs(const std::string& a, const std::string& b) {
    std::size_t n = a.size(), m = b.size();
    std::vector<std::vector<int>> dp(n + 1, std::vector<int>(m + 1, 0));
    for (std::size_t i = 1; i <= n; ++i)
        for (std::size_t j = 1; j <= m; ++j)
            dp[i][j] = a[i - 1] == b[j - 1] ? dp[i - 1][j - 1] + 1
                                            : std::max(dp[i - 1][j], dp[i][j - 1]);
    std::string out;
    for (std::size_t i = n, j = m; i > 0 && j > 0;) {
        if (a[i - 1] == b[j - 1]) { out += a[i - 1]; --i; --j; }
        else if (dp[i - 1][j] >= dp[i][j - 1]) --i;
        else --j;
    }
    std::reverse(out.begin(), out.end());
    return out;
}

int main() {
    std::cout << lcs("ABCBDAB", "BDCABA") << ' ' << lcs("programmazione", "grammatica") << '\n';
}
```

```
BCBA grammai
```

La stessa tabella, con costi diversi, risolve l'**edit distance** (LeetCode 72) ed e' alla base di `diff` e degli strumenti di confronto tra file.

Esercizi (soluzioni nell'Appendice)

Esercizio 14.1 [Base] - Min Cost Climbing Stairs (LeetCode 746) con O(1) spazio.

Esercizio 14.2 [Base] - House Robber (LeetCode 198): massimo bottino senza svaligiare due case adiacenti. Scrivi la ricorrenza prima del codice.

Esercizio 14.3 [Medio] - Edit Distance (LeetCode 72): numero minimo di inserimenti, cancellazioni e sostituzioni per trasformare una parola in un'altra.

Esercizio 14.4 [Medio] - Unique Paths II (LeetCode 63): numero di percorsi (solo destra o giu') in una griglia con ostacoli, usando un solo vettore di dimensione pari alle colonne.

Esercizio 14.5 [Avanzato] - Longest Increasing Subsequence (LeetCode 300) in O(n log n) con `std::lower_bound` sul vettore delle "code" minime.

Esercizio 14.6 [Avanzato] - Partition Equal Subset Sum (LeetCode 416) come zaino 0/1 booleano; poi riscrivilo con `std::bitset` e spiega perche' e' molto piu' veloce.

Capitolo 15 - Algoritmi su grafi

Lezione 15.1 - Cammini minimi, ordinamento topologico, MST

- **Dijkstra**: cammini minimi con pesi non negativi, O((V+E) log V) con `priority_queue`.
- **Bellman-Ford**: pesi negativi e rilevamento di cicli negativi, O(VE).
- **Floyd-Warshall**: tutte le coppie, O(V^3).
- **Ordinamento topologico**: Kahn (BFS) o DFS su DAG.
- **MST**: Kruskal (con DSU) e Prim.

- **Altri**: componenti fortemente connesse (Tarjan, Kosaraju), ponti e punti di articolazione, flusso massimo (Dinic), A* per la ricerca di percorsi nei giochi.

```
using P = std::pair<long long, int>;
std::vector<long long> dijkstra(const std::vector<std::vector<std::pair<int,int>>>& g, int s) {
    std::vector<long long> d(g.size(), LLONG_MAX);
    std::priority_queue<P, std::vector<P>, std::greater<>> pq;
    d[s] = 0; pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du > d[u]) continue;
        for (auto [v, w] : g[u]) if (du + w < d[v]) { d[v] = du + w; pq.push({d[v], v}); }
    }
    return d;
}
```

Esercizi consigliati su LeetCode

743 - Network Delay Time

787 - Cheapest Flights Within K Stops

1631 - Path With Minimum Effort

778 - Swim in Rising Water

1584 - Min Cost to Connect All Points

332 - Reconstruct Itinerary

269 - Alien Dictionary

802 - Find Eventual Safe States

1192 - Critical Connections in a Network

399 - Evaluate Division

695 - Max Area of Island

130 - Surrounded Regions

Lezione 15.2 - Esempi svolti ed esercizi

Esempio svolto 15.1 - Dijkstra con ricostruzione del percorso

Problema. Su una rete stradale con distanze positive, trovare la distanza minima da Milano a ogni altra citta' e il percorso verso Napoli.

Ragionamento. Dijkstra estrae sempre il nodo non definitivo con distanza minima: con pesi non negativi quella distanza non puo' piu' migliorare. Quando un arco migliora la distanza di `v` ricordiamo il **predecessore** `prec[v] = u`; il percorso si ottiene risalendo i predecessori dalla destinazione. Usiamo la variante "lazy": le voci obsolete nella coda vengono scartate controllando `du > d[u]`.

```
#include <algorithm>
#include <climits>
#include <iostream>
#include <queue>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> nome{"Milano", "Bologna", "Firenze", "Roma", "Napoli", "Genova"};
    struct Arco { int a, b, km; };
    std::vector<Arco> archi{{0, 1, 214}, {0, 5, 142}, {1, 2, 106}, {5, 2, 230},
                            {2, 3, 273}, {1, 3, 380}, {3, 4, 226}, {5, 3, 501}};
    int n = nome.size();
    std::vector<std::vector<std::pair<int, int>>> g(n);
    for (auto [a, b, km] : archi) { g[a].push_back({b, km}); g[b].push_back({a, km}); }

    std::vector<long long> d(n, LLONG_MAX);
    std::vector<int> prec(n, -1);
    using P = std::pair<long long, int>;
    std::priority_queue<P, std::vector<P>, std::greater<>> pq;
    d[0] = 0; pq.push({0, 0});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du > d[u]) continue;                        // voce obsoleta
        for (auto [v, w] : g[u])
            if (du + w < d[v]) { d[v] = du + w; prec[v] = u; pq.push({d[v], v}); }
    }

    for (int i = 0; i < n; ++i) std::cout << nome[i] << ' ' << d[i] << "  ";
    std::cout << '\n';
    std::vector<int> percorso;
    for (int v = 4; v != -1; v = prec[v]) percorso.push_back(v);
    std::reverse(percorso.begin(), percorso.end());
    for (std::size_t i = 0; i < percorso.size(); ++i)
        std::cout << (i ? " -> " : "") << nome[percorso[i]];
    std::cout << '\n';
}
```

```
Milano 0  Bologna 214  Firenze 320  Roma 593  Napoli 819  Genova 142
Milano -> Bologna -> Firenze -> Roma -> Napoli
```

Esempio svolto 15.2 - Albero ricoprente minimo con Kruskal

Problema. Collegare con cavi n edifici spendendo il meno possibile, dati i costi di ogni collegamento possibile.

Ragionamento. Kruskal ordina gli archi per costo e li aggiunge uno alla volta, **scartando** quelli che creerebbero un ciclo; per sapere in O(1) circa se due nodi sono gia' connessi si usa Union-Find. Ci si ferma quando si hanno n - 1 archi. Costo O(E log E) dominato dall'ordinamento.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <tuple>
#include <vector>

struct DSU {
    std::vector<int> p;
    explicit DSU(int n) : p(n) { std::iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unisci(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        p[b] = a;
        return true;
    }
};

int main() {
    int n = 5;
    std::vector<std::tuple<int, int, int>> archi{                  // (costo, a, b)
        {4, 0, 1}, {8, 0, 2}, {2, 1, 2}, {6, 1, 3}, {3, 2, 3}, {9, 3, 4}, {5, 2, 4}};
    std::sort(archi.begin(), archi.end());
    DSU d(n);
    int totale = 0, presi = 0;
    for (auto [c, a, b] : archi) {
        if (!d.unisci(a, b)) continue;                             // creerebbe un ciclo
        std::cout << a << '-' << b << " (" << c << ")\n";
        totale += c;
        if (++presi == n - 1) break;
    }
    std::cout << "costo totale: " << totale << '\n';
}
```

```
1-2 (2)
2-3 (3)
0-1 (4)
2-4 (5)
costo totale: 14
```

Esercizi (soluzioni nell'Appendice)

Esercizio 15.1 [Base] - Network Delay Time (LeetCode 743): il tempo perche' un segnale partito da k raggiunga tutti i nodi (massimo delle distanze di Dijkstra), o -1.

Esercizio 15.2 [Medio] - Implementa Floyd-Warshall e usalo per stampare la matrice delle distanze tra tutte le coppie del grafo dell'Esempio 15.1.

Esercizio 15.3 [Medio] - Implementa Bellman-Ford e usalo per rilevare un ciclo di peso negativo (per esempio in un grafo di tassi di cambio con pesi `-log(tasso)`).

Esercizio 15.4 [Medio] - Implementa Prim con `priority_queue` e verifica che sul grafo dell'Esempio 15.2 dia lo stesso costo di Kruskal.

Esercizio 15.5 [Avanzato] - Cheapest Flights Within K Stops (LeetCode 787): perche' Dijkstra classico non basta? Risolvilo con Bellman-Ford limitato a k+1 iterazioni.

Capitolo 16 - Stringhe, matematica e bit manipulation

Lezione 16.1 - Algoritmi su stringhe

Pattern matching: KMP (prefix function), Z-algorithm, Rabin-Karp (rolling hash), Boyer-Moore; trie, suffix array, Aho-Corasick; algoritmo di Manacher per i palindromi.

Esercizi consigliati su LeetCode

28 - Find the Index of the First Occurrence in a String

459 - Repeated Substring Pattern

214 - Shortest Palindrome

647 - Palindromic Substrings

187 - Repeated DNA Sequences

336 - Palindrome Pairs

8 - String to Integer (atoi)

43 - Multiply Strings

Lezione 16.2 - Teoria dei numeri, geometria e operazioni sui bit

MCD (Euclide), crivello di Eratostene, esponenziazione modulare, inverso modulare, coefficienti binomiali, geometria computazionale (prodotto vettoriale, convex hull), operazioni sui bit (maschere, XOR trick, `std::popcount`, `std::bit_ceil`).

```
long long potMod(long long b, long long e, long long m) {
    long long r = 1; b %= m;
    for (; e; e >>= 1, b = b * b % m) if (e & 1) r = r * b % m;
    return r;
}
```

Esercizi consigliati su LeetCode

204 - Count Primes

202 - Happy Number

69 - Sqrt(x)

172 - Factorial Trailing Zeroes

268 - Missing Number

371 - Sum of Two Integers

149 - Max Points on a Line

1071 - Greatest Common Divisor of Strings

2 - Add Two Numbers

48 - Rotate Image

54 - Spiral Matrix

73 - Set Matrix Zeroes

Lezione 16.3 - Esempi svolti ed esercizi

Esempio svolto 16.1 - Ricerca di un pattern con KMP

Problema. Trovare tutte le occorrenze di un pattern `p` in un testo `t` in tempo O(|t| + |p|).

Ragionamento. La ricerca ingenua, dopo un confronto fallito, riparte dal carattere successivo e puo' costare O(|t| |p|). KMP precalcola la **prefix function** `pi[i]`: la lunghezza del piu' lungo prefisso proprio di `p[0..i]` che e' anche suffisso. Dopo un fallimento sappiamo gia' quanti caratteri del pattern coincidono ancora, e non torniamo mai indietro nel testo. Trucco elegante: calcolare la prefix function di `p + '#' + t`; ogni posizione con valore |p| segna un'occorrenza.

```
#include <iostream>
#include <string>
#include <vector>

std::vector<int> prefixFunction(const std::string& s) {
    std::vector<int> pi(s.size(), 0);
    for (std::size_t i = 1; i < s.size(); ++i) {
        int k = pi[i - 1];
        while (k > 0 && s[i] != s[k]) k = pi[k - 1];   // ripiega sul bordo piu' corto
        if (s[i] == s[k]) ++k;
        pi[i] = k;
    }
    return pi;
}

std::vector<int> trova(const std::string& t, const std::string& p) {
    auto pi = prefixFunction(p + '#' + t);              // '#' non compare nel testo
    std::vector<int> pos;
    for (std::size_t i = p.size() + 1; i < pi.size(); ++i)
        if (pi[i] == (int)p.size()) pos.push_back(i - 2 * p.size());
    return pos;
}

int main() {
    for (int x : prefixFunction("abacaba")) std::cout << x;
    std::cout << " |";
    for (int i : trova("abababcababababab", "abab")) std::cout << ' ' << i;
    std::cout << '\n';
}
```

```
0010123 | 0 2 7 9 11
```

Esempio svolto 16.2 - Crivello di Eratostene ed esponenziazione modulare

Problema. (a) Elencare i primi fino a 100 e contare quelli sotto 10^7. (b) Calcolare 2^(10^18) mod 10^9+7 e verificare il piccolo teorema di Fermat `a^(p-1) = 1 (mod p)`.

Ragionamento. Il crivello parte da un vettore "tutti primi" e, per ogni primo `i`, cancella i multipli a partire da `i*i` (quelli piu' piccoli sono gia' stati cancellati da fattori minori): O(n log log n). Per la potenza, `potMod` della Lezione 16.2 usa O(log e) moltiplicazioni; con un modulo di circa 10^9 il prodotto di due residui sta in 64 bit.

```
#include <iostream>
#include <vector>

std::vector<bool> crivello(int n) {
    std::vector<bool> primo(n + 1, true);
    primo[0] = primo[1] = false;
    for (long long i = 2; i * i <= n; ++i)
        if (primo[i])
            for (long long j = i * i; j <= n; j += i) primo[j] = false;
    return primo;
}

long long potMod(long long b, long long e, long long m) {
    long long r = 1; b %= m;
    for (; e; e >>= 1, b = b * b % m) if (e & 1) r = r * b % m;
    return r;
}

int main() {
    auto p = crivello(100);
    for (int i = 0; i <= 100; ++i) if (p[i]) std::cout << i << ' ';
    auto grande = crivello(10'000'000);
    int conta = 0;
    for (bool b : grande) conta += b;
    std::cout << "\nprimi sotto 10^7: " << conta << '\n';

    const long long MOD = 1'000'000'007;
    std::cout << potMod(2, 1'000'000'000'000'000'000LL, MOD) << ' ' << potMod(12345, MOD - 1, MOD)
              << '\n';
}
```

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
primi sotto 10^7: 664579
719476260 1
```

`std::vector<bool>` e' specializzato per occupare un bit per elemento: 10^7 elementi occupano circa 1,2 MB invece di 10 MB.

Esercizi (soluzioni nell'Appendice)

Esercizio 16.1 [Base] - Missing Number (LeetCode 268): trova il numero mancante in `0..n` con lo XOR, senza rischio di overflow.

Esercizio 16.2 [Medio] - String to Integer (LeetCode 8): implementa `atoi` gestendo spazi iniziali, segno, cifre e saturazione a `INT_MIN`/`INT_MAX`.

Esercizio 16.3 [Medio] - Multiply Strings (LeetCode 43): moltiplica due numeri naturali rappresentati come stringhe, come si fa a mano.

Esercizio 16.4 [Medio] - Rotate Image (LeetCode 48): ruota di 90 gradi in senso orario una matrice n x n in loco (trasposta + inversione delle righe).

Esercizio 16.5 [Avanzato] - Rabin-Karp: trova le occorrenze di un pattern con un rolling hash polinomiale modulo un primo grande, verificando i candidati per evitare falsi positivi.

Parte 5 - Concorrenza, prestazioni e librerie

Capitolo 17 - Concorrenza e parallelismo

Lezione 17.1 - Thread, mutex e variabili di condizione

Dal C++11 la libreria standard include `std::thread` (e `std::jthread` in C++20, che fa join automatico), `std::mutex`, `std::lock_guard`, `std::scoped_lock`, `std::condition_variable`, `std::atomic`, `std::future`, `std::async`. Una **data race** (accesso concorrente non sincronizzato con almeno una scrittura) e' comportamento indefinito.

```
#include <thread>
#include <mutex>
#include <vector>
#include <iostream>

int main() {
    long contatore = 0;
    std::mutex m;
    std::vector<std::jthread> ts;
    for (int i = 0; i < 4; ++i)
        ts.emplace_back([&] {
            for (int k = 0; k < 100000; ++k) { std::scoped_lock lk(m); ++contatore; }
        });
    ts.clear();                       // join
    std::cout << contatore << '\n';   // 400000
}
```

Pattern: produttore/consumatore con coda bloccante, thread pool, `std::latch`/`std::barrier`/`std::semaphore` (C++20), `std::stop_token` per la cancellazione cooperativa. Per il lock-free: `std::atomic` e i memory order (relaxed, acquire, release, seq_cst).

Esercizi consigliati su LeetCode

1114 - Print in Order

1115 - Print FooBar Alternately

1116 - Print Zero Even Odd

1117 - Building H2O

1188 - Design Bounded Blocking Queue

1226 - The Dining Philosophers

1242 - Web Crawler Multithreaded

Lezione 17.2 - Async, coroutine e algoritmi paralleli

`std::async` e `std::future` offrono un modello a task. Le coroutine C++20 permettono di scrivere codice asincrono in stile sequenziale (usate da librerie come Boost.Asio e cppcoro). Gli algoritmi paralleli (`std::execution::par`) parallelizzano `sort`, `transform`, `reduce`. Librerie: Intel oneTBB, OpenMP, Taskflow, HPX; per la GPU CUDA, SYCL, OpenCL.

```
#include <future>
#include <numeric>
#include <execution>
auto f = std::async(std::launch::async, [] { return 6 * 7; });
int r = f.get();
double s = std::reduce(std::execution::par, v.begin(), v.end(), 0.0);
```

Lezione 17.3 - Esempi svolti ed esercizi

Esempio svolto 17.1 - Produttore/consumatore con una coda bloccante limitata

Problema. Due produttori generano i numeri da 1 a 1000 ciascuno; tre consumatori li sommano. La coda condivisa ha capacita' 8: i produttori devono attendere quando e' piena, i consumatori quando e' vuota. Alla fine la somma totale deve valere 2 * 500500.

Ragionamento. Un mutex protegge la coda; due `condition_variable` segnalano "non piena" e "non vuota". L'attesa va sempre fatta con un **predicato** (`wait(lock, pred)`), che gestisce i risvegli spuri. Per terminare, la coda viene "chiusa": `pop` restituisce `std::nullopt` quando e' chiusa e vuota. `std::jthread` esegue il join nel distruttore, quindi nessun thread viene dimenticato.

```
#include <atomic>
#include <condition_variable>
#include <deque>
#include <iostream>
#include <mutex>
#include <optional>
#include <thread>
#include <vector>

template <typename T>
class CodaBloccante {
    std::mutex m_;
    std::condition_variable nonPiena_, nonVuota_;
    std::deque<T> q_;
    std::size_t cap_;
    bool chiusa_ = false;
public:
    explicit CodaBloccante(std::size_t cap) : cap_(cap) {}
    void push(T x) {
        std::unique_lock lk(m_);
        nonPiena_.wait(lk, [&] { return q_.size() < cap_; });
        q_.push_back(std::move(x));
        nonVuota_.notify_one();
    }
    std::optional<T> pop() {
        std::unique_lock lk(m_);
        nonVuota_.wait(lk, [&] { return !q_.empty() || chiusa_; });
        if (q_.empty()) return std::nullopt;            // chiusa e svuotata
        T x = std::move(q_.front());
        q_.pop_front();
        nonPiena_.notify_one();
        return x;
    }
    void chiudi() {
        { std::lock_guard lk(m_); chiusa_ = true; }
        nonVuota_.notify_all();                         // sveglia tutti i consumatori
    }
};

int main() {
    CodaBloccante<int> coda(8);
    std::atomic<long> totale{0};
    {
        std::vector<std::jthread> consumatori;
        for (int c = 0; c < 3; ++c)
            consumatori.emplace_back([&] {
                long parziale = 0;
                while (auto x = coda.pop()) parziale += *x;
                totale += parziale;
            });
        {
            std::vector<std::jthread> produttori;
            for (int p = 0; p < 2; ++p)
                produttori.emplace_back([&] { for (int i = 1; i <= 1000; ++i) coda.push(i); });
        }                                   // join dei produttori
        coda.chiudi();
    }                                       // join dei consumatori
    std::cout << "totale = " << totale << " (atteso " << 2 * 500500 << ")\n";
}
```

```
totale = 1001000 (atteso 1001000)
```

Esempio svolto 17.2 - Somma parallela con std::async

Problema. Sommare 10 milioni di numeri dividendo il lavoro tra piu' task e confrontare il risultato con la versione sequenziale.

Ragionamento. Dividiamo il vettore in blocchi contigui (uno per core, ricavato con `std::thread::hardware_concurrency`); ogni blocco e' sommato da un task `std::async(std::launch::async, ...)`. I `future` raccolgono i parziali con `get()`, che attende la fine del task e propaga eventuali eccezioni. Nessuna variabile e' condivisa in scrittura, quindi non serve alcun lock: e' la forma piu' semplice (e piu' scalabile) di parallelismo.

```
#include <algorithm>
#include <future>
#include <iostream>
#include <numeric>
#include <thread>
#include <vector>

long long sommaParallela(const std::vector<int>& v) {
    std::size_t nt = std::max(1u, std::thread::hardware_concurrency());
    std::size_t blocco = (v.size() + nt - 1) / nt;
    std::vector<std::future<long long>> parziali;
    for (std::size_t inizio = 0; inizio < v.size(); inizio += blocco) {
        auto a = v.begin() + inizio, b = v.begin() + std::min(v.size(), inizio + blocco);
        parziali.push_back(std::async(std::launch::async,
                                      [a, b] { return std::accumulate(a, b, 0LL); }));
    }
    long long tot = 0;
    for (auto& f : parziali) tot += f.get();
    return tot;
}

int main() {
    std::vector<int> v(10'000'000);
    std::iota(v.begin(), v.end(), 1);
    long long seq = std::accumulate(v.begin(), v.end(), 0LL);
    std::cout << sommaParallela(v) << ' ' << (sommaParallela(v) == seq ? "ok" : "ERRORE") << '\n';
}
```

```
50000005000000 ok
```

> **Nota**
>
> Compila gli esercizi di questo capitolo anche con `-fsanitize=thread` (ThreadSanitizer, non combinabile con AddressSanitizer): segnala le data race anche quando il programma sembra funzionare.

> **Esercizi (soluzioni nell'Appendice)**
>
> **Esercizio 17.1** [Base] - Print in Order (LeetCode 1114): tre thread chiamano `primo()`, `secondo()`, `terzo()` in ordine casuale; garantisci che stampino sempre nell'ordine giusto usando due `std::promise<void>`.
>
> **Esercizio 17.2** [Base] - Ripeti il contatore della Lezione 17.1 con `std::atomic<long>` al posto del mutex e poi senza alcuna sincronizzazione. Che risultati ottieni? Cosa dice ThreadSanitizer?
>
> **Esercizio 17.3** [Medio] - Print FooBar Alternately (LeetCode 1115) con due `std::binary_semaphore` (C++20).
>
> **Esercizio 17.4** [Avanzato] - Scrivi un thread pool con un numero fisso di thread e un metodo `invia(f)` che restituisce un `std::future` del risultato di `f`. Usa `std::packaged_task` e la coda dell'Esempio 17.1.

Capitolo 18 - Prestazioni e ottimizzazione

Lezione 18.1 - Misurare, profilare, ottimizzare

Prima misura, poi ottimizza. Strumenti: `perf`, Valgrind/Callgrind, Google Benchmark, Tracy, VTune, flamegraph. Principi: scegliere l'algoritmo giusto, migliorare la localita' di cache (struct of arrays, contenitori contigui), evitare allocazioni inutili (`reserve`, `string_view`, pool/arena allocator, `std::pmr`), usare SIMD (intrinsics, `std::simd`), ottimizzazioni del compilatore (`-O3`, LTO, PGO), `constexpr` per spostare calcoli a compile time.

Lezione 18.2 - Comportamento indefinito e sicurezza

Undefined behavior: overflow dei signed, accesso fuori limiti, dereferenziazione di nullptr, uso di oggetti distrutti, data race, violazione dello strict aliasing. Difese: sanitizer (ASan, UBSan, TSan, MSan), `at()` invece di `[]` quando serve, `std::span`, analisi statica, fuzzing (libFuzzer, AFL++), hardening del compilatore (`-D_GLIBCXX_ASSERTIONS`, `-fstack-protector`, CFI).

Lezione 18.3 - Esempi svolti ed esercizi

Esempio svolto 18.1 - Misurare l'effetto della cache

Problema. Sommare tutti gli elementi di una matrice 4096 x 4096 memorizzata per righe, una volta scorrendola per righe e una per colonne. Il numero di operazioni e' identico: perche' i tempi sono cosi' diversi?

Ragionamento. La memoria viene caricata in cache a **linee** di 64 byte (16 `int`). Scorrendo per righe ogni linea caricata viene usata per intero; scorrendo per colonne si salta di 16 KB a ogni accesso e quasi ogni lettura e' un cache miss. Per misurare correttamente: compilare con `-O2`, usare `std::chrono::steady_clock`, ripetere le misure e **usare il risultato** (altrimenti il compilatore puo' eliminare il calcolo).

```
#include <chrono>
#include <iostream>
#include <vector>

template <typename F>
double millisecondi(F&& f) {
    auto t0 = std::chrono::steady_clock::now();
    f();
    return std::chrono::duration<double, std::milli>(std::chrono::steady_clock::now() - t0).count();
}

int main() {
    const std::size_t N = 4096;
    std::vector<int> m(N * N, 1);
    long long s1 = 0, s2 = 0;
    double tRighe = millisecondi([&] {
        for (std::size_t i = 0; i < N; ++i)
            for (std::size_t j = 0; j < N; ++j) s1 += m[i * N + j];
    });
    double tColonne = millisecondi([&] {
        for (std::size_t j = 0; j < N; ++j)
            for (std::size_t i = 0; i < N; ++i) s2 += m[i * N + j];
    });
    std::cout << "somme uguali: " << (s1 == s2) << '\n';
    std::cerr << "righe: " << tRighe << " ms, colonne: " << tColonne << " ms\n";
}
```

```
somme uguali: 1
```

```
$ g++ -std=c++20 -O2 cache.cpp -o cache && ./cache
somme uguali: 1
righe: 9.8 ms, colonne: 112.4 ms
```

I valori esatti dipendono dalla macchina, ma un fattore 5-20 e' tipico. La stessa ragione rende `std::vector` piu' veloce di `std::list` e motiva il layout **struct of arrays** nei motori di gioco (Capitolo 22).

Esempio svolto 18.2 - Riconoscere e correggere il comportamento indefinito

Problema. Le tre funzioni seguenti contengono comportamento indefinito. Individuarlo e correggerlo.

```
int media(int a, int b) { return (a + b) / 2; }          // 1
bool contiene(const std::vector<int>& v, int x) {          // 2
    for (int i = 0; i <= (int)v.size(); ++i) if (v[i] == x) return true;
    return false;
}
float comeFloat(std::uint32_t bits) { return *reinterpret_cast<float*>(&bits); }   // 3
```

Ragionamento. (1) `a + b` puo' superare `INT_MAX`: l'overflow dei signed e' UB e l'ottimizzatore puo' assumere che non avvenga. (2) `<=` legge `v[v.size()]`, fuori dai limiti. (3) Accedere a un `uint32_t` tramite un `float*` viola lo **strict aliasing**. Le correzioni usano strumenti standard: `std::midpoint` (C++20), un range-for o `std::ranges::find`, `std::bit_cast` (C++20).

```
#include <algorithm>
#include <bit>
#include <climits>
#include <cstdint>
#include <iostream>
#include <numeric>
#include <vector>

int media(int a, int b) { return std::midpoint(a, b); }
bool contiene(const std::vector<int>& v, int x) { return std::ranges::find(v, x) != v.end(); }
float comeFloat(std::uint32_t bits) { return std::bit_cast<float>(bits); }

int main() {
    std::cout << media(INT_MAX, INT_MAX - 2) << ' ' << contiene({1, 2, 3}, 4) << ' '
              << comeFloat(0x40490FDB) << '\n';
}
```

```
2147483646 0 3.14159
```

Esercizi (soluzioni nell'Appendice)

Esercizio 18.1 [Base] - Riempi un `std::vector` con 1000 `push_back` e stampa ogni volta che `capacity()` cambia. Ripeti dopo `reserve(1000)`. Qual e' il fattore di crescita della tua implementazione?

Esercizio 18.2 [Medio] - Confronta il tempo per aggiornare la posizione di un milione di particelle memorizzate come array di struct (`struct P { float x, y, z, vx, vy, vz; char nome[40]; }`) e come struct di array. Spiega il risultato.

Esercizio 18.3 [Medio] - Questa funzione e' chiamata milioni di volte: `bool inizia(std::string s, std::string p) { return s.substr(0, p.size()) == p; }`. Elenca le allocazioni inutili e riscrivila senza.

Esercizio 18.4 [Medio] - Spiega perche' `for (unsigned i = n - 1; i >= 0; --i)` non termina e perche' `int x = INT_MAX; if (x + 1 < x) ...` puo' essere eliminato dal compilatore. Scrivi le versioni corrette.

Capitolo 19 - Le principali librerie del C++

Lezione 19.1 - Librerie generali e utilita'

- **Boost**: Asio (rete/asincrono), Beast (HTTP/WebSocket), Spirit, Graph, Program_options, Filesystem, Multiprecision, Serialization.
- **Abseil** (Google): contenitori hash veloci, stringhe, tempo. **Folly** (Meta).
- **fmt / spdlog**: formattazione e logging (fmt e' alla base di `std::format`).
- **nlohmann/json, simdjson, RapidJSON**: JSON. **protobuf, FlatBuffers, Cap'n Proto**: serializzazione binaria.
- **CLI11, cxxopts**: argomenti da riga di comando. **Catch2, GoogleTest, doctest**: test.
- **Eigen, Armadillo, BLAS/LAPACK**: algebra lineare. **OpenCV**: visione artificiale. **Qt, wxWidgets, Dear ImGui**: interfacce grafiche.
- **SDL2, SFML, GLFW, Vulkan, OpenGL, DirectX**: grafica e multimedia.

Lezione 19.2 - Gestione delle dipendenze e build avanzato

Gestori di pacchetti: **vcpkg**, **Conan**, **CPM.cmake**/`FetchContent`. Concetti CMake moderni: target, `target_link_libraries`, `PUBLIC/PRIVATE/INTERFACE`, `find_package`, `install`, presets, toolchain file per la cross-compilazione. Altri sistemi: Meson, Bazel, Ninja. CI: GitHub Actions con matrice GCC/Clang/MSVC, sanitizer e clang-tidy.

```
include(FetchContent)
FetchContent_Declare(fmt GIT_REPOSITORY https://github.com/fmtlib/fmt GIT_TAG 10.2.1)
FetchContent_MakeAvailable(fmt)
target_link_libraries(corso PRIVATE fmt::fmt)
```

Lezione 19.3 - Test e qualita' del codice

Test unitari con GoogleTest/Catch2, mocking (gMock), TDD, code coverage (gcov/lcov), benchmark, fuzzing, analisi statica (clang-tidy, cppcheck, SonarQube), formattazione automatica con clang-format.

```
#include <catch2/catch_test_macros.hpp>
int somma(int a, int b) { return a + b; }
TEST_CASE("somma") { REQUIRE(somma(2, 3) == 5); }
```

Lezione 19.4 - Esempi svolti ed esercizi

Esempio svolto 19.1 - Un progetto CMake completo con libreria, app e test

Problema. Organizzare un progetto reale: una libreria `geometria`, un eseguibile che la usa, test con Catch2 scaricato automaticamente, e avvisi del compilatore attivi.

Ragionamento. Struttura consigliata: un target per la libreria con le intestazioni pubbliche in `include/`, l'eseguibile e i test che vi si collegano. `FetchContent` scarica Catch2 alla configurazione; `catch_discover_tests` registra ogni `TEST_CASE` in CTest, cosi' `ctest` li esegue singolarmente.

```
geometria/
  CMakeLists.txt
  include/geometria/vettore.hpp
  src/vettore.cpp
  app/main.cpp
  test/test_vettore.cpp
```

```
cmake_minimum_required(VERSION 3.24)
project(geometria LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_library(geometria src/vettore.cpp)
target_include_directories(geometria PUBLIC include)
target_compile_options(geometria PRIVATE -Wall -Wextra -Wpedantic)

add_executable(app app/main.cpp)
target_link_libraries(app PRIVATE geometria)

include(FetchContent)
FetchContent_Declare(Catch2 GIT_REPOSITORY https://github.com/catchorg/Catch2.git
                     GIT_TAG v3.5.2)
FetchContent_MakeAvailable(Catch2)
add_executable(test_geometria test/test_vettore.cpp)
target_link_libraries(test_geometria PRIVATE geometria Catch2::Catch2WithMain)
include(CTest)
include(Catch)
catch_discover_tests(test_geometria)
```

```
// test/test_vettore.cpp
#include <catch2/catch_test_macros.hpp>
#include <catch2/matchers/catch_matchers_floating_point.hpp>
#include <geometria/vettore.hpp>

TEST_CASE("lunghezza di un vettore") {
    REQUIRE_THAT(lunghezza({3, 4}), Catch::Matchers::WithinAbs(5.0, 1e-9));
}
TEST_CASE("somma e' commutativa") {
    Vettore a{1, 2}, b{3, -1};
    CHECK(a + b == b + a);
}
```

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j
ctest --test-dir build --output-on-failure
```

Esempio svolto 19.2 - Un mini framework di test senza dipendenze

Problema. Capire cosa fanno le macro di un framework come Catch2 scrivendone una versione minima: registrazione automatica dei test, controlli che riportano file e riga, riepilogo finale.

Ragionamento. Ogni `TEST(nome)` definisce una funzione e un oggetto statico il cui costruttore la registra in un vettore globale **prima** che `main` inizi. `CHECK(expr)` usa `#expr` per trasformare l'espressione in stringa e `__LINE__` per la posizione. Il `main` esegue tutti i test registrati.

```
#include <functional>
#include <iostream>
#include <string>
#include <vector>

struct Test { std::string nome; std::function<void()> f; };
std::vector<Test>& registro() { static std::vector<Test> r; return r; }   // evita il problema
int falliti = 0;                                                          // dell'ordine statico

struct Registra { Registra(std::string n, std::function<void()> f) {
    registro().push_back({std::move(n), std::move(f)}); } };

#define TEST(nome) void nome(); static Registra reg_##nome(#nome, nome); void nome()
#define CHECK(expr) do { if (!(expr)) { ++falliti; \
    std::cout << "  FALLITO: " #expr " (riga " << __LINE__ << ")\n"; } } while (0)

int fattoriale(int n) { return n <= 1 ? 1 : n * fattoriale(n - 1); }

TEST(fattoriale_base) {
    CHECK(fattoriale(0) == 1);
    CHECK(fattoriale(5) == 120);
}
TEST(fattoriale_sbagliato) {
    CHECK(fattoriale(3) == 9);              // test volutamente errato
}

int main() {
    for (const auto& t : registro()) {
        int prima = falliti;
        t.f();
        std::cout << (falliti == prima ? "[ok] " : "[KO] ") << t.nome << '\n';
    }
    std::cout << registro().size() << " test, " << falliti << " controlli falliti\n";
    return falliti ? 1 : 0;
}
```

Eseguito, il programma esce con codice 1 (un controllo e' fallito) e stampa:

```
[ok] fattoriale_base
  FALLITO: fattoriale(3) == 9 (riga 24)
[KO] fattoriale_sbagliato
2 test, 1 controlli falliti
```

Esercizi (soluzioni nell'Appendice)

Esercizio 19.1 [Base] - Aggiungi al progetto dell'Esempio 19.1 un file `CMakePresets.json` con due preset, `debug` (con `-fsanitize=address,undefined`) e `release` (`-O2`), e usali con `cmake --preset debug`.

Esercizio 19.2 [Medio] - Scrivi i casi di test (con il mini framework dell'Esempio 19.2 o con Catch2) per una funzione `std::optional<int> parseInt(std::string_view)`: numeri validi, segno, spazi, stringa vuota, overflow, caratteri spuri. Quali casi limite avevi dimenticato?

Esercizio 19.3 [Medio] - Configura un file `.clang-tidy` che abiliti `modernize-*`, `bugprone-*` e `performance-*` e trasformi in errori i controlli `bugprone-use-after-move`. Eseguilo su un file che contiene un uso dopo `std::move`.

Esercizio 19.4 [Avanzato] - Scrivi un workflow di GitHub Actions che compila il progetto con GCC e Clang (matrice), esegue i test con i sanitizer e fallisce se `clang-format --dry-run --Werror` trova file non formattati.

Parte 6 - Programmazione web e mobile

Capitolo 20 - Programmazione web con C++

Lezione 20.1 - Il C++ nel backend web

Il C++ si usa sul web dove servono latenza minima e throughput elevato: server HTTP ad alte prestazioni, API gateway, motori di ricerca, servizi di trading, proxy (Envoy, NGINX moduli), browser engine. Framework e librerie principali: **Boost.Beast** (HTTP/WebSocket su Asio), **Crow**, **Drogon** (asincrono, ORM integrato), **Oat++**, **Pistache**, **cpp-httplib** (header-only, ottimo per iniziare), **Wt**, **gRPC** e **Thrift** per RPC.

```
#include "httplib.h"
#include <nlohmann/json.hpp>
using json = nlohmann::json;

int main() {
    httplib::Server svr;
    svr.Get("/api/saluto", [](const httplib::Request& req, httplib::Response& res) {
        std::string nome = req.has_param("nome") ? req.get_param_value("nome") : "mondo";
        res.set_content(json{{"messaggio", "Ciao " + nome}}.dump(), "application/json");
    });
    svr.Post("/api/somma", [](const httplib::Request& req, httplib::Response& res) {
        auto j = json::parse(req.body);
        res.set_content(json{{"risultato", j["a"].get<int>() + j["b"].get<int>()}}.dump(), "application/json");
    });
    svr.listen("0.0.0.0", 8080);
}
```

Concetti da conoscere: HTTP/1.1, HTTP/2, REST, JSON, routing, middleware, autenticazione (JWT, OAuth2), TLS (OpenSSL), CORS, rate limiting, logging strutturato, deploy con Docker.

Esercizi consigliati su LeetCode

146 - LRU Cache (cache di risposte)

362 - Design Hit Counter

359 - Logger Rate Limiter

535 - Encode and Decode TinyURL

588 - Design In-Memory File System

981 - Time Based Key-Value Store

Lezione 20.2 - Server asincroni con Boost.Asio e coroutine

Un server scalabile non usa un thread per connessione: usa un event loop (epoll/kqueue/IOCP) e I/O asincrono. Con le coroutine C++20 il codice asincrono ritorna lineare.

```
#include <boost/asio.hpp>
namespace asio = boost::asio;
using asio::ip::tcp;

asio::awaitable<void> sessione(tcp::socket s) {
    char buf[1024];
    for (;;) {
        auto n = co_await s.async_read_some(asio::buffer(buf), asio::use_awaitable);
        co_await asio::async_write(s, asio::buffer(buf, n), asio::use_awaitable);   // echo
    }
}
asio::awaitable<void> ascolta(unsigned short porta) {
    auto ex = co_await asio::this_coro::executor;
    tcp::acceptor acc(ex, {tcp::v4(), porta});
    for (;;) asio::co_spawn(ex, sessione(co_await acc.async_accept(asio::use_awaitable)), asio::detached);
}
int main() {
    asio::io_context io;
    asio::co_spawn(io, ascolta(9000), asio::detached);
    io.run();
}
```

Lezione 20.3 - WebAssembly: C++ nel browser

Con **Emscripten** si compila C++ in WebAssembly (WASM), eseguibile nei browser a velocita' quasi nativa. Casi d'uso: giochi, editor video/immagini, CAD, codec, porting di librerie esistenti (SQLite, OpenCV, FFmpeg). Binding con `embind`.

```
#include <emscripten/bind.h>
int fib(int n) { return n < 2 ? n : fib(n - 1) + fib(n - 2); }
EMSCRIPTEN_BINDINGS(modulo) { emscripten::function("fib", &fib); }
```

```
emcc fib.cpp -O3 -lembind -o fib.js -sMODULARIZE -sEXPORT_NAME=Fib
```

Lezione 20.4 - Interfacce grafiche desktop e architettura full-stack

Per le GUI desktop: **Qt** (Widgets e QML, il framework C++ piu' completo, con segnali/slot), **Dear ImGui** (tool e debug UI), **wxWidgets**, **GTKmm**. Architettura tipica: frontend in JavaScript/TypeScript che comunica via REST/gRPC/WebSocket con un servizio C++ ad alte prestazioni; oppure moduli C++ esposti a Node.js (N-API), Python (pybind11, nanobind) e altri linguaggi tramite FFI.

```
#include <pybind11/pybind11.h>
int somma(int a, int b) { return a + b; }
PYBIND11_MODULE(esempio, m) { m.def("somma", &somma); }
```

Lezione 20.5 - Esempi svolti ed esercizi

Esempio svolto 20.1 - Parser HTTP e router, senza librerie esterne

Problema. Capire cosa fa un framework web scrivendo il nucleo di un server: interpretare una richiesta HTTP/1.1 testuale (metodo, percorso, query string, intestazioni, corpo), smistarla al gestore giusto e produrre una risposta.

Ragionamento. Una richiesta HTTP e' testo: una riga `METODO PERCORSO VERSIONE`, le intestazioni `Nome: valore` terminate da `\r\n`, una riga vuota e il corpo. Il router e' una mappa da `(metodo, percorso)` a una funzione. Separare il parsing dalla rete permette di **testarlo senza socket**; il codice di rete (Capitolo 30 o Boost.Beast) si limita a leggere i byte e a passarli a queste funzioni.

```
#include <functional>
#include <iostream>
#include <map>
#include <optional>
#include <sstream>
#include <string>

struct Richiesta {
    std::string metodo, percorso, corpo;
    std::map<std::string, std::string> query, intestazioni;
};
struct Risposta { int stato = 200; std::string tipo = "text/plain", corpo; };

std::map<std::string, std::string> parseQuery(const std::string& q) {
    std::map<std::string, std::string> out;
    std::istringstream ss(q);
    for (std::string coppia; std::getline(ss, coppia, '&');)
        if (auto eq = coppia.find('='); eq != std::string::npos)
            out[coppia.substr(0, eq)] = coppia.substr(eq + 1);
    return out;
}

std::optional<Richiesta> parseRichiesta(const std::string& raw) {
    auto fineTesta = raw.find("\r\n\r\n");
    if (fineTesta == std::string::npos) return std::nullopt;     // richiesta incompleta
    std::istringstream ss(raw.substr(0, fineTesta));
    Richiesta r;
    std::string riga, target, versione;
    std::getline(ss, riga);
    std::istringstream(riga) >> r.metodo >> target >> versione;
    if (versione.rfind("HTTP/1.", 0) != 0) return std::nullopt;
    if (auto q = target.find('?'); q != std::string::npos) {
        r.query = parseQuery(target.substr(q + 1));
        target.resize(q);
    }
    r.percorso = target;
    while (std::getline(ss, riga)) {
        if (!riga.empty() && riga.back() == '\r') riga.pop_back();
        if (auto c = riga.find(':'); c != std::string::npos)
            r.intestazioni[riga.substr(0, c)] = riga.substr(riga.find_first_not_of(' ', c + 1));
    }
    r.corpo = raw.substr(fineTesta + 4);
    return r;
}
```

```
class Router {
    std::map<std::pair<std::string, std::string>, std::function<Risposta(const Richiesta&)>> r_;
public:
    void aggiungi(std::string m, std::string p, std::function<Risposta(const Richiesta&)> f) {
        r_[{std::move(m), std::move(p)}] = std::move(f);
    }
    std::string gestisci(const std::string& raw) const {
        Risposta out;
        auto req = parseRichiesta(raw);
        if (!req) out = {400, "text/plain", "richiesta non valida"};
        else if (auto it = r_.find({req->metodo, req->percorso}); it != r_.end())
            out = it->second(*req);
        else
            out = {404, "text/plain", "non trovato"};
        return "HTTP/1.1 " + std::to_string(out.stato) + "\r\nContent-Type: " + out.tipo +
               "\r\nContent-Length: " + std::to_string(out.corpo.size()) + "\r\n\r\n" + out.corpo;
    }
};

int main() {
    Router router;
    router.aggiungi("GET", "/api/saluto", [](const Richiesta& r) {
        auto it = r.query.find("nome");
        std::string nome = it != r.query.end() ? it->second : "mondo";
        return Risposta{200, "application/json", "{\"messaggio\":\"Ciao " + nome + "\"}"};
    });
    router.aggiungi("POST", "/api/eco", [](const Richiesta& r) {
        return Risposta{201, r.intestazioni.at("Content-Type"), r.corpo};
    });
    for (std::string raw : {"GET /api/saluto?nome=Anna HTTP/1.1\r\nHost: x\r\n\r\n",
                            "POST /api/eco HTTP/1.1\r\nContent-Type: text/plain\r\n\r\nping",
                            "GET /boh HTTP/1.1\r\n\r\n"}) {
        auto risposta = router.gestisci(raw);
        std::cout << risposta.substr(0, risposta.find("\r\n")) << " | "
                  << risposta.substr(risposta.find("\r\n\r\n") + 4) << '\n';
    }
}
```

```
HTTP/1.1 200 | {"messaggio":"Ciao Anna"}
HTTP/1.1 201 | ping
HTTP/1.1 404 | non trovato
```

> **Nota**
>
> Un parser esposto a Internet deve imporre limiti (lunghezza massima di intestazioni e corpo, numero di intestazioni), rispettare `Content-Length`, gestire richieste arrivate a pezzi e richieste multiple sulla stessa connessione (keep-alive). Per un server reale usa una libreria collaudata e fai fuzzing del codice di parsing.

Esempio svolto 20.2 - Rate limiting con token bucket

Problema. Un'API pubblica deve concedere a ogni client al massimo 5 richieste di picco e 2 richieste al secondo in media; le richieste in eccesso ricevono `429 Too Many Requests`.

Ragionamento. Ogni client ha un "secchio" con al massimo `capacita'` gettoni, che si ricarica di `velocita'` gettoni al secondo; ogni richiesta consuma un gettone. Non serve un timer: i gettoni si ricalcolano pigramente al momento della richiesta, in base al tempo trascorso. Passiamo il tempo come parametro: il codice diventa deterministico e testabile (in produzione si usa `steady_clock::now()`).

```
#include <algorithm>
#include <iostream>
#include <string>
#include <unordered_map>

class TokenBucket {
    double capacita_, velocita_, gettoni_, ultimo_ = 0;
public:
    TokenBucket(double capacita, double velocita)
        : capacita_(capacita), velocita_(velocita), gettoni_(capacita) {}
    bool consenti(double ora) {
        gettoni_ = std::min(capacita_, gettoni_ + (ora - ultimo_) * velocita_);
        ultimo_ = ora;
        if (gettoni_ < 1) return false;
        gettoni_ -= 1;
        return true;
    }
};

class RateLimiter {
    std::unordered_map<std::string, TokenBucket> client_;
public:
    int stato(const std::string& ip, double ora) {
        auto [it, nuovo] = client_.try_emplace(ip, 5.0, 2.0);
        return it->second.consenti(ora) ? 200 : 429;
    }
};

int main() {
    RateLimiter rl;
    for (int i = 0; i < 7; ++i) std::cout << rl.stato("1.2.3.4", 0.0) << ' ';   // raffica
    std::cout << "| " << rl.stato("5.6.7.8", 0.0);                               // altro client
    std::cout << " | dopo 1 s: ";
    for (int i = 0; i < 3; ++i) std::cout << rl.stato("1.2.3.4", 1.0) << ' ';    // +2 gettoni
    std::cout << '\n';
}
```

```
200 200 200 200 200 429 429 | 200 | dopo 1 s: 200 200 429
```

Esercizi (soluzioni nell'Appendice)

Esercizio 20.1 [Base] - Scrivi `std::string decodificaURL(std::string_view)` che trasforma `%XX` nel carattere corrispondente e `+` in spazio, e usala in `parseQuery` dell'Esempio 20.1 (`nome=Anna%20Maria`).

Esercizio 20.2 [Medio] - Encode and Decode TinyURL (LeetCode 535): genera codici brevi in base 62 da un contatore crescente e conserva le corrispondenze in due mappe.

Esercizio 20.3 [Medio] - Time Based Key-Value Store (LeetCode 981): `set(chiave, valore, t)` e `get(chiave, t)` che restituisce il valore con il timestamp piu' grande `<= t`. Sfrutta il fatto che i timestamp arrivano crescenti.

Esercizio 20.4 [Medio] - Logger Rate Limiter (LeetCode 359): un messaggio va stampato solo se lo stesso messaggio non e' stato stampato negli ultimi 10 secondi.

Esercizio 20.5 [Avanzato] - Con cpp-httplib e nlohmann/json realizza una piccola API REST "todo" in memoria con `GET /todo`, `POST /todo` (corpo JSON `{"testo": ...}`), `DELETE /todo/:id`, protetta dal rate limiter dell'Esempio 20.2.

Capitolo 21 - Programmazione mobile

Lezione 21.1 - C++ su Android (NDK, JNI)

Android permette di scrivere parti native in C++ con l'**NDK**: motori di gioco, elaborazione audio/video, crittografia, librerie multipiattaforma. La comunicazione con Kotlin/Java avviene via **JNI**. Il build usa CMake integrato in Gradle.

```
#include <jni.h>
#include <string>
extern "C" JNIEXPORT jstring JNICALL
Java_com_esempio_app_MainActivity_saluto(JNIEnv* env, jobject /*thiz*/) {
    std::string s = "Ciao da C++";
    return env->NewStringUTF(s.c_str());
}
```

```
// Kotlin
external fun saluto(): String
companion object { init { System.loadLibrary("nativo") } }
```

Punti chiave: ABI (arm64-v8a, armeabi-v7a, x86_64), gestione dei riferimenti locali/globali JNI, thread e `AttachCurrentThread`, `AAssetManager`, Vulkan/OpenGL ES, Oboe (audio a bassa latenza), logging con `__android_log_print`.

Lezione 21.2 - C++ su iOS (Objective-C++ e Swift interop)

Su iOS il C++ si integra con **Objective-C++** (file `.mm`) e, da Swift 5.9, con l'**interoperabilita' diretta Swift/C++**. Si usa per core condivisi tra piattaforme (logica, parser, motori), con interfaccia nativa in SwiftUI/UIKit. Build con Xcode o CMake (`-G Xcode`), Metal per la grafica.

```
// motore.hpp - core condiviso
class Motore { public: int calcola(int x) const { return x * 2; } };
```

```
// Swift 5.9+ con interop C++
import Motore
let m = Motore()
print(m.calcola(21))
```

Lezione 21.3 - Framework multipiattaforma: Qt/QML, Flutter, React Native

- **Qt/QML**: un'unica base di codice C++/QML per Android, iOS, desktop, embedded.
- **Flutter** e **React Native**: UI in Dart/JS con moduli nativi in C++ (FFI di Dart, TurboModules/JSI di React Native).
- **Kotlin Multiplatform**, **Djinni**, **SWIG**: generazione di binding.
- **Cocos2d-x**, **Unreal Engine**, **Godot (GDExtension)**: giochi mobili.

Linee guida architetturali: logica di business e algoritmi in C++ portabile (C++17/20, niente dipendenze di piattaforma), strato sottile di binding, UI nativa; test del core su desktop con CI, cross-compilazione con toolchain NDK/Xcode.

Esercizi consigliati su LeetCode

355 - Design Twitter (feed)

635 - Design Log Storage System

1166 - Design File System

380 - Insert Delete GetRandom O(1)

432 - All O`one Data Structure

Lezione 21.4 - Esempi svolti ed esercizi

Esempio svolto 21.1 - Un core C++ con interfaccia C per Android, iOS e Flutter

Problema. Una libreria di calcolo deve essere usata da Kotlin (JNI), Swift e Dart (FFI). Come esporla in modo stabile e testarla sul PC prima di portarla sui dispositivi?

Ragionamento. Le FFI di quasi tutti i linguaggi parlano **C**, non C++: niente classi, template, eccezioni o `std::string` al confine. Lo schema classico e':

- la logica in C++ moderno, senza dipendenze di piattaforma;
- un'API `extern "C"` con un **handle opaco** (puntatore a una struct che il chiamante non conosce), funzioni `crea`/`distruggi` e codici di errore;
- le eccezioni catturate al confine (un'eccezione che attraversa una funzione C e' comportamento indefinito);
- le stringhe restituite copiate in un buffer fornito dal chiamante.

```
// ---- core.hpp: C++ puro ----
#include <algorithm>
#include <cstring>
#include <iostream>
#include <numeric>
#include <stdexcept>
#include <string>
#include <vector>

class Statistiche {
    std::vector<double> v_;
public:
    void aggiungi(double x) { v_.push_back(x); }
    double media() const {
        if (v_.empty()) throw std::logic_error("nessun dato");
        return std::accumulate(v_.begin(), v_.end(), 0.0) / v_.size();
    }
    std::string riepilogo() const {
        return std::to_string(v_.size()) + " valori, max " +
               std::to_string(*std::max_element(v_.begin(), v_.end()));
    }
};

// ---- api.h: confine C, l'unica cosa che vedono Kotlin/Swift/Dart ----
extern "C" {
typedef struct StatHandle StatHandle;                    // opaco
enum { STAT_OK = 0, STAT_ERRORE = 1, STAT_BUFFER_CORTO = 2 };

StatHandle* stat_crea();
void stat_distruggi(StatHandle* h);
void stat_aggiungi(StatHandle* h, double x);
int stat_media(const StatHandle* h, double* out);
int stat_riepilogo(const StatHandle* h, char* buf, unsigned len);
}

// ---- api.cpp ----
struct StatHandle { Statistiche s; };

extern "C" StatHandle* stat_crea() { return new (std::nothrow) StatHandle{}; }
extern "C" void stat_distruggi(StatHandle* h) { delete h; }
extern "C" void stat_aggiungi(StatHandle* h, double x) { h->s.aggiungi(x); }
extern "C" int stat_media(const StatHandle* h, double* out) {
    try { *out = h->s.media(); return STAT_OK; }
    catch (...) { return STAT_ERRORE; }                  // nessuna eccezione oltre il confine
}
```

```
extern "C" int stat_riepilogo(const StatHandle* h, char* buf, unsigned len) {
    try {
        std::string r = h->s.riepilogo();
        if (r.size() + 1 > len) return STAT_BUFFER_CORTO;
        std::memcpy(buf, r.c_str(), r.size() + 1);
        return STAT_OK;
    } catch (...) { return STAT_ERRORE; }
}

// ---- test sul PC, come farebbe il codice nativo dell'app ----
int main() {
    StatHandle* h = stat_crea();
    double m;
    std::cout << "media senza dati -> codice " << stat_media(h, &m) << '\n';
    for (double x : {4.0, 8.0, 6.0}) stat_aggiungi(h, x);
    char buf[64], piccolo[4];
    if (stat_media(h, &m) == STAT_OK && stat_riepilogo(h, buf, sizeof buf) == STAT_OK)
        std::cout << "media " << m << ", " << buf << '\n';
    std::cout << "buffer piccolo -> codice " << stat_riepilogo(h, piccolo, sizeof piccolo) << '\n';
    stat_distruggi(h);
}
```

```
media senza dati -> codice 1
media 6, 3 valori, max 8.000000
buffer piccolo -> codice 2
```

Lato Dart basta `DynamicLibrary.open("libstat.so").lookupFunction<...>("stat_media")`; lato Swift si importa `api.h` con un bridging header; lato Android si scrive un sottile strato JNI che chiama queste funzioni (Esempio 21.2).

Esempio svolto 21.2 - Uno strato JNI robusto

Problema. Esporre `Statistiche` a Kotlin conservando l'oggetto C++ tra una chiamata e l'altra e convertendo correttamente stringhe ed eccezioni.

Ragionamento. Il puntatore all'oggetto C++ viene restituito a Kotlin come `Long` e ripassato a ogni chiamata. Le stringhe Java si convertono con `GetStringUTFChars`/`ReleaseStringUTFChars`, da accoppiare sempre: un piccolo wrapper RAII lo garantisce. Le eccezioni C++ si traducono in eccezioni Java con `ThrowNew`.

```
#include <jni.h>
#include <stdexcept>
#include <string>
#include "core.hpp"

class StringaJni {                              // RAII per GetStringUTFChars
    JNIEnv* env_; jstring s_; const char* c_;
public:
    StringaJni(JNIEnv* e, jstring s) : env_(e), s_(s), c_(e->GetStringUTFChars(s, nullptr)) {}
    ~StringaJni() { if (c_) env_->ReleaseStringUTFChars(s_, c_); }
    std::string str() const { return c_ ? c_ : ""; }
};

extern "C" JNIEXPORT jlong JNICALL
Java_com_esempio_Stat_crea(JNIEnv*, jclass) {
    return reinterpret_cast<jlong>(new Statistiche());
}
extern "C" JNIEXPORT void JNICALL
Java_com_esempio_Stat_distruggi(JNIEnv*, jclass, jlong h) {
    delete reinterpret_cast<Statistiche*>(h);
}
extern "C" JNIEXPORT jdouble JNICALL
Java_com_esempio_Stat_media(JNIEnv* env, jclass, jlong h) {
    try { return reinterpret_cast<Statistiche*>(h)->media(); }
    catch (const std::exception& e) {
        env->ThrowNew(env->FindClass("java/lang/IllegalStateException"), e.what());
        return 0;
    }
}
```

```
class Stat : AutoCloseable {
    private val h = crea()
    fun media(): Double = media(h)
    override fun close() = distruggi(h)     // usato con: Stat().use { ... }
    companion object {
        init { System.loadLibrary("stat") }
        @JvmStatic external fun crea(): Long
        @JvmStatic external fun distruggi(h: Long)
        @JvmStatic external fun media(h: Long): Double
    }
}
```

Esercizi (soluzioni nell'Appendice)

Esercizio 21.1 [Base] - Aggiungi all'API C dell'Esempio 21.1 la funzione `int stat_conta(const StatHandle*, unsigned* out)` e una funzione `stat_versione()` che restituisce una stringa costante. Perche' una stringa costante si puo' restituire come `const char*` senza problemi di proprieta'?

Esercizio 21.2 [Medio] - Insert Delete GetRandom O(1) (LeetCode 380): un insieme con inserimento, rimozione ed estrazione casuale in O(1) medio, usando un vettore e una hash map valore -> indice.

Esercizio 21.3 [Medio] - Progetta un'interfaccia `Archivio` (salva/leggi coppie chiave-valore) con un'implementazione su file per desktop e una "finta" in memoria per i test. Il core dell'app riceve l'archivio per riferimento: dimostra che si testa senza toccare il disco.

Esercizio 21.4 [Avanzato] - Scrivi il `CMakeLists.txt` che compila il core come libreria condivisa per Android (con la toolchain dell'NDK, ABI `arm64-v8a`) e come framework statico per iOS, e spiega quali file vanno consegnati ai team mobile.

Parte 7 - Videogiochi e grafica

Capitolo 22 - Fondamenti di game programming

Lezione 22.1 - Il game loop e l'architettura di un gioco

Il cuore di un videogioco e' il **game loop**: ripete lettura input, aggiornamento della simulazione, rendering. Per la stabilita' della fisica si usa un passo fisso (fixed timestep) disaccoppiato dal rendering.

```
#include <chrono>
using clock_t_ = std::chrono::steady_clock;

void gameLoop() {
    constexpr double dt = 1.0 / 60.0;
    double accumulatore = 0;
    auto precedente = clock_t_::now();
    bool inEsecuzione = true;
    while (inEsecuzione) {
        auto ora = clock_t_::now();
        accumulatore += std::chrono::duration<double>(ora - precedente).count();
        precedente = ora;
        processaInput(inEsecuzione);
        while (accumulatore >= dt) { aggiorna(dt); accumulatore -= dt; }
        disegna(accumulatore / dt);          // interpolazione
    }
}
```

Lezione 22.2 - Entity Component System (ECS) e gestione della memoria

L'architettura **ECS** separa dati (componenti), identita' (entita') e logica (sistemi), favorendo la cache e la composizione rispetto all'ereditarieta'. Librerie: **EnTT**, **flecs**. Tecniche: object pooling, allocatori ad arena/linear, data-oriented design (struct of arrays), spatial partitioning (griglia, quadtree, octree, BVH) per collisioni e culling.

```
#include <entt/entt.hpp>
struct Posizione { float x, y; };
struct Velocita { float dx, dy; };
int main() {
    entt::registry reg;
    auto e = reg.create();
    reg.emplace<Posizione>(e, 0.f, 0.f);
    reg.emplace<Velocita>(e, 1.f, 2.f);
    reg.view<Posizione, Velocita>().each([](auto& p, const auto& v) { p.x += v.dx; p.y += v.dy; });
}
```

Esercizi consigliati su LeetCode

1197 - Minimum Knight Moves

289 - Game of Life

1091 - Shortest Path in Binary Matrix

773 - Sliding Puzzle

529 - Minesweeper

79 - Word Search

782 - Transform to Chessboard

Lezione 22.3 - Matematica per i giochi: vettori, matrici, quaternioni

Servono: vettori (somma, prodotto scalare e vettoriale, normalizzazione), matrici 4x4 (traslazione, rotazione, scala, proiezione), quaternioni (rotazioni senza gimbal lock), interpolazione (lerp, slerp), collisioni (AABB, sfere, SAT, raycast). Librerie: **GLM**, **Eigen**, DirectXMath.

```
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
glm::mat4 model = glm::translate(glm::mat4(1.f), glm::vec3(1, 0, 0));
model = glm::rotate(model, glm::radians(45.f), glm::vec3(0, 1, 0));
glm::mat4 proj = glm::perspective(glm::radians(60.f), 16.f / 9.f, 0.1f, 100.f);
```

Lezione 22.4 - Esempi svolti ed esercizi

Esempio svolto 22.1 - Fisica a passo fisso e indipendenza dal frame rate

Problema. Una palla cade da 10 m con gravita' 9,81 m/s^2 e rimbalza perdendo il 30% della velocita'. Mostrare che con un **passo fisso** la simulazione da' lo stesso risultato sia a 30 sia a 144 frame al secondo, mentre con un passo variabile no.

Ragionamento. Con il passo variabile `aggiorna(dtFrame)` l'integrazione dipende dalla durata dei frame: macchine diverse producono traiettorie diverse (e in multiplayer i client divergono). Con l'accumulatore della Lezione 22.1 la fisica avanza sempre di `dt = 1/120 s`, indipendentemente da quanti frame vengono disegnati. Per rendere l'esempio riproducibile simuliamo l'orologio invece di leggerlo, e misuriamo il tempo in **tick interi** (qui 1/720 di secondo), come fa `std::chrono`: accumulare durate in virgola mobile introdurrebbe errori di arrotondamento che fanno saltare o raddoppiare un passo.

```
#include <iomanip>
#include <iostream>

struct Palla {
    double y = 10, v = 0;
    void aggiorna(double dt) {
        v -= 9.81 * dt;                       // Eulero semi-implicito
        y += v * dt;
        if (y < 0) { y = -y; v = -v * 0.7; }  // rimbalzo con perdita di energia
    }
};

constexpr long TICK_AL_SECONDO = 720;           // divisibile per 30, 144 e 120

Palla simulaPassoFisso(long fps, long secondi) {
    const long passo = TICK_AL_SECONDO / 120;     // fisica a 120 Hz
    Palla p;
    long accumulatore = 0;
    for (long frame = 0; frame < fps * secondi; ++frame) {   // un giro = un frame disegnato
        accumulatore += TICK_AL_SECONDO / fps;
        while (accumulatore >= passo) { p.aggiorna(1.0 / 120); accumulatore -= passo; }
    }
    return p;
}

Palla simulaPassoVariabile(long fps, long secondi) {
    Palla p;
    for (long frame = 0; frame < fps * secondi; ++frame) p.aggiorna(1.0 / fps);
    return p;
}

int main() {
    std::cout << std::fixed << std::setprecision(3);
    for (long fps : {30, 144})
        std::cout << fps << " fps -> fisso y=" << simulaPassoFisso(fps, 3).y
                  << "  variabile y=" << simulaPassoVariabile(fps, 3).y << '\n';
}
```

```
30 fps -> fisso y=3.199  variabile y=3.436
144 fps -> fisso y=3.199  variabile y=3.369
```

Esempio svolto 22.2 - Un mini ECS scritto da zero

Problema. Capire come funzionano librerie come EnTT implementando un Entity Component System minimo: entita' come semplici numeri, componenti memorizzati in array contigui, sistemi che iterano solo sulle entita' che hanno i componenti richiesti.

Ragionamento. Per ogni tipo di componente usiamo un **sparse set**: un vettore denso dei componenti (contiguo, ottimo per la cache), un vettore delle entita' corrispondenti e un indice sparso entita' -> posizione. Rimuovere un componente scambia l'elemento con l'ultimo: O(1) e l'array resta compatto. Un sistema itera sul componente meno numeroso e controlla la presenza degli altri.

```
#include <cstdint>
#include <iostream>
#include <vector>

using Entita = std::uint32_t;

template <typename C>
class Pool {
    std::vector<C> denso_;
    std::vector<Entita> entita_;
    std::vector<std::int32_t> sparso_;              // entita -> indice in denso_, -1 se assente
public:
    void aggiungi(Entita e, C c) {
        if (e >= sparso_.size()) sparso_.resize(e + 1, -1);
        sparso_[e] = static_cast<std::int32_t>(denso_.size());
        denso_.push_back(c);
        entita_.push_back(e);
    }
    bool ha(Entita e) const { return e < sparso_.size() && sparso_[e] >= 0; }
    C& get(Entita e) { return denso_[sparso_[e]]; }
    void rimuovi(Entita e) {
        auto i = sparso_[e];
        denso_[i] = denso_.back(); entita_[i] = entita_.back();   // l'ultimo prende il posto
        sparso_[entita_[i]] = i;
        denso_.pop_back(); entita_.pop_back();
        sparso_[e] = -1;
    }
    const std::vector<Entita>& entita() const { return entita_; }
};

struct Posizione { float x, y; };
struct Velocita { float dx, dy; };

struct Mondo {
    Entita prossima = 0;
    Pool<Posizione> pos;
    Pool<Velocita> vel;
    Entita crea() { return prossima++; }
};

void sistemaMovimento(Mondo& m, float dt) {
    for (Entita e : m.vel.entita())
        if (m.pos.ha(e)) {
            auto& p = m.pos.get(e); const auto& v = m.vel.get(e);
            p.x += v.dx * dt; p.y += v.dy * dt;
```

```
        }
}

int main() {
    Mondo m;
    Entita giocatore = m.crea(), albero = m.crea(), proiettile = m.crea();
    m.pos.aggiungi(giocatore, {0, 0});   m.vel.aggiungi(giocatore, {1, 0});
    m.pos.aggiungi(albero, {5, 5});                                  // statico: niente velocita'
    m.pos.aggiungi(proiettile, {0, 0});  m.vel.aggiungi(proiettile, {10, 2});
    for (int i = 0; i < 3; ++i) sistemaMovimento(m, 0.5f);
    m.vel.rimuovi(giocatore);                                        // il giocatore si ferma
    sistemaMovimento(m, 0.5f);
    for (Entita e : {giocatore, albero, proiettile})
        std::cout << e << ": (" << m.pos.get(e).x << ", " << m.pos.get(e).y << ")\n";
}
```

```
0: (1.5, 0)
1: (5, 5)
2: (20, 4)
```

Esempio svolto 22.3 - Vettori 2D e collisioni AABB

Problema. Scrivere un tipo `Vec2` con le operazioni essenziali e verificare le collisioni tra rettangoli allineati agli assi (AABB), calcolando anche di quanto separarli.

Ragionamento. Due AABB si sovrappongono se e solo se si sovrappongono le loro proiezioni su **entrambi** gli assi. La penetrazione su un asse e' `(meta' larghezza A + meta' larghezza B) - distanza tra i centri`; per separare i corpi si spinge lungo l'asse con penetrazione minima (minimum translation vector).

```
#include <cmath>
#include <iostream>
#include <optional>

struct Vec2 {
    float x = 0, y = 0;
    Vec2 operator+(Vec2 o) const { return {x + o.x, y + o.y}; }
    Vec2 operator-(Vec2 o) const { return {x - o.x, y - o.y}; }
    Vec2 operator*(float k) const { return {x * k, y * k}; }
    float dot(Vec2 o) const { return x * o.x + y * o.y; }
    float lunghezza() const { return std::sqrt(dot(*this)); }
    Vec2 normalizzato() const { float l = lunghezza(); return l > 0 ? *this * (1 / l) : Vec2{}; }
};

struct AABB { Vec2 centro, meta; };                // meta = meta' larghezza e altezza

std::optional<Vec2> collisione(const AABB& a, const AABB& b) {   // vettore per separare a
    Vec2 d = b.centro - a.centro;
    float px = a.meta.x + b.meta.x - std::fabs(d.x);
    float py = a.meta.y + b.meta.y - std::fabs(d.y);
    if (px <= 0 || py <= 0) return std::nullopt;
    if (px < py) return Vec2{d.x < 0 ? px : -px, 0};
    return Vec2{0, d.y < 0 ? py : -py};
}

int main() {
    Vec2 v{3, 4};
    std::cout << v.lunghezza() << ' ' << v.normalizzato().x << ' ' << v.dot({1, 0}) << '\n';
    AABB giocatore{{0, 0}, {1, 1}}, muro{{1.5f, 0.2f}, {1, 2}}, lontano{{10, 0}, {1, 1}};
    if (auto mtv = collisione(giocatore, muro))
        std::cout << "spingi di " << mtv->x << ", " << mtv->y;
    std::cout << " | lontano: " << collisione(giocatore, lontano).has_value() << '\n';
}
```

```
5 0.6 3
spingi di -0.5, 0 | lontano: 0
```

Esercizi (soluzioni nell'Appendice)

Esercizio 22.1 [Base] - Game of Life (LeetCode 289): calcola la generazione successiva di una griglia in loco, codificando lo stato vecchio e nuovo nello stesso intero (bit 0 = vecchio, bit 1 = nuovo).

Esercizio 22.2 [Base] - Estendi `Vec2` a `Vec3` con prodotto vettoriale e verifica che `(1,0,0) x (0,1,0) = (0,0,1)` e che il risultato sia perpendicolare a entrambi.

Esercizio 22.3 [Medio] - Scrivi un `ObjectPool<T, N>` per i proiettili: `acquisisci()` restituisce un puntatore a un oggetto libero (o `nullptr`), `rilascia(p)` lo rimette disponibile, tutto in O(1) e senza allocazioni dopo la costruzione (usa una free list di indici).

Esercizio 22.4 [Medio] - Minimum Knight Moves (LeetCode 1197): il numero minimo di mosse di un cavallo da (0,0) a (x,y) su una scacchiera infinita, con una BFS.

Esercizio 22.5 [Avanzato] - Implementa una griglia spaziale uniforme (celle di lato fisso, `unordered_map` da cella a lista di entita') e usala per trovare le coppie di AABB in collisione tra 10.000 oggetti. Confronta il numero di test con l'approccio O(n^2).

Capitolo 23 - Librerie e motori

Lezione 23.1 - SDL2, SFML e un primo gioco 2D

SDL2 e **SFML** forniscono finestra, input, audio e rendering 2D su Windows, Linux, macOS (e SDL anche su mobile e web).

```
#include <SFML/Graphics.hpp>
int main() {
    sf::RenderWindow w(sf::VideoMode({800, 600}), "Pong");
    w.setFramerateLimit(60);
    sf::CircleShape palla(10.f); palla.setPosition({400, 300});
    sf::Vector2f vel{4.f, 3.f};
    while (w.isOpen()) {
        while (auto ev = w.pollEvent()) if (ev->is<sf::Event::Closed>()) w.close();
        palla.move(vel);
        auto p = palla.getPosition();
        if (p.x < 0 || p.x > 780) vel.x = -vel.x;
        if (p.y < 0 || p.y > 580) vel.y = -vel.y;
        w.clear(); w.draw(palla); w.display();
    }
}
```

Progetti proposti: Pong, Snake, Tetris, Breakout, platform 2D con tilemap, shoot'em up, gioco a turni con IA.

Lezione 23.2 - Grafica 3D: OpenGL, Vulkan, DirectX, Metal

La pipeline grafica: vertex shader, rasterizzazione, fragment shader, depth test, blending. **OpenGL** (semplice, ottimo per imparare), **Vulkan** (basso livello, multipiattaforma, esplicito), **DirectX 12** (Windows/Xbox), **Metal** (Apple), **WebGPU**. Concetti: buffer (VBO/VAO/UBO), texture, framebuffer, illuminazione (Phong, PBR), shadow mapping, normal mapping, instancing, deferred rendering, compute shader. Librerie di supporto: GLFW, GLAD, stb_image, Assimp (caricamento modelli), bgfx, Dear ImGui.

```
const char* vs = R"(#version 330 core
layout(location=0) in vec3 pos;
uniform mat4 mvp;
void main() { gl_Position = mvp * vec4(pos, 1.0); })";
```

Lezione 23.3 - Unreal Engine, Godot e altri motori

Unreal Engine si programma in C++ (con Blueprint): `UCLASS`, `UPROPERTY`, `UFUNCTION`, Actor, Component, Gameplay Framework, garbage collector per gli `UObject`, replicazione di rete. **Godot** supporta C++ tramite GDExtension. Altri: **O3DE**, **Cocos2d-x**, **Urho3D**, **Ogre3D**, **raylib** (semplice e didattico), **Box2D/Bullet/PhysX/Jolt** per la fisica, **FMOD/OpenAL/miniaudio** per l'audio.

```
UCLASS()
class AMioPersonaggio : public ACharacter {
    GENERATED_BODY()
public:
    UPROPERTY(EditAnywhere, BlueprintReadWrite) float Salute = 100.f;
    UFUNCTION(BlueprintCallable) void SubisciDanno(float Danno);
};
```

Lezione 23.4 - Esempi svolti ed esercizi

Esempio svolto 23.1 - Snake con la logica separata dal rendering

Problema. Scrivere la logica completa di Snake (movimento, crescita, cibo, collisioni) in modo che sia testabile senza finestra e poi collegabile a SFML, SDL o al terminale.

Ragionamento. Il modello di gioco non sa nulla di pixel e tastiera: riceve comandi (`gira`), avanza di un passo (`passo`) e espone lo stato. Il serpente e' una `std::deque` di celle: si aggiunge la nuova testa davanti e si toglie la coda dietro (a meno che abbia mangiato). Un generatore casuale con seme fisso rende la partita riproducibile, utile per i test e per i replay.

```
#include <algorithm>
#include <deque>
#include <iostream>
#include <random>
#include <string>
#include <vector>

struct Cella { int x, y; bool operator==(const Cella&) const = default; };
enum class Dir { Su, Giu, Sx, Dx };

class Snake {
    int w_, h_;
    std::deque<Cella> corpo_;
    Dir dir_ = Dir::Dx;
    Cella cibo_{};
    std::mt19937 rng_;
    bool vivo_ = true;
    int punti_ = 0;

    void nuovoCibo() {
        std::uniform_int_distribution<int> rx(0, w_ - 1), ry(0, h_ - 1);
        do cibo_ = {rx(rng_), ry(rng_)};
        while (std::find(corpo_.begin(), corpo_.end(), cibo_) != corpo_.end());
    }
public:
    Snake(int w, int h, unsigned seme) : w_(w), h_(h), rng_(seme) {
        corpo_ = {{2, h / 2}, {1, h / 2}, {0, h / 2}};
        nuovoCibo();
    }
    void gira(Dir d) {                       // vietato invertire la marcia
        bool opposta = (d == Dir::Su && dir_ == Dir::Giu) || (d == Dir::Giu && dir_ == Dir::Su) ||
                       (d == Dir::Sx && dir_ == Dir::Dx) || (d == Dir::Dx && dir_ == Dir::Sx);
        if (!opposta) dir_ = d;
    }
    void passo() {
        if (!vivo_) return;
        Cella t = corpo_.front();
        if (dir_ == Dir::Su) --t.y; else if (dir_ == Dir::Giu) ++t.y;
        else if (dir_ == Dir::Sx) --t.x; else ++t.x;
        bool mangia = t == cibo_;
        if (!mangia) corpo_.pop_back();      // la coda si sposta (prima del test di collisione)
        if (t.x < 0 || t.y < 0 || t.x >= w_ || t.y >= h_ ||
            std::find(corpo_.begin(), corpo_.end(), t) != corpo_.end()) { vivo_ = false; return; }
        corpo_.push_front(t);
        if (mangia) { ++punti_; nuovoCibo(); }
```

```
    }
    void forzaCibo(Cella c) { cibo_ = c; }   // per i test
    bool vivo() const { return vivo_; }
    int punti() const { return punti_; }
    std::size_t lunghezza() const { return corpo_.size(); }
    std::string disegna() const {            // "renderer" testuale
        std::vector<std::string> g(h_, std::string(w_, '.'));
        g[cibo_.y][cibo_.x] = '*';
        for (const auto& c : corpo_) g[c.y][c.x] = 'o';
        g[corpo_.front().y][corpo_.front().x] = '@';
        std::string out;
        for (auto& r : g) out += r + '\n';
        return out;
    }
};

int main() {
    Snake s(8, 5, 7);
    s.forzaCibo({4, 2});
    s.passo(); s.passo();                    // mangia al secondo passo
    s.gira(Dir::Su); s.passo();
    std::cout << s.disegna();
    std::cout << "punti " << s.punti() << ", lunghezza " << s.lunghezza() << '\n';
    s.passo(); s.passo(); s.passo();         // esce dal bordo superiore
    std::cout << "vivo? " << s.vivo() << '\n';
}
```

```
........
....@.*.
..ooo...
........
........
punti 1, lunghezza 4
vivo? 0
```

Collegarlo a SFML significa solo tradurre gli eventi tastiera in `gira(...)`, chiamare `passo()` a intervalli fissi (per esempio 8 volte al secondo con l'accumulatore dell'Esempio 22.1) e disegnare un `sf::RectangleShape` per cella al posto della stringa.

Esercizi (soluzioni nell'Appendice)

Esercizio 23.1 [Base] - Per Pong: scrivi la funzione che, quando la palla colpisce la racchetta, calcola la nuova direzione in base al punto d'impatto (al centro rimbalza orizzontale, ai bordi con angolo massimo di 60 gradi) mantenendo costante la velocita'.

Esercizio 23.2 [Medio] - Tetris: rappresenta un pezzo come matrice 4x4 di booleani, scrivi la rotazione di 90 gradi e la funzione `collide(griglia, pezzo, x, y)`. Poi la rimozione delle righe complete.

Esercizio 23.3 [Medio] - Collega lo Snake dell'Esempio 23.1 a SFML 3 (o raylib): finestra 640x400, celle da 40 pixel, frecce per girare, 8 passi al secondo, schermata di game over con il punteggio.

Esercizio 23.4 [Avanzato] - Scrivi una classe `Animazione` che, dato un elenco di frame con durate diverse e un tempo trascorso, restituisce il frame corrente, gestendo sia la riproduzione in loop sia quella singola.

Capitolo 24 - IA, rete e ottimizzazione nei giochi

Lezione 24.1 - Intelligenza artificiale per i giochi

Pathfinding (A*, navmesh, flow field, Dijkstra map), macchine a stati finiti, behavior tree, utility AI, GOAP, steering behaviors (seek, flee, flocking), minimax con potatura alpha-beta (scacchi, tris), Monte Carlo Tree Search, generazione procedurale (rumore di Perlin, WFC, dungeon generation).

Esercizi consigliati su LeetCode

1091 - Shortest Path in Binary Matrix

505 - The Maze II

490 - The Maze

913 - Cat and Mouse

464 - Can I Win

486 - Predict the Winner

877 - Stone Game

292 - Nim Game

348 - Design Tic-Tac-Toe

130 - Surrounded Regions

Lezione 24.2 - Multiplayer e ottimizzazione

Networking: UDP vs TCP, modelli client-server autoritativo, client-side prediction, riconciliazione, interpolazione delle entita', lag compensation, serializzazione compatta, librerie (ENet, GameNetworkingSockets, Photon). Ottimizzazione: profiling frame per frame, batching, culling (frustum, occlusion), LOD, streaming asset, multithreading con job system, evitare allocazioni durante il frame, SIMD.

Lezione 24.3 - Esempi svolti ed esercizi

Esempio svolto 24.1 - Pathfinding con A*

Problema. Trovare il percorso piu' breve tra due celle di una mappa a griglia con muri (movimenti in 4 direzioni) e disegnarlo.

Ragionamento. A* e' Dijkstra guidato da un'**euristica** `h(n)` che stima la distanza residua: espande per primo il nodo con `f = g + h` minima, dove `g` e' il costo gia' percorso. Con la distanza di Manhattan l'euristica non sovrastima mai (e' **ammissibile**), quindi il percorso trovato e' ottimo; su mappe grandi e aperte vengono esplorate molte meno celle che con Dijkstra, perche' la ricerca "punta" verso l'obiettivo.

```
#include <algorithm>
#include <cstdlib>
#include <iostream>
#include <queue>
#include <string>
#include <tuple>
#include <vector>

int main() {
    std::vector<std::string> mappa{
        "S.....#...",
        ".####.#.#.",
        "....#...#.",
        "###.#####.",
        "........#G",
    };
    int H = mappa.size(), W = mappa[0].size();
    int sr = 0, sc = 0, gr = 4, gc = 9;
    auto h = [&](int r, int c) { return std::abs(r - gr) + std::abs(c - gc); };

    std::vector<int> g(H * W, 1 << 30), prec(H * W, -1);
    using N = std::tuple<int, int, int>;                        // (f, g, cella)
    std::priority_queue<N, std::vector<N>, std::greater<>> aperti;
    g[sr * W + sc] = 0;
    aperti.push({h(sr, sc), 0, sr * W + sc});
    int espansi = 0;
    const int dr[] = {1, -1, 0, 0}, dc[] = {0, 0, 1, -1};
    while (!aperti.empty()) {
        auto [f, gu, u] = aperti.top(); aperti.pop();
        if (gu > g[u]) continue;
        ++espansi;
        if (u == gr * W + gc) break;
        for (int d = 0; d < 4; ++d) {
            int r = u / W + dr[d], c = u % W + dc[d];
            if (r < 0 || c < 0 || r >= H || c >= W || mappa[r][c] == '#') continue;
            int v = r * W + c;
            if (gu + 1 < g[v]) {
                g[v] = gu + 1;
                prec[v] = u;
                aperti.push({g[v] + h(r, c), g[v], v});
            }
        }
    }
    for (int v = prec[gr * W + gc]; v != sr * W + sc; v = prec[v]) mappa[v / W][v % W] = '*';
    for (const auto& riga : mappa) std::cout << riga << '\n';
```

```
    std::cout << "lunghezza " << g[gr * W + gc] << ", celle espanse " << espansi << '\n';
}
```

```
S*****#***
.####*#*#*
....#***#*
###.#####*
........#G
lunghezza 17, celle espanse 31
```

Esempio svolto 24.2 - Minimax per il tris

Problema. Scrivere un giocatore imbattibile di tris e verificare che, con gioco perfetto da entrambe le parti, la partita finisce in pareggio.

Ragionamento. Minimax esplora l'albero delle mosse: il giocatore di turno sceglie la mossa che **massimizza** il proprio risultato sapendo che l'avversario sceglie quella che lo **minimizza**. Valutiamo +1 la

vittoria di X, -1 quella di O, 0 il pareggio. Nel tris l'albero completo ha poco piu' di mezzo milione di nodi, quindi si esplora tutto; per giochi piu' grandi si aggiungono potatura alfa-beta, profondita' limitata e funzioni di valutazione.

```
#include <array>
#include <iostream>

using Tavola = std::array<char, 9>;

char vincitore(const Tavola& t) {
    static const int linee[8][3] = {{0, 1, 2}, {3, 4, 5}, {6, 7, 8}, {0, 3, 6},
                                    {1, 4, 7}, {2, 5, 8}, {0, 4, 8}, {2, 4, 6}};
    for (auto& l : linee)
        if (t[l[0]] != ' ' && t[l[0]] == t[l[1]] && t[l[1]] == t[l[2]]) return t[l[0]];
    return ' ';
}

long nodi = 0;
int minimax(Tavola& t, char turno) {
    ++nodi;
    if (char w = vincitore(t); w != ' ') return w == 'X' ? 1 : -1;
    int migliore = turno == 'X' ? -2 : 2;
    bool mossaFatta = false;
    for (int i = 0; i < 9; ++i) {
        if (t[i] != ' ') continue;
        mossaFatta = true;
        t[i] = turno;
        int v = minimax(t, turno == 'X' ? 'O' : 'X');
        t[i] = ' ';
        migliore = turno == 'X' ? std::max(migliore, v) : std::min(migliore, v);
    }
    return mossaFatta ? migliore : 0;                  // tavola piena: pareggio
}

int main() {
    Tavola t;
    t.fill(' ');
    std::cout << "valore della partita: " << minimax(t, 'X') << " (nodi visitati " << nodi << ")\n";
    t = {'X', ' ', ' ', ' ', 'O', ' ', ' ', ' ', 'X'};  // O deve evitare l'angolo
    nodi = 0;
    for (int i : {1, 2})
        if (t[i] == ' ') {
            t[i] = 'O';
            std::cout << "O in " << i << " -> " << minimax(t, 'X') << '\n';
            t[i] = ' ';
        }
}
```

```
valore della partita: 0 (nodi visitati 549946)
O in 1 -> 0
O in 2 -> 1
```

Con X negli angoli opposti e O al centro, giocare O in un angolo (casella 2) perde: X crea una doppia minaccia. Giocare su un lato (casella 1) pareggia.

Esercizi (soluzioni nell'Appendice)

Esercizio 24.1 [Base] - Scrivi la macchina a stati di un nemico (`Pattuglia`, `Insegui`, `Attacca`, `Fuga`) con transizioni basate su distanza dal giocatore e salute, usando `enum class` e uno `switch`. Simula 10 tick e stampa lo stato.

Esercizio 24.2 [Medio] - Aggiungi la potatura alfa-beta al minimax dell'Esempio 24.2 e confronta il numero di nodi visitati partendo dalla tavola vuota.

Esercizio 24.3 [Medio] - Nim Game (LeetCode 292) e Predict the Winner (LeetCode 486): per il secondo scrivi la DP su intervalli `dp[i][j]` = vantaggio massimo del giocatore di turno sui numeri da i a j.

Esercizio 24.4 [Medio] - Interpolazione delle entita' in multiplayer: il client riceve dal server snapshot `(tempo, posizione)` a 20 Hz e disegna con 100 ms di ritardo. Scrivi la funzione che, dati gli snapshot e il tempo di rendering, interpola linearmente la posizione.

Esercizio 24.5 [Avanzato] - Implementa i tre comportamenti del flocking di Reynolds (separazione, allineamento, coesione) per 50 boid in 2D usando `Vec2` dell'Esempio 22.3, e verifica che dopo molti passi le velocita' tendano ad allinearsi.

Parte 8 - Sistemi embedded

Capitolo 25 - Fondamenti dell'embedded con C++

Lezione 25.1 - Hardware, microcontrollori e toolchain

Un sistema embedded e' un computer dedicato a un compito specifico, con risorse limitate (kB di RAM, MHz di CPU, bassi consumi). Famiglie: **ARM Cortex-M** (STM32, nRF52, RP2040), **AVR** (Arduino Uno), **ESP32** (Xtensa/RISC-V con Wi-Fi/BLE), **RISC-V**, PIC, MSP430. Toolchain: **arm-none-eabi-gcc**, PlatformIO, Zephyr, ESP-IDF, STM32CubeIDE, Pico SDK. Strumenti: debugger JTAG/SWD (OpenOCD, J-Link, ST-Link), oscilloscopio, analizzatore logico.

Cosa evitare/limitare in ambito embedded: eccezioni (`-fno-exceptions`), RTTI (`-fno-rtti`), allocazione dinamica e frammentazione dell'heap, iostream pesante. Cosa usare: `constexpr`, template, `std::array`, `std::span`, `std::optional`, `etl::` (Embedded Template Library), allocatori statici, `static_assert`.

Lezione 25.2 - Accesso ai registri e GPIO

I periferici sono mappati in memoria: si leggono e scrivono registri tramite puntatori `volatile`. `volatile` impedisce al compilatore di eliminare o riordinare gli accessi.

```
#include <cstdint>

struct GPIO {
    volatile std::uint32_t MODER, OTYPER, OSPEEDR, PUPDR, IDR, ODR;
};
inline GPIO& GPIOA = *reinterpret_cast<GPIO*>(0x40020000);

template <typename Port, unsigned Pin>
struct Led {
    static void init() { Port::MODER |= (1u << (Pin * 2)); }
    static void toggle() { Port::ODR ^= (1u << Pin); }
};
```

Un approccio C++ idiomatico incapsula i registri in classi con template a costo zero, trasformando errori di configurazione in errori di compilazione.

```
// Blink su Arduino
void setup() { pinMode(LED_BUILTIN, OUTPUT); }
void loop() { digitalWrite(LED_BUILTIN, HIGH); delay(500); digitalWrite(LED_BUILTIN, LOW); delay(500); }
```

Lezione 25.3 - Interrupt, timer e vincoli real-time

Le **ISR** (Interrupt Service Routine) devono essere brevi: impostano flag o riempiono buffer, il lavoro pesante si fa nel main loop. Variabili condivise: `volatile` + sezioni critiche o `std::atomic`. Timer e PWM per temporizzazione e controllo motori; watchdog per il recupero da blocchi; modalita' a basso consumo (sleep). Real-time: determinismo, latenza massima, priorita', analisi WCET.

```
volatile bool tastoPremuto = false;
extern "C" void EXTI0_IRQHandler() { tastoPremuto = true; EXTI->PR |= 1; }
int main() {
    init();
    for (;;) { if (tastoPremuto) { tastoPremuto = false; gestisci(); } __WFI(); }
}
```

Esercizi consigliati su LeetCode

622 - Design Circular Queue (ring buffer)

641 - Design Circular Deque

346 - Moving Average from Data Stream

1472 - Design Browser History

191 - Number of 1 Bits

190 - Reverse Bits

461 - Hamming Distance

78 - Subsets (bitmask)

89 - Gray Code

1318 - Minimum Flips to Make a OR b Equal to c

Lezione 25.4 - Esempi svolti ed esercizi

Esempio svolto 25.1 - Registri type-safe a costo zero, testati sul PC

Problema. Configurare un pin GPIO scrivendo campi di bit dentro registri a 32 bit, in modo che un errore (pin inesistente, campo troppo largo) sia un errore di **compilazione**, e poter testare il driver sul PC senza la scheda.

Ragionamento. Un campo di bit si descrive con posizione e larghezza note a compile time: `Campo<Offset, Larghezza>`. Le funzioni `constexpr` calcolano maschere e valori, e `static_assert` rifiuta i campi impossibili. L'indirizzo del registro e' l'unica cosa che cambia tra scheda e PC: sul microcontrollore e' un indirizzo fisso, nel test punta a un array. Con `-O2` tutto si riduce alle stesse istruzioni che si scriverebbero a mano.

```
#include <bit>
#include <cstdint>
#include <iostream>

template <unsigned Offset, unsigned Larghezza>
struct Campo {
    static_assert(Larghezza > 0 && Offset + Larghezza <= 32, "campo fuori dal registro");
    static constexpr std::uint32_t maschera = ((1ull << Larghezza) - 1) << Offset;
    static constexpr std::uint32_t valore(std::uint32_t v) { return (v << Offset) & maschera; }
};

struct Registro {
    volatile std::uint32_t* indirizzo;
    template <typename C> void scrivi(std::uint32_t v) {          // read-modify-write
        *indirizzo = (*indirizzo & ~C::maschera) | C::valore(v);
    }
    template <typename C> std::uint32_t leggi() const {
        return (*indirizzo & C::maschera) >> std::countr_zero(C::maschera);
    }
};

enum class Modo : std::uint32_t { Ingresso = 0, Uscita = 1, Alternativo = 2, Analogico = 3 };

template <unsigned Pin>
struct PinGPIO {
    static_assert(Pin < 16, "la porta ha solo 16 pin");
    using CampoModo = Campo<Pin * 2, 2>;                          // MODER: 2 bit per pin
    using CampoUscita = Campo<Pin, 1>;                            // ODR: 1 bit per pin
    Registro moder, odr;
    void configura(Modo m) { moder.scrivi<CampoModo>(static_cast<std::uint32_t>(m)); }
    void imposta(bool alto) { odr.scrivi<CampoUscita>(alto); }
    void inverti() { odr.scrivi<CampoUscita>(!odr.leggi<CampoUscita>()); }
};

int main() {
    // Sulla scheda: reinterpret_cast<volatile std::uint32_t*>(0x40020000) ecc.
    volatile std::uint32_t registriFinti[2] = {0xABCD0000, 0};   // MODER, ODR simulati
    PinGPIO<5> led{{&registriFinti[0]}, {&registriFinti[1]}};
    led.configura(Modo::Uscita);
    led.imposta(true);
    led.inverti(); led.inverti();
    std::cout << std::hex << "MODER=0x" << registriFinti[0] << " ODR=0x" << registriFinti[1]
              << '\n';
    // PinGPIO<16> errato;   // errore di compilazione: "la porta ha solo 16 pin"
}
```

```
MODER=0xabcd0400 ODR=0x20
```

I 2 bit del pin 5 (bit 10-11 di MODER) valgono `01` = uscita e gli altri bit non sono stati toccati; in ODR e' acceso il bit 5.

Esempio svolto 25.2 - Antirimbalzo di un pulsante come macchina a stati

Problema. Un pulsante meccanico, quando viene premuto, oscilla per qualche millisecondo tra 0 e 1 ("rimbalzi"). Leggendolo ogni millisecondo da un interrupt di timer, generare un solo evento "premuto" e un solo evento "rilasciato" per ogni pressione reale.

Ragionamento. Un nuovo livello viene accettato solo se resta stabile per N campioni consecutivi. La macchina a stati e' una funzione pura che riceve un campione e restituisce l'evento: niente `delay`, niente dipendenze dall'hardware, quindi la si testa sul PC con una sequenza di campioni registrata. Nel firmware `aggiorna` viene chiamata dall'ISR del timer e l'evento viene messo in una coda per il main loop.

```
#include <cstdint>
#include <iostream>
#include <string>

enum class Evento { Nessuno, Premuto, Rilasciato };

class Antirimbalzo {
    bool stabile_ = false, ultimo_ = false;
    std::uint8_t contatore_ = 0;
    static constexpr std::uint8_t N = 4;            // 4 ms di stabilita'
public:
    Evento aggiorna(bool campione) {
        if (campione != ultimo_) { ultimo_ = campione; contatore_ = 0; return Evento::Nessuno; }
        if (campione == stabile_ || ++contatore_ < N) return Evento::Nessuno;
        stabile_ = campione;
        return stabile_ ? Evento::Premuto : Evento::Rilasciato;
    }
};

int main() {
    std::string campioni = "0000101101111111111010010000000";   // un campione al ms
    Antirimbalzo a;
    for (std::size_t t = 0; t < campioni.size(); ++t) {
        Evento e = a.aggiorna(campioni[t] == '1');
        if (e == Evento::Premuto) std::cout << "t=" << t << " premuto\n";
        if (e == Evento::Rilasciato) std::cout << "t=" << t << " rilasciato\n";
    }
}
```

```
t=13 premuto
t=28 rilasciato
```

Esempio svolto 25.3 - Coda lock-free tra interrupt e main loop

Problema. L'ISR della UART riceve byte e li deve passare al main loop senza disabilitare gli interrupt e senza perdere dati.

Ragionamento. Con **un solo produttore** (l'ISR) e **un solo consumatore** (il main loop) basta un ring buffer con due indici atomici: solo l'ISR scrive `testa`, solo il main scrive `coda`. Con una capacita' potenza di 2 il modulo diventa un AND. Gli ordini di memoria `release`/`acquire` garantiscono che il dato sia scritto prima che l'altro lato veda l'indice aggiornato (su un Cortex-M single core servono comunque a impedire al compilatore di riordinare).

```
#include <array>
#include <atomic>
#include <cstddef>
#include <cstdint>
#include <iostream>
#include <optional>

template <typename T, std::size_t N>
class RingSPSC {
    static_assert((N & (N - 1)) == 0, "N deve essere una potenza di 2");
    std::array<T, N> buf_{};
    std::atomic<std::size_t> testa_{0}, coda_{0};   // testa: scrive l'ISR, coda: il main
public:
    bool push(T x) {                                // chiamata dall'ISR
        auto t = testa_.load(std::memory_order_relaxed);
        if (t - coda_.load(std::memory_order_acquire) == N) return false;   // piena
        buf_[t & (N - 1)] = x;
        testa_.store(t + 1, std::memory_order_release);
        return true;
    }
    std::optional<T> pop() {                        // chiamata dal main loop
        auto c = coda_.load(std::memory_order_relaxed);
        if (c == testa_.load(std::memory_order_acquire)) return std::nullopt;   // vuota
        T x = buf_[c & (N - 1)];
        coda_.store(c + 1, std::memory_order_release);
        return x;
    }
};

RingSPSC<std::uint8_t, 8> rx;
unsigned persi = 0;
void UART_IRQHandler(std::uint8_t byte) { if (!rx.push(byte)) ++persi; }   // simulata

int main() {
    for (char c : std::string_view("ciao, mondo!")) UART_IRQHandler(c);    // raffica di 12 byte
    std::cout << "persi: " << persi << ", ricevuti: ";
    while (auto b = rx.pop()) std::cout << static_cast<char>(*b);
    std::cout << '\n';
}
```

```
persi: 4, ricevuti: ciao, mo
```

Con una raffica piu' lunga della coda i byte in eccesso si perdono: il conteggio `persi` serve proprio a dimensionare il buffer durante i test.

Esercizi (soluzioni nell'Appendice)

Esercizio 25.1 [Base] - Scrivi le funzioni `constexpr impostaBit`, `azzeraBit`, `invertiBit` e `leggiBit` su `std::uint32_t` e verificale con `static_assert`.

Esercizio 25.2 [Base] - Design Circular Queue (LeetCode 622) senza allocazione dinamica: capacita' come parametro template e storage in `std::array`.

Esercizio 25.3 [Medio] - Moving Average (LeetCode 346) in aritmetica intera: media mobile degli ultimi 8 campioni di un ADC a 12 bit, aggiornata in O(1) mantenendo la somma corrente. Perche' una finestra potenza di 2 e' vantaggiosa?

Esercizio 25.4 [Medio] - Gray Code (LeetCode 89): genera la sequenza di Gray a n bit con la formula `i ^ (i >> 1)` e scrivi la funzione inversa. Dove si usano i codici di Gray negli encoder?

Esercizio 25.5 [Medio] - Spiega perche' `volatile` non rende atomico `contatore++` condiviso tra ISR e main, e mostra due modi corretti per farlo su un Cortex-M.

Capitolo 26 - Protocolli, RTOS e sistemi connessi

Lezione 26.1 - Bus e protocolli: UART, I2C, SPI, CAN

- **UART**: seriale asincrona (debug, GPS, modem).
- **I2C**: bus a 2 fili con indirizzi (sensori, EEPROM, display).
- **SPI**: full-duplex veloce con chip select (memorie flash, display, SD).
- **CAN/CAN-FD**: bus robusto per automotive e industria.
- **1-Wire, USB, Ethernet, Modbus, LIN**.
- **Wireless**: BLE, Wi-Fi, LoRa, Zigbee, Thread, MQTT/CoAP a livello applicativo.

```
std::uint8_t leggiRegistroI2C(std::uint8_t addr, std::uint8_t reg) {
    i2c.start(); i2c.write(addr << 1); i2c.write(reg);
    i2c.restart(); i2c.write((addr << 1) | 1);
    auto v = i2c.read(/*nack*/true); i2c.stop();
    return v;
}
```

Driver ben progettati separano l'accesso all'hardware (HAL) dalla logica (pattern: interfacce astratte o concepts, iniezione delle dipendenze, test su PC con mock).

Lezione 26.2 - RTOS: FreeRTOS e Zephyr

Un **RTOS** offre task con priorita', scheduler preemptive, code, semafori, mutex (con priority inheritance), timer software, event groups. Problemi tipici: inversione di priorita', deadlock, overflow dello stack di un task, race condition tra ISR e task.

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"

QueueHandle_t coda;
void taskSensore(void*) {
    for (;;) { int v = leggiSensore(); xQueueSend(coda, &v, portMAX_DELAY); vTaskDelay(pdMS_TO_TICKS(100)); }
}
void taskElabora(void*) {
    int v; for (;;) if (xQueueReceive(coda, &v, portMAX_DELAY)) elabora(v);
}
int main() {
    coda = xQueueCreate(8, sizeof(int));
    xTaskCreate(taskSensore, "sens", 256, nullptr, 2, nullptr);
    xTaskCreate(taskElabora, "elab", 256, nullptr, 1, nullptr);
    vTaskStartScheduler();
}
```

Lezione 26.3 - Bootloader, OTA, sicurezza e Embedded Linux

Bootloader (MCUboot, U-Boot), aggiornamenti OTA firmati, secure boot, crittografia (mbedTLS), gestione di flash e wear leveling, diagnostica e log in-field. Per sistemi piu' potenti si usa **Embedded Linux** (Yocto, Buildroot, Raspberry Pi): accesso a GPIO via `libgpiod`, `/dev/i2c`, `/dev/spidev`, driver in userspace, systemd, cross-compilazione con CMake toolchain file. Standard di codifica per sistemi critici: **MISRA C++**, **AUTOSAR C++14**, **CERT C++**, ISO 26262 (automotive), DO-178C (aerospaziale), IEC 61508.

Lezione 26.4 - Progetti pratici embedded

- LED lampeggiante con timer e interrupt (senza `delay`).
- Lettura di un sensore di temperatura I2C con filtro a media mobile e display OLED.
- Macchina a stati per un distributore (stati, eventi, transizioni).
- Datalogger su scheda SD con FreeRTOS.
- Stazione meteo ESP32 che pubblica su MQTT.

- Controllo PID di un motore DC con encoder.
- Driver SPI generico basato su template e concepts, testato su PC.

Esercizi consigliati su LeetCode

232 - Implement Queue using Stacks

1670 - Design Front Middle Back Queue

295 - Find Median from Data Stream

480 - Sliding Window Median

362 - Design Hit Counter

1429 - First Unique Number

Lezione 26.5 - Esempi svolti ed esercizi

Esempio svolto 26.1 - Un protocollo seriale con framing e CRC

Problema. Un sensore invia misure su UART in pacchetti `0x7E | lunghezza | payload | CRC-8`. Il ricevitore riceve un byte alla volta (dall'ISR, Esempio 25.3) e deve ricostruire i pacchetti, scartando quelli corrotti e risincronizzandosi dopo un errore.

Ragionamento. Il parser e' una macchina a stati: `AttesaInizio -> Lunghezza -> Payload -> Crc`. Un byte di inizio fisso permette di ritrovare l'allineamento; la lunghezza massima protegge da valori assurdi; il CRC-8 (polinomio 0x07) rileva le corruzioni. Il parser non sa nulla della UART: riceve byte e restituisce pacchetti completi.

```
#include <array>
#include <cstdint>
#include <iostream>
#include <optional>
#include <vector>

std::uint8_t crc8(const std::uint8_t* d, std::size_t n) {
    std::uint8_t crc = 0;
    for (std::size_t i = 0; i < n; ++i) {
        crc ^= d[i];
        for (int b = 0; b < 8; ++b) crc = crc & 0x80 ? std::uint8_t((crc << 1) ^ 0x07) : crc << 1;
    }
    return crc;
}

std::vector<std::uint8_t> incapsula(const std::vector<std::uint8_t>& payload) {
    std::vector<std::uint8_t> f{0x7E, std::uint8_t(payload.size())};
    f.insert(f.end(), payload.begin(), payload.end());
    f.push_back(crc8(payload.data(), payload.size()));
    return f;
}

class Parser {
    enum class Stato { AttesaInizio, Lunghezza, Payload, Crc } s_ = Stato::AttesaInizio;
    std::array<std::uint8_t, 32> buf_{};
    std::uint8_t len_ = 0, ricevuti_ = 0;
public:
    unsigned errori = 0;
    std::optional<std::vector<std::uint8_t>> byte(std::uint8_t b) {
        switch (s_) {
            case Stato::AttesaInizio: if (b == 0x7E) s_ = Stato::Lunghezza; break;
            case Stato::Lunghezza:
                if (b == 0 || b > buf_.size()) { ++errori; s_ = Stato::AttesaInizio; break; }
                len_ = b; ricevuti_ = 0; s_ = Stato::Payload; break;
            case Stato::Payload:
                buf_[ricevuti_++] = b;
                if (ricevuti_ == len_) s_ = Stato::Crc;
                break;
            case Stato::Crc:
                s_ = Stato::AttesaInizio;
                if (b != crc8(buf_.data(), len_)) { ++errori; break; }
                return std::vector<std::uint8_t>(buf_.begin(), buf_.begin() + len_);
        }
        return std::nullopt;
    }
};

int main() {
    std::vector<std::uint8_t> flusso{0x13, 0x37};                    // rumore iniziale
    for (auto p : {std::vector<std::uint8_t>{21, 5}, {22, 0}, {23, 9}}) {
        auto f = incapsula(p);
        flusso.insert(flusso.end(), f.begin(), f.end());
    }
    flusso[2 + 4 + 3] ^= 0x01;                                       // corrompe il 2o pacchetto
    Parser parser;
    for (auto b : flusso)
        if (auto p = parser.byte(b))
            std::cout << "temperatura " << int((*p)[0]) << '.' << int((*p)[1]) << " C\n";
    std::cout << "pacchetti scartati: " << parser.errori << '\n';
}
```

```
temperatura 21.5 C
temperatura 23.9 C
pacchetti scartati: 1
```

Nota

In un protocollo reale il byte 0x7E potrebbe comparire anche nel payload: si usa il **byte stuffing** (come in HDLC/PPP o COBS) per renderlo univoco, oppure un timeout tra i byte per risincronizzarsi.

Esempio svolto 26.2 - Controllo PID di un motore simulato

Problema. Portare la velocita' di un motore DC a 100 giri/s e mantenerla, confrontando un controllo solo proporzionale con un PID completo.

Ragionamento. Il PID calcola il comando come `Kp * errore + Ki * integrale(errore) + Kd * derivata(errore)`. Il solo termine P lascia un **errore a regime** (al motore serve un comando non nullo per girare, quindi l'errore non puo' essere zero); il termine I lo elimina accumulando l'errore residuo; il termine D smorza le oscillazioni. Il motore e' simulato come sistema del primo ordine: `dv/dt = (K * comando - v) / tau`. Limitare il comando (saturazione) e congelare l'integrale quando si e' saturi (anti-windup) e' indispensabile sull'hardware reale.

```
#include <algorithm>
#include <iomanip>
#include <iostream>

struct PID {
    double kp, ki, kd, integrale = 0, errorePrec = 0;
    double aggiorna(double obiettivo, double misura, double dt) {
        double e = obiettivo - misura;
        double derivata = (e - errorePrec) / dt;
        errorePrec = e;
        double u = kp * e + ki * (integrale + e * dt) + kd * derivata;
        double saturato = std::clamp(u, 0.0, 12.0);            // tensione 0-12 V
        if (u == saturato) integrale += e * dt;                // anti-windup
        return saturato;
    }
};

double simula(PID pid, bool stampa) {
    double v = 0;                                              // giri/s
    const double dt = 0.01, K = 10, tau = 0.5;
    for (int passo = 1; passo <= 300; ++passo) {
        double u = pid.aggiorna(100, v, dt);
        v += (K * u - v) / tau * dt;
        if (stampa && passo % 50 == 0) std::cout << std::setw(6) << v;
    }
    if (stampa) std::cout << '\n';
    return v;
}

int main() {
    std::cout << std::fixed << std::setprecision(1);
    std::cout << "solo P:";  simula({0.05, 0, 0}, true);
    std::cout << "PID:   ";  simula({0.05, 0.2, 0.001}, true);
}
```

```
solo P:  26.1  31.7  33.0  33.3  33.3  33.3
PID:     51.3  85.7 100.3 103.4 102.5 101.1
```

Con il solo P la velocita' si ferma a 33 giri/s: il comando `0.05 * errore` e' sufficiente solo quando l'errore e' ancora grande. Il PID raggiunge l'obiettivo con una piccola sovraelongazione (103,4) che poi si smorza. Aumentando `ki` si arriva prima ma la sovraelongazione cresce; diminuendolo sparisce, ma la risposta diventa piu' lenta: la taratura (manuale, Ziegler-Nichols o automatica) e' un compromesso tra rapidita' e stabilita'.

Esercizi (soluzioni nell'Appendice)

Esercizio 26.1 [Base] - Calcola il CRC-16/CCITT-FALSE (polinomio 0x1021, valore iniziale 0xFFFF) della stringa `"123456789"`: il risultato deve essere `0x29B1`.

Esercizio 26.2 [Medio] - Definisci un concept `BusI2C` (metodi `scrivi(indirizzo, dati)` e `leggi(indirizzo, n)`) e scrivi un driver `SensoreTemperatura<Bus>` che lo usa. Testalo sul PC con un bus finto che registra le transazioni.

Esercizio 26.3 [Medio] - Implementa la macchina a stati di un semaforo pedonale (verde auto, giallo, rosso, verde pedoni, con prenotazione tramite pulsante) guidata da tick di 100 ms, e simulane un minuto.

Esercizio 26.4 [Medio] - Progetta l'architettura FreeRTOS di una stazione meteo: quali task, con quali priorita', quali code o semafori, e chi gestisce Wi-Fi e scheda SD? Giustifica le scelte rispetto a inversione di priorita' e consumo.

Esercizio 26.5 [Avanzato] - Sliding Window Median (LeetCode 480): mediana degli ultimi k campioni, utile come filtro anti-picchi. Usa un `std::multiset` con un iteratore alla mediana.

Parte 9 - Sistemi operativi

Capitolo 27 - Programmazione di sistema (POSIX/Linux e Windows)

Lezione 27.1 - Processi, fork/exec, segnali

Un sistema operativo gestisce processi, memoria, file, dispositivi e rete. In C++ si usano le API di sistema: POSIX su Linux/macOS, Win32 su Windows. Un processo ha un PID, uno spazio di indirizzi, file descriptor. `fork()` clona il processo, `exec*()` ne sostituisce l'immagine, `waitpid()` attende il figlio.

```
#include <unistd.h>
#include <sys/wait.h>
#include <csignal>
#include <iostream>

int main() {
    pid_t pid = fork();
    if (pid == 0) { execlp("ls", "ls", "-l", nullptr); _exit(127); }
    int stato; waitpid(pid, &stato, 0);
    std::cout << "figlio terminato con " << WEXITSTATUS(stato) << '\n';
}
```

Segnali: `SIGINT`, `SIGTERM`, `SIGSEGV`, `SIGCHLD`; con `sigaction` si installano handler (solo funzioni async-signal-safe al loro interno).

Lezione 27.2 - File descriptor, pipe, IPC e memoria condivisa

Tutto e' un file: file regolari, pipe, socket, dispositivi. API: `open`, `read`, `write`, `close`, `dup2`, `pipe`, `mmap`, `fcntl`. IPC: pipe e FIFO, socket Unix, code di messaggi, memoria condivisa (`shm_open` + `mmap`), semafori, segnali. Wrapper RAII: classe `FileDescriptor` che chiama `close()` nel distruttore.

```
class Fd {
    int fd_;
public:
    explicit Fd(int fd = -1) : fd_(fd) {}
    ~Fd() { if (fd_ >= 0) ::close(fd_); }
    Fd(Fd&& o) noexcept : fd_(std::exchange(o.fd_, -1)) {}
    Fd& operator=(Fd&& o) noexcept { std::swap(fd_, o.fd_); return *this; }
    Fd(const Fd&) = delete; Fd& operator=(const Fd&) = delete;
    int get() const { return fd_; }
};
```

Lezione 27.3 - Una mini shell

Esercizio completo: leggi una riga, tokenizza, `fork`+`exec`, gestisci pipe (`a | b`), redirezioni (`>`, `<`), comandi interni (`cd`, `exit`), job in background (`&`) e `SIGINT`.

Esercizi consigliati su LeetCode

71 - Simplify Path

588 - Design In-Memory File System

609 - Find Duplicate File in System

1166 - Design File System

636 - Exclusive Time of Functions

862 - Shortest Subarray with Sum at Least K

Lezione 27.4 - Esempi svolti ed esercizi

Esempio svolto 27.1 - Due processi che dialogano con le pipe

Problema. Il processo padre invia delle parole a un processo figlio, che le restituisce in maiuscolo; il padre stampa le risposte e attende la terminazione del figlio.

Ragionamento. Una pipe e' unidirezionale, quindi ne servono due: padre -> figlio e figlio -> padre. Dopo `fork()` ciascun processo **chiude le estremita' che non usa**: e' fondamentale, perche' chi legge riceve fine-file (read che restituisce 0) solo quando **tutte** le estremita' di scrittura sono chiuse. Il figlio termina con `_exit` per non eseguire distruttori e flush ereditati dal padre.

```
#include <sys/wait.h>
#include <unistd.h>
#include <cctype>
#include <cstring>
#include <iostream>
#include <string>

int main() {
    int versoFiglio[2], versoPadre[2];
    if (pipe(versoFiglio) < 0 || pipe(versoPadre) < 0) { perror("pipe"); return 1; }

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {                                  // ---- figlio ----
        close(versoFiglio[1]); close(versoPadre[0]);
        char buf[256];
        ssize_t n;
        while ((n = read(versoFiglio[0], buf, sizeof buf)) > 0) {
            for (ssize_t i = 0; i < n; ++i)
                buf[i] = std::toupper(static_cast<unsigned char>(buf[i]));
            if (write(versoPadre[1], buf, n) != n) _exit(1);
        }
        close(versoFiglio[0]); close(versoPadre[1]);
        _exit(0);                                    // read == 0: il padre ha chiuso
    }
    close(versoFiglio[0]); close(versoPadre[1]);     // ---- padre ----
    const char* messaggio = "ciao figlio, come stai?";
    if (write(versoFiglio[1], messaggio, std::strlen(messaggio)) < 0) return 1;
    close(versoFiglio[1]);                           // segnala la fine dei dati

    std::string risposta;
    char buf[256];
    ssize_t n;
    while ((n = read(versoPadre[0], buf, sizeof buf)) > 0) risposta.append(buf, n);
    close(versoPadre[0]);
    int stato;
    waitpid(pid, &stato, 0);
    std::cout << "risposta: " << risposta << '\n'
              << "figlio uscito con " << WEXITSTATUS(stato) << '\n';
}
```

```
risposta: CIAO FIGLIO, COME STAI?
figlio uscito con 0
```

Se il padre dimenticasse `close(versoFiglio[1])`, il figlio resterebbe bloccato nella `read` e il padre nella sua: un **deadlock** classico.

Esempio svolto 27.2 - Il cuore di una shell: eseguire una pipeline

Problema. Eseguire una riga come `printf "pera\nmela\nuva\nmela\n" | sort | uniq -c`, collegando l'uscita di ogni comando all'ingresso del successivo, come fa `bash`.

Ragionamento. Per n comandi servono n-1 pipe. Ogni figlio usa `dup2` per sostituire il proprio stdin (fd 0) con l'estremita' di lettura della pipe precedente e il proprio stdout (fd 1) con l'estremita' di scrittura della successiva, poi chiude **tutti** i descrittori delle pipe ed esegue il programma con `execvp`. Il padre chiude le sue copie e attende tutti i figli. Il tokenizzatore qui e' volutamente semplice (spazi e virgolette doppie).

```
#include <sys/wait.h>
#include <unistd.h>
#include <iostream>
#include <string>
#include <vector>

std::vector<std::string> tokenizza(const std::string& s) {
    std::vector<std::string> t;
    std::string cur;
    bool virgolette = false, inToken = false;
    for (std::size_t i = 0; i < s.size(); ++i) {
        char c = s[i];
        if (c == '"') { virgolette = !virgolette; inToken = true; }
        else if (c == '\\' && virgolette && i + 1 < s.size() && s[i + 1] == 'n') {
            cur += '\n'; ++i;                                   // \n tra virgolette
        } else if (c == ' ' && !virgolette) {
            if (inToken) t.push_back(cur);
            cur.clear(); inToken = false;
        }
        else { cur += c; inToken = true; }
    }
    if (inToken) t.push_back(cur);
    return t;
}

int eseguiPipeline(const std::string& riga) {
    std::vector<std::vector<std::string>> comandi(1);
    for (auto& tok : tokenizza(riga))
        if (tok == "|") comandi.emplace_back(); else comandi.back().push_back(tok);

    std::size_t n = comandi.size();
    std::vector<int> fd(2 * (n - 1));
    for (std::size_t i = 0; i + 1 < n; ++i)
        if (pipe(&fd[2 * i]) < 0) return -1;

    std::vector<pid_t> figli;
    for (std::size_t i = 0; i < n; ++i) {
        pid_t pid = fork();
        if (pid == 0) {
            if (i > 0) dup2(fd[2 * (i - 1)], 0);         // stdin dalla pipe precedente
            if (i + 1 < n) dup2(fd[2 * i + 1], 1);       // stdout verso la successiva
            for (int f : fd) close(f);
            std::vector<char*> argv;
            for (auto& a : comandi[i]) argv.push_back(a.data());
            argv.push_back(nullptr);
```

```
            execvp(argv[0], argv.data());
            perror(argv[0]);                              // raggiunto solo se exec fallisce
            _exit(127);
        }
        figli.push_back(pid);
    }
    for (int f : fd) close(f);
    int stato = 0;
    for (pid_t p : figli) waitpid(p, &stato, 0);
    return WEXITSTATUS(stato);                            // come bash: stato dell'ultimo comando
}

int main() {
    std::cout.flush();
    int s = eseguiPipeline(R"(printf "pera\nmela\nuva\nmela\n" | sort | uniq -c)");
    std::cout << "stato: " << s << '\n';
}
```

```
      2 mela
      1 pera
      1 uva
stato: 0
```

Per una shell completa (Lezione 27.3) mancano il ciclo di lettura con prompt, i comandi interni (`cd` deve essere eseguito dal processo shell stesso, non da un figlio, altrimenti cambierebbe la directory del figlio), le redirezioni e la gestione di `SIGINT`.

Esercizi (soluzioni nell'Appendice)

Esercizio 27.1 [Base] - Simplify Path (LeetCode 71): semplifica un percorso Unix assoluto gestendo `.`, `..` e barre ripetute, usando un vettore come pila.

Esercizio 27.2 [Base] - Scrivi `std::string leggiTutto(int fd)` che legge un descrittore fino alla fine gestendo letture parziali ed `EINTR`, e usala con la classe RAII `Fd` della Lezione 27.2 per leggere `/proc/self/status` (solo su Linux).

Esercizio 27.3 [Medio] - Installa con `sigaction` un gestore di `SIGINT` che imposta un flag `volatile std::sig_atomic_t`; il ciclo principale deve terminare in modo ordinato. Perche' non si puo' chiamare `std::cout` o `malloc` dentro il gestore?

Esercizio 27.4 [Medio] - Aggiungi a `eseguiPipeline` le redirezioni `> file` e `< file` con `open` e `dup2`.

Esercizio 27.5 [Avanzato] - Exclusive Time of Functions (LeetCode 636): dati i log `id:start:t` e `id:end:t` di un processore single core, calcola il tempo esclusivo di ogni funzione con una pila.

Capitolo 28 - Concetti di sistemi operativi

Lezione 28.1 - Scheduling, memoria virtuale, paginazione

Scheduling della CPU: FCFS, SJF, Round Robin, priorita', MLFQ, CFS di Linux. Memoria virtuale: tabelle delle pagine, TLB, page fault, swapping, algoritmi di sostituzione (FIFO, LRU, Clock). Allocatori: first-fit/best-fit, buddy system, slab; `malloc` implementato con liste libere. Implementare un allocatore e' un ottimo esercizio.

Esercizi consigliati su LeetCode

146 - LRU Cache

460 - LFU Cache

621 - Task Scheduler

253 - Meeting Rooms II

56 - Merge Intervals

1834 - Single-Threaded CPU

1962 - Remove Stones to Minimize the Total

358 - Rearrange String k Distance Apart

Lezione 28.2 - Sincronizzazione e deadlock

Race condition, sezioni critiche, mutex, semafori, monitor, spinlock, lock-free. Problemi classici: produttore-consumatore, lettori-scrittori, filosofi a cena. Deadlock: condizioni di Coffman (mutua esclusione, hold and wait, no preemption, attesa circolare); prevenzione con ordinamento delle risorse (`std::scoped_lock`/`std::lock` acquisiscono piu' mutex senza deadlock).

Esercizi consigliati su LeetCode

1226 - The Dining Philosophers

1195 - Fizz Buzz Multithreaded

1279 - Traffic Light Controlled Intersection

207 - Course Schedule (rilevare cicli di dipendenza)

Lezione 28.3 - File system e I/O

Struttura di un file system: superblocco, inode, blocchi dati, directory, journaling (ext4), copy-on-write (Btrfs, ZFS, APFS), FAT. Operazioni di I/O: buffered vs direct, `mmap`, `io_uring`, `epoll`, I/O asincrono. Implementa un mini file system in un file immagine.

Lezione 28.4 - Esempi svolti ed esercizi

Esempio svolto 28.1 - Simulare lo scheduler Round Robin

Problema. Quattro processi arrivano con tempi di arrivo e di CPU noti. Con Round Robin e quanto di 2 unita', calcolare il diagramma di esecuzione (Gantt), il tempo di completamento (turnaround) e di attesa di ciascuno.

Ragionamento. Una coda FIFO contiene i processi pronti. Il processo in testa esegue per `min(quanto, residuo)`; i processi arrivati **durante** il quanto entrano in coda **prima** che il processo interrotto vi rientri (e' la convenzione dei libri di testo). Turnaround = completamento - arrivo; attesa = turnaround - burst.

```
#include <algorithm>
#include <iostream>
#include <queue>
#include <string>
#include <vector>

struct Processo { std::string nome; int arrivo, burst, residuo = 0, fine = 0; };

int main() {
    std::vector<Processo> p{{"P1", 0, 5}, {"P2", 1, 3}, {"P3", 2, 1}, {"P4", 4, 2}};
    for (auto& x : p) x.residuo = x.burst;
    const int quanto = 2;
    std::queue<int> pronti;
    std::size_t prossimo = 0;
    int t = 0;
    auto arrivi = [&](int finoA) {
        while (prossimo < p.size() && p[prossimo].arrivo <= finoA) pronti.push(prossimo++);
    };
    arrivi(t);
    std::string gantt;
    while (!pronti.empty() || prossimo < p.size()) {
        if (pronti.empty()) { t = p[prossimo].arrivo; arrivi(t); }   // CPU inattiva
        int i = pronti.front(); pronti.pop();
        int esegui = std::min(quanto, p[i].residuo);
        gantt += "|" + std::to_string(t) + " " + p[i].nome + " ";
        t += esegui;
        p[i].residuo -= esegui;
        arrivi(t);                                  // prima i nuovi arrivati...
        if (p[i].residuo > 0) pronti.push(i);       // ...poi chi e' stato interrotto
        else p[i].fine = t;
    }
    std::cout << gantt << "|" << t << '\n';
    double attesaTot = 0;
    for (auto& x : p) {
        int tat = x.fine - x.arrivo, attesa = tat - x.burst;
        attesaTot += attesa;
        std::cout << x.nome << ": turnaround " << tat << ", attesa " << attesa << '\n';
    }
    std::cout << "attesa media " << attesaTot / p.size() << '\n';
}
```

```
|0 P1 |2 P2 |4 P3 |5 P1 |7 P4 |9 P2 |10 P1 |11
P1: turnaround 11, attesa 6
P2: turnaround 9, attesa 6
P3: turnaround 3, attesa 2
P4: turnaround 5, attesa 3
attesa media 4.25
```

Un quanto piccolo migliora la reattivita' ma aumenta i cambi di contesto (ognuno costa microsecondi e svuota cache e TLB); un quanto enorme degenera in FCFS.

Esempio svolto 28.2 - Sostituzione delle pagine: FIFO, LRU e l'anomalia di Belady

Problema. Contare i page fault della sequenza di riferimenti `1 2 3 4 1 2 5 1 2 3 4 5` con 3 e 4 frame, usando FIFO e LRU.

Ragionamento. FIFO espelle la pagina caricata da piu' tempo; LRU quella usata meno di recente (la stessa idea della LRU cache, Soluzione 6.5). Con FIFO puo' succedere che **piu' memoria produca piu' fault**: e' l'anomalia di Belady. LRU e' un algoritmo "a pila" e non ne soffre.

```
#include <algorithm>
#include <deque>
#include <iostream>
#include <list>
#include <vector>

int faultFIFO(const std::vector<int>& rif, std::size_t frame) {
    std::deque<int> mem;
    int fault = 0;
    for (int p : rif) {
        if (std::find(mem.begin(), mem.end(), p) != mem.end()) continue;
        ++fault;
        if (mem.size() == frame) mem.pop_front();      // la piu' vecchia
        mem.push_back(p);
    }
    return fault;
}

int faultLRU(const std::vector<int>& rif, std::size_t frame) {
    std::list<int> mem;                                // in testa la piu' recente
    int fault = 0;
    for (int p : rif) {
        auto it = std::find(mem.begin(), mem.end(), p);
        if (it != mem.end()) { mem.splice(mem.begin(), mem, it); continue; }
        ++fault;
        if (mem.size() == frame) mem.pop_back();       // la meno recente
        mem.push_front(p);
    }
    return fault;
}

int main() {
    std::vector<int> rif{1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    for (std::size_t f : {3, 4})
        std::cout << f << " frame: FIFO " << faultFIFO(rif, f) << ", LRU " << faultLRU(rif, f)
                  << '\n';
}
```

```
3 frame: FIFO 9, LRU 10
4 frame: FIFO 10, LRU 8
```

I sistemi reali non possono aggiornare una lista a ogni accesso alla memoria: approssimano LRU con il **bit di accesso** delle pagine e l'algoritmo Clock (Esercizio 28.2).

Esercizi (soluzioni nell'Appendice)

Esercizio 28.1 [Base] - Ripeti l'Esempio 28.1 con SJF non preemptive (tra i processi arrivati si sceglie quello con burst minore) e confronta l'attesa media.

Esercizio 28.2 [Medio] - Implementa l'algoritmo Clock (seconda possibilita') e conta i fault sulla sequenza dell'Esempio 28.2 con 3 e 4 frame.

Esercizio 28.3 [Medio] - Rilevare un deadlock: dato un grafo di attesa "il thread A aspetta una risorsa tenuta da B", trova un ciclo con una DFS a tre colori e stampalo.

Esercizio 28.4 [Medio] - The Dining Philosophers (LeetCode 1226): cinque filosofi e cinque forchette con `std::mutex`. Mostra la soluzione con `std::scoped_lock` e quella con l'ordinamento delle risorse.

Esercizio 28.5 [Avanzato] - Scrivi un allocatore first-fit su un buffer statico di 4 KB: ogni blocco ha un'intestazione con dimensione e flag "libero"; `alloca(n)` divide i blocchi troppo grandi, `libera(p)` fonde i blocchi liberi adiacenti.

Capitolo 29 - Costruire un sistema operativo

Lezione 29.1 - Boot, bare metal e kernel minimale

Per sviluppare un OS in C++ si lavora in ambiente **freestanding**: niente libreria standard completa, niente eccezioni/RTTI (o implementati da te), cross-compiler (`x86_64-elf-g++` o `riscv64-elf-g++`), emulatore **QEMU**. Sequenza di avvio: firmware (BIOS/UEFI) -> bootloader (GRUB, limine) -> kernel. Tappe di un kernel didattico:

- Boot con header Multiboot2, passaggio in modalita' 64 bit.
- Output su console VGA/seriale.
- GDT, IDT, gestione delle eccezioni e degli interrupt (PIC/APIC), timer PIT/HPET.
- Gestore di memoria fisica (bitmap/buddy) e virtuale (paginazione), heap del kernel.
- Tastiera (PS/2), driver ATA/AHCI/NVMe, file system.
- Scheduler cooperativo poi preemptive, context switch in assembly.
- Syscall (`syscall`/`sysret`), modalita' utente, caricamento di eseguibili ELF.
- Shell in user space, libc minimale, rete (driver e1000 + TCP/IP).

```
extern "C" void kmain(std::uint32_t magic, void* info) {
    Console::pulisci();
    Console::stampa("Kernel avviato\n");
    Gdt::init(); Idt::init(); Pic::init(); Timer::avvia(100);
    Paging::init(info); Heap::init();
    asm volatile("sti");
    for (;;) asm volatile("hlt");
}
```

```
// global constructors e operatori new nel kernel
void* operator new(std::size_t n) { return Heap::alloca(n); }
void operator delete(void* p) noexcept { Heap::libera(p); }
```

Risorse: OSDev wiki, "Operating Systems: Three Easy Pieces", xv6 (MIT), "Writing an OS in Rust/C++" tutorial, SerenityOS (un OS completo in C++ moderno, ottimo da leggere), Linux kernel (C, con Rust), Haiku e Fuchsia/Zircon (C++).

Lezione 29.2 - Driver, moduli e C++ nel kernel

Scrivere driver per Windows (WDK, KMDF) o per Linux (moduli in C; C++ non e' ammesso nel kernel Linux upstream). In user space con **FUSE** puoi implementare file system, con **libusb** driver USB, con **DPDK** e **io_uring** I/O ad alte prestazioni. Concetti: MMIO, DMA, interrupt handler top/bottom half, concorrenza nel kernel, sicurezza (ring 0 / ring 3, ASLR, NX, SMEP).

Esercizi consigliati su LeetCode

146 - LRU Cache

380 - Insert Delete GetRandom O(1)

705 - Design HashSet

622 - Design Circular Queue

1396 - Design Underground System

1348 - Tweet Counts Per Frequency

Lezione 29.3 - Esempi svolti ed esercizi

Esempio svolto 29.1 - Allocatore di frame fisici a bitmap

Problema. Il kernel deve tenere traccia di quali frame fisici da 4 KB sono liberi. Scrivere un allocatore a bitmap che supporti l'allocazione di un frame e di N frame contigui (per i buffer DMA), testandolo in user space.

Ragionamento. Un bit per frame: con 4 GB di RAM e frame da 4 KB servono 1.048.576 bit = 128 KB. Per trovare velocemente un frame libero si scorre la bitmap a parole di 64 bit, saltando quelle piene (`~0`), e dentro una parola si usa `std::countr_one` per trovare il primo bit a 0. La classe non usa heap ne' eccezioni: e' utilizzabile in ambiente freestanding, dove la bitmap viene posizionata in una zona di memoria nota.

```
#include <bit>
#include <cstddef>
#include <cstdint>
#include <iostream>
#include <optional>

template <std::size_t NFrame>
class AllocatoreFrame {
    static constexpr std::size_t PAROLE = (NFrame + 63) / 64;
    std::uint64_t bit_[PAROLE] = {};
    bool occupato(std::size_t f) const { return bit_[f / 64] >> (f % 64) & 1; }
    void segna(std::size_t f, bool v) {
        if (v) bit_[f / 64] |= 1ull << (f % 64); else bit_[f / 64] &= ~(1ull << (f % 64));
    }
public:
    static constexpr std::size_t DIM_FRAME = 4096;
    void riserva(std::size_t da, std::size_t n) {
        for (std::size_t f = da; f < da + n; ++f) segna(f, true);
    }

    std::optional<std::size_t> alloca() {
        for (std::size_t w = 0; w < PAROLE; ++w) {
            if (bit_[w] == ~0ull) continue;                       // 64 frame occupati: salta
            std::size_t f = w * 64 + std::countr_one(bit_[w]);    // primo bit a 0
            if (f >= NFrame) break;
            segna(f, true);
            return f;
        }
        return std::nullopt;
    }
    std::optional<std::size_t> allocaContigui(std::size_t n) {
        std::size_t corsa = 0;
        for (std::size_t f = 0; f < NFrame; ++f) {
            corsa = occupato(f) ? 0 : corsa + 1;
            if (corsa == n) { riserva(f + 1 - n, n); return f + 1 - n; }
        }
        return std::nullopt;
    }
    void libera(std::size_t f, std::size_t n = 1) {
        for (std::size_t i = f; i < f + n; ++i) segna(i, false);
    }
    std::size_t liberi() const {
        std::size_t occ = 0;
        for (auto w : bit_) occ += std::popcount(w);
        return NFrame - occ;
```

```
    }
};

int main() {
    AllocatoreFrame<256> pmm;                    // 1 MB di RAM simulata
    pmm.riserva(0, 70);                          // kernel e strutture del boot
    auto a = pmm.alloca(), b = pmm.alloca();
    auto dma = pmm.allocaContigui(16);
    pmm.libera(*a);
    auto c = pmm.alloca();                       // riusa il frame appena liberato
    std::cout << "a=" << *a << " b=" << *b << " dma=" << *dma << " c=" << *c
              << " liberi=" << pmm.liberi() << " indirizzo dma=0x" << std::hex
              << *dma * pmm.DIM_FRAME << '\n';
}
```

```
a=70 b=71 dma=72 c=70 liberi=168 indirizzo dma=0x48000
```

Esempio svolto 29.2 - Console di testo in stile VGA, testabile sul PC

Problema. Il primo driver di ogni kernel didattico scrive caratteri nella memoria video VGA in modalita' testo (80 x 25 celle a partire da `0xB8000`, ogni cella = carattere + attributo colore). Scrivere la console con a capo e scorrimento, in modo che si possa provare senza QEMU.

Ragionamento. Ogni cella e' un `uint16_t`: byte basso il carattere, byte alto il colore. Quando il cursore supera l'ultima riga si spostano tutte le righe in su (`memmove`) e si pulisce l'ultima. La classe riceve il puntatore al buffer: nel kernel `reinterpret_cast<volatile std::uint16_t*>(0xB8000)`, nei test un array. Qui usiamo una console piccola (20 x 4) per vedere lo scorrimento.

```
#include <cstdint>
#include <cstring>
#include <iostream>

template <int COL, int RIG>
class Console {
    std::uint16_t* buf_;
    int x_ = 0, y_ = 0;
    std::uint8_t colore_ = 0x07;                  // grigio su nero
    void scorri() {
        std::memmove(buf_, buf_ + COL, sizeof(std::uint16_t) * COL * (RIG - 1));
        for (int i = 0; i < COL; ++i) buf_[COL * (RIG - 1) + i] = ' ' | colore_ << 8;
        y_ = RIG - 1;
    }
public:
    explicit Console(std::uint16_t* b) : buf_(b) {
        for (int i = 0; i < COL * RIG; ++i) b[i] = ' ' | 0x0700;
    }
    void colore(std::uint8_t c) { colore_ = c; }
    void put(char c) {
        if (c == '\n') { x_ = 0; ++y_; }
        else {
            buf_[y_ * COL + x_] = static_cast<std::uint8_t>(c) | colore_ << 8;
            if (++x_ == COL) { x_ = 0; ++y_; }
        }
        if (y_ == RIG) scorri();
    }
    void stampa(const char* s) { while (*s) put(*s++); }
};

int main() {
    std::uint16_t memoriaVideo[20 * 4];           // nel kernel: 0xB8000, 80 x 25
    Console<20, 4> con(memoriaVideo);
    con.stampa("Kernel avviato\n");
    con.colore(0x0C);                             // rosso chiaro
    con.stampa("Errore: page fault\n");
    con.colore(0x07);
    con.stampa("riga tre\nriga quattro, troppo lunga\n");
    for (int r = 0; r < 4; ++r) {
        std::cout << '[';
        for (int c = 0; c < 20; ++c)
            std::cout << static_cast<char>(memoriaVideo[r * 20 + c] & 0xFF);
        std::cout << "] attr 0x" << std::hex << (memoriaVideo[r * 20] >> 8) << std::dec << '\n';
    }
}
```

```
[riga tre            ] attr 0x7
[riga quattro, troppo] attr 0x7
[ lunga              ] attr 0x7
[                    ] attr 0x7
```

Le prime due righe (compresa quella rossa) sono uscite dallo schermo per lo scorrimento; la riga troppo lunga e' andata a capo da sola dopo 20 caratteri e il `\n` finale ha lasciato il cursore su una riga vuota. Nel kernel lo stesso codice viene compilato con `-ffreestanding -fno-exceptions -fno-rtti` e `memmove` va fornita dal kernel stesso.

Esercizi (soluzioni nell'Appendice)

Esercizio 29.1 [Medio] - Dato un indirizzo virtuale x86_64 a 48 bit, estrai i quattro indici delle tabelle delle pagine (PML4, PDPT, PD, PT, 9 bit ciascuno) e l'offset (12 bit). Verifica con l'indirizzo `0xFFFF800000201ABC`.

Esercizio 29.2 [Medio] - Scrivi `kprintf` freestanding (senza libreria standard) che supporti `%d`, `%u`, `%x`, `%s`, `%c` e `%%`, usando `va_list` e scrivendo carattere per carattere con la `put` della console.

Esercizio 29.3 [Avanzato] - Simula in user space il cambio di contesto di uno scheduler cooperativo con `<ucontext.h>` (`getcontext`, `makecontext`, `swapcontext`): tre "thread" che stampano e cedono la CPU a turno.

Esercizio 29.4 [Avanzato] - Implementa un buddy allocator su 1 MB con blocchi da 4 KB a 1 MB: allocazione arrotondata alla potenza di 2, divisione dei blocchi e fusione con il "compagno" (`indirizzo ^ dimensione`) alla liberazione.

Parte 10 - Programmazione di reti

Capitolo 30 - Fondamenti di networking

Lezione 30.1 - Modello ISO/OSI e TCP/IP

Livelli TCP/IP: collegamento (Ethernet, Wi-Fi), rete (IP, ICMP, ARP), trasporto (TCP, UDP, QUIC), applicazione (HTTP, DNS, TLS, SMTP, SSH, MQTT). Concetti: indirizzi IPv4/IPv6, subnet, porte, NAT, routing, MTU, handshake a tre vie di TCP, controllo di flusso e di congestione. Strumenti: `ping`, `traceroute`, `ss`, `nc`, `tcpdump`, **Wireshark**, `curl`, `nmap`.

Endianness: la rete usa **big endian** (network byte order). Funzioni `htons`, `htonl`, `ntohs`, `ntohl`, oppure `std::byteswap` (C++23) e `std::endian` (C++20).

Lezione 30.2 - Socket BSD in C++

Server TCP sequenziale con socket POSIX:

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <cstring>
#include <iostream>

int main() {
    int srv = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1; setsockopt(srv, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof opt);
    sockaddr_in a{}; a.sin_family = AF_INET; a.sin_addr.s_addr = INADDR_ANY; a.sin_port = htons(9000);
    bind(srv, (sockaddr*)&a, sizeof a);
    listen(srv, 16);
    for (;;) {
        int c = accept(srv, nullptr, nullptr);
        char buf[1024];
        ssize_t n;
        while ((n = recv(c, buf, sizeof buf, 0)) > 0) send(c, buf, n, 0);   // echo
        close(c);
    }
}
```

Client TCP: `socket` -> `getaddrinfo` -> `connect` -> `send/recv` -> `close`. Per UDP: `SOCK_DGRAM`, `sendto/recvfrom`. Gestire sempre: letture/scritture parziali, `EINTR`, timeout, chiusura ordinata (`shutdown`), `SIGPIPE`.

Lezione 30.3 - Esempi svolti ed esercizi

Esempio svolto 30.1 - Client e server TCP nello stesso programma

Problema. Scrivere un server echo e un client che si collega, invia tre messaggi e legge le risposte, facendo girare il server in un thread dello stesso programma su una porta scelta dal sistema operativo.

Ragionamento. Facendo `bind` sulla porta 0 il kernel assegna una porta libera, che si legge con `getsockname`: niente conflitti con altri programmi. Il server viene messo in ascolto (`listen`) **prima** di avviare il client, cosi' la `connect` non puo' fallire. TCP e' un flusso di byte, non di messaggi: una `recv` puo' restituire meno byte di quelli inviati, per questo usiamo `inviaTutto` e leggiamo finche' non abbiamo la risposta completa.

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <unistd.h>
#include <iostream>
#include <string>
#include <thread>

bool inviaTutto(int fd, const std::string& s) {
    std::size_t inviati = 0;
    while (inviati < s.size()) {
        ssize_t n = send(fd, s.data() + inviati, s.size() - inviati, MSG_NOSIGNAL);
        if (n <= 0) return false;
        inviati += n;
    }
    return true;
}

int main() {
    int srv = socket(AF_INET, SOCK_STREAM, 0);
    sockaddr_in a{};
    a.sin_family = AF_INET;
    a.sin_addr.s_addr = htonl(INADDR_LOOPBACK);             // 127.0.0.1
    a.sin_port = 0;                                         // porta scelta dal kernel
    if (bind(srv, (sockaddr*)&a, sizeof a) < 0 || listen(srv, 4) < 0) { perror("srv"); return 1; }
    socklen_t len = sizeof a;
    getsockname(srv, (sockaddr*)&a, &len);
    unsigned short porta = ntohs(a.sin_port);

    std::jthread server([srv] {
        int c = accept(srv, nullptr, nullptr);
        char buf[512];
        ssize_t n;
        while ((n = recv(c, buf, sizeof buf, 0)) > 0)       // 0 = il client ha chiuso
            inviaTutto(c, "eco: " + std::string(buf, n));
        close(c);
    });

    int cli = socket(AF_INET, SOCK_STREAM, 0);
    sockaddr_in dest{};
    dest.sin_family = AF_INET;
    dest.sin_port = htons(porta);
    inet_pton(AF_INET, "127.0.0.1", &dest.sin_addr);
    if (connect(cli, (sockaddr*)&dest, sizeof dest) < 0) { perror("connect"); return 1; }
    for (std::string msg : {"ciao", "come va?", "fine"}) {
```

```
        inviaTutto(cli, msg);
        std::string risposta;
        char buf[512];
        while (risposta.size() < msg.size() + 5) {          // "eco: " + messaggio
            ssize_t n = recv(cli, buf, sizeof buf, 0);
            if (n <= 0) break;
            risposta.append(buf, n);
        }
        std::cout << risposta << '\n';
    }
    close(cli);                                             // il server vede recv == 0
    server.join();
    close(srv);
    std::cout << "porta usata: " << (porta > 1024 ? "effimera" : "privilegiata") << '\n';
}
```

```
eco: ciao
eco: come va?
eco: fine
porta usata: effimera
```

> **Nota**
>
> Qui il client conosce la lunghezza della risposta; in generale serve un **protocollo di framing** (delimitatore o prefisso di lunghezza, Esempio 31.1) per sapere dove finisce un messaggio.

Esempio svolto 30.2 - Indirizzi IPv4, maschere e byte order

Problema. Data la notazione CIDR `192.168.10.77/26`, calcolare indirizzo di rete, broadcast, primo e ultimo host e numero di host; mostrare poi come lo stesso intero appare in memoria in host order e in network order.

Ragionamento. Un indirizzo IPv4 e' un intero a 32 bit; `/26` significa maschera con i 26 bit alti a 1. Rete = `ip & maschera`, broadcast = `rete | ~maschera`. Sulle CPU x86 e ARM (little endian) il byte meno significativo sta all'indirizzo piu' basso, mentre in rete si trasmette per primo il piu' significativo (big endian): `htonl` converte, `std::endian` permette di saperlo a compile time.

```
#include <arpa/inet.h>
#include <bit>
#include <cstdint>
#include <cstdio>
#include <cstring>
#include <iostream>
#include <string>

std::uint32_t parseIPv4(const std::string& s) {
    unsigned a, b, c, d;
    std::sscanf(s.c_str(), "%u.%u.%u.%u", &a, &b, &c, &d);
    return a << 24 | b << 16 | c << 8 | d;
}
std::string comeTesto(std::uint32_t ip) {
    return std::to_string(ip >> 24) + '.' + std::to_string(ip >> 16 & 255) + '.' +
           std::to_string(ip >> 8 & 255) + '.' + std::to_string(ip & 255);
}

int main() {
    std::string cidr = "192.168.10.77/26";
    auto barra = cidr.find('/');
    std::uint32_t ip = parseIPv4(cidr.substr(0, barra));
    int prefisso = std::stoi(cidr.substr(barra + 1));
    std::uint32_t maschera = prefisso == 0 ? 0 : ~0u << (32 - prefisso);
    std::uint32_t rete = ip & maschera, broadcast = rete | ~maschera;
    std::cout << "maschera  " << comeTesto(maschera) << "\nrete      " << comeTesto(rete)
              << "\nbroadcast " << comeTesto(broadcast) << "\nhost      " << comeTesto(rete + 1)
              << " - " << comeTesto(broadcast - 1) << " (" << broadcast - rete - 1 << ")\n";

    unsigned char host[4], net[4];
    std::uint32_t inRete = htonl(ip);
    std::memcpy(host, &ip, 4);
    std::memcpy(net, &inRete, 4);
    std::cout << "little endian? " << (std::endian::native == std::endian::little)
              << " | in memoria host: " << int(host[0]) << ' ' << int(host[3])
              << " | network: " << int(net[0]) << ' ' << int(net[3]) << '\n';
}
```

```
maschera  255.255.255.192
rete      192.168.10.64
broadcast 192.168.10.127
host      192.168.10.65 - 192.168.10.126 (62)
little endian? 1 | in memoria host: 77 192 | network: 192 77
```

Esercizi (soluzioni nell'Appendice)

Esercizio 30.1 [Base] - Scrivi un server e un client UDP (`SOCK_DGRAM`, `sendto`, `recvfrom`) nello stesso programma, sul modello dell'Esempio 30.1. Che cosa cambia nella gestione dei messaggi rispetto a TCP?

Esercizio 30.2 [Base] - Scrivi `bool riceviTutto(int fd, char* buf, std::size_t n)` che legge esattamente n byte gestendo letture parziali, `EINTR` e la chiusura della connessione.

Esercizio 30.3 [Medio] - Usa `getaddrinfo` per risolvere `localhost` e stampa tutti gli indirizzi restituiti (IPv4 e IPv6) con `inet_ntop`. Ricordati di `freeaddrinfo` (meglio: un `unique_ptr` con deleter).

Esercizio 30.4 [Medio] - Implementa il checksum di Internet (RFC 1071) usato da IP, ICMP, TCP e UDP: somma in complemento a uno delle parole di 16 bit. Verifica che ricalcolando il checksum su un'intestazione che lo contiene gia' si ottenga 0.

Capitolo 31 - Server scalabili

Lezione 31.1 - Modelli di concorrenza dei server

1. **Un processo/thread per connessione**: semplice, non scala. 2. **Thread pool** con coda di lavori. 3. **Multiplexing** con `select`/`poll` e soprattutto `epoll` (Linux), `kqueue` (BSD/macOS), IOCP (Windows), `io_uring` (Linux moderno). 4. **Reactor** (event loop) o **Proactor** (completamento asincrono). 5. **Coroutine** sopra un event loop (Asio, libuv, liburing).

Il problema C10K/C10M: gestire decine di migliaia di connessioni con pochi thread, buffer non bloccanti, backpressure e timeout.

```
int ep = epoll_create1(0);
epoll_event ev{}; ev.events = EPOLLIN; ev.data.fd = srv;
epoll_ctl(ep, EPOLL_CTL_ADD, srv, &ev);
epoll_event eventi[64];
for (;;) {
    int n = epoll_wait(ep, eventi, 64, -1);
    for (int i = 0; i < n; ++i) {
        int fd = eventi[i].data.fd;
        if (fd == srv) { int c = accept4(srv, nullptr, nullptr, SOCK_NONBLOCK); /* registra c */ }
        else gestisciLettura(fd);
    }
}
```

Lezione 31.2 - Protocolli applicativi e serializzazione

Progettare un protocollo: framing (prefisso di lunghezza, delimitatori), versioning, checksum, gestione di messaggi parziali. Formati: JSON, Protocol Buffers, FlatBuffers, MessagePack, CBOR. Il codice di parsing deve essere robusto contro input malevoli (limiti di lunghezza, validazione, fuzzing).

```
struct Intestazione { std::uint32_t lunghezza; std::uint16_t tipo; std::uint16_t versione; };
// invio: lunghezza in network byte order, poi payload
```

Lezione 31.3 - TLS, sicurezza e RPC

TLS con OpenSSL, BoringSSL, mbedTLS, wolfSSL o `Boost.Asio::ssl`: certificati, catena di fiducia, SNI, ALPN, verifica dell'hostname. Evitare di scrivere crittografia in proprio. Altri temi: autenticazione, rate limiting, SYN flood, buffer overflow nei parser, DoS. **gRPC** (HTTP/2 + protobuf), **WebSocket**, **QUIC/HTTP3** (msquic, ngtcp2), **ZeroMQ**, **nanomsg** per messaggistica distribuita.

Esercizi consigliati su LeetCode

535 - Encode and Decode TinyURL

271 - Encode and Decode Strings (framing)
359 - Logger Rate Limiter
362 - Design Hit Counter
1146 - Snapshot Array
1244 - Design A Leaderboard
981 - Time Based Key-Value Store
1396 - Design Underground System
1500 - Design a File Sharing System

Lezione 31.4 - Esempi svolti ed esercizi

Esempio svolto 31.1 - Framing con prefisso di lunghezza e messaggi spezzati

Problema. Su una connessione TCP si inviano messaggi di testo. Il ricevitore riceve i byte in blocchi arbitrari (un messaggio puo' arrivare a pezzi, o piu' messaggi in un solo blocco) e deve ricostruire i messaggi originali.

Ragionamento. Ogni messaggio e' preceduto dalla sua lunghezza in 4 byte big endian. Il ricevitore accumula i byte in un buffer e, finche' il buffer contiene almeno un'intestazione **e** il payload corrispondente, estrae un messaggio completo. Un limite alla lunghezza massima impedisce a un client malevolo di far allocare gigabyte annunciando una lunghezza enorme. La classe non usa socket: la si testa simulando la frammentazione.

```
#include <cstdint>
#include <iostream>
#include <optional>
#include <stdexcept>
#include <string>
#include <vector>

std::string incapsula(const std::string& msg) {
    std::uint32_t n = msg.size();
    std::string out{char(n >> 24), char(n >> 16), char(n >> 8), char(n)};   // big endian
    return out + msg;
}

class Deframer {
    std::string buf_;
    static constexpr std::uint32_t MAX = 1 << 20;            // 1 MB
public:
    void ricevi(const std::string& blocco) { buf_ += blocco; }
    std::optional<std::string> prossimo() {
        if (buf_.size() < 4) return std::nullopt;
        auto b = [&](int i) { return std::uint32_t(static_cast<unsigned char>(buf_[i])); };
        std::uint32_t n = b(0) << 24 | b(1) << 16 | b(2) << 8 | b(3);
        if (n > MAX) throw std::runtime_error("messaggio troppo lungo: chiudo la connessione");
        if (buf_.size() < 4 + n) return std::nullopt;        // payload incompleto
        std::string msg = buf_.substr(4, n);
        buf_.erase(0, 4 + n);
        return msg;
    }
};

int main() {
    std::string flusso = incapsula("ciao") + incapsula("") + incapsula("messaggio piu' lungo");
    Deframer d;
    for (std::size_t dim : {3, 5, 1, 30}) {                  // blocchi come li darebbe recv()
        std::string blocco = flusso.substr(0, dim);
        flusso.erase(0, dim);
        d.ricevi(blocco);
        std::cout << "ricevuti " << blocco.size() << " byte:";
        while (auto m = d.prossimo()) std::cout << " [" << *m << ']';
        std::cout << '\n';
    }
}
```

```
ricevuti 3 byte:
ricevuti 5 byte: [ciao]
ricevuti 1 byte:
ricevuti 27 byte: [] [messaggio piu' lungo]
```

Esempio svolto 31.2 - Un server epoll che gestisce piu' client con un solo thread

Problema. Scrivere un server che serve contemporaneamente molti client con un unico thread, usando socket non bloccanti ed `epoll`, e verificarlo con tre client concorrenti.

Ragionamento. Il socket di ascolto e quelli dei client sono registrati in un'unica istanza epoll. `epoll_wait` restituisce solo i descrittori **pronti**: sul socket di ascolto si accettano le nuove connessioni, sugli altri si legge. Con socket non bloccanti una `recv` senza dati restituisce `EAGAIN` invece di bloccare l'intero server. Ogni client invia un numero e riceve il doppio; quando tutti hanno chiuso, il server termina. Per semplicita' si assume che ogni messaggio arrivi in una sola `recv` (vero su loopback con messaggi piccoli; in produzione si usa il framing dell'Esempio 31.1 e un buffer per connessione).

```
#include <arpa/inet.h>
#include <fcntl.h>
#include <netinet/in.h>
#include <sys/epoll.h>
#include <sys/socket.h>
#include <unistd.h>
#include <iostream>
#include <string>
#include <thread>
#include <vector>

int main() {
    int srv = socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK, 0);
    sockaddr_in a{};
    a.sin_family = AF_INET;
    a.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    bind(srv, (sockaddr*)&a, sizeof a);
    listen(srv, 16);
    socklen_t len = sizeof a;
    getsockname(srv, (sockaddr*)&a, &len);

    std::vector<std::string> risultati(3);
    std::vector<std::jthread> client;
    for (int i = 0; i < 3; ++i)
        client.emplace_back([&, i, porta = a.sin_port] {
            int c = socket(AF_INET, SOCK_STREAM, 0);
            sockaddr_in d{};
            d.sin_family = AF_INET; d.sin_port = porta; d.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
            connect(c, (sockaddr*)&d, sizeof d);
            std::string msg = std::to_string((i + 1) * 10);
            send(c, msg.data(), msg.size(), 0);
            char buf[64];
            ssize_t n = recv(c, buf, sizeof buf, 0);
            risultati[i] = msg + " -> " + std::string(buf, n > 0 ? n : 0);
            close(c);
        });

    int ep = epoll_create1(0);
    epoll_event ev{};
    ev.events = EPOLLIN; ev.data.fd = srv;
    epoll_ctl(ep, EPOLL_CTL_ADD, srv, &ev);
    int aperti = 0, chiusi = 0;
    epoll_event eventi[16];
    while (chiusi < 3) {
        int n = epoll_wait(ep, eventi, 16, 5000);
```

```
        if (n <= 0) break;                                          // timeout di sicurezza
        for (int i = 0; i < n; ++i) {
            int fd = eventi[i].data.fd;
            if (fd == srv) {                                        // nuove connessioni
                int c;
                while ((c = accept4(srv, nullptr, nullptr, SOCK_NONBLOCK)) >= 0) {
                    epoll_event e{};
                    e.events = EPOLLIN; e.data.fd = c;
                    epoll_ctl(ep, EPOLL_CTL_ADD, c, &e);
                    ++aperti;
                }
            } else {
                char buf[64];
                ssize_t r = recv(fd, buf, sizeof buf, 0);
                if (r > 0) {
                    std::string risposta = std::to_string(2 * std::stoi(std::string(buf, r)));
                    send(fd, risposta.data(), risposta.size(), MSG_NOSIGNAL);
                } else if (r == 0) {                                // il client ha chiuso
                    epoll_ctl(ep, EPOLL_CTL_DEL, fd, nullptr);
                    close(fd);
                    ++chiusi;
                }                                                   // r < 0 con EAGAIN: niente
            }
        }
    }
    client.clear();                                                 // join dei client
    for (auto& r : risultati) std::cout << r << '\n';
    std::cout << "connessioni servite: " << aperti << " con un solo thread\n";
    close(ep); close(srv);
}
```

```
10 -> 20
20 -> 40
30 -> 60
connessioni servite: 3 con un solo thread
```

Esercizi (soluzioni nell'Appendice)

Esercizio 31.1 [Base] - Encode and Decode Strings (LeetCode 271): codifica un vettore di stringhe arbitrarie (che possono contenere qualsiasi carattere) in una sola stringa e viceversa, usando il formato `lunghezza#contenuto`.

Esercizio 31.2 [Medio] - Estendi il parser HTTP dell'Esempio 20.1 per supportare il keep-alive: da un buffer che contiene piu' richieste consecutive, estrai una richiesta completa usando `Content-Length` per sapere dove finisce il corpo.

Esercizio 31.3 [Medio] - Design Hit Counter (LeetCode 362): conta gli accessi negli ultimi 300 secondi con memoria costante, usando due array circolari di 300 posizioni.

Esercizio 31.4 [Medio] - Snapshot Array (LeetCode 1146): un array con `set`, `snap` e `get(indice, idSnap)`, memorizzando per ogni cella solo le modifiche (la stessa idea delle versioni MVCC nei database).

Esercizio 31.5 [Avanzato] - Modifica il server dell'Esempio 31.2 perche' usi un buffer per connessione e il framing dell'Esempio 31.1, e gestisca la scrittura non bloccante registrando `EPOLLOUT` quando `send` non riesce a inviare tutto.

Capitolo 32 - Algoritmi e progetti di rete

Lezione 32.1 - Algoritmi di rete

Instradamento (Dijkstra e Bellman-Ford sono alla base di OSPF e RIP), spanning tree (STP), flusso massimo, load balancing (round robin, least connections, consistent hashing), rate limiting (token bucket,

leaky bucket, sliding window), controllo di congestione (AIMD, CUBIC, BBR), compressione (Huffman, LZ77), checksum e CRC, codici di correzione degli errori.

Esercizi consigliati su LeetCode

743 - Network Delay Time

1334 - Find the City With the Smallest Number of Neighbors at a Threshold Distance

1135 - Connecting Cities With Minimum Cost

1319 - Number of Operations to Make Network Connected

1202 - Smallest String With Swaps

886 - Possible Bipartition

1584 - Min Cost to Connect All Points

1976 - Number of Ways to Arrive at Destination

2092 - Find All People With Secret

Lezione 32.2 - Progetti di rete

- Chat multiutente con epoll o Asio (stanze, nickname, comandi).
- Client/server HTTP minimale con parser delle richieste e file statici.
- Port scanner e traceroute con raw socket (solo su reti tue!).
- Proxy TCP/SOCKS5 e load balancer.
- Server DNS ricorsivo semplice con cache.
- Trasferimento file affidabile sopra UDP (ritrasmissioni, finestra scorrevole).
- Client MQTT e broker minimale.
- Gioco multiplayer client-server autoritativo (vedi Parte 7).

Lezione 32.3 - Esempi svolti ed esercizi

Esempio svolto 32.1 - Consistent hashing per distribuire le chiavi tra i server

Problema. Una cache distribuita assegna ogni chiave a uno tra N server. Con `hash(chiave) % N`, aggiungendo un server quasi tutte le chiavi cambiano posto (e la cache si svuota). Mostrare che con il consistent hashing se ne sposta solo circa 1/(N+1).

Ragionamento. Server e chiavi vengono mappati su un anello di valori hash; ogni chiave appartiene al primo server che si incontra procedendo in senso orario. Aggiungendo un server, solo le chiavi dell'arco che esso "conquista" cambiano proprietario. Ogni server e' inserito piu' volte (**nodi virtuali**) per distribuire il carico in modo uniforme. L'anello e' una `std::map` ordinata: il proprietario si trova con `lower_bound` in O(log n). Usiamo FNV-1a come funzione di hash, perche' `std::hash` non garantisce lo stesso risultato su compilatori diversi.

```
#include <cstdint>
#include <iostream>
#include <map>
#include <string>
#include <vector>

std::uint32_t fnv1a(const std::string& s) {
    std::uint32_t h = 2166136261u;
    for (unsigned char c : s) { h ^= c; h *= 16777619u; }
    h ^= h >> 15; h *= 0x2c1b3c6dU; h ^= h >> 12;           // mescola meglio i bit alti
    return h;
}

class Anello {
    std::map<std::uint32_t, std::string> nodi_;
public:
    void aggiungi(const std::string& server, int virtuali = 100) {
        for (int i = 0; i < virtuali; ++i) nodi_[fnv1a(server + "#" + std::to_string(i))] = server;
    }
    const std::string& proprietario(const std::string& chiave) const {
        auto it = nodi_.lower_bound(fnv1a(chiave));
        return it == nodi_.end() ? nodi_.begin()->second : it->second;   // l'anello si chiude
    }
};

int main() {
    std::vector<std::string> chiavi;
    for (int i = 0; i < 10000; ++i) chiavi.push_back("utente:" + std::to_string(i));

    Anello anello;
    for (auto s : {"cache-a", "cache-b", "cache-c", "cache-d"}) anello.aggiungi(s);
    std::map<std::string, int> carico;
    std::vector<std::string> prima;
    for (auto& k : chiavi) { prima.push_back(anello.proprietario(k)); ++carico[prima.back()]; }
    anello.aggiungi("cache-e");

    int spostateAnello = 0, spostateModulo = 0;
    for (std::size_t i = 0; i < chiavi.size(); ++i) {
        spostateAnello += anello.proprietario(chiavi[i]) != prima[i];
        spostateModulo += fnv1a(chiavi[i]) % 4 != fnv1a(chiavi[i]) % 5;
    }
    std::cout << "carico con 4 server:";
    for (auto& [s, n] : carico) std::cout << ' ' << n;
    std::cout << "\nchiavi spostate aggiungendo un server: anello " << spostateAnello / 100.0
              << "%, modulo " << spostateModulo / 100.0 << "%\n";
```

```
}
```

```
carico con 4 server: 2197 2885 2313 2605
chiavi spostate aggiungendo un server: anello 19.36%, modulo 79.61%
```

Con l'anello si sposta circa il 20% delle chiavi (il minimo possibile: 1/5 deve andare sul nuovo server), con il modulo circa l'80%. Il carico non e' perfettamente uniforme (da 2197 a 2885 chiavi): aumentando i nodi virtuali per server le differenze si riducono, al prezzo di un anello piu' grande.

Esempio svolto 32.2 - Compressione di Huffman

Problema. Comprimere un testo assegnando ai caratteri frequenti codici binari corti e a quelli rari codici lunghi, senza ambiguita' in decodifica.

Ragionamento. L'algoritmo greedy di Huffman unisce ripetutamente i due nodi meno frequenti (una `priority_queue`) finche' resta un solo albero; il codice di un carattere e' il percorso dalla radice (0 = sinistra, 1 = destra). Poiche' i caratteri stanno solo nelle foglie, nessun codice e' prefisso di un altro: la decodifica scorre l'albero bit per bit. Huffman e' usato in DEFLATE (zip, gzip, PNG), JPEG e HTTP/2

(HPACK).

```
#include <iostream>
#include <map>
#include <memory>
#include <queue>
#include <string>
#include <vector>

struct Nodo {
    int freq; char c; std::unique_ptr<Nodo> sx, dx;
    bool foglia() const { return !sx; }
};

void codici(const Nodo* n, const std::string& pref, std::map<char, std::string>& out) {
    if (n->foglia()) { out[n->c] = pref.empty() ? "0" : pref; return; }
    codici(n->sx.get(), pref + '0', out);
    codici(n->dx.get(), pref + '1', out);
}

int main() {
    std::string testo = "abracadabra alla cabala";
    std::map<char, int> freq;
    for (char c : testo) ++freq[c];

    auto cmp = [](const Nodo* a, const Nodo* b) {          // min-heap, spareggio deterministico
        return a->freq != b->freq ? a->freq > b->freq : a->c > b->c;
    };
    std::priority_queue<Nodo*, std::vector<Nodo*>, decltype(cmp)> pq(cmp);
    for (auto [c, f] : freq) pq.push(new Nodo{f, c, nullptr, nullptr});
    while (pq.size() > 1) {
        Nodo* a = pq.top(); pq.pop();
        Nodo* b = pq.top(); pq.pop();
        pq.push(new Nodo{a->freq + b->freq, std::min(a->c, b->c),
                         std::unique_ptr<Nodo>(a), std::unique_ptr<Nodo>(b)});
    }
    std::unique_ptr<Nodo> radice(pq.top());                // da qui l'albero si libera da solo

    std::map<char, std::string> cod;
    codici(radice.get(), "", cod);
    std::string bit;
    for (char c : testo) bit += cod[c];
    for (auto& [c, s] : cod) std::cout << "'" << c << "'=" << s << ' ';

    std::string decodificato;
    for (const Nodo* n = radice.get(); char b : bit) {
        n = b == '0' ? n->sx.get() : n->dx.get();
        if (n->foglia()) { decodificato += n->c; n = radice.get(); }
    }
    std::cout << "\n" << testo.size() * 8 << " bit -> " << bit.size() << " bit, decodifica "
              << (decodificato == testo ? "corretta" : "ERRATA") << '\n';
}
```

```
' '=1001 'a'=0 'b'=101 'c'=1110 'd'=1000 'l'=110 'r'=1111
184 bit -> 56 bit, decodifica corretta
```

Esercizi (soluzioni nell'Appendice)

Esercizio 32.1 [Base] - Number of Operations to Make Network Connected (LeetCode 1319): con quanti spostamenti di cavi si collegano tutti i computer? Usa Union-Find: servono `componenti - 1` cavi, se ce ne sono almeno `n - 1`.

Esercizio 32.2 [Medio] - Implementa un leaky bucket (coda a uscita costante) e confrontalo con il token bucket dell'Esempio 20.2 sulla stessa raffica di richieste: quale dei due ammette i picchi?

Esercizio 32.3 [Medio] - Calcola il CRC-32 (polinomio riflesso 0xEDB88320, usato da Ethernet, zip e PNG) con una tabella di 256 voci precalcolata a compile time. Il check value di `"123456789"` e' `0xCBF43926`.

Esercizio 32.4 [Medio] - Scrivi due bilanciatori di carico, round robin e least connections, e simula 20 richieste di durata variabile su 3 backend: quale distribuisce meglio il lavoro quando le durate sono molto diverse?

Esercizio 32.5 [Avanzato] - Simula un protocollo affidabile Go-Back-N sopra un canale che perde pacchetti in modo deterministico (per esempio ogni quinto pacchetto): mittente con finestra di 4, ACK cumulativi, ritrasmissione allo scadere del timeout. Conta le trasmissioni totali per consegnare 20 pacchetti.

Parte 11 - Programmazione di database

Capitolo 33 - Usare database da C++

Lezione 33.1 - SQL e SQLite

Un database relazionale organizza i dati in tabelle; SQL permette di interrogarli (`SELECT`, `JOIN`, `GROUP BY`), modificarli (`INSERT`, `UPDATE`, `DELETE`) e definirne lo schema. Concetti: chiavi primarie/esterne, indici, normalizzazione, transazioni e proprieta' **ACID**, livelli di isolamento (read committed, repeatable read, serializable), query plan (`EXPLAIN`).

SQLite e' la via piu' semplice per cominciare (libreria C embedded, zero configurazione).

```
#include <sqlite3.h>
#include <iostream>
#include <memory>

int main() {
    sqlite3* raw = nullptr;
    sqlite3_open("corso.db", &raw);
    std::unique_ptr<sqlite3, decltype(&sqlite3_close)> db(raw, &sqlite3_close);

    sqlite3_exec(db.get(), "CREATE TABLE IF NOT EXISTS utenti(id INTEGER PRIMARY KEY, nome TEXT, eta INT);", 0, 0, 0);

    sqlite3_stmt* st;
    sqlite3_prepare_v2(db.get(), "INSERT INTO utenti(nome, eta) VALUES(?, ?);", -1, &st, nullptr);
    sqlite3_bind_text(st, 1, "Anna", -1, SQLITE_TRANSIENT);   // query parametrizzata: niente SQL injection
    sqlite3_bind_int(st, 2, 30);
    sqlite3_step(st); sqlite3_finalize(st);

    sqlite3_prepare_v2(db.get(), "SELECT nome, eta FROM utenti;", -1, &st, nullptr);
    while (sqlite3_step(st) == SQLITE_ROW)
        std::cout << sqlite3_column_text(st, 0) << ' ' << sqlite3_column_int(st, 1) << '\n';
    sqlite3_finalize(st);
}
```

Nota

Usa sempre query parametrizzate (prepared statement): concatenare stringhe nelle query espone ad attacchi di SQL injection.

Esercizi consigliati su LeetCode

175 - Combine Two Tables (SQL)

176 - Second Highest Salary (SQL)

181 - Employees Earning More Than Their Managers (SQL)

182 - Duplicate Emails (SQL)

183 - Customers Who Never Order (SQL)

184 - Department Highest Salary (SQL)

262 - Trips and Users (SQL)

185 - Department Top Three Salaries (SQL)

Lezione 33.2 - PostgreSQL, MySQL e ORM

Client C++ per i principali DBMS: **libpqxx** (PostgreSQL), **MySQL Connector/C++** e **MariaDB Connector**, **ODBC/nanodbc**, **SOCI** (astrazione multi-database), **sqlpp11**, **ODB**, **Drogon ORM**, **QxOrm**; per NoSQL:

mongocxx (MongoDB), **hiredis/redis-plus-plus** (Redis), **cassandra-cpp-driver**, **cpp-driver per Couchbase**. Pool di connessioni, transazioni RAII, gestione degli errori, mapping tra tipi SQL e C++ (`std::optional` per NULL).

```
#include <pqxx/pqxx>
int main() {
    pqxx::connection c("dbname=corso user=app host=localhost");
    pqxx::work tx(c);
    pqxx::result r = tx.exec_params("SELECT nome FROM utenti WHERE eta > $1", 18);
    for (auto riga : r) std::cout << riga[0].as<std::string>() << '\n';
    tx.exec_params("UPDATE utenti SET eta = eta + 1 WHERE id = $1", 7);
    tx.commit();    // senza commit: rollback automatico nel distruttore
}
```

Lezione 33.3 - Esempi svolti ed esercizi

Esempio svolto 33.1 - Un wrapper RAII per SQLite con transazioni

Problema. L'API C di SQLite richiede di ricordarsi `sqlite3_finalize`, `sqlite3_close`, `COMMIT` e `ROLLBACK` e di controllare ogni codice di ritorno. Scrivere un piccolo strato C++ che renda impossibile dimenticarli, e usarlo per un registro di ordini.

Ragionamento. Tre classi RAII: `Db` (chiude la connessione), `Statement` (finalizza la query preparata, con `bind` variadico) e `Transazione` (esegue `BEGIN` nel costruttore e `ROLLBACK` nel distruttore se nessuno ha chiamato `commit`). Gli errori diventano eccezioni con il messaggio di SQLite. Si usa un database `:memory:`, cosi' l'esempio non lascia file.

```
// compila: -lsqlite3
#include <sqlite3.h>
#include <iostream>
#include <stdexcept>
#include <string>

class Db {
    sqlite3* db_ = nullptr;
public:
    explicit Db(const char* percorso) {
        if (sqlite3_open(percorso, &db_) != SQLITE_OK)
            throw std::runtime_error(sqlite3_errmsg(db_));
    }
    ~Db() { sqlite3_close(db_); }
    Db(const Db&) = delete; Db& operator=(const Db&) = delete;
    sqlite3* get() const { return db_; }
    void esegui(const char* sql) {
        char* err = nullptr;
        if (sqlite3_exec(db_, sql, nullptr, nullptr, &err) != SQLITE_OK) {
            std::string m = err; sqlite3_free(err);
            throw std::runtime_error(m);
        }
    }
};

class Statement {
    sqlite3_stmt* st_ = nullptr;
    sqlite3* db_;
    int i_ = 0;
    void lega(int v) { sqlite3_bind_int(st_, ++i_, v); }
    void lega(double v) { sqlite3_bind_double(st_, ++i_, v); }
    void lega(const std::string& v) {
        sqlite3_bind_text(st_, ++i_, v.c_str(), -1, SQLITE_TRANSIENT);   // SQLite copia il testo
    }
public:
    Statement(Db& db, const char* sql) : db_(db.get()) {
        if (sqlite3_prepare_v2(db_, sql, -1, &st_, nullptr) != SQLITE_OK)
            throw std::runtime_error(sqlite3_errmsg(db_));
    }
    ~Statement() { sqlite3_finalize(st_); }
    Statement(const Statement&) = delete; Statement& operator=(const Statement&) = delete;
    template <typename... A> Statement& bind(const A&... a) { (lega(a), ...); return *this; }
    bool passo() {                                      // true se c'e' una riga
        int rc = sqlite3_step(st_);
        if (rc == SQLITE_ROW) return true;
```

```
        if (rc == SQLITE_DONE) return false;
        throw std::runtime_error(sqlite3_errmsg(db_));
    }
    void reset() { sqlite3_reset(st_); sqlite3_clear_bindings(st_); i_ = 0; }
    std::string testo(int c) const {
        return reinterpret_cast<const char*>(sqlite3_column_text(st_, c));
    }
    double reale(int c) const { return sqlite3_column_double(st_, c); }
};

class Transazione {
    Db& db_;
    bool fatta_ = false;
public:
    explicit Transazione(Db& db) : db_(db) { db_.esegui("BEGIN"); }
    void commit() { db_.esegui("COMMIT"); fatta_ = true; }
    ~Transazione() { if (!fatta_) try { db_.esegui("ROLLBACK"); } catch (...) {} }
};

int main() {
    Db db(":memory:");
    db.esegui("CREATE TABLE ordini(id INTEGER PRIMARY KEY, cliente TEXT NOT NULL,"
              " importo REAL CHECK(importo > 0))");
    {
        Transazione tx(db);
        Statement ins(db, "INSERT INTO ordini(cliente, importo) VALUES(?, ?)");
        for (auto [c, imp] : {std::pair{"Anna", 30.5}, {"Luca", 12.0}, {"Anna", 7.25}}) {
            ins.bind(std::string(c), imp).passo();
            ins.reset();
        }
        tx.commit();
    }
    try {                                                  // il secondo inserimento viola il CHECK
        Transazione tx(db);
        Statement ins(db, "INSERT INTO ordini(cliente, importo) VALUES(?, ?)");
        ins.bind(std::string("Sara"), 50.0).passo(); ins.reset();
        ins.bind(std::string("Sara"), -5.0).passo();
        tx.commit();
    } catch (const std::exception& e) {
        std::cout << "errore: " << e.what() << " -> rollback\n";
    }
    Statement q(db, "SELECT cliente, SUM(importo) FROM ordini GROUP BY cliente ORDER BY cliente");
    while (q.passo()) std::cout << q.testo(0) << ": " << q.reale(1) << '\n';
}
```

```
errore: CHECK constraint failed: importo > 0 -> rollback
Anna: 37.75
Luca: 12
```

Sara non compare: anche il suo primo ordine, valido, e' stato annullato dal rollback, perche' la transazione e' **atomica**.

Esempio svolto 33.2 - Query SQL classiche, eseguite da C++

Problema. Su una tabella `dipendenti(nome, reparto, stipendio, capo)` rispondere a: (a) il secondo stipendio piu' alto (LeetCode 176); (b) chi guadagna piu' del proprio capo (LeetCode 181); (c) i due stipendi piu' alti di ogni reparto (LeetCode 185).

Ragionamento. (a) `DISTINCT` + `ORDER BY ... LIMIT 1 OFFSET 1`, avvolto in una sottoquery perche' restituisca `NULL` se non esiste. (b) Un **self-join**: la tabella compare due volte, una come dipendente e una come capo. (c) Le **window function** (`DENSE_RANK() OVER (PARTITION BY ...)`) numerano le righe all'interno di ciascun gruppo. SQLite le supporta dalla versione 3.25.

```
// compila: -lsqlite3
#include <sqlite3.h>
#include <iostream>

int stampa(void*, int n, char** valori, char**) {
    for (int i = 0; i < n; ++i) std::cout << (i ? " | " : "") << (valori[i] ? valori[i] : "NULL");
    std::cout << '\n';
    return 0;
}

int main() {
    sqlite3* db;
    sqlite3_open(":memory:", &db);
    sqlite3_exec(db, R"(
        CREATE TABLE dipendenti(id INTEGER PRIMARY KEY, nome TEXT, reparto TEXT,
                                stipendio INT, capo INT REFERENCES dipendenti(id));
        INSERT INTO dipendenti VALUES
            (1, 'Rita', 'IT', 5000, NULL), (2, 'Ugo', 'IT', 5200, 1), (3, 'Ines', 'IT', 4000, 1),
            (4, 'Gino', 'Vendite', 3500, NULL), (5, 'Lea', 'Vendite', 3900, 4),
            (6, 'Ada', 'Vendite', 3500, 4), (7, 'Teo', 'IT', 4000, 1);)",
        nullptr, nullptr, nullptr);

    const char* query[] = {
        "SELECT (SELECT DISTINCT stipendio FROM dipendenti ORDER BY stipendio DESC"
        " LIMIT 1 OFFSET 1) AS secondo",

        "SELECT d.nome, d.stipendio, c.nome FROM dipendenti d"
        " JOIN dipendenti c ON d.capo = c.id WHERE d.stipendio > c.stipendio",

        "SELECT reparto, nome, stipendio FROM ("
        "  SELECT *, DENSE_RANK() OVER (PARTITION BY reparto ORDER BY stipendio DESC) AS r"
        "  FROM dipendenti) WHERE r <= 2 ORDER BY reparto, stipendio DESC, nome",
    };
    for (const char* q : query) {
        sqlite3_exec(db, q, stampa, nullptr, nullptr);
        std::cout << "--\n";
    }
    sqlite3_close(db);
}
```

```
5000
--
Ugo | 5200 | Rita
Lea | 3900 | Gino
--
IT | Ugo | 5200
IT | Rita | 5000
Vendite | Lea | 3900
Vendite | Ada | 3500
Vendite | Gino | 3500
--
```

`DENSE_RANK` assegna lo stesso rango ai pari merito (Ada e Gino), quindi in Vendite compaiono tre persone; con `ROW_NUMBER` se ne vedrebbero esattamente due, scelte arbitrariamente.

Esercizi (soluzioni nell'Appendice)

Esercizio 33.1 [Base] - Scrivi la query di LeetCode 183 (clienti che non hanno mai ordinato) in due modi: con `LEFT JOIN ... WHERE o.id IS NULL` e con `NOT EXISTS`. Eseguile con il wrapper dell'Esempio 33.1.

Esercizio 33.2 [Base] - Aggiungi a `Statement` un metodo `std::optional<std::string> testoOpzionale(int colonna)` che restituisce `std::nullopt` per i valori `NULL` (`sqlite3_column_type == SQLITE_NULL`).

Esercizio 33.3 [Medio] - Mostra con un esempio concreto perche' `"SELECT * FROM utenti WHERE nome = '" + input + "'"` e' vulnerabile all'SQL injection (che cosa succede con `input = "x' OR '1'='1"`?) e verifica che la versione con `?` e `bind` non lo sia.

Esercizio 33.4 [Medio] - Inserisci 100.000 righe con e senza una transazione esplicita e misura i tempi. Perche' la differenza e' cosi' grande? Poi crea un indice e confronta l'output di `EXPLAIN QUERY PLAN` di una ricerca prima e dopo.

Capitolo 34 - Costruire un database: strutture e algoritmi

Lezione 34.1 - Storage engine: pagine, B-tree, LSM-tree

Un DBMS e' fatto di: parser SQL, planner/ottimizzatore, executor, **storage engine**, buffer pool, gestore di transazioni, log (WAL). Molti database sono scritti in C++: MySQL, MongoDB, ClickHouse, RocksDB, LevelDB, DuckDB, ScyllaDB, FoundationDB.

- **Pagine** di dimensione fissa (4-16 KB) su disco; **buffer pool** con politica LRU/Clock.
- **B+tree**: indice ordinato con letture ottimizzate per disco; usato da InnoDB e PostgreSQL.
- **LSM-tree**: memtable in RAM + SSTable su disco con compaction (RocksDB, Cassandra); ottimo per scritture.
- **Hash index**, **bitmap index**, **R-tree** (geospaziale), **inverted index** (full-text).
- **WAL** (Write-Ahead Logging) per durabilita' e recupero dai crash (ARIES).
- **MVCC** per letture non bloccanti; lock a due fasi per l'isolamento.
- **Colonnare** vs **riga**: analitica (OLAP) vs transazionale (OLTP).

```
// Scheletro di una pagina slotted
struct Pagina {
    static constexpr std::size_t DIM = 4096;
    std::uint16_t nSlot, offsetLibero;
    std::byte dati[DIM - 4];
};
```

Esercizi consigliati su LeetCode

146 - LRU Cache (buffer pool)

460 - LFU Cache

981 - Time Based Key-Value Store

1146 - Snapshot Array (MVCC)

355 - Design Twitter

981 - Time Based Key-Value Store

706 - Design HashMap

208 - Implement Trie (Prefix Tree)

307 - Range Sum Query - Mutable

315 - Count of Smaller Numbers After Self

23 - Merge k Sorted Lists (merge delle SSTable)

Lezione 34.2 - Parser e ottimizzatore di query

Un parser SQL si costruisce con lexer + parser a discesa ricorsiva (o ANTLR, Bison, PEGTL). Il risultato e' un AST che il planner trasforma in un piano logico e poi fisico (scelta di indici, ordine dei join, algoritmi

nested-loop/hash/merge join). Un'ottimizzazione basata sui costi usa statistiche e istogrammi.

```
struct Espr { virtual ~Espr() = default; virtual double valuta() const = 0; };
// Parser a discesa ricorsiva per: expr := term (('+'|'-') term)*
```

Esercizi consigliati su LeetCode

224 - Basic Calculator

227 - Basic Calculator II

772 - Basic Calculator III

150 - Evaluate Reverse Polish Notation

394 - Decode String

726 - Number of Atoms

1096 - Brace Expansion II

Lezione 34.3 - Progetti con database

- Rubrica / gestionale con SQLite e interfaccia a riga di comando.
- Key-value store persistente con WAL e compattazione (stile Bitcask).
- Piccolo B-tree su file con pagine e buffer pool.
- Mini SQL engine: CREATE/INSERT/SELECT con WHERE su tabelle in memoria, poi su disco.
- API REST (Parte 6) collegata a PostgreSQL con pool di connessioni.
- Cache distribuita con protocollo Redis (RESP) sopra il server epoll della Parte 10.

Lezione 34.4 - Esempi svolti ed esercizi

Esempio svolto 34.1 - Un mini LSM-tree: memtable, SSTable, tombstone e compaction

Problema. Realizzare il cuore di uno storage engine stile LevelDB/RocksDB: scritture veloci in memoria, scarico periodico su "file" ordinati immutabili, letture che consultano i livelli dal piu' recente, cancellazioni tramite **tombstone** e compattazione.

Ragionamento. La **memtable** e' una `std::map` ordinata: quando supera una soglia diventa una SSTable immutabile (qui un vettore ordinato; su disco sarebbe un file). Una lettura cerca prima nella memtable, poi nelle SSTable dalla piu' nuova alla piu' vecchia, con ricerca binaria; il primo valore trovato vince. Cancellare scrive un tombstone (`std::nullopt`), perche' le SSTable non si modificano. La **compaction** fonde tutte le SSTable tenendo solo la versione piu' recente di ogni chiave e scartando i tombstone.

```
#include <algorithm>
#include <iostream>
#include <map>
#include <optional>
#include <string>
#include <utility>
#include <vector>

using Valore = std::optional<std::string>;                    // nullopt = tombstone
using SSTable = std::vector<std::pair<std::string, Valore>>;  // ordinata per chiave

class LSM {
    std::map<std::string, Valore> memtable_;
    std::vector<SSTable> livelli_;                            // dal piu' vecchio al piu' nuovo
    std::size_t soglia_;
    void scarica() {
        livelli_.emplace_back(memtable_.begin(), memtable_.end());
        memtable_.clear();
    }
public:
    explicit LSM(std::size_t soglia) : soglia_(soglia) {}
    void put(const std::string& k, std::string v) {
        memtable_[k] = std::move(v);
        if (memtable_.size() >= soglia_) scarica();
    }
    void del(const std::string& k) {
        memtable_[k] = std::nullopt;
        if (memtable_.size() >= soglia_) scarica();
    }
    Valore get(const std::string& k) const {
        if (auto it = memtable_.find(k); it != memtable_.end()) return it->second;
        for (auto t = livelli_.rbegin(); t != livelli_.rend(); ++t) {
            auto it = std::ranges::lower_bound(*t, k, {}, &SSTable::value_type::first);
            if (it != t->end() && it->first == k) return it->second;
        }
        return std::nullopt;
    }
    void compatta() {
        std::map<std::string, Valore> fuso;
        for (const auto& t : livelli_)                          // i piu' nuovi sovrascrivono
            for (const auto& [k, v] : t) fuso[k] = v;
        SSTable unica;
        for (auto& [k, v] : fuso) if (v) unica.emplace_back(k, v);   // via i tombstone
        livelli_.assign(1, std::move(unica));
    }
```

```
    std::size_t numeroSSTable() const { return livelli_.size(); }
    std::size_t vociSuDisco() const {
        std::size_t n = 0;
        for (auto& t : livelli_) n += t.size();
        return n;
    }
};

int main() {
    LSM db(3);
    db.put("mela", "1"); db.put("pera", "2"); db.put("uva", "3");      // -> SSTable 1
    db.put("mela", "10"); db.del("pera"); db.put("kiwi", "4");         // -> SSTable 2
    db.put("fico", "5");                                               // resta in memtable
    auto mostra = [&](const char* k) { std::cout << k << '=' << db.get(k).value_or("-") << ' '; };
    for (auto k : {"mela", "pera", "uva", "kiwi", "fico"}) mostra(k);
    std::cout << "| sstable " << db.numeroSSTable() << ", voci " << db.vociSuDisco() << '\n';
    db.compatta();
    for (auto k : {"mela", "pera", "uva", "kiwi", "fico"}) mostra(k);
    std::cout << "| sstable " << db.numeroSSTable() << ", voci " << db.vociSuDisco() << '\n';
}
```

```
mela=10 pera=- uva=3 kiwi=4 fico=5 | sstable 2, voci 6
mela=10 pera=- uva=3 kiwi=4 fico=5 | sstable 1, voci 3
```

Nei motori reali ogni SSTable ha anche un **Bloom filter** che evita di leggerla quando la chiave non c'e' di sicuro, la memtable e' protetta da un WAL (Esercizio 34.2) e la compattazione e' organizzata a livelli per limitare il lavoro.

Esempio svolto 34.2 - Parser a discesa ricorsiva per espressioni e clausole WHERE

Problema. Costruire l'albero sintattico (AST) di espressioni come `eta >= 18 AND (citta = 'Roma' OR punti * 2 > 100)` e valutarlo su righe di una tabella in memoria: e' cio' che fa la parte "parser + executor" di un DBMS.

Ragionamento. La grammatica definisce le precedenze, e ogni regola diventa una funzione:

- `or := and ('OR' and)*`
- `and := cmp ('AND' cmp)*`
- `cmp := somma (('=' | '<' | '>' | '>=' | '<=') somma)?`
- `somma := prod (('+' | '-') prod)*`, `prod := prim (('*' | '/') prim)*`
- `prim := numero | 'stringa' | colonna | '(' or ')'`

Il lexer trasforma il testo in token. I nodi dell'AST sono classi con un metodo virtuale `valuta(riga)` (la classe `Espr` della Lezione 34.2); i valori sono `std::variant<double, std::string>`.

```
#include <cctype>
#include <iostream>
#include <map>
#include <memory>
#include <stdexcept>
#include <string>
#include <variant>
#include <vector>

using Valore = std::variant<double, std::string>;
using Riga = std::map<std::string, Valore>;

struct Espr {
    virtual ~Espr() = default;
    virtual Valore valuta(const Riga& r) const = 0;
};
using P = std::unique_ptr<Espr>;

struct Costante : Espr {
    Valore v; explicit Costante(Valore x) : v(std::move(x)) {}
    Valore valuta(const Riga&) const override { return v; }
};
struct Colonna : Espr {
    std::string n; explicit Colonna(std::string x) : n(std::move(x)) {}
    Valore valuta(const Riga& r) const override { return r.at(n); }
};
struct Binaria : Espr {
    std::string op; P a, b;
    Binaria(std::string o, P x, P y) : op(std::move(o)), a(std::move(x)), b(std::move(y)) {}
    Valore valuta(const Riga& r) const override {
        Valore x = a->valuta(r), y = b->valuta(r);
        auto num = [](const Valore& v) { return std::get<double>(v); };   // lancia se e' testo
        if (op == "AND") return double(num(x) != 0 && num(y) != 0);
        if (op == "OR") return double(num(x) != 0 || num(y) != 0);
        if (op == "=") return double(x == y);
        if (op == "<") return double(x < y);
        if (op == ">") return double(x > y);
        if (op == "<=") return double(x <= y);
        if (op == ">=") return double(x >= y);
        if (op == "+") return num(x) + num(y);
        if (op == "-") return num(x) - num(y);
        if (op == "*") return num(x) * num(y);
        return num(x) / num(y);
    }
};
```

```
class Parser {
    std::vector<std::string> t_;
    std::size_t i_ = 0;
    bool vede(const std::string& s) const { return i_ < t_.size() && t_[i_] == s; }
    bool accetta(const std::string& s) { if (vede(s)) { ++i_; return true; } return false; }
public:
    explicit Parser(const std::string& s) {                    // lexer
        for (std::size_t i = 0; i < s.size();) {
            char c = s[i];
            if (std::isspace(static_cast<unsigned char>(c))) { ++i; continue; }
            auto parola = [](char x) {
                return std::isalnum(static_cast<unsigned char>(x)) || x == '.';
            };
            std::size_t j = i + 1;
            if (parola(c))
                while (j < s.size() && parola(s[j])) ++j;
            else if (c == '\'') {
                j = s.find('\'', i + 1) + 1;
                if (j == 0) throw std::runtime_error("' mancante");
            } else if ((c == '<' || c == '>') && i + 1 < s.size() && s[i + 1] == '=') j = i + 2;
            t_.push_back(s.substr(i, j - i));
            i = j;
        }
    }
    P parseOr() {
        P a = parseAnd();
        while (accetta("OR")) a = std::make_unique<Binaria>("OR", std::move(a), parseAnd());
        return a;
    }
    P parseAnd() {
        P a = parseCmp();
        while (accetta("AND")) a = std::make_unique<Binaria>("AND", std::move(a), parseCmp());
        return a;
    }
    P parseCmp() {
        P a = parseSomma();
        for (const char* op : {"=", "<", ">", "<=", ">="})
            if (accetta(op)) return std::make_unique<Binaria>(op, std::move(a), parseSomma());
        return a;
    }
    P parseSomma() {
        P a = parseProd();
        for (;;) {
            if (accetta("+")) a = std::make_unique<Binaria>("+", std::move(a), parseProd());
```

```
            else if (accetta("-")) a = std::make_unique<Binaria>("-", std::move(a), parseProd());
            else return a;
        }
    }
    P parseProd() {
        P a = parsePrim();
        for (;;) {
            if (accetta("*")) a = std::make_unique<Binaria>("*", std::move(a), parsePrim());
            else if (accetta("/")) a = std::make_unique<Binaria>("/", std::move(a), parsePrim());
            else return a;
        }
    }
    P parsePrim() {
        if (i_ >= t_.size()) throw std::runtime_error("espressione incompleta");
        if (accetta("(")) {
            P e = parseOr();
            if (!accetta(")")) throw std::runtime_error("')' attesa");
            return e;
        }
        std::string t = t_[i_++];
        if (t[0] == '\'') return std::make_unique<Costante>(t.substr(1, t.size() - 2));
        if (std::isdigit(static_cast<unsigned char>(t[0])))
            return std::make_unique<Costante>(std::stod(t));
        return std::make_unique<Colonna>(t);
    }
    P parse() {
        P e = parseOr();
        if (i_ != t_.size()) throw std::runtime_error("token inatteso: " + t_[i_]);
        return e;
    }
};

int main() {
    std::vector<Riga> tabella{
        {{"nome", "Anna"}, {"eta", 34.0}, {"citta", "Roma"}, {"punti", 20.0}},
        {{"nome", "Bruno"}, {"eta", 17.0}, {"citta", "Roma"}, {"punti", 90.0}},
        {{"nome", "Carla"}, {"eta", 41.0}, {"citta", "Milano"}, {"punti", 75.0}},
        {{"nome", "Dario"}, {"eta", 25.0}, {"citta", "Torino"}, {"punti", 10.0}},
    };
    for (std::string where : {"eta >= 18 AND (citta = 'Roma' OR punti * 2 > 100)",
                              "punti - eta < 0", "eta >= 18 AND (citta = 'Roma'"}) {
        std::cout << "WHERE " << where << " ->";
        try {
            P filtro = Parser(where).parse();
            for (const auto& r : tabella)
```

```
                if (std::get<double>(filtro->valuta(r)) != 0)
                    std::cout << ' ' << std::get<std::string>(r.at("nome"));
        } catch (const std::exception& e) { std::cout << " errore: " << e.what(); }
        std::cout << '\n';
    }
}
```

```
WHERE eta >= 18 AND (citta = 'Roma' OR punti * 2 > 100) -> Anna Carla
WHERE punti - eta < 0 -> Anna Dario
WHERE eta >= 18 AND (citta = 'Roma' -> errore: ')' attesa
```

Un DBMS vero fa un passo in piu': prima di valutare, il **planner** analizza l'AST e, per esempio, nota che `citta = 'Roma'` puo' usare un indice su `citta` invece di scorrere tutta la tabella.

Esercizi (soluzioni nell'Appendice)

Esercizio 34.1 [Medio] - Basic Calculator II (LeetCode 227): valuta un'espressione con `+ - * /` tra interi non negativi senza parentesi, in una passata e con una pila (o con il parser dell'Esempio 34.2 ridotto alle regole `somma` e `prod`).

Esercizio 34.2 [Medio] - Aggiungi al mini LSM un **Write-Ahead Log**: ogni `put`/`del` viene prima accodato a un file di testo (`P chiave valore` o `D chiave`); scrivi la funzione di recupero che, dopo un "crash", ricostruisce la memtable rileggendo il log.

Esercizio 34.3 [Medio] - Evaluate Reverse Polish Notation (LeetCode 150), poi scrivi la funzione che converte un AST dell'Esempio 34.2 in notazione polacca inversa con una visita post-order.

Esercizio 34.4 [Avanzato] - Buffer pool: una cache di N pagine da 4 KB con politica LRU in cui le pagine **fissate** (pin count > 0) non possono essere espulse e le pagine **sporche** vengono riscritte sul "disco" prima dell'espulsione. Conta letture e scritture su disco in uno scenario di prova.

Esercizio 34.5 [Avanzato] - Merge k Sorted Lists (LeetCode 23) applicato alla compaction: fondi k SSTable ordinate in una sola con una `priority_queue`, in O(n log k), mantenendo la versione piu' recente in caso di chiavi uguali.

Parte 12 - Percorso di studio e progetti finali

Capitolo 35 - Piano di studio e preparazione ai colloqui

Lezione 35.1 - Piano di studio suggerito (24 settimane)

- Settimane 1-4: Parte 1 (basi) + 30 problemi LeetCode Easy su array, stringhe, hash map.
- Settimane 5-8: Parte 2 (OOP, template, C++ moderno) + liste concatenate, stack, queue.
- Settimane 9-12: Parte 3 (STL e strutture dati) + alberi, heap, grafi (Medium).
- Settimane 13-16: Parte 4 (algoritmi): ricerca binaria, backtracking, DP, grafi.
- Settimane 17-18: Parte 5 (concorrenza, prestazioni, librerie, test).
- Settimane 19-24: scegli un percorso di specializzazione (Parti 6-11) e realizza un progetto completo.

Tecnica per LeetCode: leggi il problema, scrivi esempi, individua il pattern (hash map, two pointers, sliding window, stack, binary search, BFS/DFS, DP, heap, backtracking, union-find), stima la complessita', implementa, testa casi limite. Liste utili: **Blind 75**, **NeetCode 150**, **Grind 75**. Obiettivo: 150-300 problemi risolti in modo ragionato, non 1000 a memoria.

Esercizi consigliati su LeetCode

Blind 75 - lista curata di problemi fondamentali

NeetCode 150 - percorso per pattern

Grind 75 - piano di studio a settimane

LeetCode Explore Cards - "Arrays 101", "Linked List", "Binary Tree", "Graph", "Dynamic Programming"

Lezione 35.2 - Progetti finali consigliati

1. **Libreria di strutture dati** generiche con concepts e test (Catch2). 2. **Server HTTP** asincrono con Asio e database PostgreSQL. 3. **Gioco 2D** con ECS (EnTT) e SDL/SFML, poi port su WebAssembly. 4. **Firmware** per ESP32/STM32 con FreeRTOS, sensore e MQTT. 5. **Mini kernel** x86_64/RISC-V che stampa su seriale, gestisce interrupt e memoria. 6. **Key-value store** persistente con protocollo di rete. 7. **App mobile** con core C++ condiviso tra Android (JNI) e iOS (Swift interop).

Lezione 35.3 - Risorse per continuare

- Libri: "A Tour of C++" (Stroustrup), "Effective Modern C++" (Meyers), "C++ Concurrency in Action" (Williams), "C++ Templates: The Complete Guide", "The C++ Programming Language", "Programming: Principles and Practice Using C++", "Game Programming Patterns", "Computer Systems: A Programmer's Perspective", "Designing Data-Intensive Applications", "Introduction to Algorithms (CLRS)".
- Siti: cppreference.com, isocpp.org, C++ Core Guidelines, Compiler Explorer (godbolt.org), learncpp.com, cp-algorithms.com.
- Conferenze e canali: CppCon, C++Now, Meeting C++, The Cherno, Jason Turner (C++ Weekly).
- Pratica: Exercism, Codeforces, AtCoder, Advent of Code, contributi open source (LLVM, Qt, Boost, SerenityOS).

Lezione 35.4 - Esempio svolto di colloquio ed esercizi di riepilogo

Esempio svolto 35.1 - Un problema da colloquio affrontato passo passo

Problema. Longest Consecutive Sequence (LeetCode 128): dato un vettore non ordinato di interi, trovare la lunghezza della piu' lunga sequenza di numeri consecutivi (in qualunque posizione). Esempio: `[100, 4, 200, 1, 3, 2]` -> 4 (la sequenza 1, 2, 3, 4). Richiesto O(n).

Ragionamento. Ecco come conviene ragionare **ad alta voce** durante un colloquio.

- **Chiarire**: ci sono duplicati? (si', vanno ignorati); il vettore puo' essere vuoto? (si', risposta 0); i numeri possono essere negativi? (si').
- **Soluzione ovvia**: ordinare e contare le corse consecutive. O(n log n): corretta, ma non rispetta il vincolo. Dirlo comunque mostra che si ha un punto di partenza.
- **Osservazione chiave**: una sequenza e' identificata dal suo **inizio**, cioe' un numero `x` tale che `x - 1` non e' presente. Con un `unordered_set` il test di presenza costa O(1).
- **Algoritmo**: per ogni `x` che e' un inizio, contare `x + 1, x + 2, ...` finche' sono presenti.
- **Complessita'**: sembra O(n^2) per il ciclo annidato, ma ogni numero viene visitato dal ciclo interno **una sola volta** (solo partendo dal proprio inizio): O(n) totale.
- **Test**: vettore vuoto, un solo elemento, duplicati, negativi, tutto consecutivo.

```
#include <algorithm>
#include <iostream>
#include <unordered_set>
#include <vector>

int sequenzaPiuLunga(const std::vector<int>& a) {
    std::unordered_set<int> s(a.begin(), a.end());
    int best = 0;
    for (int x : s) {
        if (s.count(x - 1)) continue;                // non e' l'inizio di una sequenza
        int lung = 1;
        while (s.count(x + lung)) ++lung;
        best = std::max(best, lung);
    }
    return best;
}

int main() {
    for (const auto& v : std::vector<std::vector<int>>{
             {100, 4, 200, 1, 3, 2}, {}, {7}, {0, 3, 7, 2, 5, 8, 4, 6, 0, 1}, {-2, -1, 0, 5, 5}})
        std::cout << sequenzaPiuLunga(v) << ' ';
    std::cout << '\n';
}
```

```
4 0 1 9 3
```

> **Nota**
>
> Iterare su `s` e non su `a` evita di ripetere il lavoro per i duplicati: con un input come un milione di copie dello stesso numero, iterare su `a` resterebbe O(n), ma con piu' controlli inutili.

> **Esercizi (soluzioni nell'Appendice)**
>
> **Esercizio 35.1** [Medio] - Trapping Rain Water (LeetCode 42): quanta acqua resta intrappolata tra le barre di un istogramma? Parti dalla soluzione O(n) con due vettori di massimi (sinistra e destra), poi riducila a O(1) spazio con due puntatori.
>
> **Esercizio 35.2** [Medio] - Word Break (LeetCode 139): una stringa si puo' dividere in parole di un dizionario? Formula la DP `ok[i]` = "i primi i caratteri sono divisibili" e spiega come la descriveresti in un colloquio.
>
> **Esercizio 35.3** [Medio] - Merge k Sorted Lists (LeetCode 23): confronta tre approcci (fondere a coppie in sequenza, divide et impera, min-heap) indicando la complessita' di ciascuno con k liste e n elementi totali.

Esercizio 35.4 [Avanzato] - Progetta (senza scrivere tutto il codice) un servizio di accorciamento URL capace di gestire 10.000 richieste al secondo: API, schema dei dati, generazione dei codici (Soluzione 20.2), cache (Esempio 32.1 e Soluzione 6.5), rate limiting (Esempio 20.2), stima di memoria e di spazio su disco per cinque anni.

Appendice - Soluzioni degli esercizi

Questa appendice contiene le soluzioni degli esercizi proposti nelle lezioni "Esempi svolti ed esercizi" di ogni capitolo. Prova sempre a risolvere l'esercizio da solo prima di leggere la soluzione: anche un tentativo fallito rende la soluzione molto piu' istruttiva. Le soluzioni non sono le uniche possibili: confronta la tua con quella proposta su correttezza, casi limite, complessita' e leggibilita'.

Tutti i programmi completi (quelli con una funzione `main`) di esempi e soluzioni sono stati compilati con GCC 13 (`-std=c++20 -Wall -Wextra -fsanitize=address,undefined`) ed eseguiti, verificando che l'output coincida con quello riportato nel testo; i programmi che richiedono librerie esterne (SQLite, cpp-httplib, SFML) o hardware specifico sono indicati nel codice. Lo script `verifica_esempi.py` ripete il controllo: `python3 verifica_esempi.py src/*.txt`. Gli output indicati come "tipici" (tempi di esecuzione) dipendono dalla macchina. Lo script segnala anche una decina di frammenti delle lezioni originali che richiedono librerie esterne (Boost.Asio, SFML, EnTT, FreeRTOS, libpqxx), `<print>` del C++23 o un server in ascolto senza fine: per quelli e' normale.

Soluzioni - Parte 1

Capitolo 1 - Introduzione e ambiente di lavoro

Soluzione 1.1

```
#include <iostream>
#include <string_view>

int main(int argc, char* argv[]) {
    std::string_view nome = argc > 1 ? argv[1] : "mondo";
    std::cout << "Ciao, " << nome << "!\n";
}
```

```
Ciao, mondo!
```

Soluzione 1.2

Gli avvisi sono due: `y` e' una variabile inutilizzata (`-Wunused-variable`) e la funzione puo' terminare senza `return` quando `x <= 0` (`-Wreturn-type`). Il secondo e' grave: uscire da una funzione non-void senza `return` e' comportamento indefinito. Correzione: `int f(int x) { return x > 0 ? x : 0; }`. Conviene aggiungere `-Werror=return-type` alle opzioni di compilazione.

Soluzione 1.3

```
cmake_minimum_required(VERSION 3.20)
project(esempio LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_library(matematica src/matematica.cpp)
target_include_directories(matematica PUBLIC include)   # PUBLIC: vale anche per chi la usa
target_compile_options(matematica PRIVATE -Wall -Wextra)

add_executable(app src/main.cpp)
target_link_libraries(app PRIVATE matematica)           # eredita include/ automaticamente
```

Soluzione 1.4

```
g++ -E main.cpp -o main.ii   # preprocessore: include espansi, macro sostituite (testo C++)
g++ -S main.ii -o main.s     # compilazione: codice assembly della piattaforma
g++ -c main.s  -o main.o     # assemblaggio: file oggetto binario con simboli non risolti
g++ main.o -o main           # linking: risolve i simboli (std::cout...), crea l'eseguibile
```

Il file `.ii` di un semplice "Ciao, mondo" supera le 30.000 righe: e' il contenuto di `<iostream>` incluso testualmente, motivo per cui i moduli C++20 velocizzano la compilazione. Con `nm main.o` si vedono i simboli marcati `U` (undefined) che il linker deve risolvere.

Capitolo 2 - Tipi, variabili e operatori

Soluzione 2.1

```
#include <iomanip>
#include <iostream>

int main() {
    for (int c = -20; c <= 40; c += 10) {
        double f = c * 9.0 / 5.0 + 32;    // 9.0 forza la divisione in virgola mobile
        std::cout << std::setw(4) << c << " C = " << std::setw(5) << f << " F\n";
    }
}
```

```
 -20 C =    -4 F
 -10 C =    14 F
   0 C =    32 F
  10 C =    50 F
  20 C =    68 F
  30 C =    86 F
  40 C =   104 F
```

Soluzione 2.2

```
#include <iostream>

constexpr bool bisestile(int anno) {
    return (anno % 4 == 0 && anno % 100 != 0) || anno % 400 == 0;
}
static_assert(bisestile(2024) && bisestile(2000) && !bisestile(1900) && !bisestile(2023));

int main() { std::cout << std::boolalpha << bisestile(2100) << '\n'; }
```

```
false
```

Soluzione 2.3

`n - 1` spegne il bit a 1 meno significativo e accende tutti quelli a 0 alla sua destra; l'AND con `n` quindi azzera esattamente quel bit. Ogni iterazione elimina un bit acceso, percio' il ciclo gira `popcount(n)` volte invece di 32.

```
#include <bit>
#include <cstdint>
#include <iostream>

int contaBit(std::uint32_t n) {
    int c = 0;
    for (; n; n &= n - 1) ++c;
    return c;
}

int main() {
    for (std::uint32_t n : {0u, 11u, 255u, 0xFFFFFFFFu})
        std::cout << n << ": " << contaBit(n) << " (" << std::popcount(n) << ")\n";
}
```

```
0: 0 (0)
11: 3 (3)
255: 8 (8)
4294967295: 32 (32)
```

Soluzione 2.4

```
#include <iostream>
int main() {
    std::cout << (-1 < 0u) << ' ' << (7 / 2) << ' ' << (-7 / 2) << ' ' << (-7 % 3) << '\n';
}
```

```
0 3 -3 -1
```

- `-1 < 0u` e' falso: nel confronto con un `unsigned` il `-1` viene convertito in `4294967295`. Il compilatore avvisa con `-Wsign-compare`; in C++20 si usa `std::cmp_less(-1, 0u)`, che restituisce il risultato matematicamente corretto.
- `7 / 2` vale 3: la divisione tra interi tronca.
- `-7 / 2` vale -3: dal C++11 la divisione tronca **verso zero**.
- `-7 % 3` vale -1: il resto ha il segno del dividendo, cosi' che `(a / b) * b + a % b == a`.

Soluzione 2.5

Si scorre la stringa: se il simbolo corrente vale meno del successivo (come la `C` in `CM`) si sottrae, altrimenti si somma.

```
#include <iostream>
#include <string_view>

int valore(char c) {
    switch (c) {
        case 'I': return 1;   case 'V': return 5;   case 'X': return 10;
        case 'L': return 50;  case 'C': return 100; case 'D': return 500;
        case 'M': return 1000;
        default: return 0;
    }
}

int romanoInIntero(std::string_view s) {
    int tot = 0;
    for (std::size_t i = 0; i < s.size(); ++i) {
        int v = valore(s[i]);
        if (i + 1 < s.size() && v < valore(s[i + 1])) tot -= v;
        else tot += v;
    }
    return tot;
}

int main() {
    std::cout << romanoInIntero("MCMXCIV") << ' ' << romanoInIntero("LVIII") << '\n';
}
```

```
1994 58
```

Capitolo 3 - Controllo di flusso e funzioni

Soluzione 3.1

Dividere prima riduce il rischio di overflow: `a * b` potrebbe superare il limite del tipo anche quando il mcm ci sta. Dal C++17 esistono anche `std::gcd` e `std::lcm` in `<numeric>`.

```
#include <iostream>

long long mcd(long long a, long long b) {
    while (b != 0) { long long r = a % b; a = b; b = r; }
    return a < 0 ? -a : a;
}
long long mcm(long long a, long long b) { return a / mcd(a, b) * b; }

int main() { std::cout << mcd(84, 36) << ' ' << mcm(4, 6) << ' ' << mcm(21, 6) << '\n'; }
```

```
12 12 42
```

Soluzione 3.2

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> parole{"sole", "montagna", "re", "programmazione", "mare"};
    std::size_t k = 4;
    auto lunghe = std::count_if(parole.begin(), parole.end(),
                                [k](const std::string& p) { return p.size() > k; });
    std::cout << lunghe << '\n';
}
```

```
2
```

Soluzione 3.3

La versione con hash map memorizza, per ogni valore visto, il suo indice: per l'elemento corrente basta cercare il "complemento" `target - x`. Una sola passata: O(n) tempo, O(n) spazio.

```
#include <iostream>
#include <unordered_map>
#include <utility>
#include <vector>

std::pair<int, int> twoSumLento(const std::vector<int>& a, int target) {
    for (int i = 0; i < (int)a.size(); ++i)
        for (int j = i + 1; j < (int)a.size(); ++j)
            if (a[i] + a[j] == target) return {i, j};
    return {-1, -1};
}

std::pair<int, int> twoSum(const std::vector<int>& a, int target) {
    std::unordered_map<int, int> visti;            // valore -> indice
    for (int i = 0; i < (int)a.size(); ++i) {
        if (auto it = visti.find(target - a[i]); it != visti.end()) return {it->second, i};
        visti[a[i]] = i;
    }
    return {-1, -1};
}

int main() {
    std::vector<int> a{2, 7, 11, 15};
    auto [i, j] = twoSum(a, 18);
    auto [p, q] = twoSumLento(a, 18);
    std::cout << i << ' ' << j << " | " << p << ' ' << q << '\n';
}
```

```
1 2 | 1 2
```

Soluzione 3.4

Per spostare `n` dischi da A a C: sposta `n-1` dischi su B, sposta il disco grande su C, riporta gli `n-1` da B a C. Le mosse sono `2^n - 1`.

```
#include <iostream>

int mosse = 0;
void hanoi(int n, char da, char a, char via) {
    if (n == 0) return;
    hanoi(n - 1, da, via, a);
    std::cout << "disco " << n << ": " << da << " -> " << a << '\n';
    ++mosse;
    hanoi(n - 1, via, a, da);
}

int main() {
    hanoi(3, 'A', 'C', 'B');
    std::cout << "mosse: " << mosse << '\n';
}
```

```
disco 1: A -> C
disco 2: A -> B
disco 1: C -> B
disco 3: A -> C
disco 1: B -> A
disco 2: B -> C
disco 1: A -> C
mosse: 7
```

Soluzione 3.5

Esponenziazione per quadrati: `x^n = (x^2)^(n/2)`, moltiplicando per `x` quando il bit corrente di `n` e' 1. Per `n` negativo si usa `1/x` e `-n`; ma `-LLONG_MIN` va in overflow, quindi lavoriamo con un `unsigned long long`.

```
#include <climits>
#include <iostream>

double potenza(double x, long long n) {
    unsigned long long e = n < 0 ? 0ULL - static_cast<unsigned long long>(n) : n;
    if (n < 0) x = 1 / x;
    double r = 1;
    for (; e; e >>= 1, x *= x)
        if (e & 1) r *= x;
    return r;
}

int main() {
    std::cout << potenza(2, 10) << ' ' << potenza(2, -2) << ' '
              << potenza(1, LLONG_MIN) << '\n';
}
```

```
1024 0.25 1
```

Capitolo 4 - Puntatori, riferimenti e memoria

Soluzione 4.1

I riferimenti sono piu' sicuri: non possono essere nulli e non vanno dereferenziati esplicitamente; con i puntatori il chiamante potrebbe passare `nullptr`. In pratica si usa `std::swap`.

```
#include <iostream>

void scambia(int* a, int* b) { if (a && b) { int t = *a; *a = *b; *b = t; } }
void scambia(int& a, int& b) { int t = a; a = b; b = t; }

int main() {
    int x = 1, y = 2;
    scambia(&x, &y);  std::cout << x << y << ' ';
    scambia(x, y);    std::cout << x << y << '\n';
}
```

```
21 12
```

Soluzione 4.2

`s` e' una variabile locale distrutta all'uscita dalla funzione: il riferimento restituito e' **pendente** (dangling) e usarlo e' comportamento indefinito. GCC e Clang avvisano con `-Wreturn-local-addr` /

`-Wreturn-stack-address`. Correzione: restituire per valore, `std::string saluto() { return "ciao"; }`; grazie alla copy elision non costa nulla.

Soluzione 4.3

```
#include <iostream>
#include <utility>

template <typename F>
class ScopeGuard {
    F f_;
    bool attivo_ = true;
public:
    explicit ScopeGuard(F f) : f_(std::move(f)) {}
    ~ScopeGuard() { if (attivo_) f_(); }
    void annulla() noexcept { attivo_ = false; }
    ScopeGuard(const ScopeGuard&) = delete;
    ScopeGuard& operator=(const ScopeGuard&) = delete;
};

int livelloLog = 1;

void operazioneVerbosa(bool conferma) {
    int vecchio = livelloLog;
    livelloLog = 3;
    ScopeGuard g([&] { livelloLog = vecchio; });   // CTAD: deduce il tipo della lambda
    std::cout << "durante: " << livelloLog << '\n';
    if (conferma) g.annulla();                     // mantiene il nuovo valore
}

int main() {
    operazioneVerbosa(false);
    std::cout << "dopo: " << livelloLog << '\n';
    operazioneVerbosa(true);
    std::cout << "dopo: " << livelloLog << '\n';
}
```

```
durante: 3
dopo: 1
durante: 3
dopo: 3
```

Soluzione 4.4

Quando il puntatore veloce raggiunge la fine, quello lento e' a meta'. Con un numero pari di nodi restituisce il secondo dei due centrali, come richiesto da LeetCode.

```
#include <iostream>

struct Nodo { int val; Nodo* next; };

const Nodo* centro(const Nodo* testa) {
    const Nodo *lento = testa, *veloce = testa;
    while (veloce && veloce->next) { lento = lento->next; veloce = veloce->next->next; }
    return lento;
}

int main() {
    Nodo n6{6, nullptr}, n5{5, &n6}, n4{4, &n5}, n3{3, &n4}, n2{2, &n3}, n1{1, &n2};
    std::cout << centro(&n2)->val << ' ' << centro(&n1)->val << '\n';   // 5 nodi, 6 nodi
}
```

```
4 4
```

Soluzione 4.5

Con due `shared_ptr` che si puntano a vicenda il contatore di ciascuno non scende mai a zero: memory leak. Il figlio deve **osservare** il genitore con un `weak_ptr`, che non incrementa il contatore; `lock()` lo trasforma temporaneamente in `shared_ptr`.

```
#include <iostream>
#include <memory>

struct Genitore;
struct Figlio {
    std::weak_ptr<Genitore> genitore;        // con shared_ptr: ciclo e leak
    ~Figlio() { std::cout << "~Figlio\n"; }
};
struct Genitore {
    std::shared_ptr<Figlio> figlio;
    ~Genitore() { std::cout << "~Genitore\n"; }
};

int main() {
    auto g = std::make_shared<Genitore>();
    g->figlio = std::make_shared<Figlio>();
    g->figlio->genitore = g;
    if (auto p = g->figlio->genitore.lock()) std::cout << "uso: " << p.use_count() << '\n';
}
```

```
uso: 2
~Genitore
~Figlio
```

Capitolo 5 - Stringhe, I/O e file

Soluzione 5.1

```
#include <cctype>
#include <iostream>
#include <string_view>

bool palindroma(std::string_view s) {
    auto ok = [](char c) { return std::isalnum(static_cast<unsigned char>(c)) != 0; };
    auto low = [](char c) { return std::tolower(static_cast<unsigned char>(c)); };
    std::size_t i = 0, j = s.size();
    while (i < j) {
        if (!ok(s[i])) { ++i; continue; }
        if (!ok(s[j - 1])) { --j; continue; }
        if (low(s[i]) != low(s[j - 1])) return false;
        ++i; --j;
    }
    return true;
}

int main() {
    std::cout << palindroma("A man, a plan, a canal: Panama")
              << palindroma("race a car") << '\n';
}
```

```
10
```

Soluzione 5.2

Le `string_view` puntano ai caratteri della stringa originale: se questa viene distrutta (o riallocata modificandola) le viste diventano pendenti. Il chiamante deve garantire che la stringa sopravviva al risultato; per esempio `dividi(std::string("a,b"), ',')` restituisce viste su un temporaneo gia' distrutto.

```
#include <iostream>
#include <string_view>
#include <vector>

std::vector<std::string_view> dividi(std::string_view s, char sep) {
    std::vector<std::string_view> out;
    std::size_t inizio = 0;
    for (std::size_t i = 0; i <= s.size(); ++i)
        if (i == s.size() || s[i] == sep) {
            out.push_back(s.substr(inizio, i - inizio));
            inizio = i + 1;
        }
    return out;
}

int main() {
    for (auto p : dividi("rosso,,verde,blu", ',')) std::cout << '[' << p << ']';
    std::cout << '\n';
}
```

```
[rosso][][verde][blu]
```

Soluzione 5.3

```
#include <iostream>
#include <string>

std::string cifra(std::string s, int k) {
    k = ((k % 26) + 26) % 26;                  // normalizza anche k negativi
    for (char& c : s) {
        if (c >= 'a' && c <= 'z') c = static_cast<char>('a' + (c - 'a' + k) % 26);
        else if (c >= 'A' && c <= 'Z') c = static_cast<char>('A' + (c - 'A' + k) % 26);
    }
    return s;
}
std::string decifra(const std::string& s, int k) { return cifra(s, -k); }

int main() {
    auto c = cifra("Veni, vidi, vici!", 3);
    std::cout << c << " | " << decifra(c, 3) << '\n';
}
```

```
Yhql, ylgl, ylfl! | Veni, vidi, vici!
```

Soluzione 5.4

```
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>

int main() {
    { std::ofstream("testo.txt") << "uno due tre\nquattro\n\ncinque sei\n"; }
    std::ifstream in("testo.txt");
    std::size_t righe = 0, parole = 0, caratteri = 0;
    std::string riga, p;
    while (std::getline(in, riga)) {
        ++righe;
        caratteri += riga.size() + 1;          // +1 per il '\n'
        std::istringstream ss(riga);
        while (ss >> p) ++parole;              // >> salta gli spazi
    }
    std::cout << righe << ' ' << parole << ' ' << caratteri << '\n';
}
```

```
4 6 32
```

Soluzione 5.5

Ordinare ogni parola costa O(L log L); in totale O(n L log L) con n parole di lunghezza massima L. Si puo' scendere a O(n L) usando come chiave il conteggio delle 26 lettere.

```
#include <algorithm>
#include <iostream>
#include <map>
#include <string>
#include <vector>

int main() {
    std::vector<std::string> parole{"eat", "tea", "tan", "ate", "nat", "bat"};
    std::map<std::string, std::vector<std::string>> gruppi;
    for (const auto& p : parole) {
        std::string chiave = p;
        std::sort(chiave.begin(), chiave.end());
        gruppi[chiave].push_back(p);
    }
    for (const auto& [chiave, g] : gruppi) {
        for (const auto& p : g) std::cout << p << ' ';
        std::cout << '\n';
    }
}
```

```
bat
eat tea ate
tan nat
```

Soluzioni - Parte 2

Capitolo 6 - Programmazione orientata agli oggetti

Soluzione 6.1

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <utility>

class Conto {
public:
    explicit Conto(std::string titolare, double saldo = 0.0)
        : titolare_(std::move(titolare)), saldo_(saldo) {}
    void versa(double v) {
        if (v <= 0) throw std::invalid_argument("importo non positivo");
        saldo_ += v;
    }
    void preleva(double v) {
        if (v <= 0) throw std::invalid_argument("importo non positivo");
        if (v > saldo_) throw std::runtime_error("saldo insufficiente");
        saldo_ -= v;
    }
    double saldo() const noexcept { return saldo_; }
private:
    std::string titolare_;
    double saldo_;
};

int main() {
    Conto c("Anna", 100);
    for (double importo : {30.0, -5.0, 500.0}) {
        try {
            c.preleva(importo);
            std::cout << "prelevati " << importo << ", saldo " << c.saldo() << '\n';
        } catch (const std::invalid_argument& e) {
            std::cout << "argomento non valido: " << e.what() << '\n';
        } catch (const std::runtime_error& e) {
            std::cout << "operazione rifiutata: " << e.what() << '\n';
        }
    }
}
```

```
prelevati 30, saldo 70
argomento non valido: importo non positivo
operazione rifiutata: saldo insufficiente
```

L'ordine dei `catch` conta: vanno dal piu' specifico al piu' generale. Qui sono indipendenti perche' `invalid_argument` deriva da `logic_error`, non da `runtime_error`.

Soluzione 6.2

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

struct Animale {
    virtual ~Animale() = default;
    virtual std::string nome() const = 0;
    virtual std::string verso() const = 0;
};
struct Cane : Animale {
    std::string nome() const override { return "cane"; }
    std::string verso() const override { return "bau"; }
};
struct Gatto : Animale {
    std::string nome() const override { return "gatto"; }
    std::string verso() const override { return "miao"; }
};
struct Mucca : Animale {
    std::string nome() const override { return "mucca"; }
    std::string verso() const override { return "muu"; }
};

int main() {
    std::vector<std::unique_ptr<Animale>> fattoria;
    fattoria.push_back(std::make_unique<Cane>());
    fattoria.push_back(std::make_unique<Gatto>());
    fattoria.push_back(std::make_unique<Mucca>());
    for (const auto& a : fattoria) std::cout << a->nome() << ": " << a->verso() << '\n';
}
```

```
cane: bau
gatto: miao
mucca: muu
```

Soluzione 6.3

```
#include <iostream>
#include <utility>
#include <vector>

struct Traccia {
    Traccia() { std::cout << "ctor "; }
    ~Traccia() { std::cout << "dtor "; }
    Traccia(const Traccia&) { std::cout << "copia "; }
    Traccia& operator=(const Traccia&) { std::cout << "=copia "; return *this; }
    Traccia(Traccia&&) noexcept { std::cout << "move "; }
    Traccia& operator=(Traccia&&) noexcept { std::cout << "=move "; return *this; }
};

Traccia crea() { return Traccia{}; }   // copy elision garantita (C++17)

int main() {
    std::vector<Traccia> v;
    v.reserve(4);                          // niente riallocazioni: output piu' leggibile
    Traccia t;              std::cout << "| ";
    v.push_back(t);         std::cout << "| ";   // copia dell'lvalue
    v.push_back(std::move(t)); std::cout << "| "; // move
    v.emplace_back();       std::cout << "| ";   // costruito direttamente nel vettore
    Traccia r = crea();     std::cout << "| ";   // nessuna copia ne' move
    r = Traccia{};          std::cout << "| ";   // ctor del temporaneo, =move, dtor
    std::cout << '\n';
}
```

```
ctor | copia | move | ctor | ctor | ctor =move dtor |
dtor dtor dtor dtor dtor
```

L'ultima riga contiene i cinque `dtor` di fine `main`: `r`, `t` e i tre elementi del vettore. Togliendo `noexcept` dal costruttore di move e rimuovendo `reserve`, le riallocazioni del vettore userebbero la **copia**: `std::vector` sposta gli elementi solo se il move non puo' lanciare eccezioni.

Soluzione 6.4

`in` riceve i `push`; quando `out` e' vuota vi si travasano tutti gli elementi di `in`, invertendone l'ordine. Ogni elemento viene spostato al massimo una volta da `in` a `out`, quindi n operazioni costano O(n) in totale: O(1) ammortizzato ciascuna.

```
#include <iostream>
#include <stack>

class Coda {
    std::stack<int> in_, out_;
    void travasa() {
        if (out_.empty())
            while (!in_.empty()) { out_.push(in_.top()); in_.pop(); }
    }
public:
    void push(int x) { in_.push(x); }
    int pop() { travasa(); int x = out_.top(); out_.pop(); return x; }
    int peek() { travasa(); return out_.top(); }
    bool empty() const { return in_.empty() && out_.empty(); }
};

int main() {
    Coda q;
    q.push(1); q.push(2); q.push(3);
    std::cout << q.pop() << ' ';
    q.push(4);
    while (!q.empty()) std::cout << q.pop() << ' ';
    std::cout << '\n';
}
```

```
1 2 3 4
```

Soluzione 6.5

La lista mantiene l'ordine d'uso (in testa il piu' recente); la mappa associa a ogni chiave l'iteratore del suo nodo nella lista. `splice` sposta un nodo in testa in O(1) senza invalidare gli iteratori.

```
#include <iostream>
#include <list>
#include <unordered_map>
#include <utility>

class LRUCache {
    std::size_t cap_;
    std::list<std::pair<int, int>> lista_;                          // (chiave, valore)
    std::unordered_map<int, std::list<std::pair<int, int>>::iterator> pos_;
public:
    explicit LRUCache(std::size_t cap) : cap_(cap) {}
    int get(int k) {
        auto it = pos_.find(k);
        if (it == pos_.end()) return -1;
        lista_.splice(lista_.begin(), lista_, it->second);         // diventa il piu' recente
        return it->second->second;
    }
    void put(int k, int v) {
        if (auto it = pos_.find(k); it != pos_.end()) {
            it->second->second = v;
            lista_.splice(lista_.begin(), lista_, it->second);
            return;
        }
        if (lista_.size() == cap_) {                               // espelle il meno recente
            pos_.erase(lista_.back().first);
            lista_.pop_back();
        }
        lista_.emplace_front(k, v);
        pos_[k] = lista_.begin();
    }
};

int main() {
    LRUCache c(2);
    c.put(1, 1); c.put(2, 2);
    std::cout << c.get(1) << ' ';   // 1, ora 2 e' il meno recente
    c.put(3, 3);                    // espelle 2
    std::cout << c.get(2) << ' ';
    c.put(4, 4);                    // espelle 1
    std::cout << c.get(1) << ' ' << c.get(3) << ' ' << c.get(4) << '\n';
}
```

```
1 -1 -1 3 4
```

Capitolo 7 - Template e programmazione generica

Soluzione 7.1

```
#include <iostream>
#include <string>

template <typename T, typename U>
struct Coppia {
    T primo;
    U secondo;
    Coppia<U, T> scambiata() const { return {secondo, primo}; }
};

int main() {
    Coppia c{1, 2.5};                    // CTAD per aggregati (C++20): Coppia<int, double>
    auto s = c.scambiata();              // Coppia<double, int>
    Coppia n{std::string("Anna"), 30};
    std::cout << s.primo << ' ' << s.secondo << ' ' << n.primo << '\n';
}
```

```
2.5 1 Anna
```

Soluzione 7.2

```
#include <concepts>
#include <iostream>
#include <span>
#include <vector>

template <typename T>
concept Numerico = (std::integral<T> || std::floating_point<T>) && !std::same_as<T, bool>;

template <Numerico T>
double media(std::span<const T> v) {
    if (v.empty()) return 0.0;
    double s = 0;
    for (T x : v) s += x;
    return s / v.size();
}

int main() {
    std::vector<int> a{1, 2, 3, 4};
    double b[] = {2.5, 3.5};
    std::cout << media<int>(a) << ' ' << media<double>(b) << '\n';
    // media<bool>(...)  -> errore: bool non soddisfa Numerico
}
```

```
2.5 3
```

L'argomento template va indicato esplicitamente (`media<int>`) perche' la deduzione non considera la conversione da `vector` a `span`.

Soluzione 7.3

```
#include <array>
#include <cstddef>
#include <iostream>
#include <optional>

template <typename T, std::size_t N>
class Pila {
    static_assert(N > 0, "la capacita' deve essere positiva");
    std::array<T, N> d_{};
    std::size_t n_ = 0;
public:
    bool push(const T& x) {
        if (n_ == N) return false;
        d_[n_++] = x;
        return true;
    }
    std::optional<T> pop() {
        if (n_ == 0) return std::nullopt;
        return d_[--n_];
    }
    std::size_t size() const { return n_; }
};

int main() {
    Pila<int, 2> p;
    std::cout << p.push(1) << p.push(2) << p.push(3) << ' ';
    while (auto x = p.pop()) std::cout << *x;
    std::cout << '\n';
    // Pila<int, 0> q;   // errore di compilazione con il messaggio dello static_assert
}
```

```
110 21
```

Soluzione 7.4

Si mantiene un min-heap con i k elementi piu' grandi visti finora: la cima e' il k-esimo piu' grande. Ogni `aggiungi` costa O(log k).

```
#include <functional>
#include <iostream>
#include <queue>
#include <vector>

template <typename T, typename Cmp = std::less<T>>
class KthLargest {
    std::size_t k_;
    // con Cmp = less vogliamo un min-heap: invertiamo gli argomenti del comparatore
    struct Inverso { Cmp c; bool operator()(const T& a, const T& b) const { return c(b, a); } };
    std::priority_queue<T, std::vector<T>, Inverso> heap_;
public:
    KthLargest(std::size_t k, const std::vector<T>& iniziali) : k_(k) {
        for (const T& x : iniziali) aggiungi(x);
    }
    const T& aggiungi(const T& x) {
        heap_.push(x);
        if (heap_.size() > k_) heap_.pop();
        return heap_.top();
    }
};

int main() {
    KthLargest<int> k3(3, {4, 5, 8, 2});
    for (int x : {3, 5, 10, 9, 4}) std::cout << k3.aggiungi(x) << ' ';
    std::cout << '\n';
}
```

```
4 5 5 8 8
```

Soluzione 7.5

```
#include <array>
#include <iostream>

consteval std::array<long long, 20> tabellaFib() {
    std::array<long long, 20> t{};
    t[1] = 1;
    for (std::size_t i = 2; i < t.size(); ++i) t[i] = t[i - 1] + t[i - 2];
    return t;
}

constexpr auto FIB = tabellaFib();     // calcolata dal compilatore
static_assert(FIB[19] == 4181);

int main() {
    for (auto x : FIB) std::cout << x << ' ';
    std::cout << '\n';
}
```

```
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
```

`consteval` obbliga la valutazione a compile time: se la funzione contenesse un'operazione non ammessa (per esempio un accesso fuori limiti) si otterrebbe un errore di compilazione invece di un bug a runtime.

Capitolo 8 - Il C++ moderno

Soluzione 8.1

```
#include <iostream>
#include <map>
#include <string>
#include <vector>

int main() {
    const std::map<std::string, std::vector<int>> m{{"a", {1, 2}}, {"b", {3}}};
    for (const auto& [nome, valori] : m)          // range-for + structured bindings
        std::cout << nome << valori.size();       // nessuna allocazione: new/delete inutili
    std::cout << '\n';
}
```

```
a2b1
```

Soluzione 8.2

```
#include <iostream>
#include <ranges>

int main() {
    namespace vw = std::views;
    for (int q : vw::iota(11)                                    // sequenza infinita e pigra
                 | vw::filter([](int x) { return x % 2 == 1; })
                 | vw::transform([](int x) { return x * x; })
                 | vw::take(5))
        std::cout << q << ' ';
    std::cout << '\n';
}
```

```
121 169 225 289 361
```

Soluzione 8.3

```
// compila: -std=c++23
#include <charconv>
#include <expected>
#include <iostream>
#include <string>
#include <string_view>

std::expected<int, std::string> parseInt(std::string_view s) {
    int v = 0;
    auto [p, e] = std::from_chars(s.data(), s.data() + s.size(), v);
    if (e == std::errc::result_out_of_range) return std::unexpected("fuori intervallo");
    if (e != std::errc{} || p != s.data() + s.size())
        return std::unexpected("'" + std::string(s) + "' non e' un numero");
    return v;
}

int main() {
    for (auto s : {"42", "12abc", "99999999999"}) {
        if (auto r = parseInt(s)) std::cout << "ok " << *r << '\n';
        else std::cout << "errore: " << r.error() << '\n';
    }
}
```

```
ok 42
errore: '12abc' non e' un numero
errore: fuori intervallo
```

A differenza di `std::optional`, `std::expected` trasporta il **motivo** dell'errore; a differenza delle eccezioni, l'errore fa parte del tipo di ritorno e il chiamante non puo' ignorarlo per sbaglio.

Soluzione 8.4

```
#include <algorithm>
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> magazzino{{"viti", 120}, {"bulloni", 340}, {"dadi", 95}};
    auto it = std::ranges::max_element(magazzino, {}, [](const auto& kv) { return kv.second; });
    const auto& [prodotto, quantita] = *it;
    std::cout << prodotto << ' ' << quantita << '\n';
}
```

```
bulloni 340
```

Soluzione 8.5

Con `std::variant` l'insieme dei tipi e' chiuso e noto a compile time: niente allocazioni sull'heap, niente puntatori, e il compilatore segnala se una visita dimentica un tipo. Aggiungere una nuova **operazione** e' facile (un nuovo visitatore), aggiungere un nuovo **tipo** obbliga a modificare tutte le visite. Con l'ereditarieta' vale l'opposto: nuovi tipi si aggiungono senza toccare il codice esistente, nuove operazioni richiedono un nuovo metodo virtuale in tutta la gerarchia.

```
#include <cmath>
#include <iostream>
#include <numbers>
#include <variant>
#include <vector>

struct Cerchio { double r; };
struct Rettangolo { double w, h; };
struct Triangolo { double a, b, c; };
using Forma = std::variant<Cerchio, Rettangolo, Triangolo>;

double area(const Forma& f) {
    return std::visit([](const auto& x) -> double {
        using T = std::decay_t<decltype(x)>;
        if constexpr (std::is_same_v<T, Cerchio>) return std::numbers::pi * x.r * x.r;
        else if constexpr (std::is_same_v<T, Rettangolo>) return x.w * x.h;
        else {                                              // formula di Erone
            double s = (x.a + x.b + x.c) / 2;
            return std::sqrt(s * (s - x.a) * (s - x.b) * (s - x.c));
        }
    }, f);
}

int main() {
    std::vector<Forma> forme{Cerchio{1}, Rettangolo{3, 4}, Triangolo{3, 4, 5}};
    for (const auto& f : forme) std::cout << area(f) << ' ';
    std::cout << '\n';
}
```

```
3.14159 12 6
```

Soluzioni - Parte 3

Capitolo 9 - La Standard Template Library

Soluzione 9.1

```
#include <algorithm>
#include <iostream>
#include <vector>

void spostaZeri(std::vector<int>& a) {
    std::size_t w = 0;                                  // prossima posizione da scrivere
    for (int x : a) if (x != 0) a[w++] = x;
    std::fill(a.begin() + w, a.end(), 0);
}

int main() {
    std::vector<int> a{0, 1, 0, 3, 12}, b = a;
    spostaZeri(a);
    std::stable_partition(b.begin(), b.end(), [](int x) { return x != 0; });   // versione STL
    for (int x : a) std::cout << x << ' ';
    std::cout << "| ";
    for (int x : b) std::cout << x << ' ';
    std::cout << '\n';
}
```

```
1 3 12 0 0 | 1 3 12 0 0
```

`std::stable_partition` mantiene l'ordine relativo; `std::partition` sarebbe piu' veloce ma non lo garantisce.

Soluzione 9.2

```
#include <iostream>
#include <unordered_map>
#include <vector>

bool vicini(const std::vector<int>& a, int k) {
    std::unordered_map<int, int> ultimo;                // valore -> ultimo indice visto
    for (int i = 0; i < (int)a.size(); ++i) {
        if (auto it = ultimo.find(a[i]); it != ultimo.end() && i - it->second <= k) return true;
        ultimo[a[i]] = i;
    }
    return false;
}

int main() {
    std::cout << vicini({1, 2, 3, 1}, 3) << vicini({1, 2, 3, 1, 2, 3}, 2) << '\n';
}
```

```
10
```

Soluzione 9.3

La finestra `[inizio, i]` non contiene ripetizioni. `ultimo[c]` ricorda dove abbiamo visto `c` l'ultima volta: se e' dentro la finestra, l'inizio salta subito dopo. Ogni carattere entra ed esce una volta: O(n).

```
#include <algorithm>
#include <array>
#include <iostream>
#include <string_view>

int piuLunga(std::string_view s) {
    std::array<int, 256> ultimo;
    ultimo.fill(-1);
    int inizio = 0, best = 0;
    for (int i = 0; i < (int)s.size(); ++i) {
        auto c = static_cast<unsigned char>(s[i]);
        if (ultimo[c] >= inizio) inizio = ultimo[c] + 1;
        ultimo[c] = i;
        best = std::max(best, i - inizio + 1);
    }
    return best;
}

int main() {
    std::cout << piuLunga("abcabcbb") << piuLunga("bbbbb") << piuLunga("pwwkew") << '\n';
}
```

```
313
```

Soluzione 9.4

`unordered_set` richiede `operator==` e una funzione di hash; `std::set` richiede invece un ordinamento stretto (`operator<` o `<=>`). L'hash combina i due campi: moltiplicare per un numero primo evita che `(1,2)` e `(2,1)` collidano sempre.

```
#include <compare>
#include <functional>
#include <iostream>
#include <set>
#include <unordered_set>
#include <vector>

struct Punto {
    int x, y;
    bool operator==(const Punto&) const = default;
    auto operator<=>(const Punto&) const = default;          // serve a std::set
};
template <> struct std::hash<Punto> {
    std::size_t operator()(const Punto& p) const noexcept {
        return std::hash<int>{}(p.x) * 31 + std::hash<int>{}(p.y);
    }
};

int main() {
    std::vector<Punto> v{{1, 2}, {2, 1}, {1, 2}, {0, 0}, {2, 1}};
    std::unordered_set<Punto> u(v.begin(), v.end());
    std::set<Punto> s(v.begin(), v.end());
    std::cout << u.size() << ' ' << s.size() << " primo: " << s.begin()->x << s.begin()->y << '\n';
}
```

```
3 3 primo: 00
```

Soluzione 9.5

`out[i]` deve valere (prodotto a sinistra di i) * (prodotto a destra di i). Prima passata: riempiamo `out` con i prodotti a sinistra; seconda passata, da destra, moltiplichiamo per un accumulatore dei prodotti a destra.

```
#include <iostream>
#include <vector>

std::vector<int> prodottoTranne(const std::vector<int>& a) {
    std::vector<int> out(a.size(), 1);
    for (std::size_t i = 1; i < a.size(); ++i) out[i] = out[i - 1] * a[i - 1];
    int destra = 1;
    for (std::size_t i = a.size(); i-- > 0;) { out[i] *= destra; destra *= a[i]; }
    return out;
}

int main() {
    for (int x : prodottoTranne({1, 2, 3, 4})) std::cout << x << ' ';
    std::cout << '\n';
}
```

```
24 12 8 6
```

Capitolo 10 - Strutture dati classiche

Soluzione 10.1

Con due indici e un contatore si distinguono i casi "piena" e "vuota" (che altrimenti avrebbero `testa == coda` in entrambi i casi).

```
#include <iostream>
#include <optional>
#include <vector>

template <typename T>
class CodaCircolare {
    std::vector<T> d_;
    std::size_t testa_ = 0, n_ = 0;
public:
    explicit CodaCircolare(std::size_t cap) : d_(cap) {}
    bool vuota() const { return n_ == 0; }
    bool piena() const { return n_ == d_.size(); }
    bool push(const T& x) {
        if (piena()) return false;
        d_[(testa_ + n_) % d_.size()] = x;
        ++n_;
        return true;
    }
    std::optional<T> pop() {
        if (vuota()) return std::nullopt;
        T x = d_[testa_];
        testa_ = (testa_ + 1) % d_.size();
        --n_;
        return x;
    }
    const T& front() const { return d_[testa_]; }
};

int main() {
    CodaCircolare<int> q(3);
    std::cout << q.push(1) << q.push(2) << q.push(3) << q.push(4) << ' ';
    std::cout << *q.pop() << ' ';
    q.push(5);                                      // riusa la cella liberata
    while (auto x = q.pop()) std::cout << *x << ' ';
    std::cout << '\n';
}
```

```
1110 1 2 3 5
```

Soluzione 10.2

```
#include <array>
#include <iostream>
#include <memory>
#include <string_view>

class Trie {
    struct Nodo {
        std::array<std::unique_ptr<Nodo>, 26> figli;
        bool fine = false;
    };
    Nodo radice_;
    const Nodo* trova(std::string_view s) const {
        const Nodo* n = &radice_;
        for (char c : s) {
            n = n->figli[c - 'a'].get();
            if (!n) return nullptr;
        }
        return n;
    }
public:
    void inserisci(std::string_view s) {
        Nodo* n = &radice_;
        for (char c : s) {
            auto& f = n->figli[c - 'a'];
            if (!f) f = std::make_unique<Nodo>();
            n = f.get();
        }
        n->fine = true;
    }
    bool cerca(std::string_view s) const { auto n = trova(s); return n && n->fine; }
    bool iniziaCon(std::string_view p) const { return trova(p) != nullptr; }
};

int main() {
    Trie t;
    t.inserisci("casa"); t.inserisci("caso"); t.inserisci("cane");
    std::cout << t.cerca("casa") << t.cerca("cas") << t.iniziaCon("cas") << t.iniziaCon("co")
              << '\n';
}
```

```
1010
```

Soluzione 10.3

```
#include <iostream>
#include <numeric>
#include <utility>
#include <vector>

struct DSU {
    std::vector<int> p, r;
    explicit DSU(int n) : p(n), r(n, 0) { std::iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unisci(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (r[a] < r[b]) std::swap(a, b);
        p[b] = a;
        if (r[a] == r[b]) ++r[a];
        return true;
    }
};

int province(const std::vector<std::vector<int>>& m) {
    int n = m.size(), componenti = n;
    DSU d(n);
    for (int i = 0; i < n; ++i)
        for (int j = i + 1; j < n; ++j)
            if (m[i][j] && d.unisci(i, j)) --componenti;   // ogni unione riuscita ne toglie una
    return componenti;
}

int main() {
    std::cout << province({{1, 1, 0}, {1, 1, 0}, {0, 0, 1}}) << ' '
              << province({{1, 0, 0}, {0, 1, 0}, {0, 0, 1}}) << '\n';
}
```

```
2 3
```

Soluzione 10.4

La pila contiene gli indici dei giorni che aspettano ancora una temperatura piu' alta, con temperature decrescenti dal fondo alla cima. Quando arriva un giorno piu' caldo, risolve tutti quelli in cima piu' freddi. Ogni indice entra ed esce una volta: O(n).

```
#include <iostream>
#include <stack>
#include <vector>

std::vector<int> attese(const std::vector<int>& t) {
    std::vector<int> out(t.size(), 0);
    std::stack<int> s;
    for (int i = 0; i < (int)t.size(); ++i) {
        while (!s.empty() && t[s.top()] < t[i]) { out[s.top()] = i - s.top(); s.pop(); }
        s.push(i);
    }
    return out;
}

int main() {
    for (int x : attese({73, 74, 75, 71, 69, 72, 76, 73})) std::cout << x << ' ';
    std::cout << '\n';
}
```

```
1 1 4 2 1 1 0 0
```

Soluzione 10.5

Il vincolo del BST riguarda **tutto** il sottoalbero: nell'albero `5 -> (sx 4, dx 6 -> (sx 3))` ogni nodo rispetta i propri figli, ma 3 sta nel sottoalbero destro di 5. Si passa quindi lungo la ricorsione l'intervallo `(min, max)` ammesso.

```
#include <iostream>
#include <optional>

struct Nodo { int val; Nodo *sx = nullptr, *dx = nullptr; };

bool valido(const Nodo* n, std::optional<int> lo = {}, std::optional<int> hi = {}) {
    if (!n) return true;
    if ((lo && n->val <= *lo) || (hi && n->val >= *hi)) return false;
    return valido(n->sx, lo, n->val) && valido(n->dx, n->val, hi);
}

int main() {
    Nodo tre{3}, sei{6, &tre}, quattro{4}, cinque{5, &quattro, &sei};
    std::cout << valido(&cinque) << ' ';
    sei.sx = nullptr;
    std::cout << valido(&cinque) << '\n';
}
```

```
0 1
```

Usare `std::optional` invece di `INT_MIN`/`INT_MAX` evita errori quando l'albero contiene proprio quei valori.

Soluzione 10.6

```
#include <iostream>
#include <queue>
#include <string>
#include <utility>
#include <vector>

int isole(std::vector<std::string> g) {
    int righe = g.size(), col = g[0].size(), n = 0;
    const int dr[] = {1, -1, 0, 0}, dc[] = {0, 0, 1, -1};
    for (int r = 0; r < righe; ++r)
        for (int c = 0; c < col; ++c) {
            if (g[r][c] != '1') continue;
            ++n;
            std::queue<std::pair<int, int>> q;
            q.push({r, c}); g[r][c] = '0';            // marca subito come visitata
            while (!q.empty()) {
                auto [x, y] = q.front(); q.pop();
                for (int d = 0; d < 4; ++d) {
                    int nx = x + dr[d], ny = y + dc[d];
                    if (nx >= 0 && ny >= 0 && nx < righe && ny < col && g[nx][ny] == '1') {
                        g[nx][ny] = '0';
                        q.push({nx, ny});
                    }
                }
            }
        }
    return n;
}

int main() {
    std::cout << isole({"11000", "11000", "00100", "00011"}) << '\n';
}
```

```
3
```

Soluzioni - Parte 4

Capitolo 11 - Analisi e tecniche di base

Soluzione 11.1

```
#include <algorithm>
#include <iostream>
#include <random>
#include <vector>

std::size_t lowerBound(const std::vector<int>& a, int x) {
    std::size_t lo = 0, hi = a.size();            // risposta in [lo, hi]
    while (lo < hi) {
        std::size_t mid = lo + (hi - lo) / 2;
        if (a[mid] < x) lo = mid + 1; else hi = mid;
    }
    return lo;
}

int main() {
    std::mt19937 rng(1);
    std::uniform_int_distribution<int> val(0, 20), len(0, 30);
    int errori = 0;
    for (int t = 0; t < 10000; ++t) {
        std::vector<int> a(len(rng));
        for (int& x : a) x = val(rng);
        std::sort(a.begin(), a.end());
        int x = val(rng);
        if (lowerBound(a, x) != std::size_t(std::lower_bound(a.begin(), a.end(), x) - a.begin()))
            ++errori;
    }
    std::cout << "errori: " << errori << '\n';
}
```

```
errori: 0
```

Soluzione 11.2

Se `a[i] + a[j] < target`, nessuna coppia `(i, k)` con `k <= j` puo' funzionare (gli `a[k]` sono ancora piu' piccoli): l'indice `i` si puo' scartare per sempre. Il ragionamento simmetrico vale per `j`. Ogni passo elimina un candidato: O(n).

```
#include <iostream>
#include <utility>
#include <vector>

std::pair<int, int> twoSumOrdinato(const std::vector<int>& a, int target) {
    int i = 0, j = (int)a.size() - 1;
    while (i < j) {
        int s = a[i] + a[j];
        if (s == target) return {i + 1, j + 1};        // LeetCode usa indici da 1
        if (s < target) ++i; else --j;
    }
    return {-1, -1};
}

int main() {
    auto [i, j] = twoSumOrdinato({2, 7, 11, 15}, 9);
    auto [p, q] = twoSumOrdinato({-1, 0, 3, 5, 8}, 8);
    std::cout << i << ' ' << j << " | " << p << ' ' << q << '\n';
}
```

```
1 2 | 2 5
```

Soluzione 11.3

L'area e' limitata dalla parete piu' bassa. Spostare quella piu' alta non puo' migliorare (la larghezza cala e l'altezza resta limitata dalla bassa), quindi si sposta sempre la piu' bassa.

```
#include <algorithm>
#include <iostream>
#include <vector>

int massimaAcqua(const std::vector<int>& h) {
    int i = 0, j = (int)h.size() - 1, best = 0;
    while (i < j) {
        best = std::max(best, (j - i) * std::min(h[i], h[j]));
        if (h[i] < h[j]) ++i; else --j;
    }
    return best;
}

int main() { std::cout << massimaAcqua({1, 8, 6, 2, 5, 4, 8, 3, 7}) << '\n'; }
```

```
49
```

Soluzione 11.4

```
#include <iostream>
#include <numeric>
#include <vector>

int pivot(const std::vector<int>& a) {
    long long totale = std::accumulate(a.begin(), a.end(), 0LL), sinistra = 0;
    for (int i = 0; i < (int)a.size(); ++i) {
        if (sinistra == totale - sinistra - a[i]) return i;
        sinistra += a[i];
    }
    return -1;
}

int main() { std::cout << pivot({1, 7, 3, 6, 5, 6}) << ' ' << pivot({1, 2, 3}) << '\n'; }
```

```
3 -1
```

Soluzione 11.5

- (a) Il corpo viene eseguito 0 + 1 + ... + (n-1) = n(n-1)/2 volte: **O(n^2)**.
- (b) `i` raddoppia: 1, 2, 4, ... fino a n, cioe' circa log2(n) iterazioni: **O(log n)**.
- (c) n iterazioni da O(log n) ciascuna: **O(n log n)**.
- (d) i sottoinsiemi sono 2^n e scriverne uno costa fino a O(n): **O(n 2^n)**. Con n = 20 sono circa 20 milioni di operazioni (ok), con n = 40 mille miliardi (troppe).

Soluzione 11.6

```
#include <array>
#include <iostream>
#include <string>
#include <vector>

std::vector<int> anagrammi(const std::string& s, const std::string& p) {
    std::vector<int> out;
    if (s.size() < p.size()) return out;
    std::array<int, 26> cp{}, cw{};
    for (char c : p) ++cp[c - 'a'];
    for (std::size_t i = 0; i < s.size(); ++i) {
        ++cw[s[i] - 'a'];                                  // entra a destra
        if (i >= p.size()) --cw[s[i - p.size()] - 'a'];     // esce a sinistra
        if (i + 1 >= p.size() && cw == cp) out.push_back(i + 1 - p.size());
    }
    return out;
}

int main() {
    for (int i : anagrammi("cbaebabacd", "abc")) std::cout << i << ' ';
    std::cout << "| ";
    for (int i : anagrammi("abab", "ab")) std::cout << i << ' ';
    std::cout << '\n';
}
```

```
0 6 | 0 1 2
```

Il confronto `cw == cp` costa 26 operazioni, una costante: il totale resta O(|s|).

Capitolo 12 - Ordinamento

Soluzione 12.1

Insertion sort fa zero spostamenti su un vettore ordinato (O(n)), circa n^2/4 su uno casuale e n(n-1)/2 su uno al contrario (O(n^2)). Per questo e' ottimo su dati quasi ordinati ed e' usato da introsort per i sottovettori piccoli.

```
#include <algorithm>
#include <iostream>
#include <numeric>
#include <random>
#include <vector>

long long insertionSort(std::vector<int>& a) {
    long long spostamenti = 0;
    for (std::size_t i = 1; i < a.size(); ++i) {
        int x = a[i];
        std::size_t j = i;
        while (j > 0 && a[j - 1] > x) { a[j] = a[j - 1]; --j; ++spostamenti; }
        a[j] = x;
    }
    return spostamenti;
}

int main() {
    std::vector<int> ord(1000);
    std::iota(ord.begin(), ord.end(), 0);
    auto inv = ord;  std::reverse(inv.begin(), inv.end());
    auto cas = ord;  std::shuffle(cas.begin(), cas.end(), std::mt19937(7));
    std::cout << insertionSort(ord) << ' ' << insertionSort(inv) << ' '
              << (insertionSort(cas) > 200000 ? "~n^2/4" : "?") << '\n';
}
```

```
0 499500 ~n^2/4
```

Soluzione 12.2

Durante la fusione, quando si prende un elemento dalla meta' destra, esso e' minore di tutti quelli rimasti nella meta' sinistra: ciascuno forma con lui un'inversione.

```
#include <iostream>
#include <vector>

long long ordinaEConta(std::vector<int>& a, std::vector<int>& tmp, int l, int r) {   // [l, r)
    if (r - l < 2) return 0;
    int m = (l + r) / 2;
    long long inv = ordinaEConta(a, tmp, l, m) + ordinaEConta(a, tmp, m, r);
    int i = l, j = m, k = l;
    while (i < m || j < r) {
        if (j == r || (i < m && a[i] <= a[j])) tmp[k++] = a[i++];
        else { inv += m - i; tmp[k++] = a[j++]; }
    }
    for (int x = l; x < r; ++x) a[x] = tmp[x];
    return inv;
}

int main() {
    std::vector<int> a{8, 4, 2, 1}, b{3, 1, 2}, t(4);
    std::cout << ordinaEConta(a, t, 0, 4) << ' ' << ordinaEConta(b, t, 0, 3) << '\n';
}
```

```
6 2
```

Soluzione 12.3

Tre zone: `[0, basso)` contiene 0, `[basso, i)` contiene 1, `(alto, fine]` contiene 2; `[i, alto]` e' ancora da esaminare. Quando si scambia con `alto` non si avanza `i`, perche' l'elemento arrivato non e' ancora stato esaminato.

```
#include <iostream>
#include <utility>
#include <vector>

void ordinaColori(std::vector<int>& a) {
    int basso = 0, i = 0, alto = (int)a.size() - 1;
    while (i <= alto) {
        if (a[i] == 0) std::swap(a[i++], a[basso++]);
        else if (a[i] == 2) std::swap(a[i], a[alto--]);
        else ++i;
    }
}

int main() {
    std::vector<int> a{2, 0, 2, 1, 1, 0};
    ordinaColori(a);
    for (int x : a) std::cout << x;
    std::cout << '\n';
}
```

```
001122
```

Soluzione 12.4

Il candidato guadagna un voto quando ritorna e ne perde uno per ogni elemento diverso; se arriva a zero si cambia candidato. L'elemento di maggioranza (piu' di n/2 occorrenze) non puo' essere "annullato" del tutto, quindi resta come candidato finale.

```
#include <iostream>
#include <vector>

int maggioranza(const std::vector<int>& a) {
    int candidato = 0, voti = 0;
    for (int x : a) {
        if (voti == 0) candidato = x;
        voti += x == candidato ? 1 : -1;
    }
    return candidato;   // se la maggioranza non e' garantita va verificato con un conteggio
}

int main() { std::cout << maggioranza({2, 2, 1, 1, 1, 2, 2}) << maggioranza({3, 3, 4}) << '\n'; }
```

```
23
```

Soluzione 12.5

```
#include <algorithm>
#include <cstdint>
#include <iostream>
#include <random>
#include <vector>

void radixSort(std::vector<std::uint32_t>& a) {
    std::vector<std::uint32_t> tmp(a.size());
    for (int shift = 0; shift < 32; shift += 8) {        // 4 passate, un byte alla volta
        std::size_t conta[257] = {};
        for (auto x : a) ++conta[((x >> shift) & 0xFF) + 1];
        for (int i = 0; i < 256; ++i) conta[i + 1] += conta[i];   // posizioni di partenza
        for (auto x : a) tmp[conta[(x >> shift) & 0xFF]++] = x;   // stabile
        a.swap(tmp);
    }
}

int main() {
    std::mt19937 rng(3);
    std::vector<std::uint32_t> a(1'000'000);
    for (auto& x : a) x = rng();
    auto b = a;
    radixSort(a);
    std::sort(b.begin(), b.end());
    std::cout << (a == b ? "ordinamento corretto" : "ERRORE") << '\n';
}
```

```
ordinamento corretto
```

Misurando con `std::chrono` (compilato con `-O2`, senza sanitizer) su 10 milioni di elementi il radix sort e' tipicamente 3-5 volte piu' veloce di `std::sort`: fa 4 passate lineari invece di circa n log n confronti, al prezzo di un vettore temporaneo grande quanto l'input.

Capitolo 13 - Ricorsione, backtracking, greedy

Soluzione 13.1

```
#include <algorithm>
#include <iostream>
#include <vector>

void permuta(std::vector<int>& a, std::size_t k, std::vector<std::vector<int>>& out) {
    if (k == a.size()) { out.push_back(a); return; }
    for (std::size_t i = k; i < a.size(); ++i) {
        std::swap(a[k], a[i]);          // fissa a[i] in posizione k
        permuta(a, k + 1, out);
        std::swap(a[k], a[i]);          // ripristina
    }
}

int main() {
    std::vector<int> a{1, 2, 3};
    std::vector<std::vector<int>> out, attese;
    permuta(a, 0, out);
    do attese.push_back(a); while (std::next_permutation(a.begin(), a.end()));
    std::sort(out.begin(), out.end());
    std::cout << out.size() << ' ' << (out == attese ? "uguali" : "diverse") << '\n';
}
```

```
6 uguali
```

Soluzione 13.2

Per non generare `[2,3]` e `[3,2]` si impone un ordine: ogni chiamata puo' usare solo candidati dall'indice corrente in poi. Ordinare i candidati permette anche di interrompere il ciclo appena si supera il target.

```
#include <algorithm>
#include <iostream>
#include <vector>

void combina(const std::vector<int>& c, std::size_t da, int resto, std::vector<int>& cur,
             std::vector<std::vector<int>>& out) {
    if (resto == 0) { out.push_back(cur); return; }
    for (std::size_t i = da; i < c.size() && c[i] <= resto; ++i) {
        cur.push_back(c[i]);
        combina(c, i, resto - c[i], cur, out);    // i, non i+1: riuso consentito
        cur.pop_back();
    }
}

int main() {
    std::vector<int> c{2, 3, 6, 7}, cur;
    std::sort(c.begin(), c.end());
    std::vector<std::vector<int>> out;
    combina(c, 0, 7, cur, out);
    for (auto& v : out) { for (int x : v) std::cout << x; std::cout << ' '; }
    std::cout << '\n';
}
```

```
223 7
```

Soluzione 13.3

```
#include <algorithm>
#include <iostream>
#include <vector>

bool raggiungibile(const std::vector<int>& a) {
    int lontano = 0;
    for (int i = 0; i < (int)a.size(); ++i) {
        if (i > lontano) return false;              // non si arriva nemmeno qui
        lontano = std::max(lontano, i + a[i]);
    }
    return true;
}

int main() {
    std::cout << raggiungibile({2, 3, 1, 1, 4}) << raggiungibile({3, 2, 1, 0, 4}) << '\n';
}
```

```
10
```

Soluzione 13.4

Con {1, 3, 4} e importo 6 il greedy prende 4, poi 1 e 1: tre monete, mentre 3 + 3 ne usa due. Il greedy e' ottimo per i sistemi detti **canonici**, come quello dell'euro (1, 2, 5, 10, 20, 50 centesimi...) o {1, 5, 10, 25}: si verifica che per ogni importo il greedy coincide con la DP dell'Esempio 14.1. In generale, per un sistema arbitrario serve la programmazione dinamica.

Soluzione 13.5

Le bitmask permettono di calcolare i candidati di una casella con due OR e un NOT. Scegliere la casella con meno candidati (euristica MRV) riduce drasticamente l'albero di ricerca.

```
#include <bit>
#include <iostream>
#include <string>
#include <vector>

struct Sudoku {
    std::vector<std::string> g;
    int riga[9] = {}, col[9] = {}, box[9] = {};

    explicit Sudoku(std::vector<std::string> s) : g(std::move(s)) {
        for (int r = 0; r < 9; ++r)
            for (int c = 0; c < 9; ++c)
                if (g[r][c] != '.') segna(r, c, g[r][c] - '1');
    }
    void segna(int r, int c, int d) {
        int b = 1 << d;
        riga[r] ^= b; col[c] ^= b; box[r / 3 * 3 + c / 3] ^= b;
    }
    bool risolvi() {
        int br = -1, bc = -1, bmask = 0, bmin = 10;
        for (int r = 0; r < 9; ++r)
            for (int c = 0; c < 9; ++c) {
                if (g[r][c] != '.') continue;
                int m = ~(riga[r] | col[c] | box[r / 3 * 3 + c / 3]) & 0x1FF;
                if (std::popcount(unsigned(m)) < bmin) {
                    bmin = std::popcount(unsigned(m)); br = r; bc = c; bmask = m;
                }
            }
        if (br < 0) return true;                       // nessuna casella vuota
        for (int m = bmask; m; m &= m - 1) {
            int d = std::countr_zero(unsigned(m));
            g[br][bc] = char('1' + d); segna(br, bc, d);
            if (risolvi()) return true;
            segna(br, bc, d); g[br][bc] = '.';         // XOR annulla la scelta
        }
        return false;
    }
};

int main() {
    Sudoku s({"53..7....", "6..195...", ".98....6.", "8...6...3", "4..8.3..1",
              "7...2...6", ".6....28.", "...419..5", "....8..79"});
    if (s.risolvi()) for (auto& r : s.g) std::cout << r << '\n';
}
```

```
534678912
672195348
198342567
859761423
426853791
713924856
961537284
287419635
345286179
```

Capitolo 14 - Programmazione dinamica

Soluzione 14.1

`dp[i]` = costo minimo per arrivare al gradino i = `min(dp[i-1] + c[i-1], dp[i-2] + c[i-2])`. Servono solo gli ultimi due valori.

```
#include <algorithm>
#include <iostream>
#include <vector>

int costoMinimo(const std::vector<int>& c) {
    int a = 0, b = 0;                               // dp[i-2], dp[i-1]
    for (std::size_t i = 2; i <= c.size(); ++i) {
        int nuovo = std::min(b + c[i - 1], a + c[i - 2]);
        a = b; b = nuovo;
    }
    return b;
}

int main() {
    std::cout << costoMinimo({10, 15, 20}) << ' '
              << costoMinimo({1, 100, 1, 1, 1, 100, 1, 1, 100, 1}) << '\n';
}
```

```
15 6
```

Soluzione 14.2

Ricorrenza: `dp[i] = max(dp[i-1], dp[i-2] + a[i])` (o salto la casa i, o la svaligio e non ho toccato la i-1).

```
#include <algorithm>
#include <iostream>
#include <vector>

int bottino(const std::vector<int>& a) {
    int prec2 = 0, prec1 = 0;
    for (int x : a) {
        int cur = std::max(prec1, prec2 + x);
        prec2 = prec1; prec1 = cur;
    }
    return prec1;
}

int main() { std::cout << bottino({1, 2, 3, 1}) << ' ' << bottino({2, 7, 9, 3, 1}) << '\n'; }
```

```
4 12
```

Soluzione 14.3

`dp[i][j]` = operazioni per trasformare i primi i caratteri di `a` nei primi j di `b`. Se i caratteri finali coincidono costa `dp[i-1][j-1]`, altrimenti `1 + min(sostituzione dp[i-1][j-1], cancellazione dp[i-1][j], inserimento dp[i][j-1])`.

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>

int editDistance(const std::string& a, const std::string& b) {
    std::size_t n = a.size(), m = b.size();
    std::vector<std::vector<int>> dp(n + 1, std::vector<int>(m + 1));
    for (std::size_t i = 0; i <= n; ++i) dp[i][0] = i;     // cancellare tutto
    for (std::size_t j = 0; j <= m; ++j) dp[0][j] = j;     // inserire tutto
    for (std::size_t i = 1; i <= n; ++i)
        for (std::size_t j = 1; j <= m; ++j)
            dp[i][j] = a[i - 1] == b[j - 1]
                ? dp[i - 1][j - 1]
                : 1 + std::min({dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]});
    return dp[n][m];
}

int main() {
    std::cout << editDistance("horse", "ros") << ' ' << editDistance("intention", "execution")
              << ' ' << editDistance("gatto", "gatti") << '\n';
}
```

```
3 5 1
```

Soluzione 14.4

`dp[j]` contiene i percorsi fino alla cella `(i, j)` della riga corrente: prima dell'aggiornamento vale il valore della riga sopra, `dp[j-1]` quello della cella a sinistra.

```
#include <iostream>
#include <vector>

long long percorsi(const std::vector<std::vector<int>>& g) {
    std::vector<long long> dp(g[0].size(), 0);
    dp[0] = 1;
    for (const auto& riga : g)
        for (std::size_t j = 0; j < riga.size(); ++j) {
            if (riga[j] == 1) dp[j] = 0;            // ostacolo
            else if (j > 0) dp[j] += dp[j - 1];     // sopra + sinistra
        }
    return dp.back();
}

int main() {
    std::cout << percorsi({{0, 0, 0}, {0, 1, 0}, {0, 0, 0}}) << ' '
              << percorsi({{0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}}) << '\n';
}
```

```
2 10
```

Soluzione 14.5

`code[k]` e' il piu' piccolo elemento finale possibile di una sottosequenza crescente di lunghezza k+1. Il vettore resta ordinato, quindi per ogni nuovo elemento basta una ricerca binaria: o estende la sequenza piu' lunga, o migliora (abbassa) una coda esistente.

```
#include <algorithm>
#include <iostream>
#include <vector>

int lis(const std::vector<int>& a) {
    std::vector<int> code;
    for (int x : a) {
        auto it = std::lower_bound(code.begin(), code.end(), x);   // strettamente crescente
        if (it == code.end()) code.push_back(x); else *it = x;
    }
    return code.size();
}

int main() {
    std::cout << lis({10, 9, 2, 5, 3, 7, 101, 18}) << ' ' << lis({7, 7, 7}) << ' '
              << lis({0, 1, 0, 3, 2, 3}) << '\n';
}
```

```
4 1 4
```

Soluzione 14.6

Serve un sottoinsieme di somma `totale / 2`. `possibile[s]` dice se la somma s e' formabile; come nello zaino 0/1 si scorre all'indietro per non usare due volte lo stesso elemento. Con `std::bitset` tutta la riga si aggiorna con un'unica istruzione `bits |= bits << x`, che elabora 64 somme per operazione macchina.

```
#include <bitset>
#include <iostream>
#include <numeric>
#include <vector>

bool partizionabile(const std::vector<int>& a) {
    int tot = std::accumulate(a.begin(), a.end(), 0);
    if (tot % 2) return false;
    std::vector<bool> possibile(tot / 2 + 1, false);
    possibile[0] = true;
    for (int x : a)
        for (int s = tot / 2; s >= x; --s)
            if (possibile[s - x]) possibile[s] = true;
    return possibile[tot / 2];
}

bool partizionabileBitset(const std::vector<int>& a) {
    std::bitset<20001> bits(1);                    // bit s acceso = somma s formabile
    int tot = 0;
    for (int x : a) { bits |= bits << x; tot += x; }
    return tot % 2 == 0 && bits[tot / 2];
}

int main() {
    std::cout << partizionabile({1, 5, 11, 5}) << partizionabile({1, 2, 3, 5}) << ' '
              << partizionabileBitset({1, 5, 11, 5}) << partizionabileBitset({1, 2, 3, 5}) << '\n';
}
```

```
10 10
```

Capitolo 15 - Algoritmi su grafi

Soluzione 15.1

```
#include <algorithm>
#include <climits>
#include <iostream>
#include <queue>
#include <vector>

int ritardo(const std::vector<std::vector<int>>& tempi, int n, int k) {
    std::vector<std::vector<std::pair<int, int>>> g(n + 1);
    for (const auto& t : tempi) g[t[0]].push_back({t[1], t[2]});
    std::vector<int> d(n + 1, INT_MAX);
    using P = std::pair<int, int>;
    std::priority_queue<P, std::vector<P>, std::greater<>> pq;
    d[k] = 0; pq.push({0, k});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du > d[u]) continue;
        for (auto [v, w] : g[u]) if (du + w < d[v]) { d[v] = du + w; pq.push({d[v], v}); }
    }
    int m = *std::max_element(d.begin() + 1, d.end());
    return m == INT_MAX ? -1 : m;
}

int main() {
    std::cout << ritardo({{2, 1, 1}, {2, 3, 1}, {3, 4, 1}}, 4, 2) << ' '
              << ritardo({{1, 2, 1}}, 2, 2) << '\n';
}
```

```
2 -1
```

Soluzione 15.2

Floyd-Warshall prova ogni nodo `k` come intermedio: `d[i][j] = min(d[i][j], d[i][k] + d[k][j])`. L'ordine dei cicli e' essenziale: `k` deve essere il ciclo esterno.

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <vector>

int main() {
    const int INF = 1'000'000, n = 6;
    std::vector<std::vector<int>> d(n, std::vector<int>(n, INF));
    struct Arco { int a, b, km; };
    for (auto [a, b, km] : std::vector<Arco>{{0, 1, 214}, {0, 5, 142}, {1, 2, 106}, {5, 2, 230},
                                             {2, 3, 273}, {1, 3, 380}, {3, 4, 226}, {5, 3, 501}})
        d[a][b] = d[b][a] = km;
    for (int i = 0; i < n; ++i) d[i][i] = 0;
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                d[i][j] = std::min(d[i][j], d[i][k] + d[k][j]);
    for (const auto& riga : d) {
        for (int x : riga) std::cout << std::setw(5) << x;
        std::cout << '\n';
    }
}
```

```
    0  214  320  593  819  142
  214    0  106  379  605  336
  320  106    0  273  499  230
  593  379  273    0  226  501
  819  605  499  226    0  727
  142  336  230  501  727    0
```

La matrice rivela un dettaglio che Dijkstra da Milano non mostrava: tra Bologna e Genova conviene passare per Firenze (106 + 230 = 336) invece che per Milano (214 + 142 = 356).

Soluzione 15.3

Dopo `n - 1` rilassamenti di tutti gli archi le distanze sono definitive, se non ci sono cicli negativi; se un ulteriore passo migliora ancora qualcosa, un ciclo negativo esiste. Con i cambi valutari: un ciclo con prodotto dei tassi > 1 (arbitraggio) ha somma di `-log(tasso)` negativa.

```
#include <cmath>
#include <iostream>
#include <vector>

struct Arco { int a, b; double w; };

bool cicloNegativo(int n, const std::vector<Arco>& archi) {
    std::vector<double> d(n, 0.0);          // partenza "virtuale" da tutti i nodi
    for (int i = 0; i < n - 1; ++i)
        for (const auto& e : archi)
            if (d[e.a] + e.w < d[e.b]) d[e.b] = d[e.a] + e.w;
    for (const auto& e : archi)
        if (d[e.a] + e.w < d[e.b] - 1e-12) return true;
    return false;
}

int main() {
    // 0 = EUR, 1 = USD, 2 = GBP; tassi di cambio
    auto archiDa = [](double eurUsd, double usdGbp, double gbpEur) {
        return std::vector<Arco>{{0, 1, -std::log(eurUsd)}, {1, 2, -std::log(usdGbp)},
                                 {2, 0, -std::log(gbpEur)}};
    };
    std::cout << cicloNegativo(3, archiDa(1.10, 0.80, 1.12)) << ' '      // 1.10*0.80*1.12 < 1
              << cicloNegativo(3, archiDa(1.10, 0.80, 1.16)) << '\n';    // 1.0208 > 1
}
```

```
0 1
```

Soluzione 15.4

```
#include <iostream>
#include <queue>
#include <tuple>
#include <vector>

int main() {
    int n = 5;
    std::vector<std::vector<std::pair<int, int>>> g(n);        // (vicino, costo)
    for (auto [c, a, b] : std::vector<std::tuple<int, int, int>>{
             {4, 0, 1}, {8, 0, 2}, {2, 1, 2}, {6, 1, 3}, {3, 2, 3}, {9, 3, 4}, {5, 2, 4}}) {
        g[a].push_back({b, c});
        g[b].push_back({a, c});
    }
    std::vector<bool> inAlbero(n, false);
    using P = std::pair<int, int>;                              // (costo, nodo)
    std::priority_queue<P, std::vector<P>, std::greater<>> pq;
    pq.push({0, 0});
    int totale = 0;
    while (!pq.empty()) {
        auto [c, u] = pq.top(); pq.pop();
        if (inAlbero[u]) continue;
        inAlbero[u] = true;
        totale += c;
        for (auto [v, w] : g[u]) if (!inAlbero[v]) pq.push({w, v});
    }
    std::cout << "costo totale: " << totale << '\n';
}
```

```
costo totale: 14
```

Soluzione 15.5

Dijkstra fissa per ogni nodo **una** distanza, ma qui un percorso piu' caro con meno scali puo' essere l'unico che rispetta il vincolo. Bellman-Ford alla iterazione i trova i costi minimi usando al massimo i archi; ci fermiamo a k+1 archi. Ogni iterazione deve leggere i valori della precedente (copia `prec`), altrimenti un'iterazione potrebbe usare piu' archi.

```
#include <climits>
#include <iostream>
#include <vector>

int voloEconomico(int n, const std::vector<std::vector<int>>& voli, int src, int dst, int k) {
    std::vector<int> d(n, INT_MAX);
    d[src] = 0;
    for (int i = 0; i <= k; ++i) {
        auto prec = d;
        for (const auto& f : voli)
            if (prec[f[0]] != INT_MAX && prec[f[0]] + f[2] < d[f[1]]) d[f[1]] = prec[f[0]] + f[2];
    }
    return d[dst] == INT_MAX ? -1 : d[dst];
}

int main() {
    std::vector<std::vector<int>> voli{{0, 1, 100}, {1, 2, 100}, {2, 3, 100}, {0, 2, 500},
                                       {1, 3, 600}};
    std::cout << voloEconomico(4, voli, 0, 3, 1) << ' ' << voloEconomico(4, voli, 0, 3, 2) << ' '
              << voloEconomico(4, voli, 0, 3, 0) << '\n';
}
```

```
600 300 -1
```

Capitolo 16 - Stringhe, matematica e bit manipulation

Soluzione 16.1

`x ^ x = 0` e `x ^ 0 = x`: facendo lo XOR di tutti gli indici 0..n e di tutti gli elementi, ogni numero presente si annulla con se stesso e resta solo il mancante. Nessuna somma, quindi nessun overflow.

```
#include <iostream>
#include <vector>

int mancante(const std::vector<int>& a) {
    int x = a.size();
    for (int i = 0; i < (int)a.size(); ++i) x ^= i ^ a[i];
    return x;
}

int main() {
    std::cout << mancante({3, 0, 1}) << ' ' << mancante({9, 6, 4, 2, 3, 5, 7, 0, 1}) << '\n';
}
```

```
2 8
```

Soluzione 16.2

```
#include <climits>
#include <iostream>
#include <string_view>

int convertiIntero(std::string_view s) {
    std::size_t i = 0;
    while (i < s.size() && s[i] == ' ') ++i;
    int segno = 1;
    if (i < s.size() && (s[i] == '+' || s[i] == '-')) segno = s[i++] == '-' ? -1 : 1;
    long long r = 0;
    for (; i < s.size() && s[i] >= '0' && s[i] <= '9'; ++i) {
        r = r * 10 + (s[i] - '0');
        if (segno * r > INT_MAX) return INT_MAX;       // satura prima di superare long long
        if (segno * r < INT_MIN) return INT_MIN;
    }
    return static_cast<int>(segno * r);
}

int main() {
    for (auto s : {"42", "   -042", "1337c0d3", "words 1", "-91283472332", "+"})
        std::cout << convertiIntero(s) << ' ';
    std::cout << '\n';
}
```

```
42 -42 1337 0 -2147483648 0
```

> **Nota**
>
> Chiamare la funzione `atoi` sarebbe un errore insidioso: con un argomento `const char*` il compilatore preferirebbe `::atoi` della libreria C (corrispondenza esatta) alla nostra versione con `std::string_view` (che richiede una conversione), senza alcun avviso.

Soluzione 16.3

La cifra `i` del primo numero per la cifra `j` del secondo contribuisce alla posizione `i + j + 1` del risultato (che ha al massimo n + m cifre); i riporti si propagano verso sinistra.

```
#include <iostream>
#include <string>
#include <vector>

std::string moltiplica(const std::string& a, const std::string& b) {
    std::vector<int> r(a.size() + b.size(), 0);
    for (int i = (int)a.size() - 1; i >= 0; --i)
        for (int j = (int)b.size() - 1; j >= 0; --j) {
            int somma = (a[i] - '0') * (b[j] - '0') + r[i + j + 1];
            r[i + j + 1] = somma % 10;
            r[i + j] += somma / 10;
        }
    std::string out;
    for (int d : r) if (!(out.empty() && d == 0)) out += char('0' + d);   // salta zeri iniziali
    return out.empty() ? "0" : out;
}

int main() {
    std::cout << moltiplica("123", "456") << ' ' << moltiplica("99", "0") << ' '
              << moltiplica("123456789", "987654321") << '\n';
}
```

```
56088 0 121932631112635269
```

Soluzione 16.4

```
#include <algorithm>
#include <iostream>
#include <vector>

void ruota(std::vector<std::vector<int>>& m) {
    std::size_t n = m.size();
    for (std::size_t i = 0; i < n; ++i)
        for (std::size_t j = i + 1; j < n; ++j) std::swap(m[i][j], m[j][i]);   // trasposta
    for (auto& riga : m) std::reverse(riga.begin(), riga.end());               // specchio
}

int main() {
    std::vector<std::vector<int>> m{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    ruota(m);
    for (auto& r : m) { for (int x : r) std::cout << x << ' '; std::cout << '\n'; }
}
```

```
7 4 1
8 5 2
9 6 3
```

Soluzione 16.5

L'hash di una finestra di lunghezza m si aggiorna in O(1): si toglie il contributo del carattere uscente (moltiplicato per `B^(m-1)`), si moltiplica per la base e si aggiunge il carattere entrante. Uguaglianza di hash non implica uguaglianza di stringhe, quindi i candidati si verificano.

```
#include <iostream>
#include <string>
#include <vector>

std::vector<int> rabinKarp(const std::string& t, const std::string& p) {
    const unsigned long long MOD = 1'000'000'007, B = 131;
    std::size_t m = p.size();
    std::vector<int> out;
    if (m == 0 || m > t.size()) return out;
    unsigned long long hp = 0, ht = 0, potenza = 1;
    for (std::size_t i = 0; i < m; ++i) {
        hp = (hp * B + (unsigned char)p[i]) % MOD;
        ht = (ht * B + (unsigned char)t[i]) % MOD;
        if (i) potenza = potenza * B % MOD;                  // B^(m-1)
    }
    for (std::size_t i = 0;; ++i) {
        if (hp == ht && t.compare(i, m, p) == 0) out.push_back(i);
        if (i + m == t.size()) break;
        ht = (ht + MOD - (unsigned char)t[i] * potenza % MOD) % MOD;   // esce t[i]
        ht = (ht * B + (unsigned char)t[i + m]) % MOD;                 // entra t[i+m]
    }
    return out;
}

int main() {
    for (int i : rabinKarp("abababcababababab", "abab")) std::cout << i << ' ';
    std::cout << '\n';
}
```

```
0 2 7 9 11
```

Soluzioni - Parte 5

Capitolo 17 - Concorrenza e parallelismo

Soluzione 17.1

Ogni `promise` e' un segnale "una tantum": `secondo` attende che `primo` abbia finito, `terzo` attende `secondo`. I thread vengono avviati apposta in ordine inverso.

```
#include <future>
#include <iostream>
#include <thread>

class Ordinato {
    std::promise<void> p1_, p2_;
    std::shared_future<void> f1_ = p1_.get_future().share(), f2_ = p2_.get_future().share();
public:
    void primo()   { std::cout << "primo ";   p1_.set_value(); }
    void secondo() { f1_.wait(); std::cout << "secondo "; p2_.set_value(); }
    void terzo()   { f2_.wait(); std::cout << "terzo\n"; }
};

int main() {
    Ordinato o;
    std::jthread c([&] { o.terzo(); }), b([&] { o.secondo(); }), a([&] { o.primo(); });
}
```

```
primo secondo terzo
```

Soluzione 17.2

Con `std::atomic<long>` il risultato e' sempre 400000 e il codice e' piu' veloce del mutex (un'istruzione atomica `lock xadd` invece di lock/unlock). Senza sincronizzazione `++contatore` e' una sequenza leggi-modifica-scrivi non atomica: due thread leggono lo stesso valore e un incremento si perde, quindi il risultato e' quasi sempre minore di 400000 e cambia a ogni esecuzione. ThreadSanitizer segnala `WARNING: ThreadSanitizer: data race` indicando le due righe in conflitto; formalmente il programma ha comportamento indefinito, non semplicemente "un risultato sbagliato".

```
#include <atomic>
#include <iostream>
#include <thread>
#include <vector>

int main() {
    std::atomic<long> contatore{0};
    {
        std::vector<std::jthread> ts;
        for (int i = 0; i < 4; ++i)
            ts.emplace_back([&] {
                for (int k = 0; k < 100000; ++k) contatore.fetch_add(1, std::memory_order_relaxed);
            });
    }
    std::cout << contatore << '\n';
}
```

```
400000
```

`memory_order_relaxed` basta perche' il contatore non viene usato per sincronizzare altri dati: interessa solo il valore finale, letto dopo il join.

Soluzione 17.3

```
#include <iostream>
#include <semaphore>
#include <thread>

int main() {
    const int n = 3;
    std::binary_semaphore turnoFoo{1}, turnoBar{0};     // inizia foo
    std::jthread foo([&] {
        for (int i = 0; i < n; ++i) { turnoFoo.acquire(); std::cout << "foo"; turnoBar.release(); }
    });
    std::jthread bar([&] {
        for (int i = 0; i < n; ++i) { turnoBar.acquire(); std::cout << "bar"; turnoFoo.release(); }
    });
    bar.join(); foo.join();
    std::cout << '\n';
}
```

```
foobarfoobarfoobar
```

Soluzione 17.4

`std::packaged_task` avvolge una funzione e ne espone il risultato tramite un `future`. Poiche' `packaged_task` non e' copiabile, la coda contiene `std::function<void()>` che possiedono il task tramite `shared_ptr`.

```
#include <condition_variable>
#include <functional>
#include <future>
#include <iostream>
#include <memory>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>

class ThreadPool {
    std::mutex m_;
    std::condition_variable cv_;
    std::queue<std::function<void()>> lavori_;
    bool stop_ = false;
    std::vector<std::jthread> thread_;
public:
    explicit ThreadPool(unsigned n) {
        for (unsigned i = 0; i < n; ++i)
            thread_.emplace_back([this] {
                for (;;) {
                    std::function<void()> lavoro;
                    {
                        std::unique_lock lk(m_);
                        cv_.wait(lk, [this] { return stop_ || !lavori_.empty(); });
                        if (stop_ && lavori_.empty()) return;
                        lavoro = std::move(lavori_.front());
                        lavori_.pop();
                    }
                    lavoro();                          // eseguito fuori dal lock
                }
            });
    }
    template <typename F>
    auto invia(F f) -> std::future<decltype(f())> {
        auto task = std::make_shared<std::packaged_task<decltype(f())()>>(std::move(f));
        auto fut = task->get_future();
        { std::lock_guard lk(m_); lavori_.push([task] { (*task)(); }); }
        cv_.notify_one();
        return fut;
    }
    ~ThreadPool() {
        { std::lock_guard lk(m_); stop_ = true; }
        cv_.notify_all();                              // i jthread fanno join dopo
    }
};

int main() {
    ThreadPool pool(4);
    std::vector<std::future<long long>> risultati;
    for (int i = 1; i <= 8; ++i)
        risultati.push_back(pool.invia([i] {
            long long s = 0;
            for (int k = 1; k <= i * 1000; ++k) s += k;
            return s;
        }));
    for (auto& r : risultati) std::cout << r.get() << ' ';
    std::cout << '\n';
}
```

```
500500 2001000 4501500 8002000 12502500 18003000 24503500 32004000
```

Capitolo 18 - Prestazioni e ottimizzazione

Soluzione 18.1

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;
    std::size_t cap = v.capacity(), cambi = 0;
    for (int i = 0; i < 1000; ++i) {
        v.push_back(i);
        if (v.capacity() != cap) { ++cambi; cap = v.capacity(); std::cout << cap << ' '; }
    }
    std::cout << "\nriallocazioni: " << cambi << '\n';
    std::vector<int> w;
    w.reserve(1000);
    cap = w.capacity(); cambi = 0;
    for (int i = 0; i < 1000; ++i) { w.push_back(i); if (w.capacity() != cap) ++cambi; }
    std::cout << "con reserve: " << cambi << '\n';
}
```

```
1 2 4 8 16 32 64 128 256 512 1024
riallocazioni: 11
con reserve: 0
```

libstdc++ (GCC) e libc++ (Clang) raddoppiano la capacita'; MSVC la moltiplica per 1,5. La crescita geometrica e' cio' che rende `push_back` O(1) ammortizzato: il costo totale delle copie e' al massimo circa 2n.

Soluzione 18.2

Ogni particella in formato AoS occupa 64 byte (6 float + 40 char): per aggiornare le posizioni si caricano in cache anche i nomi, che non servono, e ogni linea di cache contiene una sola particella. In formato SoA i sei vettori di `float` sono contigui: ogni linea contiene 16 valori utili e il compilatore puo' vettorizzare il ciclo con istruzioni SIMD. Con `-O2` la versione SoA e' tipicamente 2-4 volte piu' veloce.

```
#include <iostream>
#include <vector>

struct P { float x, y, z, vx, vy, vz; char nome[40]; };          // array of structs

struct Particelle {                                              // struct of arrays
    std::vector<float> x, y, z, vx, vy, vz;
    explicit Particelle(std::size_t n) : x(n), y(n), z(n), vx(n, 1), vy(n, 2), vz(n, 3) {}
};

int main() {
    const std::size_t n = 1'000'000;
    const float dt = 0.5f;
    std::vector<P> aos(n, P{0, 0, 0, 1, 2, 3, {}});
    for (auto& p : aos) { p.x += p.vx * dt; p.y += p.vy * dt; p.z += p.vz * dt; }

    Particelle soa(n);
    for (std::size_t i = 0; i < n; ++i) {
        soa.x[i] += soa.vx[i] * dt; soa.y[i] += soa.vy[i] * dt; soa.z[i] += soa.vz[i] * dt;
    }
    std::cout << sizeof(P) << ' ' << aos[7].z << ' ' << soa.z[7] << '\n';
}
```

```
64 1.5 1.5
```

Per misurare i tempi si avvolgono i due cicli nella funzione `millisecondi` dell'Esempio 18.1.

Soluzione 18.3

La versione originale copia `s` e `p` (due allocazioni per il passaggio per valore) e crea una terza stringa con `substr`. Con `std::string_view` non si alloca nulla; dal C++20 esiste anche `starts_with`.

```
#include <iostream>
#include <string_view>

bool inizia(std::string_view s, std::string_view p) {
    return s.size() >= p.size() && s.compare(0, p.size(), p) == 0;   // o s.starts_with(p)
}

int main() {
    std::cout << inizia("programmazione", "progr") << inizia("ciao", "ciaone") << '\n';
}
```

```
10
```

Soluzione 18.4

`i` e' `unsigned`, quindi `i >= 0` e' sempre vero: dopo 0, `--i` produce 4294967295 e il ciclo prosegue (accedendo fuori dai limiti). Nel secondo caso l'overflow di `x + 1` e' UB: il compilatore ragiona come se non potesse accadere, deduce che `x + 1 < x` e' sempre falso e rimuove il ramo. Versioni corrette:

```
#include <climits>
#include <cstddef>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v{1, 2, 3};
    for (std::size_t i = v.size(); i-- > 0;) std::cout << v[i];   // decrementa dopo il test
    std::cout << ' ';
    int x = INT_MAX;
    bool overflow = x > INT_MAX - 1;                              // controllo prima della somma
    int r;
    bool overflow2 = __builtin_add_overflow(x, 1, &r);            // GCC/Clang
    std::cout << overflow << overflow2 << '\n';
}
```

```
321 11
```

Capitolo 19 - Le principali librerie del C++

Soluzione 19.1

```
{
  "version": 6,
  "configurePresets": [
    { "name": "debug", "binaryDir": "build/debug",
      "cacheVariables": { "CMAKE_BUILD_TYPE": "Debug",
        "CMAKE_CXX_FLAGS": "-fsanitize=address,undefined -fno-omit-frame-pointer" } },
    { "name": "release", "binaryDir": "build/release",
      "cacheVariables": { "CMAKE_BUILD_TYPE": "Release", "CMAKE_CXX_FLAGS": "-O2" } }
  ],
  "buildPresets": [
    { "name": "debug", "configurePreset": "debug" },
    { "name": "release", "configurePreset": "release" }
  ]
}
```

```
cmake --preset debug && cmake --build --preset debug && ctest --test-dir build/debug
```

Soluzione 19.2

Casi limite spesso dimenticati: stringa vuota, solo segno (`"-"`), `"+0"`, il valore esatto `INT_MIN` (il cui opposto non e' rappresentabile), spazi finali e caratteri dopo le cifre.

```
#include <charconv>
#include <iostream>
#include <optional>
#include <string_view>

std::optional<int> parseInt(std::string_view s) {
    if (!s.empty() && s[0] == '+') {                          // from_chars non accetta '+'
        s.remove_prefix(1);
        if (!s.empty() && s[0] == '-') return std::nullopt;     // "+-3" non e' valido
    }
    int v;
    auto [p, e] = std::from_chars(s.data(), s.data() + s.size(), v);
    if (e != std::errc{} || p != s.data() + s.size() || s.empty()) return std::nullopt;
    return v;
}

int falliti = 0;
#define CHECK(expr) \
    do { if (!(expr)) { ++falliti; std::cout << "FALLITO: " #expr "\n"; } } while (0)

int main() {
    CHECK(parseInt("42") == 42);
    CHECK(parseInt("-17") == -17);
    CHECK(parseInt("+5") == 5);
    CHECK(parseInt("-2147483648") == -2147483648LL);
    CHECK(!parseInt("2147483648"));          // overflow
    CHECK(!parseInt(""));
    CHECK(!parseInt("-"));
    CHECK(!parseInt(" 42"));                 // spazi: scelta di progetto, qui rifiutati
    CHECK(!parseInt("42 "));
    CHECK(!parseInt("12abc"));
    CHECK(!parseInt("+-3"));
    std::cout << (falliti ? "ci sono errori" : "tutti i test superati") << '\n';
}
```

```
tutti i test superati
```

Il caso `"+-3"` merita attenzione: nella prima stesura della soluzione il `+` veniva rimosso e `from_chars` accettava poi `-3`, restituendo -3. E' stato proprio questo test a scoprirlo: i casi limite vanno scritti **prima** di convincersi che il codice sia corretto.

Soluzione 19.3

```
# .clang-tidy
Checks: >
  -*,
  modernize-*,
  bugprone-*,
  performance-*,
  -modernize-use-trailing-return-type
WarningsAsErrors: 'bugprone-use-after-move'
HeaderFilterRegex: 'include/.*'
```

```
// non eseguire: file di prova per clang-tidy
#include <string>
#include <utility>
#include <vector>
int main() {
    std::string s = "ciao";
    std::vector<std::string> v;
    v.push_back(std::move(s));
    return static_cast<int>(s.size());     // clang-tidy: 's' used after it was moved
}
```

```
clang-tidy uso.cpp -- -std=c++20
# error: 's' used after it was moved [bugprone-use-after-move,-warnings-as-errors]
```

Soluzione 19.4

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        compilatore: [g++, clang++]
    steps:
      - uses: actions/checkout@v4
      - name: Configura
        run: >
          cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
          -DCMAKE_CXX_COMPILER=${{ matrix.compilatore }}
          -DCMAKE_CXX_FLAGS="-fsanitize=address,undefined"
      - name: Compila
        run: cmake --build build -j
      - name: Test
        run: ctest --test-dir build --output-on-failure
  formato:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: find src include test -name '*.[ch]pp' | xargs clang-format --dry-run --Werror
```

Soluzioni - Parte 6

Capitolo 20 - Programmazione web con C++

Soluzione 20.1

```
#include <cctype>
#include <iostream>
#include <string>
#include <string_view>

int esadecimale(char c) {
    if (c >= '0' && c <= '9') return c - '0';
    c = static_cast<char>(std::tolower(static_cast<unsigned char>(c)));
    return c >= 'a' && c <= 'f' ? c - 'a' + 10 : -1;
}

std::string decodificaURL(std::string_view s) {
    std::string out;
    for (std::size_t i = 0; i < s.size(); ++i) {
        if (s[i] == '+') out += ' ';
        else if (s[i] == '%' && i + 2 < s.size() && esadecimale(s[i + 1]) >= 0 &&
                 esadecimale(s[i + 2]) >= 0) {
            out += static_cast<char>(esadecimale(s[i + 1]) * 16 + esadecimale(s[i + 2]));
            i += 2;
        } else out += s[i];                       // '%' malformato: lasciato com'e'
    }
    return out;
}

int main() {
    std::cout << decodificaURL("Anna%20Maria") << " | " << decodificaURL("a+b%3Dc") << " | "
              << decodificaURL("100%") << '\n';
}
```

```
Anna Maria | a b=c | 100%
```

Nota: la decodifica va fatta **dopo** aver diviso la query su `&` e `=`, altrimenti un `%26` (che e' `&`) codificato dentro un valore verrebbe scambiato per un separatore.

Soluzione 20.2

Un contatore crescente convertito in base 62 produce codici brevi e unici senza collisioni (a differenza di un hash troncato). In un sistema reale i codici sequenziali sono prevedibili: si puo' mescolare il contatore con una permutazione invertibile.

```
#include <iostream>
#include <string>
#include <unordered_map>

class TinyURL {
    static constexpr const char* ALFABETO =
        "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
    unsigned long long prossimo_ = 100000;      // evita codici troppo corti
    std::unordered_map<std::string, std::string> daCodice_, daUrl_;
    static std::string base62(unsigned long long n) {
        std::string s;
        do { s.insert(s.begin(), ALFABETO[n % 62]); n /= 62; } while (n);
        return s;
    }
public:
    std::string codifica(const std::string& url) {
        if (auto it = daUrl_.find(url); it != daUrl_.end()) return it->second;
        std::string corto = "http://tiny.url/" + base62(prossimo_++);
        daUrl_[url] = corto;
        daCodice_[corto] = url;
        return corto;
    }
    std::string decodifica(const std::string& corto) const { return daCodice_.at(corto); }
};

int main() {
    TinyURL t;
    auto a = t.codifica("https://isocpp.org/std/the-standard");
    auto b = t.codifica("https://en.cppreference.com");
    std::cout << a << ' ' << b << ' ' << (t.codifica("https://en.cppreference.com") == b) << '\n'
              << t.decodifica(a) << '\n';
}
```

```
http://tiny.url/q0U http://tiny.url/q0V 1
https://isocpp.org/std/the-standard
```

Soluzione 20.3

Poiche' i `set` arrivano con timestamp crescenti, il vettore di ogni chiave e' gia' ordinato: `get` fa una ricerca binaria con `upper_bound` e prende l'elemento precedente.

```
#include <algorithm>
#include <iostream>
#include <string>
#include <unordered_map>
#include <utility>
#include <vector>

class TimeMap {
    std::unordered_map<std::string, std::vector<std::pair<int, std::string>>> m_;
public:
    void set(const std::string& k, std::string v, int t) { m_[k].emplace_back(t, std::move(v)); }
    std::string get(const std::string& k, int t) const {
        auto it = m_.find(k);
        if (it == m_.end()) return "";
        const auto& v = it->second;
        auto p = std::upper_bound(v.begin(), v.end(), t,
                                  [](int x, const auto& e) { return x < e.first; });
        return p == v.begin() ? "" : std::prev(p)->second;
    }
};

int main() {
    TimeMap tm;
    tm.set("prezzo", "10", 1);
    tm.set("prezzo", "12", 4);
    std::cout << '[' << tm.get("prezzo", 0) << "] [" << tm.get("prezzo", 1) << "] ["
              << tm.get("prezzo", 3) << "] [" << tm.get("prezzo", 5) << "]\n";
}
```

```
[] [10] [10] [12]
```

Soluzione 20.4

```
#include <iostream>
#include <string>
#include <unordered_map>

class Logger {
    std::unordered_map<std::string, int> prossimoConsentito_;
public:
    bool stampa(int t, const std::string& msg) {
        auto [it, nuovo] = prossimoConsentito_.try_emplace(msg, 0);
        if (t < it->second) return false;
        it->second = t + 10;
        return true;
    }
};

int main() {
    Logger l;
    std::cout << l.stampa(1, "foo") << l.stampa(2, "bar") << l.stampa(3, "foo")
              << l.stampa(8, "bar") << l.stampa(10, "foo") << l.stampa(11, "foo") << '\n';
}
```

```
110001
```

In un servizio di lunga durata la mappa crescerebbe senza limiti: si eliminano periodicamente le voci scadute, oppure si usa una coda dei messaggi recenti come in un hit counter.

Soluzione 20.5

```
// non compilare qui: richiede cpp-httplib e nlohmann/json (header-only)
#include <map>
#include <mutex>
#include <string>
#include "httplib.h"
#include <nlohmann/json.hpp>
using json = nlohmann::json;

int main() {
    std::mutex m;                                   // httplib usa piu' thread
    std::map<int, std::string> todo;
    int prossimoId = 1;
    RateLimiter limiter;                            // classe dell'Esempio 20.2
    httplib::Server svr;

    svr.set_pre_routing_handler([&](const httplib::Request& req, httplib::Response& res) {
        std::lock_guard lk(m);
        if (limiter.stato(req.remote_addr, secondiDaAvvio()) == 429) {
            res.status = 429;
            return httplib::Server::HandlerResponse::Handled;
        }
        return httplib::Server::HandlerResponse::Unhandled;
    });
    svr.Get("/todo", [&](const httplib::Request&, httplib::Response& res) {
        std::lock_guard lk(m);
        json j = json::array();
        for (auto& [id, testo] : todo) j.push_back({{"id", id}, {"testo", testo}});
        res.set_content(j.dump(), "application/json");
    });
    svr.Post("/todo", [&](const httplib::Request& req, httplib::Response& res) {
        auto j = json::parse(req.body, nullptr, false);       // false: niente eccezioni
        if (j.is_discarded() || !j.contains("testo") || !j["testo"].is_string()) {
            res.status = 400;
            return;
        }
        std::lock_guard lk(m);
        todo[prossimoId] = j["testo"];
        res.status = 201;
        res.set_content(json{{"id", prossimoId++}}.dump(), "application/json");
    });
    svr.Delete(R"(/todo/(\d+))", [&](const httplib::Request& req, httplib::Response& res) {
        std::lock_guard lk(m);
        res.status = todo.erase(std::stoi(req.matches[1])) ? 204 : 404;
    });
    svr.listen("0.0.0.0", 8080);
```

```
}
```

```
curl -X POST localhost:8080/todo -d '{"testo":"studiare C++"}'   # {"id":1}
curl localhost:8080/todo                                         # [{"id":1,"testo":"studiare C++"}]
curl -X DELETE -i localhost:8080/todo/1                          # HTTP/1.1 204 No Content
```

`secondiDaAvvio()` e' una funzione che restituisce i secondi trascorsi da uno `steady_clock::time_point` salvato all'avvio.

Capitolo 21 - Programmazione mobile

Soluzione 21.1

Una stringa letterale ha durata statica: esiste per tutta l'esecuzione del programma, quindi il chiamante puo' leggerla senza liberarla e senza rischio di puntatori pendenti. Va documentato che **non** deve chiamare `free`.

```
#include <iostream>
#include <vector>

struct StatHandle { std::vector<double> v; };
enum { STAT_OK = 0, STAT_ERRORE = 1 };

extern "C" int stat_conta(const StatHandle* h, unsigned* out) {
    if (!h || !out) return STAT_ERRORE;            // al confine C si validano i puntatori
    *out = static_cast<unsigned>(h->v.size());
    return STAT_OK;
}
extern "C" const char* stat_versione() { return "stat 1.2.0"; }

int main() {
    StatHandle h{{1, 2, 3}};
    unsigned n = 0;
    std::cout << stat_conta(&h, &n) << ' ' << n << ' ' << stat_conta(nullptr, &n) << ' '
              << stat_versione() << '\n';
}
```

```
0 3 1 stat 1.2.0
```

Soluzione 21.2

Per rimuovere in O(1) da un vettore si sposta l'ultimo elemento nella posizione da eliminare e si aggiorna il suo indice nella mappa.

```
#include <iostream>
#include <random>
#include <unordered_map>
#include <vector>

class InsiemeCasuale {
    std::vector<int> v_;
    std::unordered_map<int, std::size_t> idx_;
    std::mt19937 rng_{12345};
public:
    bool inserisci(int x) {
        if (idx_.count(x)) return false;
        idx_[x] = v_.size();
        v_.push_back(x);
        return true;
    }
    bool rimuovi(int x) {
        auto it = idx_.find(x);
        if (it == idx_.end()) return false;
        std::size_t i = it->second;
        v_[i] = v_.back();              // l'ultimo prende il posto di x
        idx_[v_[i]] = i;
        v_.pop_back();
        idx_.erase(x);
        return true;
    }
    int casuale() {
        return v_[std::uniform_int_distribution<std::size_t>(0, v_.size() - 1)(rng_)];
    }
    std::size_t size() const { return v_.size(); }
};

int main() {
    InsiemeCasuale s;
    std::cout << s.inserisci(1) << s.rimuovi(2) << s.inserisci(2) << s.inserisci(2) << s.rimuovi(1)
              << ' ' << s.casuale() << ' ' << s.size() << '\n';
}
```

```
10101 2 1
```

Soluzione 21.3

```
#include <fstream>
#include <iostream>
#include <map>
#include <optional>
#include <string>

struct Archivio {                           // astrazione senza dettagli di piattaforma
    virtual ~Archivio() = default;
    virtual void salva(const std::string& k, const std::string& v) = 0;
    virtual std::optional<std::string> leggi(const std::string& k) const = 0;
};

class ArchivioMemoria : public Archivio {               // per i test
    std::map<std::string, std::string> m_;
public:
    void salva(const std::string& k, const std::string& v) override { m_[k] = v; }
    std::optional<std::string> leggi(const std::string& k) const override {
        auto it = m_.find(k);
        return it == m_.end() ? std::nullopt : std::optional{it->second};
    }
};

class ArchivioFile : public Archivio {                  // desktop (su mobile: SharedPreferences,
    std::string percorso_;                              // UserDefaults, ecc. via binding)
public:
    explicit ArchivioFile(std::string p) : percorso_(std::move(p)) {}
    void salva(const std::string& k, const std::string& v) override {
        std::ofstream(percorso_ + "." + k) << v;
    }
    std::optional<std::string> leggi(const std::string& k) const override {
        std::ifstream in(percorso_ + "." + k);
        std::string v;
        if (!std::getline(in, v)) return std::nullopt;
        return v;
    }
};

class Contatore {                                       // logica dell'app
    Archivio& a_;
public:
    explicit Contatore(Archivio& a) : a_(a) {}
    int incrementa() {
        int n = std::stoi(a_.leggi("avvii").value_or("0")) + 1;
        a_.salva("avvii", std::to_string(n));
        return n;
```

```
    }
};

int main() {
    ArchivioMemoria finto;                              // il test non tocca il disco
    Contatore c(finto);
    c.incrementa(); c.incrementa();
    std::cout << "avvii: " << c.incrementa() << " | salvato: " << *finto.leggi("avvii") << '\n';
}
```

```
avvii: 3 | salvato: 3
```

Soluzione 21.4

```
cmake_minimum_required(VERSION 3.22)
project(statcore LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)

add_library(statcore src/core.cpp src/api.cpp)
target_include_directories(statcore PUBLIC include)          # contiene api.h
set_target_properties(statcore PROPERTIES CXX_VISIBILITY_PRESET hidden)  # esporta solo l'API C

if(ANDROID)
    set_target_properties(statcore PROPERTIES POSITION_INDEPENDENT_CODE ON)
elseif(IOS)
    set_target_properties(statcore PROPERTIES FRAMEWORK TRUE
                          PUBLIC_HEADER include/api.h MACOSX_FRAMEWORK_IDENTIFIER com.esempio.stat)
endif()
```

```
# Android: libreria condivisa per arm64-v8a
cmake -S . -B build-android -DBUILD_SHARED_LIBS=ON \
  -DCMAKE_TOOLCHAIN_FILE=$ANDROID_NDK/build/cmake/android.toolchain.cmake \
  -DANDROID_ABI=arm64-v8a -DANDROID_PLATFORM=android-24
cmake --build build-android
# iOS: framework statico
cmake -S . -B build-ios -G Xcode -DCMAKE_SYSTEM_NAME=iOS -DCMAKE_OSX_ARCHITECTURES=arm64
cmake --build build-ios --config Release
```

Da consegnare: al team Android `libstatcore.so` per ogni ABI (in `jniLibs/<abi>/`) piu' lo strato JNI e la classe Kotlin; al team iOS `statcore.framework` (o un `.xcframework` che riunisce dispositivo e simulatore) con `api.h`. Le intestazioni C++ interne non vanno distribuite: l'API C e' l'unico contratto, e la visibilita' `hidden` impedisce di dipendere per errore da simboli interni.

Soluzioni - Parte 7

Capitolo 22 - Fondamenti di game programming

Soluzione 22.1

Il bit 0 contiene lo stato attuale, il bit 1 quello futuro: contando i vicini si legge solo il bit 0, quindi le celle gia' aggiornate non disturbano. Alla fine si sposta ogni cella di un bit a destra.

```
#include <iostream>
#include <vector>

void prossimaGenerazione(std::vector<std::vector<int>>& g) {
    int R = g.size(), C = g[0].size();
    for (int r = 0; r < R; ++r)
        for (int c = 0; c < C; ++c) {
            int vicini = 0;
            for (int dr = -1; dr <= 1; ++dr)
                for (int dc = -1; dc <= 1; ++dc) {
                    int nr = r + dr, nc = c + dc;
                    bool dentro = nr >= 0 && nc >= 0 && nr < R && nc < C;
                    if ((dr || dc) && dentro) vicini += g[nr][nc] & 1;
                }
            bool viva = g[r][c] & 1;
            if ((viva && (vicini == 2 || vicini == 3)) || (!viva && vicini == 3)) g[r][c] |= 2;
        }
    for (auto& riga : g) for (int& x : riga) x >>= 1;
}

int main() {
    std::vector<std::vector<int>> g{{0, 1, 0}, {0, 0, 1}, {1, 1, 1}, {0, 0, 0}};
    prossimaGenerazione(g);
    for (auto& r : g) { for (int x : r) std::cout << x; std::cout << '\n'; }
}
```

```
000
101
011
010
```

Soluzione 22.2

```
#include <iostream>

struct Vec3 {
    float x, y, z;
    float dot(const Vec3& o) const { return x * o.x + y * o.y + z * o.z; }
    Vec3 cross(const Vec3& o) const {
        return {y * o.z - z * o.y, z * o.x - x * o.z, x * o.y - y * o.x};
    }
};

int main() {
    Vec3 a{1, 0, 0}, b{0, 1, 0}, c = a.cross(b);
    Vec3 p{1, 2, 3}, q{4, 5, 6}, n = p.cross(q);
    std::cout << c.x << c.y << c.z << ' ' << n.x << ',' << n.y << ',' << n.z << ' '
              << n.dot(p) << n.dot(q) << '\n';
}
```

```
001 -3,6,-3 00
```

Il prodotto vettoriale e' la base per calcolare le normali dei triangoli (illuminazione) e per capire se un punto sta a sinistra o a destra di una direzione.

Soluzione 22.3

La free list riusa la memoria degli oggetti liberi: ogni slot libero memorizza l'indice del prossimo slot libero. Acquisire e rilasciare sono O(1) e non allocano.

```
#include <array>
#include <cstddef>
#include <iostream>

struct Proiettile { float x = 0, y = 0, vx = 0, vy = 0; };

template <typename T, std::size_t N>
class ObjectPool {
    std::array<T, N> oggetti_{};
    std::array<int, N> prossimo_{};            // free list di indici
    std::array<bool, N> inUso_{};
    int libero_ = 0;
public:
    ObjectPool() {
        for (std::size_t i = 0; i < N; ++i) prossimo_[i] = i + 1 < N ? int(i + 1) : -1;
    }
    T* acquisisci() {
        if (libero_ < 0) return nullptr;
        int i = libero_;
        libero_ = prossimo_[i];
        inUso_[i] = true;
        oggetti_[i] = T{};
        return &oggetti_[i];
    }
    void rilascia(T* p) {
        int i = int(p - oggetti_.data());
        if (!inUso_[i]) return;                  // doppio rilascio ignorato
        inUso_[i] = false;
        prossimo_[i] = libero_;
        libero_ = i;
    }
};

int main() {
    ObjectPool<Proiettile, 2> pool;
    Proiettile* a = pool.acquisisci();
    Proiettile* b = pool.acquisisci();
    std::cout << (pool.acquisisci() == nullptr) << ' ';   // pool esaurito
    pool.rilascia(a);
    Proiettile* c = pool.acquisisci();
    std::cout << (c == a) << ' ' << (b != nullptr) << '\n';     // riusa lo stesso slot
}
```

```
1 1 1
```

Soluzione 22.4

Per simmetria ci si riduce al primo quadrante; si consente di scendere fino a -2 perche' alcuni percorsi ottimi passano brevemente per coordinate negative (per esempio per raggiungere (1,1)).

```
#include <cstdlib>
#include <iostream>
#include <queue>
#include <set>
#include <utility>

int mosseCavallo(int x, int y) {
    x = std::abs(x); y = std::abs(y);
    const int dx[] = {1, 2, 2, 1, -1, -2, -2, -1}, dy[] = {2, 1, -1, -2, -2, -1, 1, 2};
    std::queue<std::pair<int, int>> q;
    std::set<std::pair<int, int>> visti{{0, 0}};
    q.push({0, 0});
    for (int passi = 0;; ++passi)
        for (auto n = q.size(); n > 0; --n) {
            auto [cx, cy] = q.front(); q.pop();
            if (cx == x && cy == y) return passi;
            for (int d = 0; d < 8; ++d) {
                int nx = cx + dx[d], ny = cy + dy[d];
                if (nx >= -2 && ny >= -2 && visti.insert({nx, ny}).second) q.push({nx, ny});
            }
        }
}

int main() {
    std::cout << mosseCavallo(2, 1) << ' ' << mosseCavallo(5, 5) << ' '
              << mosseCavallo(1, 1) << '\n';
}
```

```
1 4 2
```

Soluzione 22.5

Ogni oggetto viene inserito nelle celle che il suo AABB tocca; si testano solo le coppie che condividono una cella. Una coppia puo' condividere piu' celle, quindi si deduplicano le coppie con un insieme. Con oggetti piccoli rispetto alla cella il costo passa da circa n^2/2 = 50 milioni di test a qualche decina di migliaia.

```
#include <algorithm>
#include <cmath>
#include <cstdint>
#include <iostream>
#include <random>
#include <unordered_map>
#include <unordered_set>
#include <vector>

struct AABB { float x0, y0, x1, y1; };
bool sovrapposti(const AABB& a, const AABB& b) {
    return a.x0 < b.x1 && b.x0 < a.x1 && a.y0 < b.y1 && b.y0 < a.y1;
}

int main() {
    std::mt19937 rng(5);
    std::uniform_real_distribution<float> pos(0, 1000), dim(1, 4);
    std::vector<AABB> ogg(10000);
    for (auto& o : ogg) {
        o.x0 = pos(rng); o.y0 = pos(rng);
        o.x1 = o.x0 + dim(rng); o.y1 = o.y0 + dim(rng);
    }

    const float LATO = 10;
    auto chiave = [](int cx, int cy) {                    // due int in un intero a 64 bit
        return (std::uint64_t(std::uint32_t(cx)) << 32) | std::uint32_t(cy);
    };
    std::unordered_map<std::uint64_t, std::vector<int>> griglia;
    for (int i = 0; i < (int)ogg.size(); ++i)
        for (int cx = int(ogg[i].x0 / LATO); cx <= int(ogg[i].x1 / LATO); ++cx)
            for (int cy = int(ogg[i].y0 / LATO); cy <= int(ogg[i].y1 / LATO); ++cy)
                griglia[chiave(cx, cy)].push_back(i);

    std::unordered_set<std::uint64_t> coppie;
    long test = 0;
    for (auto& [k, v] : griglia)
        for (std::size_t a = 0; a < v.size(); ++a)
            for (std::size_t b = a + 1; b < v.size(); ++b) {
                ++test;
                if (sovrapposti(ogg[v[a]], ogg[v[b]]))
                    coppie.insert(chiave(std::min(v[a], v[b]), std::max(v[a], v[b])));
            }

    long forzaBruta = 0;
    for (std::size_t a = 0; a < ogg.size(); ++a)
```

```
        for (std::size_t b = a + 1; b < ogg.size(); ++b) forzaBruta += sovrapposti(ogg[a], ogg[b]);
    std::cout << "collisioni: " << coppie.size() << " (forza bruta " << forzaBruta << ")\n"
              << "test: " << (test < 100000 ? "meno di 100000" : "troppi") << " contro "
              << ogg.size() * (ogg.size() - 1) / 2 << '\n';
}
```

```
collisioni: 1232 (forza bruta 1232)
test: meno di 100000 contro 49995000
```

Capitolo 23 - Librerie e motori

Soluzione 23.1

Il punto d'impatto relativo `t` va da -1 (bordo alto) a +1 (bordo basso); l'angolo e' `t * 60 gradi`. La velocita' si scompone con seno e coseno, quindi il modulo resta costante.

```
#include <cmath>
#include <iostream>
#include <numbers>

struct Vel { float vx, vy; };

Vel rimbalzo(float yPalla, float yRacchetta, float altezzaRacchetta, float velocita,
             bool versoDestra) {
    float t = (yPalla - yRacchetta) / (altezzaRacchetta / 2);    // -1 .. +1
    t = std::fmax(-1.f, std::fmin(1.f, t));
    float angolo = t * 60.f * std::numbers::pi_v<float> / 180.f;
    return {velocita * std::cos(angolo) * (versoDestra ? 1 : -1), velocita * std::sin(angolo)};
}

int main() {
    for (float y : {100.f, 140.f, 60.f}) {
        Vel v = rimbalzo(y, 100, 80, 10, true);
        std::cout << v.vx << ',' << v.vy << " |v|=" << std::hypot(v.vx, v.vy) << "  ";
    }
    std::cout << '\n';
}
```

```
10,0 |v|=10  5,8.66025 |v|=10  5,-8.66025 |v|=10
```

Soluzione 23.2

```
#include <algorithm>
#include <array>
#include <iostream>
#include <vector>

using Pezzo = std::array<std::array<bool, 4>, 4>;
using Griglia = std::vector<std::vector<int>>;

Pezzo ruota(const Pezzo& p) {                      // 90 gradi in senso orario
    Pezzo r{};
    for (int i = 0; i < 4; ++i)
        for (int j = 0; j < 4; ++j) r[j][3 - i] = p[i][j];
    return r;
}

bool collide(const Griglia& g, const Pezzo& p, int x, int y) {
    for (int i = 0; i < 4; ++i)
        for (int j = 0; j < 4; ++j) {
            if (!p[i][j]) continue;
            int r = y + i, c = x + j;
            if (c < 0 || c >= (int)g[0].size() || r >= (int)g.size()) return true;   // bordi
            if (r >= 0 && g[r][c]) return true;                                      // blocchi
        }
    return false;
}

int rimuoviRighe(Griglia& g) {
    auto piena = [](const std::vector<int>& r) {
        return std::all_of(r.begin(), r.end(), [](int x) { return x != 0; });
    };
    auto fine = std::remove_if(g.begin(), g.end(), piena);
    int n = g.end() - fine;
    g.erase(fine, g.end());
    g.insert(g.begin(), n, std::vector<int>(g.empty() ? 10 : g[0].size(), 0));  // vuote in cima
    return n;
}

int main() {
    Pezzo L{{{1, 0, 0, 0}, {1, 0, 0, 0}, {1, 1, 0, 0}, {0, 0, 0, 0}}};
    Pezzo r = ruota(L);
    for (auto& riga : r) { for (bool b : riga) std::cout << (b ? '#' : '.'); std::cout << '\n'; }
    Griglia g(4, std::vector<int>(4, 0));
    g[3] = {1, 1, 1, 1};
    g[2] = {1, 0, 1, 1};
    std::cout << collide(g, L, 0, 0) << collide(g, L, 1, -2) << collide(g, L, 3, 0) << ' '
              << rimuoviRighe(g) << " righe, ora in fondo: " << g[3][0] << g[3][1] << '\n';
}
```

```
.###
.#..
....
....
101 1 righe, ora in fondo: 10
```

Soluzione 23.3

```
// non compilare qui: richiede SFML 3 (collegare con sfml-graphics sfml-window sfml-system)
#include <SFML/Graphics.hpp>
#include "snake.hpp"                          // la classe Snake dell'Esempio 23.1

int main() {
    constexpr int CELLA = 40, W = 16, H = 10;
    sf::RenderWindow win(sf::VideoMode({W * CELLA, H * CELLA}), "Snake");
    win.setFramerateLimit(60);
    Snake gioco(W, H, std::random_device{}());
    sf::Clock orologio;
    float accumulatore = 0;
    const float PASSO = 1.f / 8;              // 8 passi al secondo

    while (win.isOpen()) {
        while (auto ev = win.pollEvent()) {
            if (ev->is<sf::Event::Closed>()) win.close();
            if (auto* k = ev->getIf<sf::Event::KeyPressed>()) {
                using K = sf::Keyboard::Key;
                if (k->code == K::Up) gioco.gira(Dir::Su);
                if (k->code == K::Down) gioco.gira(Dir::Giu);
                if (k->code == K::Left) gioco.gira(Dir::Sx);
                if (k->code == K::Right) gioco.gira(Dir::Dx);
            }
        }
        accumulatore += orologio.restart().asSeconds();
        while (accumulatore >= PASSO) { gioco.passo(); accumulatore -= PASSO; }

        win.clear(sf::Color(20, 20, 30));
        sf::RectangleShape r({CELLA - 2.f, CELLA - 2.f});
        for (const auto& c : gioco.celle()) {          // metodo aggiunto: restituisce il corpo
            r.setPosition({c.x * float(CELLA), c.y * float(CELLA)});
            r.setFillColor(sf::Color::Green);
            win.draw(r);
        }
        r.setPosition({gioco.cibo().x * float(CELLA), gioco.cibo().y * float(CELLA)});
        r.setFillColor(sf::Color::Red);
        win.draw(r);
        if (!gioco.vivo()) win.setTitle("Game over - punti: " + std::to_string(gioco.punti()));
        win.display();
    }
}
```

Alla classe `Snake` vanno aggiunti due accessori, `const std::deque<Cella>& celle() const` e `Cella cibo() const`: la logica non cambia, a conferma che separare modello e rendering paga.

Soluzione 23.4

```
#include <cmath>
#include <iostream>
#include <vector>

class Animazione {
    std::vector<float> durate_;          // durata di ciascun frame in secondi
    float totale_ = 0;
    bool loop_;
public:
    Animazione(std::vector<float> d, bool loop) : durate_(std::move(d)), loop_(loop) {
        for (float x : durate_) totale_ += x;
    }
    int frame(float t) const {
        if (loop_) t = std::fmod(t, totale_);
        else if (t >= totale_) return static_cast<int>(durate_.size()) - 1;   // resta sull'ultimo
        for (int i = 0; i < (int)durate_.size(); ++i) {
            if (t < durate_[i]) return i;
            t -= durate_[i];
        }
        return static_cast<int>(durate_.size()) - 1;
    }
};

int main() {
    Animazione corsa({0.1f, 0.1f, 0.2f}, true), salto({0.1f, 0.3f}, false);
    for (float t : {0.0f, 0.15f, 0.25f, 0.45f, 0.95f}) std::cout << corsa.frame(t);
    std::cout << ' ';
    for (float t : {0.0f, 0.2f, 5.0f}) std::cout << salto.frame(t);
    std::cout << '\n';
}
```

```
01201 011
```

Capitolo 24 - IA, rete e ottimizzazione nei giochi

Soluzione 24.1

```
#include <iostream>

enum class Stato { Pattuglia, Insegui, Attacca, Fuga };
const char* nome(Stato s) {
    switch (s) {
        case Stato::Pattuglia: return "Pattuglia";
        case Stato::Insegui: return "Insegui";
        case Stato::Attacca: return "Attacca";
        case Stato::Fuga: return "Fuga";
    }
    return "?";
}

Stato transizione(Stato s, float distanza, int salute) {
    if (salute < 30) return Stato::Fuga;                // priorita' massima
    switch (s) {
        case Stato::Pattuglia: return distanza < 10 ? Stato::Insegui : s;
        case Stato::Insegui:
            if (distanza < 2) return Stato::Attacca;
            return distanza > 15 ? Stato::Pattuglia : s;
        case Stato::Attacca: return distanza >= 2 ? Stato::Insegui : s;
        case Stato::Fuga: return salute > 60 ? Stato::Pattuglia : s;
    }
    return s;
}

int main() {
    Stato s = Stato::Pattuglia;
    float distanze[] = {20, 12, 8, 5, 1.5f, 1, 1, 3, 9, 20};
    int salute[] = {100, 100, 100, 100, 90, 50, 25, 25, 70, 70};
    for (int t = 0; t < 10; ++t) {
        s = transizione(s, distanze[t], salute[t]);
        std::cout << nome(s) << (t < 9 ? " > " : "\n");
    }
}
```

```
Pattuglia > Pattuglia > Insegui > Insegui > Attacca > Attacca > Fuga > Fuga > Pattuglia > Pattuglia
```

Soluzione 24.2

Alfa e' il miglior valore gia' garantito al massimizzatore, beta quello garantito al minimizzatore: quando `alfa >= beta` i rami rimanenti non possono influire sulla scelta e vengono tagliati. Il risultato e' identico, i nodi visitati crollano.

```
#include <algorithm>
#include <array>
#include <iostream>

using Tavola = std::array<char, 9>;
char vincitore(const Tavola& t) {
    static const int L[8][3] = {{0, 1, 2}, {3, 4, 5}, {6, 7, 8}, {0, 3, 6},
                                {1, 4, 7}, {2, 5, 8}, {0, 4, 8}, {2, 4, 6}};
    for (auto& l : L)
        if (t[l[0]] != ' ' && t[l[0]] == t[l[1]] && t[l[1]] == t[l[2]]) return t[l[0]];
    return ' ';
}

long nodi = 0;
int alfabeta(Tavola& t, char turno, int alfa, int beta) {
    ++nodi;
    if (char w = vincitore(t); w != ' ') return w == 'X' ? 1 : -1;
    if (std::find(t.begin(), t.end(), ' ') == t.end()) return 0;
    for (int i = 0; i < 9; ++i) {
        if (t[i] != ' ') continue;
        t[i] = turno;
        int v = alfabeta(t, turno == 'X' ? 'O' : 'X', alfa, beta);
        t[i] = ' ';
        if (turno == 'X') alfa = std::max(alfa, v); else beta = std::min(beta, v);
        if (alfa >= beta) break;                        // taglio
    }
    return turno == 'X' ? alfa : beta;
}

int main() {
    Tavola t;
    t.fill(' ');
    std::cout << "valore " << alfabeta(t, 'X', -2, 2) << ", nodi " << nodi << '\n';
}
```

```
valore 0, nodi 18297
```

Da 549.946 a 18.297 nodi: un fattore 30. Ordinare le mosse provando prima le piu' promettenti (centro, angoli) aumenta ancora i tagli.

Soluzione 24.3

Nel Nim con 1-3 pietre per turno perde chi si trova con un multiplo di 4: qualunque cosa prenda, l'avversario riporta il totale a un multiplo di 4. Per Predict the Winner, `dp[i][j]` = (punti del giocatore di turno) - (punti dell'avversario) sui numeri da i a j: prendendo `a[i]` il vantaggio e' `a[i] - dp[i+1][j]`.

```
#include <algorithm>
#include <iostream>
#include <vector>

bool vinceNim(int n) { return n % 4 != 0; }

bool vincePrimo(const std::vector<int>& a) {
    int n = a.size();
    std::vector<std::vector<int>> dp(n, std::vector<int>(n, 0));
    for (int i = 0; i < n; ++i) dp[i][i] = a[i];
    for (int len = 2; len <= n; ++len)
        for (int i = 0; i + len - 1 < n; ++i) {
            int j = i + len - 1;
            dp[i][j] = std::max(a[i] - dp[i + 1][j], a[j] - dp[i][j - 1]);
        }
    return dp[0][n - 1] >= 0;                          // il pareggio vale come vittoria
}

int main() {
    std::cout << vinceNim(4) << vinceNim(7) << ' '
              << vincePrimo({1, 5, 2}) << vincePrimo({1, 5, 233, 7}) << '\n';
}
```

```
01 01
```

Soluzione 24.4

```
#include <iostream>
#include <vector>

struct Snapshot { double t; float x, y; };

Snapshot interpola(const std::vector<Snapshot>& s, double ora, double ritardo = 0.1) {
    double t = ora - ritardo;                          // si disegna leggermente nel passato
    if (t <= s.front().t) return s.front();
    for (std::size_t i = 1; i < s.size(); ++i)
        if (t <= s[i].t) {
            const auto& a = s[i - 1]; const auto& b = s[i];
            float k = static_cast<float>((t - a.t) / (b.t - a.t));
            return {t, a.x + (b.x - a.x) * k, a.y + (b.y - a.y) * k};
        }
    return s.back();                                   // niente dati nuovi: si resta all'ultimo
}

int main() {
    std::vector<Snapshot> s{{1.00, 0, 0}, {1.05, 1, 0}, {1.10, 2, 1}, {1.15, 4, 1}};
    for (double ora : {1.12, 1.175, 1.20, 1.40}) {
        auto p = interpola(s, ora);
        std::cout << '(' << p.x << ',' << p.y << ") ";
    }
    std::cout << '\n';
}
```

```
(0.4,0) (1.5,0.5) (2,1) (4,1)
```

Il ritardo di 100 ms (due snapshot) garantisce quasi sempre di avere un pacchetto "dopo" il tempo di rendering anche se uno va perso; quando mancano dati si puo' estrapolare per breve tempo invece di fermarsi.

Soluzione 24.5

```
#include <cmath>
#include <iostream>
#include <random>
#include <vector>

struct Vec2 {
    float x = 0, y = 0;
    Vec2 operator+(Vec2 o) const { return {x + o.x, y + o.y}; }
    Vec2 operator-(Vec2 o) const { return {x - o.x, y - o.y}; }
    Vec2 operator*(float k) const { return {x * k, y * k}; }
    float lunghezza() const { return std::hypot(x, y); }
    Vec2 conModulo(float m) const {
        float l = lunghezza();
        return l > 0 ? *this * (m / l) : *this;
    }
};
struct Boid { Vec2 p, v; };

float allineamento(const std::vector<Boid>& b) {   // 1 = tutti nella stessa direzione
    Vec2 s;
    for (auto& x : b) s = s + x.v * (1 / x.v.lunghezza());
    return s.lunghezza() / b.size();
}

int main() {
    std::mt19937 rng(11);
    std::uniform_real_distribution<float> u(-1, 1);
    std::vector<Boid> b(50);
    for (auto& x : b) { x.p = {u(rng) * 20, u(rng) * 20}; x.v = {u(rng), u(rng)}; }
    std::cout << "prima: " << (allineamento(b) < 0.5 ? "disordinati" : "allineati");
    for (int passo = 0; passo < 500; ++passo) {
        std::vector<Boid> nuovo = b;
        for (std::size_t i = 0; i < b.size(); ++i) {
            Vec2 sep, ali, coe; int n = 0;
            for (std::size_t j = 0; j < b.size(); ++j) {
                Vec2 d = b[j].p - b[i].p;
                float dist = d.lunghezza();
                if (i == j || dist > 15) continue;
                ++n;
                ali = ali + b[j].v;                                 // allineamento
                coe = coe + b[j].p;                                 // coesione
                if (dist < 2) sep = sep - d * (1 / (dist + 0.01f)); // separazione
            }
            Vec2 acc = sep * 0.05f;
            if (n) acc = acc + (ali * (1.f / n) - b[i].v) * 0.05f
```

```
                           + (coe * (1.f / n) - b[i].p) * 0.01f;
            nuovo[i].v = (b[i].v + acc).conModulo(1.f);     // velocita' costante
            nuovo[i].p = b[i].p + nuovo[i].v * 0.1f;
        }
        b = nuovo;
    }
    std::cout << ", dopo 500 passi: " << (allineamento(b) > 0.95 ? "allineati" : "disordinati")
              << '\n';
}
```

```
prima: disordinati, dopo 500 passi: allineati
```

Due dettagli contano: il modulo della velocita' resta costante (altrimenti allineamento e coesione, che tendono a mediare le velocita', fermano lo stormo) e il raggio di vista deve essere abbastanza ampio da collegare tutti i boid, altrimenti si formano piu' stormi indipendenti con direzioni diverse.

Soluzioni - Parte 8

Capitolo 25 - Fondamenti dell'embedded con C++

Soluzione 25.1

```
#include <cstdint>
#include <iostream>

constexpr std::uint32_t impostaBit(std::uint32_t r, unsigned n) { return r | (1u << n); }
constexpr std::uint32_t azzeraBit(std::uint32_t r, unsigned n) { return r & ~(1u << n); }
constexpr std::uint32_t invertiBit(std::uint32_t r, unsigned n) { return r ^ (1u << n); }
constexpr bool leggiBit(std::uint32_t r, unsigned n) { return (r >> n) & 1u; }

static_assert(impostaBit(0b1000, 0) == 0b1001);
static_assert(azzeraBit(0b1011, 1) == 0b1001);
static_assert(invertiBit(0b1001, 3) == 0b0001);
static_assert(leggiBit(0x80000000u, 31) && !leggiBit(0x80000000u, 30));

int main() { std::cout << "tutti i controlli superati a compile time\n"; }
```

```
tutti i controlli superati a compile time
```

Si usa `1u` e non `1`: `1 << 31` su un `int` a 32 bit e' overflow di un signed (comportamento indefinito prima del C++20).

Soluzione 25.2

```
#include <array>
#include <cstddef>
#include <iostream>

template <typename T, std::size_t N>
class CodaCircolare {
    std::array<T, N> d_{};
    std::size_t testa_ = 0, n_ = 0;
public:
    bool enQueue(T x) {
        if (n_ == N) return false;
        d_[(testa_ + n_++) % N] = x;
        return true;
    }
    bool deQueue() {
        if (n_ == 0) return false;
        testa_ = (testa_ + 1) % N; --n_;
        return true;
    }
    int Front() const { return n_ ? d_[testa_] : -1; }
    int Rear() const { return n_ ? d_[(testa_ + n_ - 1) % N] : -1; }
    bool isFull() const { return n_ == N; }
};

int main() {
    CodaCircolare<int, 3> q;
    std::cout << q.enQueue(1) << q.enQueue(2) << q.enQueue(3) << q.enQueue(4) << ' ' << q.Rear()
              << q.isFull() << q.deQueue() << q.enQueue(4) << q.Rear() << q.Front() << '\n';
}
```

```
1110 311142
```

Le ultime cifre sono `Rear() = 4` e `Front() = 2`: dopo il `deQueue` la testa e' passata al secondo elemento e il 4 ha occupato la cella liberata.

Soluzione 25.3

Con una finestra di 8 = 2^3 la divisione diventa uno shift (`>> 3`), molto piu' economica su microcontrollori senza divisore hardware. La somma di 8 campioni a 12 bit sta comodamente in 16 bit (8 * 4095 = 32760).

```
#include <array>
#include <cstdint>
#include <iostream>

class MediaMobile {
    static constexpr unsigned LOG2N = 3, N = 1u << LOG2N;
    std::array<std::uint16_t, N> c_{};
    std::uint16_t somma_ = 0;
    unsigned i_ = 0, n_ = 0;
public:
    std::uint16_t aggiungi(std::uint16_t campione) {
        somma_ = somma_ - c_[i_] + campione;      // O(1): esce il vecchio, entra il nuovo
        c_[i_] = campione;
        i_ = (i_ + 1) & (N - 1);
        if (n_ < N) ++n_;
        return n_ == N ? somma_ >> LOG2N : somma_ / n_;   // finche' non e' piena: divisione vera
    }
};

int main() {
    MediaMobile m;
    for (std::uint16_t x : {4000, 4000, 4000, 4000, 4000, 4000, 4000, 4000, 0, 0, 0, 0})
        std::cout << m.aggiungi(x) << ' ';
    std::cout << '\n';
}
```

```
4000 4000 4000 4000 4000 4000 4000 4000 3500 3000 2500 2000
```

Soluzione 25.4

In un codice di Gray due valori consecutivi differiscono per **un solo bit**. Negli encoder rotativi assoluti questo evita letture spurie: se i sensori commutano in istanti leggermente diversi, il valore letto durante la transizione e' comunque uno dei due adiacenti.

```
#include <bitset>
#include <iostream>

unsigned gray(unsigned i) { return i ^ (i >> 1); }
unsigned daGray(unsigned g) {
    unsigned b = 0;
    for (; g; g >>= 1) b ^= g;          // XOR di tutti gli shift
    return b;
}

int main() {
    for (unsigned i = 0; i < 8; ++i) std::cout << std::bitset<3>(gray(i)) << ' ';
    bool ok = true;
    for (unsigned i = 0; i < 1024; ++i) ok = ok && daGray(gray(i)) == i;
    std::cout << "| inversa ok: " << ok << '\n';
}
```

```
000 001 011 010 110 111 101 100 | inversa ok: 1
```

Soluzione 25.5

`volatile` obbliga il compilatore a leggere e scrivere davvero la memoria a ogni accesso, ma `contatore++` resta tre operazioni (load, add, store): se l'interrupt arriva tra la load e la store, il suo incremento viene sovrascritto e perso. Due soluzioni corrette:

```
// non compilare qui: codice per Cortex-M con CMSIS
#include <atomic>
#include <cstdint>

// 1) sezione critica: disabilita brevemente gli interrupt
volatile std::uint32_t contatore1 = 0;
void incrementa1() {
    std::uint32_t primask = __get_PRIMASK();
    __disable_irq();
    ++contatore1;
    __set_PRIMASK(primask);            // ripristina lo stato precedente (annidabile)
}

// 2) std::atomic: su Cortex-M3/M4/M7 diventa un ciclo LDREX/STREX, senza bloccare gli IRQ
std::atomic<std::uint32_t> contatore2{0};
void incrementa2() { contatore2.fetch_add(1, std::memory_order_relaxed); }
```

Su Cortex-M0 (senza LDREX/STREX) `std::atomic` di 32 bit puo' non essere lock-free: si verifica con `static_assert(std::atomic<std::uint32_t>::is_always_lock_free)` e, se fallisce, si usa la sezione critica.

Capitolo 26 - Protocolli, RTOS e sistemi connessi

Soluzione 26.1

```
#include <cstdint>
#include <iostream>
#include <string_view>

std::uint16_t crc16ccitt(std::string_view dati) {
    std::uint16_t crc = 0xFFFF;
    for (unsigned char c : dati) {
        crc ^= std::uint16_t(c) << 8;
        for (int b = 0; b < 8; ++b)
            crc = crc & 0x8000 ? std::uint16_t((crc << 1) ^ 0x1021) : std::uint16_t(crc << 1);
    }
    return crc;
}

int main() { std::cout << std::hex << std::uppercase << "0x" << crc16ccitt("123456789") << '\n'; }
```

```
0x29B1
```

La stringa `"123456789"` e' il "check value" standard con cui si verificano tutte le implementazioni di CRC. Nei firmware si usa spesso una tabella di 256 voci per elaborare un byte alla volta, o la periferica CRC hardware del microcontrollore.

Soluzione 26.2

```
#include <concepts>
#include <cstdint>
#include <iostream>
#include <span>
#include <string>
#include <vector>

template <typename B>
concept BusI2C = requires(B b, std::uint8_t ind, std::span<const std::uint8_t> dati,
                          std::size_t n) {
    { b.scrivi(ind, dati) } -> std::same_as<bool>;
    { b.leggi(ind, n) } -> std::same_as<std::vector<std::uint8_t>>;
};

template <BusI2C Bus>
class SensoreTemperatura {                // registro 0x00: temperatura in 1/256 di grado
    Bus& bus_;
    std::uint8_t ind_;
public:
    SensoreTemperatura(Bus& b, std::uint8_t ind) : bus_(b), ind_(ind) {}
    double gradi() {
        const std::uint8_t reg[] = {0x00};
        bus_.scrivi(ind_, reg);
        auto r = bus_.leggi(ind_, 2);
        return std::int16_t((r[0] << 8) | r[1]) / 256.0;
    }
};

struct BusFinto {                         // registra le transazioni e risponde con dati fissi
    std::vector<std::string> log;
    bool scrivi(std::uint8_t ind, std::span<const std::uint8_t> d) {
        log.push_back("W " + std::to_string(ind) + " [" + std::to_string(d.size()) + "]");
        return true;
    }
    std::vector<std::uint8_t> leggi(std::uint8_t ind, std::size_t n) {
        log.push_back("R " + std::to_string(ind) + " [" + std::to_string(n) + "]");
        return {0x15, 0x80};              // 21.5 gradi
    }
};

int main() {
    BusFinto bus;
    SensoreTemperatura s(bus, 0x48);
    std::cout << s.gradi() << " C;";
    for (auto& t : bus.log) std::cout << ' ' << t;
```

```
    std::cout << '\n';
}
```

```
21.5 C; W 72 [1] R 72 [2]
```

Sul microcontrollore si passa una classe con gli stessi metodi che usa la periferica I2C reale: il driver non cambia e non ha funzioni virtuali, quindi nessun costo a runtime.

Soluzione 26.3

```
#include <iostream>

enum class Fase { VerdeAuto, Giallo, Rosso, VerdePedoni };

class Semaforo {
    Fase f_ = Fase::VerdeAuto;
    int tick_ = 0;
    bool prenotato_ = false;
public:
    void premiPulsante() { prenotato_ = true; }
    Fase fase() const { return f_; }
    void tick() {                                   // chiamata ogni 100 ms
        ++tick_;
        switch (f_) {
            case Fase::VerdeAuto:                   // almeno 10 s, poi cede se prenotato
                if (prenotato_ && tick_ >= 100) { f_ = Fase::Giallo; tick_ = 0; }
                break;
            case Fase::Giallo: if (tick_ >= 30) { f_ = Fase::Rosso; tick_ = 0; } break;
            case Fase::Rosso: if (tick_ >= 10) { f_ = Fase::VerdePedoni; tick_ = 0; } break;
            case Fase::VerdePedoni:
                if (tick_ >= 80) { f_ = Fase::VerdeAuto; tick_ = 0; prenotato_ = false; }
                break;
        }
    }
};

int main() {
    const char* nomi[] = {"verde auto", "giallo", "rosso", "verde pedoni"};
    Semaforo s;
    Fase prec = s.fase();
    for (int t = 1; t <= 600; ++t) {                // un minuto
        if (t == 50 || t == 400) s.premiPulsante();
        s.tick();
        if (s.fase() != prec) {
            prec = s.fase();
            std::cout << t / 10.0 << " s: " << nomi[static_cast<int>(prec)] << '\n';
        }
    }
}
```

```
10 s: giallo
13 s: rosso
14 s: verde pedoni
22 s: verde auto
40 s: giallo
43 s: rosso
44 s: verde pedoni
52 s: verde auto
```

La seconda prenotazione (a 40 s) arriva mentre il verde per le auto e' gia' durato piu' di 10 s, quindi il ciclo pedonale riparte subito; la prima deve attendere il verde minimo.

Soluzione 26.4

Un'architettura ragionevole:

- **Task Sensori** (priorita' alta, periodico ogni 1 s con `vTaskDelayUntil`): legge I2C, applica il filtro mediano, invia una struct `Misura` a due code.
- **Task Display** (priorita' media): riceve dalla coda e aggiorna l'OLED; non deve mai bloccare i sensori.
- **Task Log SD** (priorita' bassa): accumula le misure e scrive un blocco ogni minuto (le scritture su SD sono lente e assorbono corrente).

- **Task Rete** (priorita' bassa): gestisce Wi-Fi e MQTT; si risveglia ogni 5 minuti, si connette, pubblica le medie e spegne la radio (il Wi-Fi e' il consumo principale).
- Il bus I2C, se condiviso, e' protetto da un **mutex** (non un semaforo binario) per avere la priority inheritance ed evitare l'inversione di priorita'.
- La comunicazione avviene solo tramite code (copie dei dati, nessuna memoria condivisa); le ISR usano le varianti `FromISR` e `portYIELD_FROM_ISR`.
- In assenza di lavoro il tick-less idle di FreeRTOS lascia il processore in sleep.

Soluzione 26.5

Il `multiset` resta ordinato; l'iteratore `mid` punta all'elemento centrale e viene spostato di un passo a ogni inserimento o rimozione, a seconda del lato su cui cade l'elemento. Costo O(n log k).

```
#include <iostream>
#include <iterator>
#include <set>
#include <vector>

std::vector<double> medianeMobili(const std::vector<int>& a, int k) {
    std::multiset<int> w(a.begin(), a.begin() + k);
    auto mid = std::next(w.begin(), k / 2);
    std::vector<double> out;
    for (int i = k;; ++i) {
        out.push_back(k % 2 ? *mid : (double(*mid) + *std::prev(mid)) / 2);
        if (i == (int)a.size()) break;
        w.insert(a[i]);                                  // inserito dopo gli uguali
        if (a[i] < *mid) --mid;
        if (a[i - k] <= *mid) ++mid;                     // l'uscente e' a sinistra (o e' mid)
        w.erase(w.lower_bound(a[i - k]));
    }
    return out;
}

int main() {
    for (double m : medianeMobili({1, 3, -1, -3, 5, 3, 6, 7}, 3)) std::cout << m << ' ';
    std::cout << "| ";
    for (double m : medianeMobili({20, 21, 95, 21, 22, 22, 23}, 3)) std::cout << m << ' ';
    std::cout << '\n';
}
```

```
1 -1 -1 3 5 6 | 21 21 22 22 22
```

Il secondo caso mostra l'uso come filtro: il picco spurio 95 sparisce completamente, mentre una media mobile lo avrebbe "spalmato" su tre campioni.

Soluzioni - Parte 9

Capitolo 27 - Programmazione di sistema

Soluzione 27.1

```
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

std::string semplifica(const std::string& percorso) {
    std::vector<std::string> pila;
    std::istringstream ss(percorso);
    for (std::string parte; std::getline(ss, parte, '/');) {
        if (parte.empty() || parte == ".") continue;          // "//" e "." non cambiano nulla
        if (parte == "..") { if (!pila.empty()) pila.pop_back(); }
        else pila.push_back(parte);
    }
    std::string out;
    for (auto& p : pila) out += "/" + p;
    return out.empty() ? "/" : out;
}

int main() {
    for (auto p : {"/home/", "/../", "/home//foo/", "/a/./b/../../c/", "/a/../../b/../c//.//"})
        std::cout << semplifica(p) << ' ';
    std::cout << '\n';
}
```

```
/home / /home/foo /c /c
```

Soluzione 27.2

`read` puo' restituire meno byte di quelli chiesti (letture parziali), 0 a fine file, oppure -1 con `errno == EINTR` se un segnale l'ha interrotta: in quel caso si riprova.

```
#include <fcntl.h>
#include <unistd.h>
#include <cerrno>
#include <iostream>
#include <stdexcept>
#include <string>
#include <utility>

class Fd {
    int fd_;
public:
    explicit Fd(int fd = -1) : fd_(fd) {}
    ~Fd() { if (fd_ >= 0) ::close(fd_); }
    Fd(Fd&& o) noexcept : fd_(std::exchange(o.fd_, -1)) {}
    Fd& operator=(Fd&& o) noexcept { std::swap(fd_, o.fd_); return *this; }
    Fd(const Fd&) = delete; Fd& operator=(const Fd&) = delete;
    int get() const { return fd_; }
};

std::string leggiTutto(int fd) {
    std::string out;
    char buf[4096];
    for (;;) {
        ssize_t n = ::read(fd, buf, sizeof buf);
        if (n > 0) out.append(buf, n);
        else if (n == 0) return out;                          // fine file
        else if (errno != EINTR) throw std::runtime_error("read fallita");
    }
}

int main() {
    Fd f(::open("/proc/self/status", O_RDONLY));
    if (f.get() < 0) { std::cout << "non e' Linux\n"; return 0; }
    std::string s = leggiTutto(f.get());
    std::cout << s.substr(0, s.find('\n')).substr(0, 5) << " ... (" << (s.size() > 100) << ")\n";
}
```

```
Name: ... (1)
```

Soluzione 27.3

Il gestore puo' interrompere il programma in qualsiasi punto, anche dentro `malloc` o mentre `std::cout` aggiorna il suo buffer: richiamarli dal gestore corromperebbe il loro stato interno (non sono **async-signal-safe**). Si imposta solo un flag; `write` e `_exit` sono invece permesse.

```
#include <signal.h>
#include <unistd.h>
#include <csignal>
#include <iostream>

volatile std::sig_atomic_t fermati = 0;
extern "C" void gestore(int) { fermati = 1; }

int main() {
    struct sigaction sa{};
    sa.sa_handler = gestore;
    sigemptyset(&sa.sa_mask);
    sigaction(SIGINT, &sa, nullptr);

    int iterazioni = 0;
    while (!fermati) {
        if (++iterazioni == 3) raise(SIGINT);       // simula Ctrl+C
        usleep(1000);
    }
    std::cout << "interrotto dopo " << iterazioni << " iterazioni, pulizia eseguita\n";
}
```

```
interrotto dopo 3 iterazioni, pulizia eseguita
```

Nei programmi multithread si preferisce bloccare i segnali in tutti i thread e riceverli in un thread dedicato con `sigwait`, o con `signalfd` su Linux.

Soluzione 27.4

Le redirezioni si estraggono dai token del comando prima dell'`exec`; nel figlio si apre il file e lo si duplica su 0 o 1. La parte nuova e' tutta nel figlio:

```
#include <fcntl.h>
#include <sys/wait.h>
#include <unistd.h>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

int esegui(std::vector<std::string> argv) {
    std::string in, out;
    std::vector<std::string> args;
    for (std::size_t i = 0; i < argv.size(); ++i) {
        if (argv[i] == "<" && i + 1 < argv.size()) in = argv[++i];
        else if (argv[i] == ">" && i + 1 < argv.size()) out = argv[++i];
        else args.push_back(argv[i]);
    }
    pid_t pid = fork();
    if (pid == 0) {
        if (!in.empty()) {
            int fd = open(in.c_str(), O_RDONLY);
            if (fd < 0) { perror(in.c_str()); _exit(1); }
            dup2(fd, 0); close(fd);
        }
        if (!out.empty()) {
            int fd = open(out.c_str(), O_WRONLY | O_CREAT | O_TRUNC, 0644);
            if (fd < 0) { perror(out.c_str()); _exit(1); }
            dup2(fd, 1); close(fd);
        }
        std::vector<char*> a;
        for (auto& s : args) a.push_back(s.data());
        a.push_back(nullptr);
        execvp(a[0], a.data());
        _exit(127);
    }
    int stato;
    waitpid(pid, &stato, 0);
    return WEXITSTATUS(stato);
}

int main() {
    { std::ofstream("numeri.txt") << "3\n1\n2\n"; }
    esegui({"sort", "-n", "<", "numeri.txt", ">", "ordinati.txt"});   // sort -n < ... > ...
    std::ifstream f("ordinati.txt");
    for (std::string r; std::getline(f, r);) std::cout << r << ' ';
    std::cout << '\n';
```

```
}
```

```
1 2 3
```

In `eseguiPipeline` le redirezioni si applicano dopo i `dup2` delle pipe, cosi' che `> file` sull'ultimo comando prevalga sullo stdout ereditato.

Soluzione 27.5

La pila contiene le funzioni in esecuzione; la cima e' quella che sta usando la CPU. Quando inizia una nuova funzione, la cima smette temporaneamente di contare; `end` e' inclusivo, quindi la durata e' `t - inizio + 1`.

```
#include <iostream>
#include <sstream>
#include <stack>
#include <string>
#include <vector>

std::vector<int> tempiEsclusivi(int n, const std::vector<std::string>& log) {
    std::vector<int> tempo(n, 0);
    std::stack<int> pila;
    int precedente = 0;                                  // inizio dell'intervallo corrente
    for (const auto& riga : log) {
        std::istringstream ss(riga);
        std::string id, tipo, t;
        std::getline(ss, id, ':'); std::getline(ss, tipo, ':'); std::getline(ss, t);
        int f = std::stoi(id), ts = std::stoi(t);
        if (tipo == "start") {
            if (!pila.empty()) tempo[pila.top()] += ts - precedente;
            pila.push(f);
            precedente = ts;
        } else {
            tempo[pila.top()] += ts - precedente + 1;
            pila.pop();
            precedente = ts + 1;
        }
    }
    return tempo;
}

int main() {
    for (int x : tempiEsclusivi(2, {"0:start:0", "1:start:2", "1:end:5", "0:end:6"}))
        std::cout << x << ' ';
    std::cout << '\n';
}
```

```
3 4
```

Capitolo 28 - Concetti di sistemi operativi

Soluzione 28.1

```
#include <iostream>
#include <string>
#include <vector>

struct Processo { std::string nome; int arrivo, burst; };

int main() {
    std::vector<Processo> p{{"P1", 0, 5}, {"P2", 1, 3}, {"P3", 2, 1}, {"P4", 4, 2}};
    std::vector<bool> fatto(p.size(), false);
    int t = 0;
    double attesaTot = 0;
    for (std::size_t k = 0; k < p.size(); ++k) {
        int scelto = -1;
        for (std::size_t i = 0; i < p.size(); ++i)               // il piu' corto tra gli arrivati
            if (!fatto[i] && p[i].arrivo <= t && (scelto < 0 || p[i].burst < p[scelto].burst))
                scelto = i;
        if (scelto < 0) { --k; ++t; continue; }                  // CPU inattiva
        std::cout << "|" << t << ' ' << p[scelto].nome << ' ';
        attesaTot += t - p[scelto].arrivo;
        t += p[scelto].burst;
        fatto[scelto] = true;
    }
    std::cout << "|" << t << "\nattesa media " << attesaTot / p.size() << '\n';
}
```

```
|0 P1 |5 P3 |6 P4 |8 P2 |11
attesa media 3
```

SJF minimizza l'attesa media (3 contro 4,25 del Round Robin), ma P2 aspetta 7 unita' e un processo lungo puo' attendere all'infinito se continuano ad arrivare processi brevi (**starvation**); inoltre richiede di conoscere in anticipo la durata dei burst.

Soluzione 28.2

Ogni frame ha un bit "usato". La lancetta scorre i frame in cerchio: se il bit e' 1 lo azzera e passa oltre (seconda possibilita'), se e' 0 sostituisce quella pagina.

```
#include <iostream>
#include <vector>

int faultClock(const std::vector<int>& rif, std::size_t n) {
    std::vector<int> pagina(n, -1);
    std::vector<bool> usato(n, false);
    std::size_t lancetta = 0;
    int fault = 0;
    for (int p : rif) {
        bool trovata = false;
        for (std::size_t i = 0; i < n; ++i)
            if (pagina[i] == p) { usato[i] = true; trovata = true; break; }
        if (trovata) continue;
        ++fault;
        while (usato[lancetta]) { usato[lancetta] = false; lancetta = (lancetta + 1) % n; }
        pagina[lancetta] = p;
        usato[lancetta] = true;
        lancetta = (lancetta + 1) % n;
    }
    return fault;
}

int main() {
    std::vector<int> rif{1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    std::cout << faultClock(rif, 3) << ' ' << faultClock(rif, 4) << '\n';
}
```

```
9 10
```

Su questa sequenza Clock si comporta come FIFO (anche l'anomalia di Belady compare): ogni pagina viene "salvata" una volta sola prima di essere espulsa. Su carichi reali, con localita' dei riferimenti, si avvicina molto a LRU.

Soluzione 28.3

Bianco = non visitato, grigio = sulla pila di ricorsione, nero = completato. Un arco verso un nodo grigio chiude un ciclo; il ciclo si ricostruisce con i predecessori.

```
#include <iostream>
#include <map>
#include <string>
#include <vector>

struct Rilevatore {
    std::map<std::string, std::vector<std::string>> attende;   // A -> thread che A aspetta
    std::map<std::string, int> colore;                          // 0 bianco, 1 grigio, 2 nero
    std::vector<std::string> pila, ciclo;

    bool dfs(const std::string& u) {
        colore[u] = 1; pila.push_back(u);
        for (auto& v : attende[u]) {
            if (colore[v] == 1) {                               // ciclo trovato
                auto it = pila.begin();
                while (*it != v) ++it;
                ciclo.assign(it, pila.end());
                return true;
            }
            if (colore[v] == 0 && dfs(v)) return true;
        }
        colore[u] = 2; pila.pop_back();
        return false;
    }
};

int main() {
    Rilevatore r;
    r.attende = {{"T1", {"T2"}}, {"T2", {"T3"}}, {"T3", {"T4", "T1"}}, {"T4", {}}, {"T5", {"T1"}}};
    for (auto& [t, _] : r.attende)
        if (r.colore[t] == 0 && r.dfs(t)) break;
    std::cout << "deadlock:";
    for (auto& t : r.ciclo) std::cout << ' ' << t << " ->";
    std::cout << ' ' << r.ciclo.front() << '\n';
}
```

```
deadlock: T1 -> T2 -> T3 -> T1
```

Soluzione 28.4

Il deadlock nasce dall'attesa circolare: se ogni filosofo prende prima la forchetta sinistra, tutti possono restare con una forchetta in mano. Due rimedi: acquisire entrambe atomicamente con `std::scoped_lock` (che usa un algoritmo anti-deadlock), oppure imporre un **ordine globale** sulle risorse (sempre prima la forchetta con indice minore), rompendo il ciclo.

```
#include <algorithm>
#include <array>
#include <atomic>
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>

int main() {
    constexpr int N = 5, PASTI = 1000;
    std::array<std::mutex, N> forchette;
    std::atomic<int> pastiTotali{0};
    {
        std::vector<std::jthread> filosofi;
        for (int i = 0; i < N; ++i)
            filosofi.emplace_back([&, i] {
                int sx = i, dx = (i + 1) % N;
                for (int k = 0; k < PASTI; ++k) {
                    if (i % 2 == 0) {                    // soluzione 1: scoped_lock
                        std::scoped_lock lk(forchette[sx], forchette[dx]);
                        ++pastiTotali;
                    } else {                             // soluzione 2: ordine delle risorse
                        std::lock_guard a(forchette[std::min(sx, dx)]);
                        std::lock_guard b(forchette[std::max(sx, dx)]);
                        ++pastiTotali;
                    }
                }
            });
    }
    std::cout << "pasti: " << pastiTotali << " (nessun deadlock)\n";
}
```

```
pasti: 5000 (nessun deadlock)
```

Soluzione 28.5

```
#include <cstddef>
#include <cstdint>
#include <iostream>

class FirstFit {
    struct Blocco { std::size_t dim; bool libero; Blocco* next; };   // intestazione
    alignas(std::max_align_t) std::byte mem_[4096];
    Blocco* primo_;
    static constexpr std::size_t H = sizeof(Blocco);
public:
    FirstFit() : primo_(new (mem_) Blocco{sizeof mem_ - H, true, nullptr}) {}
    void* alloca(std::size_t n) {
        n = (n + 15) & ~std::size_t(15);                              // allineamento a 16
        for (Blocco* b = primo_; b; b = b->next) {
            if (!b->libero || b->dim < n) continue;
            if (b->dim >= n + H + 16) {                               // divide il blocco
                auto* resto = new (reinterpret_cast<std::byte*>(b) + H + n)
                    Blocco{b->dim - n - H, true, b->next};
                b->dim = n;
                b->next = resto;
            }
            b->libero = false;
            return reinterpret_cast<std::byte*>(b) + H;
        }
        return nullptr;
    }
    void libera(void* p) {
        if (!p) return;
        auto* b = reinterpret_cast<Blocco*>(static_cast<std::byte*>(p) - H);
        b->libero = true;
        for (Blocco* x = primo_; x; x = x->next)                      // fonde i vicini liberi
            while (x->libero && x->next && x->next->libero) {
                x->dim += H + x->next->dim;
                x->next = x->next->next;
            }
    }
    std::size_t bloccoLiberoMax() const {
        std::size_t m = 0;
        for (Blocco* b = primo_; b; b = b->next) if (b->libero && b->dim > m) m = b->dim;
        return m;
    }
};

int main() {
    FirstFit h;
```

```
    std::size_t iniziale = h.bloccoLiberoMax();
    void* a = h.alloca(100);
    void* b = h.alloca(200);
    void* c = h.alloca(50);
    h.libera(b);
    void* d = h.alloca(150);                      // entra nel buco lasciato da b
    std::cout << (d == b) << ' ';
    h.libera(a); h.libera(c); h.libera(d);
    std::cout << (h.bloccoLiberoMax() == iniziale) << ' ' << (h.alloca(5000) == nullptr) << '\n';
}
```

```
1 1 1
```

Dopo aver liberato tutto, la fusione ricostruisce un unico blocco grande quanto quello iniziale: senza fusione la memoria resterebbe **frammentata** in blocchi piccoli.

Capitolo 29 - Costruire un sistema operativo

Soluzione 29.1

```
#include <cstdint>
#include <iostream>

int main() {
    std::uint64_t v = 0xFFFF800000201ABC;
    unsigned pml4 = (v >> 39) & 0x1FF, pdpt = (v >> 30) & 0x1FF;
    unsigned pd = (v >> 21) & 0x1FF, pt = (v >> 12) & 0x1FF, offset = v & 0xFFF;
    std::cout << "PML4 " << pml4 << ", PDPT " << pdpt << ", PD " << pd << ", PT " << pt
              << ", offset 0x" << std::hex << offset << '\n';
}
```

```
PML4 256, PDPT 0, PD 1, PT 1, offset 0xabc
```

L'indice PML4 256 indica la meta' alta dello spazio di indirizzamento, dove i kernel x86_64 mappano se stessi; i bit 48-63 devono essere copie del bit 47 (indirizzo **canonico**), altrimenti la CPU genera un'eccezione.

Soluzione 29.2

```
#include <cstdarg>
#include <cstdio>

void put(char c) { std::putchar(c); }               // nel kernel: Console::put o la seriale

void stampaNumero(unsigned long v, unsigned base) {
    char buf[32];
    int i = 0;
    do { buf[i++] = "0123456789abcdef"[v % base]; v /= base; } while (v);
    while (i) put(buf[--i]);
}

void kprintf(const char* f, ...) {
    va_list ap;
    va_start(ap, f);
    for (; *f; ++f) {
        if (*f != '%') { put(*f); continue; }
        switch (*++f) {
            case 'd': {
                int x = va_arg(ap, int);
                if (x < 0) { put('-'); stampaNumero(0ul - static_cast<unsigned long>(x), 10); }
                else stampaNumero(x, 10);
                break;
            }
            case 'u': stampaNumero(va_arg(ap, unsigned), 10); break;
            case 'x': stampaNumero(va_arg(ap, unsigned), 16); break;
            case 's': for (const char* s = va_arg(ap, const char*); *s; ++s) put(*s); break;
            case 'c': put(static_cast<char>(va_arg(ap, int))); break;
            case '%': put('%'); break;
            case '\0': --f; break;                  // '%' finale: si ferma
            default: put('%'); put(*f);
        }
    }
    va_end(ap);
}

int main() {
    kprintf("memoria: %u KB, base 0x%x, %s %c%d%%\n", 640u, 0xB8000u, "carico", '-', -42);
}
```

```
memoria: 640 KB, base 0xb8000, carico --42%
```

Il valore negativo si converte prima a `unsigned long` (`0ul - x`): cosi' anche `INT_MIN` viene stampato correttamente, mentre `-x` sarebbe overflow.

Soluzione 29.3

`swapcontext` salva i registri del contesto corrente e carica quelli del successivo: e' esattamente cio' che fa il cambio di contesto di un kernel, ma in user space. Ogni "thread" ha il proprio stack.

```
#include <ucontext.h>
#include <iostream>
#include <vector>

ucontext_t principale, ctx[3];
int corrente = 0;
bool finito[3] = {};

void cedi() {                                   // scheduler round robin cooperativo
    int prec = corrente;
    do corrente = (corrente + 1) % 3; while (finito[corrente] && corrente != prec);
    if (corrente != prec) swapcontext(&ctx[prec], &ctx[corrente]);
}

void lavoro(int id, int passi) {
    for (int i = 0; i < passi; ++i) { std::cout << "T" << id << ":" << i << ' '; cedi(); }
    finito[id] = true;
    cedi();                                     // non ritorna se ci sono altri thread
}

int main() {
    std::vector<std::vector<char>> stack(3, std::vector<char>(64 * 1024));
    int passi[3] = {2, 3, 1};
    for (int i = 0; i < 3; ++i) {
        getcontext(&ctx[i]);
        ctx[i].uc_stack.ss_sp = stack[i].data();
        ctx[i].uc_stack.ss_size = stack[i].size();
        ctx[i].uc_link = &principale;           // dove andare quando la funzione termina
        makecontext(&ctx[i], reinterpret_cast<void (*)()>(lavoro), 2, i, passi[i]);
    }
    swapcontext(&principale, &ctx[0]);
    std::cout << "\nfine\n";
}
```

```
T0:0 T1:0 T2:0 T0:1 T1:1 T1:2
fine
```

> **Nota**
>
> AddressSanitizer puo' segnalare falsi positivi con i cambi di stack di `swapcontext`; se succede, compila questo esempio senza `-fsanitize=address`.

Soluzione 29.4

Un blocco di ordine k ha dimensione `4 KB * 2^k`. Le liste libere sono una per ordine; il "compagno" di un blocco all'offset `o` di dimensione `d` si trova all'offset `o ^ d`.

```
#include <cstddef>
#include <iostream>
#include <optional>
#include <set>

class Buddy {
    static constexpr std::size_t MIN = 4096, ORDINI = 9;     // 4 KB .. 1 MB (2^8 * 4 KB)
    std::set<std::size_t> liberi_[ORDINI];                   // offset dei blocchi liberi
public:
    Buddy() { liberi_[ORDINI - 1].insert(0); }
    static std::size_t ordine(std::size_t n) {
        std::size_t k = 0;
        while ((MIN << k) < n) ++k;
        return k;
    }
    std::optional<std::size_t> alloca(std::size_t n) {
        std::size_t k = ordine(n), j = k;
        while (j < ORDINI && liberi_[j].empty()) ++j;          // il piu' piccolo disponibile
        if (j == ORDINI) return std::nullopt;
        std::size_t off = *liberi_[j].begin();
        liberi_[j].erase(liberi_[j].begin());
        while (j > k) { --j; liberi_[j].insert(off + (MIN << j)); }   // divide il blocco
        return off;
    }
    void libera(std::size_t off, std::size_t n) {
        std::size_t k = ordine(n);
        for (; k + 1 < ORDINI; ++k) {
            std::size_t compagno = off ^ (MIN << k);
            auto it = liberi_[k].find(compagno);
            if (it == liberi_[k].end()) break;                  // compagno occupato: stop
            liberi_[k].erase(it);
            off = std::min(off, compagno);                      // fusione
        }
        liberi_[k].insert(off);
    }
    bool tuttoLibero() const { return liberi_[ORDINI - 1].count(0) == 1; }
};

int main() {
    Buddy b;
    auto a = b.alloca(4096), c = b.alloca(10000), d = b.alloca(4096);
    std::cout << *a << ' ' << *c << ' ' << *d << ' ';
    b.libera(*a, 4096); b.libera(*d, 4096); b.libera(*c, 10000);
    std::cout << b.tuttoLibero() << '\n';
}
```

```
0 16384 4096 1
```

La richiesta di 10.000 byte viene arrotondata a 16 KB: e' la **frammentazione interna** tipica del buddy allocator, ripagata da allocazione e fusione in O(log n). Linux lo usa per le pagine fisiche e vi appoggia sopra l'allocatore slab per gli oggetti piccoli.

Soluzioni - Parte 10

Capitolo 30 - Fondamenti di networking

Soluzione 30.1

Con UDP non ci sono connessione, `listen` o `accept`: ogni `recvfrom` restituisce **un datagramma intero** (o niente) insieme all'indirizzo del mittente, che si usa per rispondere. In compenso i datagrammi possono perdersi, duplicarsi o arrivare in ordine diverso: affidabilita' e ordinamento, se servono, li deve fornire l'applicazione.

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <unistd.h>
#include <iostream>
#include <string>
#include <thread>

int main() {
    int srv = socket(AF_INET, SOCK_DGRAM, 0);
    sockaddr_in a{};
    a.sin_family = AF_INET;
    a.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    bind(srv, (sockaddr*)&a, sizeof a);
    socklen_t len = sizeof a;
    getsockname(srv, (sockaddr*)&a, &len);

    std::jthread server([srv] {
        for (int i = 0; i < 2; ++i) {
            char buf[512];
            sockaddr_in mitt{};
            socklen_t l = sizeof mitt;
            ssize_t n = recvfrom(srv, buf, sizeof buf, 0, (sockaddr*)&mitt, &l);
            std::string r = "ricevuto " + std::to_string(n) + " byte";
            sendto(srv, r.data(), r.size(), 0, (sockaddr*)&mitt, l);   // risponde al mittente
        }
    });

    int cli = socket(AF_INET, SOCK_DGRAM, 0);
    for (std::string msg : {"datagramma uno", "due"}) {
        sendto(cli, msg.data(), msg.size(), 0, (sockaddr*)&a, sizeof a);
        char buf[512];
        ssize_t n = recvfrom(cli, buf, sizeof buf, 0, nullptr, nullptr);
        std::cout << std::string(buf, n) << '\n';
    }
    server.join();
    close(cli); close(srv);
}
```

```
ricevuto 14 byte
ricevuto 3 byte
```

Soluzione 30.2

```
#include <sys/socket.h>
#include <unistd.h>
#include <cerrno>
#include <cstring>
#include <iostream>

bool riceviTutto(int fd, char* buf, std::size_t n) {
    std::size_t letti = 0;
    while (letti < n) {
        ssize_t r = recv(fd, buf + letti, n - letti, 0);
        if (r > 0) letti += r;
        else if (r == 0) return false;                    // connessione chiusa prima del tempo
        else if (errno != EINTR) return false;            // errore vero
    }
    return true;
}

int main() {
    int sv[2];
    socketpair(AF_UNIX, SOCK_STREAM, 0, sv);              // due socket gia' collegati
    write(sv[0], "ab", 2);
    write(sv[0], "cdef", 4);                              // arriva in due pezzi
    char buf[7] = {};
    std::cout << riceviTutto(sv[1], buf, 6) << ' ' << buf << ' ';
    close(sv[0]);
    std::cout << riceviTutto(sv[1], buf, 1) << '\n';      // l'altro lato ha chiuso
    close(sv[1]);
}
```

```
1 abcdef 0
```

Soluzione 30.3

```
#include <arpa/inet.h>
#include <netdb.h>
#include <iostream>
#include <memory>
#include <set>
#include <string>

int main() {
    addrinfo hints{};
    hints.ai_family = AF_UNSPEC;                          // IPv4 e IPv6
    hints.ai_socktype = SOCK_STREAM;
    addrinfo* raw = nullptr;
    if (int e = getaddrinfo("localhost", "80", &hints, &raw); e != 0) {
        std::cout << "errore: " << gai_strerror(e) << '\n';
        return 1;
    }
    std::unique_ptr<addrinfo, decltype(&freeaddrinfo)> risultati(raw, &freeaddrinfo);

    std::set<std::string> indirizzi;                      // ordinati e senza duplicati
    for (addrinfo* p = risultati.get(); p; p = p->ai_next) {
        char testo[INET6_ADDRSTRLEN];
        const void* a = p->ai_family == AF_INET
            ? (const void*)&((sockaddr_in*)p->ai_addr)->sin_addr
            : (const void*)&((sockaddr_in6*)p->ai_addr)->sin6_addr;
        inet_ntop(p->ai_family, a, testo, sizeof testo);
        indirizzi.insert(testo);
    }
    std::cout << "localhost IPv4: " << indirizzi.count("127.0.0.1") << '\n';
}
```

```
localhost IPv4: 1
```

A seconda della configurazione del sistema compare anche `::1` (IPv6). Un client robusto prova gli indirizzi nell'ordine restituito finche' una `connect` riesce.

Soluzione 30.4

Si sommano le parole di 16 bit in un accumulatore a 32 bit, si ripiegano i riporti nei 16 bit bassi e si complementa. Poiche' il campo checksum fa parte dei dati, il ricalcolo su un pacchetto integro da' 0.

```
#include <cstdint>
#include <cstdio>
#include <vector>

std::uint16_t checksum(const std::vector<std::uint8_t>& d) {
    std::uint32_t somma = 0;
    for (std::size_t i = 0; i + 1 < d.size(); i += 2) somma += d[i] << 8 | d[i + 1];
    if (d.size() % 2) somma += d.back() << 8;              // byte dispari: riempito con 0
    while (somma >> 16) somma = (somma & 0xFFFF) + (somma >> 16);
    return static_cast<std::uint16_t>(~somma);
}

int main() {
    std::vector<std::uint8_t> ip{0x45, 0x00, 0x00, 0x73, 0x00, 0x00, 0x40, 0x00, 0x40, 0x11,
                                 0x00, 0x00, 0xC0, 0xA8, 0x00, 0x01, 0xC0, 0xA8, 0x00, 0xC7};
    std::uint16_t c = checksum(ip);
    ip[10] = c >> 8; ip[11] = c & 0xFF;                   // scrive il checksum nell'intestazione
    std::printf("checksum 0x%04X, verifica 0x%04X\n", c, checksum(ip));
}
```

```
checksum 0xB861, verifica 0x0000
```

Capitolo 31 - Server scalabili

Soluzione 31.1

Il prefisso di lunghezza rende la codifica indipendente dal contenuto: anche una stringa che contiene `#` o cifre viene ricostruita correttamente.

```
#include <iostream>
#include <string>
#include <vector>

std::string codifica(const std::vector<std::string>& v) {
    std::string out;
    for (const auto& s : v) out += std::to_string(s.size()) + '#' + s;
    return out;
}

std::vector<std::string> decodifica(const std::string& s) {
    std::vector<std::string> out;
    for (std::size_t i = 0; i < s.size();) {
        std::size_t h = s.find('#', i);
        std::size_t n = std::stoul(s.substr(i, h - i));
        out.push_back(s.substr(h + 1, n));
        i = h + 1 + n;
    }
    return out;
}

int main() {
    std::vector<std::string> v{"ciao", "", "3#x", "a b"};
    auto c = codifica(v);
    std::cout << c << " -> " << (decodifica(c) == v ? "ok" : "errore") << '\n';
}
```

```
4#ciao0#3#3#x3#a b -> ok
```

Soluzione 31.2

```
#include <iostream>
#include <optional>
#include <string>

std::optional<std::string> estraiRichiesta(std::string& buf) {
    auto fineTesta = buf.find("\r\n\r\n");
    if (fineTesta == std::string::npos) return std::nullopt;     // intestazioni incomplete
    std::size_t corpo = 0;
    auto p = buf.find("Content-Length:");
    if (p != std::string::npos && p < fineTesta) corpo = std::stoul(buf.substr(p + 15));
    std::size_t totale = fineTesta + 4 + corpo;
    if (buf.size() < totale) return std::nullopt;                // corpo incompleto
    std::string r = buf.substr(0, totale);
    buf.erase(0, totale);                                        // il resto e' la prossima
    return r;
}

int main() {
    std::string buf = "POST /a HTTP/1.1\r\nContent-Length: 5\r\n\r\nciao!"
                      "GET /b HTTP/1.1\r\nHost: x\r\n\r\n"
                      "POST /c HTTP/1.1\r\nContent-Length: 10\r\n\r\nparz";
    while (auto r = estraiRichiesta(buf)) std::cout << r->substr(0, r->find('\r')) << '\n';
    std::cout << "in attesa: " << buf.size() << " byte\n";
}
```

```
POST /a HTTP/1.1
GET /b HTTP/1.1
in attesa: 44 byte
```

La ricerca di `Content-Length` qui e' semplificata: un parser reale confronta i nomi delle intestazioni senza distinguere maiuscole e minuscole e valida il valore.

Soluzione 31.3

L'array `tempi[i]` ricorda a quale secondo si riferisce il contatore `colpi[i]` (con `i = t % 300`): se il secondo e' cambiato, il contatore vecchio viene azzerato.

```
#include <array>
#include <iostream>

class HitCounter {
    std::array<int, 300> tempi_{}, colpi_{};
public:
    void hit(int t) {
        int i = t % 300;
        if (tempi_[i] != t) { tempi_[i] = t; colpi_[i] = 0; }
        ++colpi_[i];
    }
    int getHits(int t) const {
        int tot = 0;
        for (int i = 0; i < 300; ++i) if (t - tempi_[i] < 300) tot += colpi_[i];
        return tot;
    }
};

int main() {
    HitCounter h;
    h.hit(1); h.hit(2); h.hit(3); h.hit(3);
    std::cout << h.getHits(4) << ' ';
    h.hit(300);
    std::cout << h.getHits(300) << ' ' << h.getHits(301) << ' ' << h.getHits(700) << '\n';
}
```

```
4 5 4 0
```

Soluzione 31.4

Ogni cella conserva la lista delle coppie `(idSnap, valore)` in cui e' stata modificata; `get` cerca con ricerca binaria l'ultima modifica con `idSnap <= richiesto`. Lo spazio e' proporzionale alle modifiche, non a `numero di snapshot * dimensione`.

```
#include <algorithm>
#include <iostream>
#include <limits>
#include <utility>
#include <vector>

class SnapshotArray {
    std::vector<std::vector<std::pair<int, int>>> storia_;
    int snap_ = 0;
public:
    explicit SnapshotArray(int n) : storia_(n, {{0, 0}}) {}
    void set(int i, int v) {
        auto& h = storia_[i];
        if (h.back().first == snap_) h.back().second = v;      // stessa versione: sovrascrive
        else h.emplace_back(snap_, v);
    }
    int snap() { return snap_++; }
    int get(int i, int id) const {
        const auto& h = storia_[i];
        auto chiave = std::pair{id, std::numeric_limits<int>::max()};
        auto it = std::upper_bound(h.begin(), h.end(), chiave);
        return std::prev(it)->second;
    }
};

int main() {
    SnapshotArray a(3);
    a.set(0, 5);
    int s0 = a.snap();
    a.set(0, 6);
    a.set(1, 9);
    int s1 = a.snap();
    std::cout << a.get(0, s0) << ' ' << a.get(0, s1) << ' ' << a.get(1, s0) << ' ' << a.get(1, s1)
              << '\n';
}
```

```
5 6 0 9
```

Soluzione 31.5

Per ogni connessione servono un buffer di ricezione (con il `Deframer` dell'Esempio 31.1) e un buffer di invio. Lo schema dell'evento di scrittura e' il seguente:

```
// non compilare qui: frammento da inserire nel server dell'Esempio 31.2
struct Connessione { Deframer in; std::string out; };
std::unordered_map<int, Connessione> conn;

void accoda(int ep, int fd, const std::string& msg) {
    auto& c = conn[fd];
    bool eraVuoto = c.out.empty();
    c.out += incapsula(msg);
    if (eraVuoto) svuota(ep, fd);                // prova subito a inviare
}

void svuota(int ep, int fd) {
    auto& c = conn[fd];
    while (!c.out.empty()) {
        ssize_t n = send(fd, c.out.data(), c.out.size(), MSG_NOSIGNAL);
        if (n > 0) { c.out.erase(0, n); continue; }
        if (errno == EAGAIN) {                   // buffer del kernel pieno: attendi EPOLLOUT
            epoll_event e{EPOLLIN | EPOLLOUT, {.fd = fd}};
            epoll_ctl(ep, EPOLL_CTL_MOD, fd, &e);
            return;
        }
        chiudi(ep, fd); return;                  // errore vero
    }
    epoll_event e{EPOLLIN, {.fd = fd}};          // tutto inviato: non serve piu' EPOLLOUT
    epoll_ctl(ep, EPOLL_CTL_MOD, fd, &e);
}

// nel ciclo eventi:
//   if (ev.events & EPOLLIN) {
//       leggi in un buffer e chiama conn[fd].in.ricevi(...);
//       while (auto m = conn[fd].in.prossimo()) accoda(ep, fd, elabora(*m));
//   }
//   if (ev.events & EPOLLOUT) svuota(ep, fd);
```

`EPOLLOUT` va registrato **solo** quando c'e' qualcosa da inviare: un socket e' quasi sempre scrivibile e lasciarlo attivo farebbe girare il ciclo a vuoto al 100% di CPU. Conviene anche limitare la dimensione di `out` (backpressure): un client che non legge non deve far crescere la memoria del server all'infinito.

Capitolo 32 - Algoritmi e progetti di rete

Soluzione 32.1

```
#include <iostream>
#include <numeric>
#include <vector>

struct DSU {
    std::vector<int> p;
    explicit DSU(int n) : p(n) { std::iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unisci(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        p[a] = b;
        return true;
    }
};

int operazioni(int n, const std::vector<std::vector<int>>& cavi) {
    if ((int)cavi.size() < n - 1) return -1;          // cavi insufficienti
    DSU d(n);
    int componenti = n;
    for (auto& c : cavi) if (d.unisci(c[0], c[1])) --componenti;
    return componenti - 1;
}

int main() {
    std::cout << operazioni(4, {{0, 1}, {0, 2}, {1, 2}}) << ' '
              << operazioni(6, {{0, 1}, {0, 2}, {0, 3}, {1, 2}, {1, 3}}) << ' '
              << operazioni(6, {{0, 1}, {0, 2}, {0, 3}, {1, 2}}) << '\n';
}
```

```
1 2 -1
```

Soluzione 32.2

Il leaky bucket mette in coda le richieste e le fa uscire a velocita' costante: **livella** il traffico e scarta solo quando la coda e' piena, ma non ammette picchi. Il token bucket accumula credito nei periodi di quiete e permette raffiche fino alla capacita'.

```
#include <algorithm>
#include <iostream>

class LeakyBucket {
    double capacita_, velocita_, livello_ = 0, ultimo_ = 0;
public:
    LeakyBucket(double c, double v) : capacita_(c), velocita_(v) {}
    bool consenti(double ora) {
        livello_ = std::max(0.0, livello_ - (ora - ultimo_) * velocita_);   // la coda si svuota
        ultimo_ = ora;
        if (livello_ + 1 > capacita_) return false;
        livello_ += 1;
        return true;
    }
};

class TokenBucket {
    double capacita_, velocita_, gettoni_, ultimo_ = 0;
public:
    TokenBucket(double c, double v) : capacita_(c), velocita_(v), gettoni_(c) {}
    bool consenti(double ora) {
        gettoni_ = std::min(capacita_, gettoni_ + (ora - ultimo_) * velocita_);
        ultimo_ = ora;
        if (gettoni_ < 1) return false;
        gettoni_ -= 1;
        return true;
    }
};

int main() {
    LeakyBucket lb(5, 2);
    TokenBucket tb(5, 2);
    int okL = 0, okT = 0;
    for (int i = 0; i < 10; ++i) { okL += lb.consenti(0); okT += tb.consenti(0); }   // raffica
    std::cout << "raffica di 10: leaky " << okL << ", token " << okT << '\n';
}
```

```
raffica di 10: leaky 5, token 5
```

Con la stessa capacita' entrambi accettano 5 richieste della raffica, ma la differenza e' **quando** vengono servite: nel leaky bucket le 5 accettate restano in coda ed escono una ogni 0,5 s (ritardo fino a 2,5 s), nel token bucket vengono servite subito. Il primo protegge un backend che non regge i picchi, il secondo offre bassa latenza tollerando raffiche brevi.

Soluzione 32.3

```
#include <array>
#include <cstdint>
#include <cstdio>
#include <string_view>

constexpr std::array<std::uint32_t, 256> tabellaCrc() {
    std::array<std::uint32_t, 256> t{};
    for (std::uint32_t i = 0; i < 256; ++i) {
        std::uint32_t c = i;
        for (int k = 0; k < 8; ++k) c = c & 1 ? 0xEDB88320u ^ (c >> 1) : c >> 1;
        t[i] = c;
    }
    return t;
}
constexpr auto TABELLA = tabellaCrc();                    // calcolata dal compilatore

constexpr std::uint32_t crc32(std::string_view s) {
    std::uint32_t c = 0xFFFFFFFFu;
    for (unsigned char b : s) c = TABELLA[(c ^ b) & 0xFF] ^ (c >> 8);
    return c ^ 0xFFFFFFFFu;
}
static_assert(crc32("123456789") == 0xCBF43926);

int main() { std::printf("0x%08X\n", crc32("123456789")); }
```

```
0xCBF43926
```

Soluzione 32.4

```
#include <algorithm>
#include <iostream>
#include <vector>

struct Richiesta { int arrivo, durata; };

std::vector<int> simula(const std::vector<Richiesta>& r, bool leastConn) {
    std::vector<std::vector<int>> fine(3);       // istanti di fine delle richieste attive
    std::vector<int> lavoro(3, 0);
    int rr = 0;
    for (const auto& q : r) {
        for (auto& f : fine)                                  // rimuove le richieste concluse
            std::erase_if(f, [&](int t) { return t <= q.arrivo; });
        int b;
        if (leastConn)
            b = std::ranges::min_element(fine, {}, &std::vector<int>::size) - fine.begin();
        else b = rr++ % 3;
        fine[b].push_back(q.arrivo + q.durata);
        lavoro[b] += q.durata;
    }
    return lavoro;
}

int main() {
    std::vector<Richiesta> r;
    for (int i = 0; i < 20; ++i) r.push_back({i, i % 3 == 0 ? 30 : 1});   // una lenta ogni tre
    for (bool lc : {false, true}) {
        auto l = simula(r, lc);
        std::cout << (lc ? "least connections:" : "round robin:      ");
        for (int x : l) std::cout << ' ' << x;
        std::cout << '\n';
    }
}
```

```
round robin:       210 7 6
least connections: 94 65 64
```

Con round robin tutte le richieste lente (una ogni tre) finiscono sul primo backend, perche' il periodo delle richieste coincide con il numero di server: e' un caso patologico ma realistico. Least connections tiene conto delle richieste ancora attive e distribuisce il lavoro molto meglio (94, 65, 64). Nella pratica si usano anche varianti pesate (per server di potenza diversa) o "power of two choices" (si scelgono a caso due server e si prende il meno carico), che evita di dover conoscere lo stato di tutti.

Soluzione 32.5

Modello semplificato a "giri": a ogni giro il mittente invia l'intera finestra a partire da `base`; il ricevitore accetta solo il pacchetto che si aspetta (in ordine) e scarta gli altri; alla fine del giro l'ACK cumulativo sposta `base`, e se qualcosa e' andato perso il timeout fa ripartire l'invio da li'.

```
#include <iostream>

int main() {
    const int N = 20, FINESTRA = 4;
    int base = 0, trasmissioni = 0, contatore = 0, giri = 0;
    int atteso = 0;                                    // prossimo numero atteso dal ricevitore
    while (base < N) {
        ++giri;
        for (int seq = base; seq < base + FINESTRA && seq < N; ++seq) {
            ++trasmissioni;
            bool perso = ++contatore % 5 == 0;         // il canale perde 1 pacchetto su 5
            if (!perso && seq == atteso) ++atteso;     // accettato solo se in ordine
        }
        base = atteso;                                 // ACK cumulativo (+ timeout se serve)
    }
    std::cout << "consegnati " << atteso << " pacchetti in " << giri << " giri con "
              << trasmissioni << " trasmissioni\n";
}
```

```
consegnati 20 pacchetti in 10 giri con 39 trasmissioni
```

Ogni perdita costringe a ritrasmettere anche i pacchetti successivi gia' inviati nella finestra, che il ricevitore aveva scartato perche' fuori ordine: e' il limite di Go-Back-N. Selective Repeat conserva i pacchetti fuori ordine e ritrasmette solo quelli persi, al prezzo di buffer anche nel ricevitore; TCP moderno usa ACK selettivi (SACK) per lo stesso motivo.

Soluzioni - Parti 11 e 12

Capitolo 33 - Usare database da C++

Soluzione 33.1

```
// compila: -lsqlite3
#include <sqlite3.h>
#include <iostream>

int stampa(void*, int, char** v, char**) { std::cout << v[0] << ' '; return 0; }

int main() {
    sqlite3* db;
    sqlite3_open(":memory:", &db);
    sqlite3_exec(db, R"(
        CREATE TABLE clienti(id INTEGER PRIMARY KEY, nome TEXT);
        CREATE TABLE ordini(id INTEGER PRIMARY KEY, cliente INT REFERENCES clienti(id));
        INSERT INTO clienti VALUES (1, 'Joe'), (2, 'Henry'), (3, 'Sam'), (4, 'Max');
        INSERT INTO ordini VALUES (1, 3), (2, 1);)", nullptr, nullptr, nullptr);
    sqlite3_exec(db, "SELECT c.nome FROM clienti c LEFT JOIN ordini o ON o.cliente = c.id"
                     " WHERE o.id IS NULL ORDER BY c.nome", stampa, nullptr, nullptr);
    std::cout << "| ";
    sqlite3_exec(db, "SELECT nome FROM clienti c WHERE NOT EXISTS"
                     " (SELECT 1 FROM ordini o WHERE o.cliente = c.id) ORDER BY nome",
                 stampa, nullptr, nullptr);
    std::cout << '\n';
    sqlite3_close(db);
}
```

```
Henry Max | Henry Max
```

Il `LEFT JOIN` conserva tutti i clienti e riempie con `NULL` le colonne degli ordini mancanti; `NOT EXISTS` esprime l'intenzione in modo piu' diretto e molti ottimizzatori lo eseguono come **anti-join**, fermandosi al primo ordine trovato.

Soluzione 33.2

```
// compila: -lsqlite3
#include <sqlite3.h>
#include <iostream>
#include <optional>
#include <string>

std::optional<std::string> testoOpzionale(sqlite3_stmt* st, int c) {
    if (sqlite3_column_type(st, c) == SQLITE_NULL) return std::nullopt;
    return reinterpret_cast<const char*>(sqlite3_column_text(st, c));
}

int main() {
    sqlite3* db;
    sqlite3_open(":memory:", &db);
    sqlite3_exec(db, "CREATE TABLE p(nome TEXT, email TEXT);"
                     "INSERT INTO p VALUES ('Anna', 'anna@x.it'), ('Ugo', NULL);",
                 nullptr, nullptr, nullptr);
    sqlite3_stmt* st;
    sqlite3_prepare_v2(db, "SELECT nome, email FROM p", -1, &st, nullptr);
    while (sqlite3_step(st) == SQLITE_ROW)
        std::cout << *testoOpzionale(st, 0) << ": " << testoOpzionale(st, 1).value_or("(nessuna)")
                  << '\n';
    sqlite3_finalize(st);
    sqlite3_close(db);
}
```

```
Anna: anna@x.it
Ugo: (nessuna)
```

Nel wrapper dell'Esempio 33.1 diventa un metodo di `Statement`. Senza questo controllo, `sqlite3_column_text` su un `NULL` restituisce un puntatore nullo, e costruire una `std::string` da

`nullptr` e' comportamento indefinito.

Soluzione 33.3

```
// compila: -lsqlite3
#include <sqlite3.h>
#include <iostream>
#include <string>

int conta(void* n, int, char**, char**) { ++*static_cast<int*>(n); return 0; }

int main() {
    sqlite3* db;
    sqlite3_open(":memory:", &db);
    sqlite3_exec(db, "CREATE TABLE utenti(nome TEXT, segreto TEXT);"
                     "INSERT INTO utenti VALUES ('anna', 's1'), ('bruno', 's2'), ('carla', 's3');",
                 nullptr, nullptr, nullptr);
    std::string input = "x' OR '1'='1";

    std::string sql = "SELECT * FROM utenti WHERE nome = '" + input + "'";   // VULNERABILE
    int righe = 0;
    sqlite3_exec(db, sql.c_str(), conta, &righe, nullptr);
    std::cout << "concatenazione: " << sql << "\n  -> righe restituite: " << righe << '\n';

    sqlite3_stmt* st;
    sqlite3_prepare_v2(db, "SELECT * FROM utenti WHERE nome = ?", -1, &st, nullptr);
    sqlite3_bind_text(st, 1, input.c_str(), -1, SQLITE_TRANSIENT);
    righe = 0;
    while (sqlite3_step(st) == SQLITE_ROW) ++righe;
    sqlite3_finalize(st);
    std::cout << "parametro: righe restituite: " << righe << '\n';
    sqlite3_close(db);
}
```

```
concatenazione: SELECT * FROM utenti WHERE nome = 'x' OR '1'='1'
  -> righe restituite: 3
parametro: righe restituite: 0
```

Concatenando, l'apice nell'input chiude la stringa SQL e il resto diventa **codice**: la condizione `'1'='1'` e' sempre vera e la query restituisce i segreti di tutti. Con il parametro l'input resta un **dato**: si cerca un utente che si chiama letteralmente `x' OR '1'='1`.

Soluzione 33.4

```
// compila: -lsqlite3
#include <sqlite3.h>
#include <chrono>
#include <cstdio>
#include <iostream>

double inserisci(sqlite3* db, int n, bool transazione) {
    auto t0 = std::chrono::steady_clock::now();
    if (transazione) sqlite3_exec(db, "BEGIN", nullptr, nullptr, nullptr);
    sqlite3_stmt* st;
    sqlite3_prepare_v2(db, "INSERT INTO t(k, v) VALUES(?, ?)", -1, &st, nullptr);
    for (int i = 0; i < n; ++i) {
        sqlite3_bind_int(st, 1, i);
        sqlite3_bind_int(st, 2, i * 7 % 1000);
        sqlite3_step(st);
        sqlite3_reset(st);
    }
    sqlite3_finalize(st);
    if (transazione) sqlite3_exec(db, "COMMIT", nullptr, nullptr, nullptr);
    return std::chrono::duration<double>(std::chrono::steady_clock::now() - t0).count();
}

int piano(void*, int, char** v, char**) { std::cout << "  " << v[3] << '\n'; return 0; }

int main() {
    std::remove("prova.db");
    sqlite3* db;
    sqlite3_open("prova.db", &db);                       // su file: ogni COMMIT tocca il disco
    sqlite3_exec(db, "CREATE TABLE t(k INT, v INT)", nullptr, nullptr, nullptr);
    double senza = inserisci(db, 300, false);            // 300 transazioni implicite
    double con = inserisci(db, 100000, true);            // una sola transazione
    std::cerr << "300 righe senza transazione: " << senza << " s, 100000 con: " << con << " s\n";

    const char* q = "EXPLAIN QUERY PLAN SELECT * FROM t WHERE v = 42";
    std::cout << "prima dell'indice:\n";
    sqlite3_exec(db, q, piano, nullptr, nullptr);
    sqlite3_exec(db, "CREATE INDEX idx_v ON t(v)", nullptr, nullptr, nullptr);
    std::cout << "dopo l'indice:\n";
    sqlite3_exec(db, q, piano, nullptr, nullptr);
    sqlite3_close(db);
}
```

```
prima dell'indice:
  SCAN t
dopo l'indice:
  SEARCH t USING INDEX idx_v (v=?)
```

Senza transazione esplicita ogni `INSERT` e' una transazione a se': SQLite deve attendere che il journal arrivi fisicamente sul disco (`fsync`) a ogni riga, e un disco completa solo qualche centinaio (HDD) o migliaio (SSD) di sincronizzazioni al secondo. In una transazione l'`fsync` si fa una volta sola al `COMMIT`: 100.000 righe richiedono una frazione di secondo, contro diversi secondi per poche centinaia di righe senza transazione. Il piano mostra il passaggio da `SCAN` (lettura di tutta la tabella, O(n)) a `SEARCH ... USING INDEX` (B-tree, O(log n)).

Capitolo 34 - Costruire un database

Soluzione 34.1

Si legge un numero alla volta insieme all'operatore che lo precede: `+` e `-` mettono il numero (con segno) sulla pila, `*` e `/` lo combinano subito con la cima, perche' hanno precedenza. Alla fine si somma la pila.

```
#include <cctype>
#include <iostream>
#include <numeric>
#include <string>
#include <vector>

int calcola(const std::string& s) {
    std::vector<long long> pila;
    long long num = 0;
    char op = '+';
    for (std::size_t i = 0; i <= s.size(); ++i) {
        char c = i < s.size() ? s[i] : '+';                 // sentinella finale
        if (std::isdigit(static_cast<unsigned char>(c))) { num = num * 10 + (c - '0'); continue; }
        if (c == ' ') continue;
        if (op == '+') pila.push_back(num);
        else if (op == '-') pila.push_back(-num);
        else if (op == '*') pila.back() *= num;
        else pila.back() /= num;                            // divisione troncata verso zero
        op = c;
        num = 0;
    }
    return static_cast<int>(std::accumulate(pila.begin(), pila.end(), 0LL));
}

int main() {
    std::cout << calcola("3+2*2") << ' ' << calcola(" 3/2 ") << ' ' << calcola("14-3/2*4") << '\n';
}
```

```
7 1 10
```

Soluzione 34.2

Ogni modifica viene **prima** scritta e resa persistente nel log, **poi** applicata in memoria: se il processo muore tra le due operazioni il dato non e' perso. Il recupero rilegge il log dall'inizio. In un motore reale si chiama `fsync` dopo ogni scrittura (o a gruppi, "group commit") e il log viene troncato dopo lo scarico della memtable.

```
#include <fstream>
#include <iostream>
#include <map>
#include <optional>
#include <sstream>
#include <string>

class MemtableConWAL {
    std::map<std::string, std::optional<std::string>> mem_;
    std::ofstream log_;
public:
    explicit MemtableConWAL(const std::string& file) : log_(file, std::ios::app) {}
    void put(const std::string& k, const std::string& v) {
        log_ << "P " << k << ' ' << v << '\n' << std::flush;     // prima il log...
        mem_[k] = v;                                            // ...poi la memoria
    }
    void del(const std::string& k) {
        log_ << "D " << k << '\n' << std::flush;
        mem_[k] = std::nullopt;
    }
    static std::map<std::string, std::optional<std::string>> recupera(const std::string& file) {
        std::map<std::string, std::optional<std::string>> m;
        std::ifstream in(file);
        for (std::string riga; std::getline(in, riga);) {
            std::istringstream ss(riga);
            std::string op, k, v;
            ss >> op >> k;
            if (op == "P" && ss >> v) m[k] = v;
            else if (op == "D") m[k] = std::nullopt;
            // una riga troncata (crash durante la scrittura) viene ignorata
        }
        return m;
    }
};

int main() {
    std::remove("wal.log");
    {
        MemtableConWAL db("wal.log");
        db.put("a", "1"); db.put("b", "2"); db.del("a"); db.put("c", "3");
    }                                                           // "crash": la memoria e' persa
    { std::ofstream("wal.log", std::ios::app) << "P d"; }       // scrittura interrotta a meta'
    for (auto& [k, v] : MemtableConWAL::recupera("wal.log"))
        std::cout << k << '=' << v.value_or("(cancellato)") << ' ';
    std::cout << '\n';
```

```
}
```

```
a=(cancellato) b=2 c=3
```

Soluzione 34.3

```
#include <iostream>
#include <stack>
#include <string>
#include <vector>

int valutaRPN(const std::vector<std::string>& t) {
    std::stack<long long> s;
    for (const auto& x : t) {
        if (x.size() == 1 && std::string("+-*/").find(x[0]) != std::string::npos) {
            long long b = s.top(); s.pop();
            long long a = s.top(); s.pop();                    // attenzione all'ordine
            s.push(x == "+" ? a + b : x == "-" ? a - b : x == "*" ? a * b : a / b);
        } else s.push(std::stoll(x));
    }
    return static_cast<int>(s.top());
}

int main() {
    std::cout << valutaRPN({"2", "1", "+", "3", "*"}) << ' '
              << valutaRPN({"4", "13", "5", "/", "+"}) << ' '
              << valutaRPN({"10", "6", "9", "3", "+", "-11", "*", "/", "*", "17", "+", "5", "+"})
              << '\n';
}
```

```
9 6 22
```

Per convertire l'AST dell'Esempio 34.2 basta aggiungere a `Espr` un metodo virtuale `void rpn(std::vector<std::string>& out) const`: `Costante` e `Colonna` accodano il proprio valore, `Binaria` chiama `a->rpn(out)`, poi `b->rpn(out)`, poi accoda `op` (visita **post-order**). La notazione polacca inversa e' il formato con cui molte macchine virtuali, compresa quella di SQLite (il "bytecode" VDBE), eseguono le espressioni.

Soluzione 34.4

```
#include <array>
#include <iostream>
#include <list>
#include <stdexcept>
#include <unordered_map>

constexpr std::size_t DIM_PAGINA = 4096;
using Pagina = std::array<char, DIM_PAGINA>;

struct Disco {                                      // "disco" simulato con contatori
    std::unordered_map<int, Pagina> pagine;
    int letture = 0, scritture = 0;
    Pagina leggi(int id) { ++letture; return pagine[id]; }
    void scrivi(int id, const Pagina& p) { ++scritture; pagine[id] = p; }
};

class BufferPool {
    struct Frame { int id; Pagina dati; int pin = 0; bool sporca = false; };
    std::size_t capacita_;
    Disco& disco_;
    std::list<Frame> lru_;                          // in testa la piu' recente
    std::unordered_map<int, std::list<Frame>::iterator> mappa_;
public:
    BufferPool(std::size_t c, Disco& d) : capacita_(c), disco_(d) {}
    Pagina& fissa(int id, bool scrittura) {
        auto it = mappa_.find(id);
        if (it == mappa_.end()) {
            if (lru_.size() == capacita_) {         // cerca la vittima meno recente non fissata
                auto v = lru_.end();
                for (auto x = lru_.rbegin(); x != lru_.rend(); ++x)
                    if (x->pin == 0) { v = std::next(x).base(); break; }
                if (v == lru_.end()) throw std::runtime_error("tutte le pagine sono fissate");
                if (v->sporca) disco_.scrivi(v->id, v->dati);
                mappa_.erase(v->id);
                lru_.erase(v);
            }
            lru_.push_front({id, disco_.leggi(id)});
            it = mappa_.emplace(id, lru_.begin()).first;
        } else lru_.splice(lru_.begin(), lru_, it->second);
        it->second->pin++;
        it->second->sporca |= scrittura;
        return it->second->dati;
    }
    void rilascia(int id) { mappa_.at(id)->pin--; }
    void scaricaTutto() {
```

```
        for (auto& f : lru_) if (f.sporca) { disco_.scrivi(f.id, f.dati); f.sporca = false; }
    }
};

int main() {
    Disco d;
    BufferPool bp(3, d);
    bp.fissa(1, false); bp.rilascia(1);
    bp.fissa(2, true)[0] = 'x'; bp.rilascia(2);             // pagina 2 sporca
    bp.fissa(3, false);                                     // la 3 resta fissata
    bp.fissa(1, false); bp.rilascia(1);                     // hit: nessuna lettura
    bp.fissa(4, false); bp.rilascia(4);                     // espelle la 2 (sporca -> scrittura)
    bp.fissa(5, false); bp.rilascia(5);                     // espelle la 1 (la 3 e' fissata)
    std::cout << "letture " << d.letture << ", scritture " << d.scritture
              << ", pagina 2 su disco: " << d.pagine[2][0] << '\n';
    bp.rilascia(3);
    bp.scaricaTutto();
}
```

```
letture 5, scritture 1, pagina 2 su disco: x
```

Soluzione 34.5

Lo heap contiene la prossima voce di ciascuna SSTable; a parita' di chiave si preferisce la tabella piu' recente (indice maggiore) e si saltano le versioni vecchie della stessa chiave.

```
#include <iostream>
#include <queue>
#include <string>
#include <tuple>
#include <utility>
#include <vector>

using SSTable = std::vector<std::pair<std::string, std::string>>;

SSTable fondi(const std::vector<SSTable>& tab) {        // tab[i] piu' recente di tab[i-1]
    using Voce = std::tuple<std::string, int, std::size_t>;   // (chiave, -tabella, posizione)
    std::priority_queue<Voce, std::vector<Voce>, std::greater<>> pq;
    for (int i = 0; i < (int)tab.size(); ++i)
        if (!tab[i].empty()) pq.push({tab[i][0].first, -i, 0});
    SSTable out;
    while (!pq.empty()) {
        auto [k, menoI, pos] = pq.top(); pq.pop();
        int i = -menoI;
        if (out.empty() || out.back().first != k) out.push_back(tab[i][pos]);   // la piu' recente
        if (pos + 1 < tab[i].size()) pq.push({tab[i][pos + 1].first, menoI, pos + 1});
    }
    return out;
}

int main() {
    std::vector<SSTable> t{{{"a", "1"}, {"c", "1"}, {"e", "1"}},
                           {{"b", "2"}, {"c", "2"}},
                           {{"a", "3"}, {"d", "3"}}};
    for (auto& [k, v] : fondi(t)) std::cout << k << '=' << v << ' ';
    std::cout << '\n';
}
```

```
a=3 b=2 c=2 d=3 e=1
```

Ogni voce entra ed esce dallo heap una volta, e lo heap ha al massimo k elementi: O(n log k). Le SSTable vengono lette in sequenza, il che su disco e' molto piu' veloce di accessi casuali.

Capitolo 35 - Piano di studio e preparazione ai colloqui

Soluzione 35.1

L'acqua sopra la barra i vale `min(maxSinistra, maxDestra) - h[i]`. Con due puntatori: il lato con il massimo minore e' quello che limita, e il suo contributo si puo' calcolare subito.

```
#include <algorithm>
#include <iostream>
#include <vector>

int acqua(const std::vector<int>& h) {
    int i = 0, j = (int)h.size() - 1, maxSx = 0, maxDx = 0, tot = 0;
    while (i < j) {
        if (h[i] < h[j]) { maxSx = std::max(maxSx, h[i]); tot += maxSx - h[i]; ++i; }
        else { maxDx = std::max(maxDx, h[j]); tot += maxDx - h[j]; --j; }
    }
    return tot;
}

int main() {
    std::cout << acqua({0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1}) << ' '
              << acqua({4, 2, 0, 3, 2, 5}) << '\n';
}
```

```
6 9
```

Soluzione 35.2

In un colloquio: "`ok[i]` dice se il prefisso di lunghezza i si puo' spezzare; `ok[0]` e' vero (stringa vuota); `ok[i]` e' vero se esiste `j < i` con `ok[j]` vero e `s[j..i)` nel dizionario. La risposta e' `ok[n]`. Con un `unordered_set` di parole e limitando j alle lunghezze delle parole il costo e' O(n * L) controlli, dove L e' la lunghezza massima di una parola."

```
#include <algorithm>
#include <iostream>
#include <string>
#include <unordered_set>
#include <vector>

bool spezzabile(const std::string& s, const std::vector<std::string>& dizionario) {
    std::unordered_set<std::string> d(dizionario.begin(), dizionario.end());
    std::size_t maxL = 0;
    for (auto& p : dizionario) maxL = std::max(maxL, p.size());
    std::vector<bool> ok(s.size() + 1, false);
    ok[0] = true;
    for (std::size_t i = 1; i <= s.size(); ++i)
        for (std::size_t l = 1; l <= std::min(i, maxL) && !ok[i]; ++l)
            ok[i] = ok[i - l] && d.count(s.substr(i - l, l));
    return ok[s.size()];
}

int main() {
    std::cout << spezzabile("leetcode", {"leet", "code"})
              << spezzabile("applepenapple", {"apple", "pen"})
              << spezzabile("catsandog", {"cats", "dog", "sand", "and", "cat"}) << '\n';
}
```

```
110
```

Soluzione 35.3

- **A coppie in sequenza** (fondi la prima con la seconda, il risultato con la terza...): le prime liste vengono ricopiate a ogni passo, O(n k).
- **Divide et impera** (fondi a coppie come nel merge sort: k/2 fusioni, poi k/4...): log k livelli, ciascuno O(n): **O(n log k)**, nessuna struttura extra.

- **Min-heap** con la testa di ogni lista: ogni elemento entra ed esce dallo heap di dimensione k, **O(n log k)**, O(k) spazio extra; e' anche la soluzione naturale quando le liste sono flussi (Soluzione 34.5).

In un colloquio conviene citare la prima, spiegare perche' e' lenta e implementare una delle altre due.

Soluzione 35.4

Una traccia di progetto:

- **API**: `POST /api/link {url}` restituisce `{codice}`; `GET /{codice}` risponde `301` con `Location`. Autenticazione con chiave API per la creazione.
- **Dati**: tabella `link(codice PRIMARY KEY, url, creato, proprietario)`. 10.000 richieste al secondo sono per la maggior parte **letture** (rapporto tipico 100:1).
- **Codici**: contatore distribuito (blocchi di ID assegnati a ogni server, cosi' non serve coordinarsi a ogni richiesta) convertito in base 62; 7 caratteri bastano per 62^7 = 3,5 * 10^12 link.
- **Cache**: le letture passano da una cache LRU in memoria su ogni nodo e da una cache distribuita (Redis/memcached) con consistent hashing; un piccolo insieme di link molto popolari genera la maggior parte del traffico, quindi il tasso di hit e' alto.
- **Rate limiting**: token bucket per chiave API e per IP davanti agli endpoint di creazione.
- **Stime**: 100 creazioni al secondo per cinque anni sono circa 1,6 * 10^10 link; con circa 500 byte per riga (URL, indici, metadati) servono circa 8 TB, da partizionare (sharding per codice). Per la cache, tenere in memoria il 20% dei link letti ogni giorno richiede poche decine di GB.
- **Scalabilita'**: server applicativi senza stato dietro un bilanciatore, repliche di sola lettura del database, CDN per le redirezioni piu' frequenti.